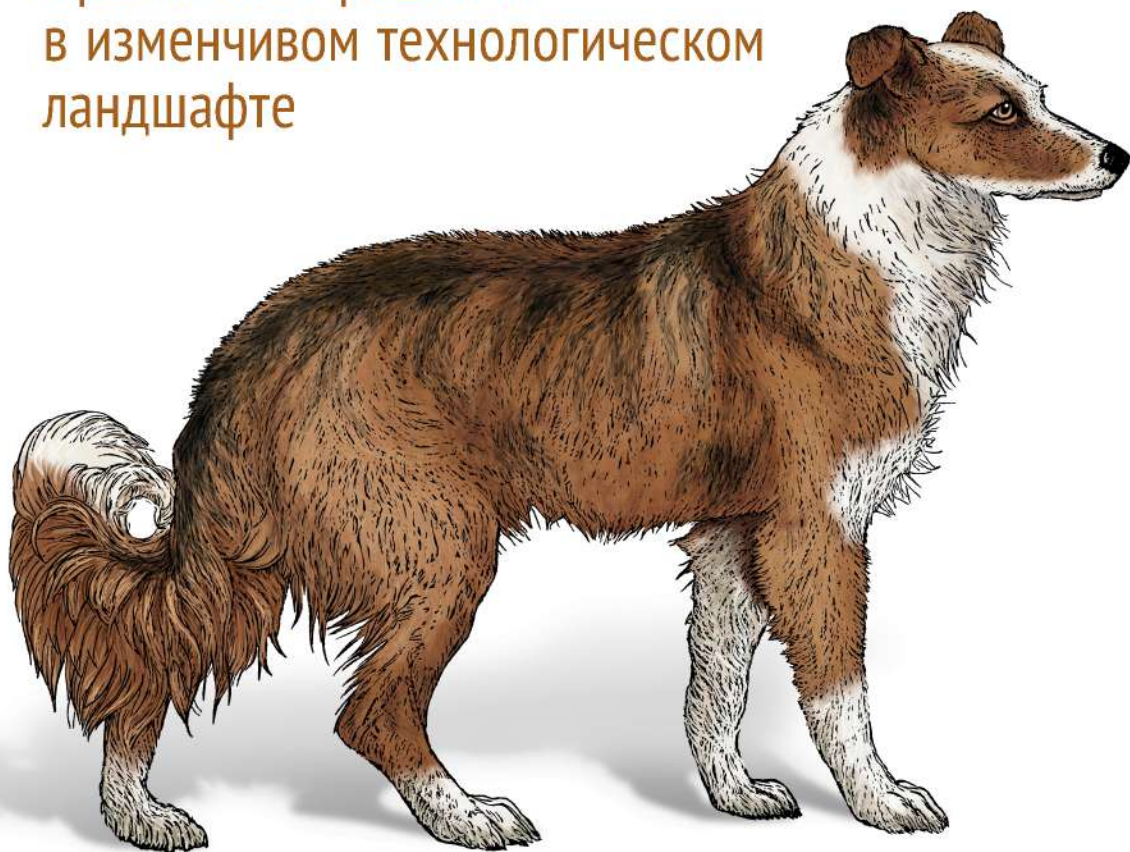


O'REILLY®

2-е издание

Непрерывное развитие API

Правильные решения
в изменчивом технологическом
ландшафте



Мехди Меджуи, Эрик Уайлд
Ронни Митра, Майк Амундсен



Впечатляет, что авторам удалось сделать эту книгу об API в целом, а не о конкретных технологиях. Независимо от выбранной вами технологии API, вы определенно получите ценные рекомендации.

Стефан Тильков, генеральный директор
и главный консультант INNOQ

API — это основа современного предприятия. Эта книга станет вашим руководством по внедрению вездесущих API и управлению ими.

Грегор Хохпе, автор книги *The Software Architect Elevator*

«Непрерывное развитие API» — отличное руководство для тех, кто отвечает за создание и масштабирование своей программы API. От практических советов до глубокого погружения во все аспекты реализации программы API — эта книга станет незаменимым ресурсом для всех, от руководителей до специалистов-практиков API.

Джеймс Хиггинботэм, консультант по API
и автор книги *Principles of Web API Design*

Многочисленные печатные издания подробно описывают тонкости создания веб-API. Однако книга «Непрерывное развитие API» стоит особняком как целостное руководство. Она обязательна к прочтению для руководителей проектов.

Мэтью Рейнболд, автор информационного
бюллетеня *Net API Notes* и директор по экосистемам
API и цифровой трансформации в Postman

Майк, Мехди, Ронни и Эрик написали потрясающую книгу, в которой отражено то, что необходимо для создания и развития сложных систем API, процветающих в цифровом мире.

Хибри Марзук, главный консультант
компании Contino

«Непрерывное развитие API» — всеобъемлющая книга, посвященная управлению продуктами API. Она полна практических рекомендаций, и я знаю, что множество организаций использовали информацию из нее для продвижения своих цифровых стратегий с использованием API.

Мэтт Макларти, руководитель по стратегии API
в MuleSoft, компания Salesforce

SECOND EDITION

Continuous API Management

*Making the Right Decisions
in an Evolving Landscape*

*Mehdi Medjaoui, Erik Wilde,
Ronnie Mitra, and Mike Amundsen*

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Непрерывное развитие API

Правильные решения
в изменчивом технологическом
ландшафте

Мехди Меджуи, Эрик Уайлд,
Ронни Митра, Майк Амундсен

2-е издание



Санкт-Петербург • Москва • Минск

2023

ББК 32.988.02-018
УДК 004.438.5
Н53

Меджуи Мехди, Уайлд Эрик, Митра Ронни, Амундсен Майк

Н53 Непрерывное развитие API. Правильные решения в изменчивом технологическом ландшафте. 2-е изд. — СПб.: Питер, 2023. — 368 с.: ил. — (Серия «Бестселлеры O'Reilly»).

ISBN 978-5-4461-2023-9

Для реализации API необходимо провести большую работу, но эти усилия не всегда окупаются. Чрезмерное планирование может стать пустой тратой сил, а его недостаток приводит к катастрофическим последствиям. Во втором издании представлены решения для отдельных API и систем из нескольких API, которые позволят вам распределить необходимые ресурсы и достичь требуемого уровня эффективности за оптимальное время.

Как соблюсти баланс гибкости и производительности, сохранив надежность и простоту настройки? Четыре эксперта по API объясняют разработчикам, руководителям продуктов и проектам, как максимально увеличить ценность их API, управляя интерфейсами как продуктами с непрерывным жизненным циклом.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.438.5

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1098103521 англ.

Authorized Russian translation of the English edition of Continuous API Management, 2E ISBN 9781098103521 © 2022 Mehdi Medjaoui, Build Digital GmbH, Kudo & Leap Ltd., and Amundsen.com, Inc.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same

ISBN 978-5-4461-2023-9

© Перевод на русский язык ООО «Прогресс книга», 2022

© Издание на русском языке, оформление ООО «Прогресс книга», 2022

© Серия «Бестселлеры O'Reilly», 2022

Краткое содержание

Предисловие к первому изданию	14
Вступление	16
Благодарности	20
От издательства	21
Глава 1. Сложности и перспективы управления API.....	22
Глава 2. Руководство API.....	40
Глава 3. API как продукт	72
Глава 4. Основы API-продукта	111
Глава 5. Непрерывное улучшение API.....	155
Глава 6. Стили API	176
Глава 7. Жизненный цикл API-продуктов	192
Глава 8. Команды API	228
Глава 9. Ландшафты API	258
Глава 10. Путешествие по ландшафту API.....	285
Глава 11. Управление жизненным циклом API в меняющемся ландшафте	314
Глава 12. Продолжение путешествия	362
Об авторах	366
Иллюстрация на обложке	367

Оглавление

Предисловие к первому изданию	14
Вступление	16
Для кого эта книга	16
Что вас ждет в книге	17
Структура издания	17
Чего вы здесь не найдете	19
Условные обозначения	19
Благодарности	20
От издательства	21
Глава 1. Сложности и перспективы управления API	22
Что такое развитие API	24
API в бизнесе	24
Что такое API	26
Стили API	28
Больше чем просто API	28
Стадии API	29
Больше одного API	30
Сложности в управлении API	30
Объем	31
Масштаб	32
Стандарты	33
Управление ландшафтом API	34
Технология	35
Команды	36
Руководство	37
Итоги главы	38

Глава 2. Руководство API	40
Что такое руководство API	41
Решения	41
Управление принятием решений	42
Руководство сложными системами	43
Управление решениями	46
Централизация и децентрализация.....	47
Элементы решения	52
Планирование решений.....	58
Проектирование решений на практике	60
Разработка системы руководства	61
Шаблон руководства 1: ответственные разработчики	63
Шаблон руководства 2: внедренные централизованные эксперты	64
Шаблон руководства 3: влияние на самоуправление	66
Внедрение моделей руководства	67
Разработка решения	67
Наблюдаемость и наглядность.....	69
Рабочие модели	69
Разработка стратегии управления стандартами.....	70
Итоги главы	71
Глава 3. API как продукт	72
Программируемая экономика, основанная на API.....	73
Цена, продвижение, продукт, место → везде	74
Дизайн-мышление	76
Соответствие требованиям пользователей	77
Конкурентная бизнес-стратегия.....	78
Устав Безоса	78
Применение дизайн-мышления к API	80
Поддержка новых клиентов	81
Время до «вау!»	82
Поддержка новых клиентов API	84
Опыт разработчика	86
Познакомьтесь с целевой аудиторией	87
Проще и безопаснее	92

Почему разработчики так важны в экономике API	95
Отношения с разработчиками API.....	97
Монетизация и ценообразование API как продукта.....	106
Итоги главы	109
Глава 4. Основы API-продукта	111
Знакомство со столпами	112
Стратегия	112
Дизайн	116
Документация	119
Разработка	122
Тестирование.....	126
Развертывание.....	129
Безопасность	132
Мониторинг.....	136
Обнаружение.....	138
Управление изменениями	141
Совместное использование столпов	142
Применение столпов во время планирования	143
Использование столпов для реализации.....	146
Использование столпов для управления и запуска.....	150
Итоги главы	154
Глава 5. Непрерывное улучшение API	155
Непрерывное управление изменениями	156
Постепенное улучшение	157
Скорость изменения API	162
Изменения API	164
Жизненный цикл релиза API	164
Изменение модели интерфейса	166
Изменения в реализации.....	168
Изменения в экземпляре.....	169
Изменения в ресурсах поддержки.....	169
Улучшение изменяемости API	170
Стоимость ресурсов	171

Альтернативные издержки	171
Затраты из-за связанности	172
Разве все это не просто BDUF?	174
Итоги главы	175
Глава 6. Стили API	176
API — это языки.....	177
Пять стилей API.....	179
Туннельный стиль.....	179
Ресурсный стиль	181
Гипермедийный стиль	183
Стиль запросов.....	185
Событийный стиль	187
Как выбрать стиль и технологию API.....	188
Не загоняйте себя в угол стиля	190
Итоги главы	191
Глава 7. Жизненный цикл API-продуктов	192
Измерения и ключевые этапы.....	193
OKR и KPI	193
Определение цели API	195
Определение измеримых результатов.....	196
Жизненный цикл API-продукта	198
Стадия 1: создание	199
Стадия 2: публикация.....	203
Стадия 3: окупаемость.....	207
Стадия 4: поддержка	210
Стадия 5: удаление	212
Применение жизненного цикла продукта к столпам.....	215
Создание	216
Публикация	219
Окупаемость	223
Поддержка.....	225
Удаление	226
Итоги главы	227

Глава 8. Команды API	228
Обязанности в команде API	230
Бизнес-роли	231
Технические обязанности	233
Команды API	235
Команды и зрелость API	236
Масштабирование команд	243
Команды и обязанности в Spotify	243
Особенности вашего подхода к масштабированию	245
Культура и команды	247
Закон Конвея	249
Разумное использование чисел Данбара	250
Работа с культурной мозаикой по Александру	252
Поддержка экспериментов	254
Итоги главы	256
Глава 9. Ландшафты API	258
Археология API	260
Управление API в больших масштабах	262
Принцип платформы	263
Принципы, протоколы и шаблоны	265
Ландшафты API как языковые системы	267
API через API	268
Понимание ландшафта	270
Восемь аспектов ландшафта API	271
Разнообразие	271
Словари	273
Объем	277
Скорость	278
Уязвимость	279
Видимость	280
Контроль версий	281
Изменяемость	283
Итоги главы	284

Глава 10. Путешествие по ландшафту API	285
Структурирование руководства в ландшафте API	286
Жизненный цикл руководства	289
Центр поддержки	290
Команда C4E и контекст	292
Зрелость и восемь аспектов.....	294
Разнообразие	295
Словари.....	297
Объем.....	300
Скорость	302
Уязвимость.....	304
Видимость	307
Контроль версий.....	309
Изменяемость	311
Итоги главы	313
 Глава 11. Управление жизненным циклом API	
в меняющемся ландшафте	314
Управление развивающимся ландшафтом на практике	315
Организуите в коллективе свои «красные линии»	315
Платформы важнее проектов (в конечном итоге).....	316
Дизайн для потребителей, производителей и спонсоров	318
Тестировать, измерять и учиться	319
Продукты API и столпы жизненного цикла	320
Ландшафты API.....	321
Момент принятия решения и развитие	322
Аспекты ландшафта и столпы жизненного цикла API	322
Стратегия	324
Дизайн	326
Документация	329
Разработка	333
Тестирование.....	336
Развертывание.....	342
Безопасность	346

Мониторинг.....	350
Обнаружение.....	352
Управление изменениями.....	357
Итоги главы	360
Глава 12. Продолжение путешествия	362
Продолжайте готовиться к будущему.....	364
Не останавливайтесь	364
Об авторах.....	366
Иллюстрация на обложке.....	367

Моим коллегам, которые помогали мне быть полезным в отрасли, Кину Лейну, который заразил меня своей страстью к API, и всем специалистам по API, которые поделились со мной своими практиками API, вдохновившими меня на создание этой книги. Моим родителям.

Мехди Меджуи

Всем людям в моей жизни, благодаря которым эта книга стала возможной. Это было то еще приключение!

Эрик Уайлд

Кайраву — за то, что помог мне написать это посвящение.

Ронни Митра

Всем компаниям, которые пригласили нас поделиться тем, чему мы научились, и в процессе научили нас столькому, что мы должны были попытаться отразить это в этой книге.

Майк Амундсен

Предисловие к первому изданию

API нужны любой компании, организации, учреждению или государственному органу, чтобы правильно управлять своими цифровыми ресурсами в условиях постоянно изменяющейся цифровой среды.

Диджитал-трансформация, происходящая последние пять лет, привела к изменениям в среде API. Компании перестали задавать вопрос: *«Нужно ли нам использовать API?»* и начали искать информацию, *как правильно это делать*. В организациях понимают, что недостаточно просто создать API. Многое зависит от разработки всего его жизненного цикла. Авторы книги «Непрерывное развитие API» хорошо понимают, что нужно для последовательного, масштабного и воспроизводимого перехода API от идеи к реализации. Это позволяет им стать уникальными наставниками в данной сфере.

Большинство специалистов работают с ландшафтом API, охватывающим только один набор API. Авторы этой книги обладают уникальным опытом работы с системой ландшафта API, охватывающей тысячи API, несколько отраслей и крупнейших современных предприятий. Высокоталантливых специалистов в этой области по всему миру можно пересчитать по пальцам. И Меджуи, Уайлд, Митра и Амундсен всегда будут первыми, кого я назову. Благодаря своему богатому опыту авторы помогают понять, как провести API от идеи до проекта, от разработки до производства. Нет другой команды экспертов по API, обладающей такими же уникальными знаниями. Эта книга может стать вашей настольной — тем зачитанным до дыр томом, к которому вы будете возвращаться снова и снова.

Я прочел много книг по техническим аспектам создания API, включая литературу на такие темы, как гипермедиа и все, что нужно знать о REST, или как реализовать свое видение с помощью разных языков программирования и платформ. Но это первая прочитанная мной книга, подходящая для разработки API целиком. Она учитывает не только технологические детали, но и важные моменты использования API в бизнесе, включая человеческий фактор при их изучении, реализацию и активацию API на крупных предприятиях.

В издании подробно описаны элементы, важные для любого разработчика при создании надежных, безопасных и целостных API в нужном масштабе. Книга

поможет любой команде по созданию API определить требуемое число операций и обдумать улучшения и изменения API с критической точки зрения. Это пособие устанавливает и определяет структурированный, но гибкий подход к выпуску API по одному стандарту для разных команд.

Прочитав эту книгу, я почувствовал, что по-новому взглянул на современный жизненный цикл API. У меня появилось много идей о том, как я могу оценить и измерить свои операции API и стратегию их жизненного цикла, которую сейчас использую для управления этими операциями. Несмотря на мою высокую компетенцию в данной области, книга заставила меня посмотреть на эту среду по-новому. Я почувствовал, что наполнился информацией, которая не только освежила мои знания, но и изменила представления о том, что мне *казалось* известным. Она заставила меня расти над собой и развиваться в своей сфере деятельности.

На пути разработки API для меня главное — постоянно бросать себе вызов, учиться, планировать, выполнять, оценивать и повторять, пока не добьешься желаемого результата. «Непрерывное развитие API» отражает реальность разработки API. Это руководство по технологии, бизнесу и политике создания API в масштабе целого предприятия.

Советую прочитать эту книгу несколько раз. Прочтите, а потом реализуйте свои идеи. Улучшите свою стратегию API, определите собственную версию жизненного цикла API, используя то, что узнали у Меджуи, Уайлда, Митры и Амундсена. По всей книге разбросаны маленькие бриллианты знаний, новые грани которых вы будете открывать при каждом прочтении. Это поможет лучше понять происходящее в среде API и увереннее заявить о себе.

Ким Лейн, евангелист API

Вступление

Добро пожаловать на страницы второго издания «Непрерывного развития API»! Во вступлении издания 2018 года говорилось:

Когда общество и бизнес естественным образом тесно переплелись с цифровыми технологиями, спрос на взаимосвязанное программное обеспечение резко вырос. В свою очередь, интерфейс для программирования приложений (API) стал важным ресурсом для современных компаний, потому что он упрощает установление связей между ПО. Но управление этими API стало вызывать новые сложности. Чтобы получить от API максимум, необходимо научиться управлять их разработкой, запуском, ростом, качеством и безопасностью, учитывая сложные факторы контекста, времени и масштаба.

Сейчас мало что изменилось, когда речь идет о росте API и проблемах управления ими. Хорошая новость: за годы с момента выхода нашего первого издания появилось большое количество инструментов и обучающих материалов, что помогло расширить и развить пространство управления API. Плохая новость: множество организаций все еще сталкиваются с трудностями, когда пытаются подключить людей, услуги и компании с помощью API. В новом издании мы хотим поделиться свежей информацией о том, как развиваются компании, рассказать о новых историях успеха и уточнить некоторые материалы из книги 2018 года.

Мы добавили новые примеры и обновили существующие, но по-прежнему сохранили тот же базовый подход и основные положения. Надеемся, эти изменения помогут вам расширить свои познания на пути к непрерывному управлению API.

Для кого эта книга

Если вы только начинаете разрабатывать программу внедрения и использования API и вам нужно понять, что вас ждет, или у вас уже есть API, но вы хотите научиться лучше управлять ими, — эта книга для вас.

Здесь мы попытались создать структуру управления API, которую можно применять более чем к одному контексту. На страницах вы найдете руководство, способное помочь вам управлять одним API, которым вы хотите поделиться

с разработчиками по всему миру, и советы по созданию сложного набора API в микросервисной архитектуре, предназначенной только для внутренних разработчиков.

Мы постарались сделать книгу максимально технологически нейтральной. Советы и анализ, которыми мы делимся, подходят для любой архитектуры на основе API, включая HTTP (HyperText Transfer Protocol — протокол передачи гипертекста), CRUD (акроним, обозначающий четыре основные функции, используемые при работе с базами данных: создание (create), чтение (read), модификация (update), удаление (delete)), REST (Representational State Transfer — передача репрезентативного состояния), GraphQL и событийно-ориентированные стили взаимодействия. Это книга для тех, кто хочет принимать более эффективные решения при работе с API.

Что вас ждет в книге

Здесь изложен весь наш многолетний опыт проектирования, разработки и улучшения API — как своих, так и чужих. Мы определили два ключевых фактора эффективной разработки API (продуктоориентированный подход и создание правильной команды) и три фактора управления этой работой: руководство, развитие продукта и разработка ландшафта API.

Это пять базовых составляющих успешной программы по управлению API. Мы знакомим читателя со всеми этими темами и помогаем вписать их в контекст организации.

Структура издания

Книга построена так, чтобы список важных вопросов рос по мере чтения. Мы начнем с основных понятий управления на основе решений и концепции «API как продукт», а затем сделаем обзор всей работы для создания API-продукта.

Далее к простому обзору одного API добавим аспект времени и тщательнее рассмотрим принцип управления изменениями API и то, как на это влияет развитие API. После этого вас ждет подробное рассмотрение команд и специалистов, занимающихся этими изменениями. Вторая половина книги посвящена сложности масштаба ландшафта API и управления им.

Ознакомимся с кратким содержанием всех глав.

- Глава 1 «Сложности и перспективы управления API» знакомит вас со сферой управления API.

- В главе 2 «Руководство API» управление рассматривается с точки зрения принятия решений — базовой концепции управления API.
- В главе 3 «API как продукт» мы рассказываем о важности концепции «API как продукт» для любой стратегии API.
- Глава 4 «Основы API-продукта» освещает десять базовых принципов работы в сфере API-продуктов, которые формируют систему задач по принятию решений, которыми нужно управлять.
- Глава 5 «Непрерывное улучшение API» дает представление о том, что означает постоянное изменение API. Мы рассказываем о необходимости принять идею постоянных изменений и знакомим с разными типами изменений в API и их последствиями.
- Глава 6 «Стили API» — новая глава в этом издании. В ней рассматриваются пять самых распространенных стилей API, встречающихся в компаниях по всему миру, и анализируются их сильные и слабые стороны, чтобы вы смогли выбрать подходящий для каждого конкретного случая.
- Глава 7 «Жизненный цикл API-продуктов» рассматривает жизненный цикл продукта API — структуру, которая поможет вам управлять работой API по десяти основным направлениям в течение всего времени работы API-продукта.
- Глава 8 «Команды API» рассказывает о человеческом факторе управления API — типичных ролях, зонах ответственности и схемах разработки для команды, разрабатывающей API.
- Глава 9 «Ландшафты API» добавляет перспективу масштабирования к проблеме управления API. В ней приведены «восемь V» — разнообразие (variety), словари (vocabulary), объем (volume), скорость (velocity), уязвимость (vulnerability), видимость (visibility), контроль версий (versioning) и изменчивость (volatility), — которые нужно учитывать при одновременном изменении нескольких API.
- В главе 10 «Путешествие по ландшафту API» мы освещаем подход непрерывной разработки системы для постоянного и согласованного управления изменениями API.
- Глава 11 «Управление жизненным циклом API в меняющемся ландшафте» сопоставляет перспективу ландшафта с перспективой API как продукта и определяет, как меняется работа API, когда вокруг него развивается ландшафт.
- Глава 12 «Продолжение путешествия» суммирует изложенную информацию по управлению API и дает советы о том, как подготовиться к будущей работе и сразу же начать ее.

Чего вы здесь не найдете

Сфера управления API обширна и содержит огромное множество вариантов контекстов, платформ и протоколов. К сожалению, в эту книгу просто не поместятся все конкретные способы реализации API. Это не руководство по разработке REST API или выбору шлюза безопасности. Здесь вы не найдете пошаговую инструкцию по написанию кода для API или разработке HTTP API.

Хоть мы и приводим примеры конкретных практик, наша книга не о реализации API — в этом вам помогут другие источники. Она затрагивает редко упоминаемую проблему: как эффективно управлять работой по созданию API в сложной и меняющейся организационной системе.

Условные обозначения

В книге применяются следующие обозначения.

Курсив

Обозначает новые термины и слова.

Рубленый шрифт

Им набраны URL, адреса электронной почты.

Моноширинный шрифт

Обозначает программные элементы: названия переменных или функций, типы данных, утверждения и ключевые слова, названия и расширения файлов.



Обозначает совет или предложение.



Обозначает примечание.



Обозначает предупреждение.

Благодарности

Еще раз хотим поблагодарить многих за помощь и поддержку, которую мы получили, собирая новые материалы для этого издания.

В первую очередь благодарим всех, с кем консультировались и у кого нам повезло взять интервью, а также тех, кто посетил наши семинары и вебинары. Мы получили от вас прекрасную обратную связь и на каждой встрече узнавали что-то новое.

Дополнительная благодарность сотрудникам NGINX, которые вдохновили нас на доработку этой книги и помогли с финансированием. Отдельное спасибо всем, кто читал ранние черновики и помог нам сформировать окончательный вариант книги.

Мы также хотели бы поблагодарить Джеймса Хиггинботэма, Хибри Марзука, Марьюкку Нииниойя и Мэтью Рейнболда за все то время, которое они потратили на чтение и рецензирование нашей работы.

И конечно, все это было бы невозможно без поддержки сотрудников O'Reilly Media. Мы благодарим Мелиссу Даффилд, Гэри О'Брайена, Кейт Гэллоуэй, Ким Уимпсетт и многих других, кто посвятил свое время и талант тому, чтобы помочь нам собрать все воедино.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

ГЛАВА 1

Сложности и перспективы управления API

Управление — это прежде всего занятие на стыке искусства, науки и ремесла.

Генри Минцберг

Согласно отчету IDC за 2019 год, 75 % опрошенных компаний ожидают «цифровой трансформации» в следующем десятилетии и что 90 % всех новых приложений будут иметь микросервисную архитектуру на базе API¹. Было также отмечено, что для организаций, ориентированных на API, до 30 % дохода генерируется через цифровые каналы. В то же время эти компании определили ключевые препятствия для внедрения API — «сложность», «безопасность» и «управление».

Вот один из важных итоговых выводов: «Определение правильной архитектуры приложений требует глубокого понимания проблем, связанных с регулированием, управлением и координацией этих основных технологических компонентов»².

Это исследование, как и исследование Коулмана Паркса (<https://oreil.ly/pAWGs>), которое мы приводили в первом издании, одновременно и ободряет, и предупреждает.

¹ Thomson J., Mironescu G. APIs: The Determining Agents Between Success or Failure of Digital Business («API: факторы, определяющие успех или провал цифрового бизнеса») // IDC, <https://oreil.ly/9yshw>.

² Там же.

Интересная тенденция, которую мы наблюдаем последние несколько лет, — увеличивающийся разрыв между «имеющими API» и «не имеющими API». Например, на вопрос «Есть ли у вашей компании платформа для управления API?» 72 % компаний в сфере медиа и услуг ответили «да», тогда как в производственном секторе утвердительно ответили только 46 % компаний¹. Все указывает на то, что API будут и дальше стимулировать рост бизнеса, и крайне важно, чтобы компании из всех ниш приняли вызов цифровой трансформации.

Хорошая новость: многие организации успешно управляют своими программами API. Новость похуже: либо их опыт и знания тяжело передать, либо они не всегда доступны. Чаще всего фирмам, успешно управляющим API, просто некогда поделиться опытом с другими.

Мы имели опыт общения с компаниями, которые неохотно делятся своими знаниями в этой области. Они уверены, что навыки в этой сфере — конкурентное преимущество, и редко открывают свои тайны.

И даже когда организации делятся опытом на конференциях, в статьях или блогах, эта информация обычно очень узконаправленная и для других фирм, как правило, бесполезна.

Здесь мы попытаемся разобраться с последней из этих проблем — превращением специфичных примеров в опыт, которым могут воспользоваться любые организации. Мы посетили десятки компаний, разговаривали со многими специалистами по API и попытались найти связующие нити между примерами, которыми они поделились с нами и общественностью. Вся книга посвящена рассмотрению нескольких тем, с которыми вы ознакомитесь в этой главе.

Главная сложность — понять, что именно люди подразумевают под API. Во-первых, термин API может относиться просто к *интерфейсу* (например, URL HTTP-запроса и возвращаемый JSON). Во-вторых, он может применяться к коду и элементам развертывания, необходимым для создания доступного сервиса (например, *customerOnBoarding API*). И в-третьих, иногда этот термин относится к конкретному *экземпляру приложения*, предоставляющего доступ через API (например, *customerOnBoarding API*, запущенный в облаке AWS, в отличие от *customerOnBoarding API*, запущенного в облаке Azure).

Другая важная проблема в управлении API — это разница между работой по проектированию, созданию и выпуску *одного API* и поддержкой и управлением *множества API* — так называемым *ландшафтом API*. В книге мы подробно

¹ Там же.

рассмотрим все эти проблемы. В работе с одним API могут помочь такие понятия, как «API как продукт» (API as a product, AaaP), и важные навыки для создания и поддержки API (они же *базовые принципы API*). Мы поговорим и о таких важных аспектах, как роль модели зрелости API и работа с постепенными его изменениями.

Другая сторона проблемы — управление ландшафтом API. В данном случае ландшафт — это API всех бизнес-доменов, запущенные на всех платформах и управляемые всеми командами, разрабатывающими API в вашей компании. Есть несколько аспектов этой ландшафтной проблемы, в том числе то, как масштаб и контекст изменяют способ разработки и реализации API, и то, как большие экосистемы могут создавать дополнительную нестабильность и уязвимость только из-за своего размера.

Наконец, мы коснемся принятия решений во время управления экосистемой API. Это ключ к созданию успешного плана *управления* вашими API. Способ принятия решений должен меняться вместе со средой: ваша верность старым моделям управления может помешать программе достичь успеха и поставить под удар существующие API.

Прежде чем начать изучение способов работы с индивидуальными API и ландшафтом API, рассмотрим два важных вопроса: что такое управление API и в чем его сложность?

Что такое развитие API

Как уже говорилось, развитие (управление) API включает в себя не только управление разработкой, внедрением и реализацией одного API. В него также входят и управление экосистемой API, распределение решений внутри организации и даже перемещение уже существующих API в ваш растущий ландшафт. В этом разделе мы остановимся на каждом из пунктов, но сначала важно пояснить главную причину внедрения API — *API в бизнесе*.

API в бизнесе

Помимо деталей создания API и управления ими важно помнить, что вся эта работа предназначена для поддержания бизнес-целей и задач. API — это не только технические детали JSON или XML, синхронности или асинхронности и т. д. Это способ соединять подразделения бизнеса, делиться важными знаниями и функциями, увеличивая эффективность работы компании. Часто API — это

возможность раскрыть то ценное, что уже есть в организации, например, при создании новых приложений, открытии новых источников дохода и создании нового бизнеса.

Такой способ мышления фокусируется на нуждах потребителей API, а не тех, кто их создает и публикует. Этот клиентоориентированный подход обычно называют «работа, которая должна быть выполнена», или JTBD (Jobs to Be Done). Его предложил Клейтон Кристенсен из Гарвардской школы бизнеса, и в своих книгах *The Innovator's Dilemma*¹ и *The Innovator's Solution*² (Harvard Business Review Press) он тщательно исследует возможности такого подхода. Его основная идея в том, что для запуска успешной API-программы и управления ею нужно помнить: API важны в первую очередь для решения задач бизнеса. По нашему опыту, компании, которые умеют применять API для бизнеса, относятся к своим API как к продуктам для «выполнения работы» в том же смысле, в каком JTBD Кристенсена решает проблемы потребителей.

- *Доступ к данным.* Один из способов, которым API может помочь бизнесу, — это упрощение доступа к важным данным о клиентах или рынке, которые можно сопоставить с возникающими тенденциями или уникальным поведением в новых нишах рынка. API обеспечивает безопасный и легкий доступ к этим данным (анонимизированным и отфильтрованным), помогая вашему бизнесу открывать новые возможности, реализовывать новые продукты или услуги или даже создавать новые инициативы быстрее и дешевле.
- *Доступ к продуктам.* Еще один способ, которым программа API может помочь бизнесу, — создание гибкого набора «инструментов» (API) для новых решений без больших затрат. Например, если у вас есть API онлайн-продаж **OnlineSales**, позволяющий важным партнерам отслеживать активность своих продаж и управлять ею, и API маркетинга и акций **MarketingPromotions**, позволяющий специалистам разрабатывать и отслеживать рекламные кампании продукта, то у вас есть возможность создать новое партнерское решение: приложение для отслеживания продаж и рекламы **SalesAndPromotions**.
- *Доступ к нововведениям.* Во многих компаниях есть внутренние процессы, практики и производственные конвейеры, которые эффективны, но нецелесообразны. Некоторые из них существуют уже давно (иногда — десятилетия). Нам встречались случаи, когда никто не мог вспомнить дату или причину внедрения в организацию того или иного процесса.

¹ Кристенсен К. Дилемма инноватора. Как из-за новых технологий погибают сильные компании.

² Кристенсен К., Рейнор М. Решение проблемы инноваций в бизнесе. Как создать растущий бизнес и успешно поддерживать его рост.

Изменить существующие процессы нелегко и временами очень дорого. Создавая инфраструктуру API в своей компании, вы сможете раскрыть творческий потенциал своей организации и обойти механизмы контроля доступа. Это позволит повысить эффективность внутри вашей компании.

Эти важные аспекты (АааР) мы рассматриваем в главе 3. Но сначала давайте кратко пройдемся по термину *API*.

Что такое API

Иногда, используя термин API, люди подразумевают не только интерфейс, но и функциональность — код, скрывающийся за интерфейсом. Например, можно сказать: «Нам нужно поскорее выпустить обновленный клиентский API, чтобы другие команды смогли задействовать новые функции поиска, которые мы добавили». В других случаях этот термин может применяться только в отношении деталей самого интерфейса. Например, сотрудник вашей команды может сказать: «Я бы хотел разработать новый JSON API для существующих SOAP-сервисов добавления клиентов в систему (customer onboarding)». Оба случая употребления термина верны, и в обоих понятно, что имелось в виду, но иногда можно и запутаться.

Чтобы было проще вести разговор об интерфейсе и функциональности, мы введем несколько дополнительных терминов: интерфейс, реализация и экземпляр (приложения).

Интерфейс, реализация и экземпляр. Сокращение API обозначает «*программный интерфейс приложения*». Мы используем интерфейс для получения доступа к тому, что работает «под капотом» API. Например, у вас может быть API, отображающий задачи управления учетными записями пользователей. Этот интерфейс может позволить разработчикам:

- открыть новую учетную запись;
- редактировать профиль уже существующей учетной записи;
- изменить (заморозить или активировать) статус учетной записи.

Такой интерфейс обычно предоставляется через такие общепотребимые протоколы, как HTTP, MQTT, Thrift, TCP/IP и т. д., и опирается на такие стандартизированные форматы, как JSON, XML, YAML или HTML.

Но это только интерфейс. Нужно еще что-то для выполнения задач. Это что-то мы в дальнейшем будем называть *реализацией*. Реализация — та часть системы, которая обеспечивает фактическую функциональность. Обычно она написана на языке программирования (Java, C#, Ruby, Python и т. п.).

В рамках примера с пользовательской учетной записью реализация **UserManagement** (Управление пользователями) может содержать возможность создавать, добавлять, редактировать и удалять пользователей. Затем эта функциональность передается с помощью вышеупомянутого интерфейса.



Отделение интерфейса от реализации

Обратите внимание, что функциональность описанной реализации — это простой набор действий с использованием шаблона «создать, прочесть, обновить, удалить» (CRUD), а в упомянутом интерфейсе есть три действия: **OnboardAccount** (Активировать учетную запись), **EditAccount** (Редактировать учетную запись) и **ChangeAccountStatus** (Изменить статус учетной записи). Это кажущееся несовпадение между реализацией и интерфейсом широко распространено и может оказаться полезным. Оно разъединяет конкретную реализацию каждого сервиса и интерфейс, используемый для доступа к нему, упрощая изменения без прерывания работы.

Третий термин в нашем списке — *экземпляр*. Экземпляр приложения с API — это сочетание интерфейса и реализации. Это удобный способ рассказать о реальном работающем API, который был запущен в производство. Экземплярами управляют с помощью системы показателей, благодаря которым можно убедиться в их работоспособности. Экземпляры регистрируют и документируют, чтобы разработчикам было проще найти и использовать API для решения реальных проблем. Их защищают, чтобы только авторизованные пользователи могли выполнять действия и читать/записывать необходимые данные.

На рис. 1.1 можно увидеть отношения между тремя этими составляющими. В нашей книге под API чаще всего подразумевается экземпляр API — полностью рабочее сочетание интерфейса и реализации. Если же мы захотим сделать акцент *только* на интерфейсе или *только* на реализации, мы подчеркнем это в тексте.

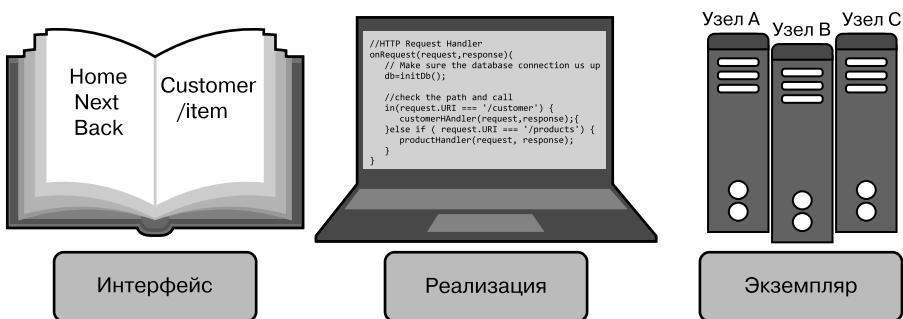


Рис. 1.1. Три элемента API

Стили API

Еще один важный элемент API — то, что можно назвать *стилем*. Как и стили в других областях (живопись, декор, мода и архитектура), стили API — это последовательные идентифицируемые подходы к созданию и использованию API. Важно знать, какой стиль API хотят использовать ваши клиентские приложения, и обеспечить его последовательное применение при создании своих реализаций API.

Самый распространенный стиль API на сегодняшний день — REST, или RESTful API. Но это только один из множества вариантов. На самом деле сегодня все чаще можно наблюдать, как организации разных размеров используют интерфейсы, отличные от REST и не связанные с HTTP. Развитие событийно-ориентированной архитектуры (EDA) — один из примеров этой новой реальности для управления API.



Несмотря на существование множества уникальных стилей, есть пять общепринятых, о которых важно знать при управлении программой API. Мы расскажем о важности стилей API и рассмотрим каждый из них в главе 6.

Компания редко может обойтись только одним стилем API. И даже если вы выберете какой-то универсальный, вряд ли он просуществует вечно. Учет стиля при проектировании, внедрении и управлении экосистемой API очень важен для успеха и стабильности вашей программы API.

Такая многостилевая реальность API приводит к другому важному аспекту успешных API-программ: способности согласованно и последовательно управлять множеством API.

Больше чем просто API

В книге мы не ограничиваемся только рассмотрением API — техническими деталями его интерфейса и реализации. Конечно, традиционные элементы «проект — разработка — реализация» важны для ваших API. Но *управление* API также подразумевает и тестирование, документирование и публикацию их на портале, чтобы пользователи (разработчики организации, партнеры, сторонние анонимные разработчики приложений и т. д.) могли их найти и правильно применить.

API важно защищать, следить за их работой и поддерживать ее (включая все изменения) в течение всего срока их службы. Такие дополнительные действия мы называем *базовыми принципами API*. Это элементы, необходимые для всех API, с которыми приходится сталкиваться специалистам в этой области. Базовые принципы мы подробнее рассмотрим в главе 4, где изучим список из десяти основных методик, необходимых для создания и поддержки качественных API.

Плюс этих методик в том, что они применимы к любым API. Например, навык хорошего документирования API может передаваться от одной команды к другой. То же и с изучением навыков тестирования, схем безопасности и т. д. Это значит, что даже если для каждого домена API будет своя команда (по продажам, по продукту, бэк-офис и т. д.), то ее члены все равно будут связаны между собой, так как их интересы будут пересекаться¹.

Создание и организация команд, разрабатывающих API, — еще один важный аспект управления ими. Подробнее об этом мы поговорим в главе 8.

Стадии API

Понимание базовых принципов API — это еще не все. Каждый API в вашей программе проходит серию определенных полезных этапов. Зная, в какой точке пути находитесь, вы сможете определить, сколько времени и ресурсов вкладывать в API в данный момент. Понимание принципа *развития* API позволяет вам распознавать одинаковые этапы для широкого спектра API и помогает верно реагировать на меняющиеся запросы ко времени и энергии на каждом.

Может показаться правильным при планировании, разработке и релизе API учитывать все их базовые принципы. Но это не так. Часто на ранних этапах лучше всего уделить внимание планированию и разработке, затрачивая меньше сил, допустим, на документацию.

На других этапах (например, когда прототип оказывается в руках бета-тестеров) важнее уделять больше времени мониторингу использования API и его защите от неправомерного применения. Чем лучше вы будете ориентироваться в этих этапах, тем проще вам будет распределить свои ресурсы для достижения максимального результата. Мы подробнее поговорим об этом в главе 7.

¹ На стриминговом музыкальном сервисе Spotify эти пересекающиеся группы называются гильдиями. Больше информации вы найдете в разделе «Масштабирование команд» на с. 243.

Больше одного API

Многие знают, что управление большим количеством API влечет за собой серьезные изменения. У нас есть клиенты, которым нужно разработать и отслеживать тысячи API и управлять ими в течение долгого времени. В такой ситуации люди меньше сосредоточены на деталях реализации отдельного API и больше — на том, как все эти API сосуществуют в постоянно растущей динамичной экосистеме. Как уже упоминалось, мы называем эту экосистему ландшафтом API. Ей будут посвящены несколько глав во второй половине книги.

В такой ситуации сложно обеспечить некоторый уровень согласованности, не вызывая задержек из-за централизованного управления и проверки всех деталей API. Обычно такого уровня достигают, передавая ответственность за эти вопросы отдельным командам API и сосредотачивая усилия центрального управления на упорядочении взаимодействия API между собой. Это обеспечивает наличие базового набора общих сервисов или инфраструктуры (безопасность, контроль и т. д.) и их доступность для всех команд API. Более автономным командам предоставляются общее руководство и поддержка. Другими словами, часто приходится отходить от привычной централизованно-административной модели.

Одна из сложностей при передаче командам прав на самостоятельное принятие решений в том, что вышестоящее руководство может легко упустить из виду важные действия, происходящие на уровне этих команд. Раньше сотрудникам приходилось просить разрешение на выполнение определенных действий, но теперь компании, предоставляющие дополнительную автономию отдельным командам, будут побуждать их действовать, не дожидаясь проверки и разрешения «сверху».

Сложность управления средой API связана в основном с *масштабом* и *объемом*. Оказывается, по мере роста ваша API-программа еще и меняет *форму*. Это мы и обсудим далее.

Сложности в управлении API

Как мы уже упоминали в начале главы, хотя большинство компаний запустили программу API, некоторые секторы экономики более продвинуты в своих API, чем другие. В чем причина? Почему одни компании справляются с этим лучше других? Каковы общие проблемы и как вы можете помочь своей организации преодолеть их?

Мы выделили несколько основных тем, которые затрагиваем на встречах с компаниями по всему миру, когда обсуждаем управление жизненным циклом API.

- *Объем.* На чем должны сосредоточиться команды по архитектуре центрального программного обеспечения при долгосрочном управлении API?
- *Масштаб.* Зачастую то, что работает, когда компании только начинают заниматься API, перестает работать, когда программа из нескольких небольших команд разрастается до глобальной инициативы.
- *Стандарты.* По мере развития программ усилия по управлению и руководству надо перенести с подробных рекомендаций по проектированию и внедрению API на более общую стандартизацию ландшафта API. Это позволит командам принимать больше собственных решений на конкретном уровне.

Именно постоянный баланс объема, масштаба и стандартов создает жизнеспособную и растущую программу по управлению API. Поэтому стоит рассмотреть каждую составляющую подробнее.

Объем

Одна из самых больших сложностей в работе с исправной программой управления API — достичь оптимального уровня централизованного контроля и, что еще сложнее, — чтобы оптимальный уровень менялся вместе с развитием программы.

На ранних этапах лучше всего сосредоточиться на элементах проектирования API. В случаях, когда API еще в зачаточном состоянии, эти элементы могут исходить напрямую от команды создателей. Они изучают существующие программы «в полевых условиях», выбирают инструменты и библиотеки, подходящие для стиля того API, который они планируют создать, и затем разрабатывают его.

Именно на этом начальном этапе жизни API в организации подробные рекомендации и четкое распределение ролей помогут вам скорее добиться успеха. В главах 3 и 4 вы найдете множество материалов, полезных для компаний — новичков в этой области.

На первоначальном этапе развития API все в новинку. Все проблемы возникают (и решаются) впервые. Этот опыт часто в конечном итоге записывается в «лучшие практики API» или документацию компании и т. д. Он важен для маленькой команды, работающей над несколькими API в первый раз. Но эти руководства могут оказаться неполными.

С увеличением количества API-команд в компании растет и разнообразие стилей, ситуаций и точек зрения. Становится сложнее поддерживать стабильность во всех командах, и не только потому, что некоторые не соблюдают инструкции компании. Возможно, следовать этим инструкциям мешает работа с другим набором готовых продуктов. Возможно, причина в том, что члены команды не работают в среде с потоком событий, а поддерживают API, основанные на XML, в режиме «запрос — ответ». Конечно, им тоже нужно руководство, но оно должно соответствовать *их* сфере и потребностям *их* клиентов.

На «промежуточном этапе» вашей программы управления API инструкции должны перейти от конкретных предписаний по разработке и внедрению API к более общим рекомендациям по их жизненному циклу и способам взаимодействия. В главах 6 и 7 вы узнаете, что делают успешные организации для программ API на среднем этапе.

Конечно, есть универсальные инструкции для всех команд, но они должны соответствовать сферам их задач и запросам пользователей их API. С расширением вашего сообщества увеличивается и вариативность. И попытка ее уничтожить ознаменует ваш провал. Здесь вам следует сместить фокус контроля с *директивных распоряжений* (например, «все API *должны* использовать следующие схемы URL...») на указание *направлений* (например, «API, работающие на HTTP, *могут* использовать один из следующих шаблонов URL...»).

На «позднем этапе» перспектива управления расширяется сильнее и фокусируется на том, как ваши API взаимодействуют между собой и с API других компаний в вашем отраслевом секторе. Главы 9, 10 и 11 отражают тип мышления «видения целостной картины», важный для поддержания здоровой и стабильной экосистемы API в будущем.

Как видите, с расширением объемов вашей программы необходимо расширять и инструкции. Это особенно важно для международных предприятий, где местная культура, язык и история играют важную роль в том, как команды думают, творят и решают задачи.

Это подводит нас к следующему ключевому элементу — масштабу.

Масштаб

Еще одна серьезная проблема при создании и поддержании работоспособной программы управления API связана с изменением масштаба со временем. Как мы обсудили ранее, рост числа команд и созданных ими API может вызывать трудности. Процессы, необходимые для мониторинга и управления API во время выполнения программы, тоже будут меняться по мере развития системы.

Инструменты для отслеживания нескольких API, созданных одной командой в одном месте, сильно отличаются от инструментов для отслеживания сотен или тысяч точек входа API, разбросанных по разным часовым поясам и странам.

В книге мы рассказываем об этом аспекте управления API как о ландшафте. По мере расширения вашей программы вам нужно будет следить за многими процессами у многих команд во множестве локаций. Вы будете больше полагаться на мониторинг поведения во время выполнения, чтобы получить представление об уровне работоспособности вашей системы в разные моменты. Во второй части этой книги (начиная с главы 9) мы рассмотрим, как понятие управления ландшафтом API поможет вам понять, какие элементы заслуживают вашего внимания и какие инструменты и процессы могут пригодиться для контроля платформы API.

Ландшафт API ставит перед вами новые задачи. Процессы, используемые для разработки, реализации и поддержки одного API, не всегда совпадают с теми, которые нужны при расширении экосистемы. Здесь работает закон больших чисел: чем больше API в вашей системе, тем вероятнее их взаимодействие друг с другом. Это создает риск того, что некоторые из таких взаимодействий приведут к неожиданным последствиям (или ошибкам). Так работают все крупные системы: больше взаимодействий — больше неожиданных результатов. Попытки избавиться от них решат только часть проблемы. Удалить все баги невозможно.

И это приводит к третьей проблеме, с которой сталкивается большинство растущих программ API: как уменьшить количество неожиданных изменений, применяя в программе соответствующий уровень стандартов?

Стандарты

Первое, что меняется при переходе с уровня API на уровень ландшафта, — это изменение числа стандартов при непрерывном управлении командами, разрабатывающими, реализующими и использующими API в вашей организации.

Чем больше в компании рабочих групп, тем выше будут расходы на координацию их совместных действий (см. раздел «Решения» на с. 41). Увеличение масштаба требует изменений в объеме работ. Разобраться с этим будет проще, полагаясь на общие стандарты, а не на конкретные ограничения.

Например, одна из причин, по которой интернет продолжает хорошо работать с момента создания в 1990 году, в том, что его разработчики изначально решили полагаться на общие стандарты, применимые ко всем типам программных платформ и языков, а не создавать узкоспециальные инструкции

по реализации. Это позволяет креативным командам изобретать новые языки, архитектурные шаблоны и среды выполнения, не нарушая работу существующих программ.

Все долгосрочные стандарты, которые помогали интернету успешно функционировать, объединяет фокус на стандартизации *взаимодействия* между компонентами и системами. Вместо того чтобы стандартизировать способы внутренней реализации компонентов (например, используйте эту библиотеку, эту модель данных и т. д.), веб-стандарты стремятся облегчить взаимопонимание сторон. Когда ваша API-программа поднимается на более высокий уровень, указания вашему сообществу API тоже должны быть сосредоточены на общих стандартах взаимодействия, а не на конкретных деталях реализации.

Этот переход может быть сложным, но он важен для приближения к зрелому ландшафту, в котором команды смогут разрабатывать API, легко взаимодействующие как с уже существующими, так и с будущими API.

Управление ландшафтом API

Как уже было сказано, две самые сложные задачи в работе с API — это управление жизненным циклом одного API и управление ландшафтом API. Посещая разные компании и исследуя эту сферу в целом, мы видели много версий управления одним API. Нам известно множество примеров жизненных циклов и моделей зрелости, помогающих распознать сложности создания, разработки и развертывания API и справиться с ними. Но мы практически не нашли рекомендаций, касающихся экосистемы (мы называем ее ландшафтом) API.

У ландшафтов API есть свои сложности, свое поведение и тенденции. То, что вам нужно учитывать при разработке одного API, отличается от того, что вы должны учитывать, когда вам нужно поддерживать десятки, сотни или даже тысячи. В экосистеме появляются проблемы с *масштабом*, которых не возникало в ходе работы с одной программой или реализацией API. Мы подробнее остановимся на ландшафте позже, а сейчас хотим упомянуть три области, в которых появляются уникальные для управления API:

- масштабирование технологий;
- масштабирование команд;
- масштабирование руководства.

Рассмотрим каждый из этих аспектов в отношении к ландшафтам.

Технология

Во время первого запуска программы API нужно принять серию технических решений, которые повлияют на все ваши API. Тот факт, что «все» ваши API — это лишь небольшой набор, сейчас не важен. Важно то, что у вас есть стабильный набор инструментов и технологий, на которые вы полагаетесь при разработке изначальной API-программы.

Когда вы доберетесь до рассмотрения деталей жизненного цикла API (см. главу 7) и развития API, то увидите, что стоимость таких программ весьма высока и вам нужно тщательно контролировать свои затраты времени и энергии на деятельность, которая серьезно повлияет на успех вашего API. Обычно это значит, что нужно выбрать и поддерживать небольшой набор инструментов и предоставить четкие и детализированные документы с указаниями, чтобы помочь вашим специалистам создать и разработать те API, которые решат проблемы бизнеса и будут хорошо сочетаться друг с другом. Иначе говоря, на ранних этапах можно сэкономить, ограничивая технический объем.

На первых порах это хорошо работает по всем вышеупомянутым причинам. Но как только объем (см. раздел «Объем» на с. 277) программы увеличивается (например, больше команд начинают разрабатывать больше API, чтобы обслужить больше сфер бизнеса и т. д.), появляются сложности. Привязка к ограниченному набору инструментов и технологий может сильно замедлить работу. Вначале, при небольшом количестве команд, ограничение выбора ускорило процесс. Но когда команд много, ограничения — затратное и рискованное предприятие, особенно если они находятся далеко друг от друга или же если вы создаете новые бизнес-подразделения или приобретаете новые компании, чтобы добавить их в свой ландшафт API. Здесь разнообразие (см. раздел «Разнообразие» на с. 271) становится более важным фактором для успешного развития вашей экосистемы.

Итак, в управлении технологиями ландшафта API важно понять, когда он стал достаточно большим, чтобы начать увеличивать разнообразие технологий, а не ограничивать их. Если ландшафт API должен поддерживать существующие в организации сервисы SOAP-over-TCP/IP, нельзя требовать, чтобы все эти сервисы использовали одно и то же руководство по URL, которое вы составили для новых API на CRUD-over-HTTP. То же самое касается создания сервисов для новых событийно-ориентированных реализаций Angular или устаревших реализаций удаленного вызова процедур (RPC).

Более широкий охват означает большее технологическое разнообразие в вашем ландшафте.

Команды

Технологии — не единственный аспект управления API, который сталкивается с новыми задачами при росте программы. Состав самих команд тоже должен корректироваться по мере изменения ландшафта. Опять же в начале работы над программой с API можно работать всего с несколькими универсальными специалистами. К ним относятся full-stack-разработчик, или MEAN-разработчик (MongoDB, Express.js, Angular.js, Node.js), или какой-либо другой вариант идеи одного разработчика с навыками для всех аспектов вашей программы API. Вы также, скорее всего, слышали о стартап-командах или автономных командах. Все это сводится к наличию необходимых вам навыков у членов одной команды.

Прибегать к услугам таких специалистов имеет смысл, когда у вас мало API и все они разрабатываются и реализуются с помощью одного набора инструментов (см. раздел «Технология» на с. 35). Но при увеличении масштаба и объема программы растет и число необходимых навыков для ее разработки и поддержки. Уже нельзя ожидать, что в каждой команде по API будет нужное количество людей, имеющих навыки в областях разработки баз данных, серверных и клиентских приложений, тестирования и развертывания программы. У вас может трудиться группа специалистов, чьей задачей будет разработать интерфейс панели управления, ориентированный на обработку данных, который станут применять другие команды. Их навыки должны включать в себя, например, использование всех форматов данных и необходимых инструментов для их сбора. Или у вас может быть команда для разработки мобильных приложений, применяющих одну технологию, например GraphQL или другую библиотеку, ориентированную на запросы. По мере роста технологического разнообразия вашим командам может потребоваться специализация. Эту тему мы подробнее рассмотрим в главе 8.

Другая область, где группам разработчиков придется измениться с ростом ландшафта API, — это их участие в ежедневных процессах принятия решений. Когда у вас работают несколько неопытных команд, разумнее всего будет доверить принятие решений одной руководящей группе. В больших организациях она часто называется группой архитектуры предприятия. Это работает при небольших масштабах и объемах, но оказывается серьезной проблемой, когда экосистема становится более разнообразной.

Технологии становятся все более вовлеченными, и одна группа вряд ли сможет следить за деталями каждого инструмента и фреймворка. Так что с добавлением новых команд должно быть перераспределено само принятие решений. Центральный штаб редко разбирается в подробностях всех повседневных операций в международной компании.

От этой проблемы можно избавиться, разбив процесс принятия решений на так называемые *элементы решения* (см. раздел «Элементы решения» на с. 52) и распределив их на соответствующих уровнях внутри компании. Рост экосистемы означает, что команды должны стать более специализированными на техническом уровне и более ответственными на уровне принятия решений.

Руководство

Последняя область, которую мы здесь рассмотрим, — общий подход к *руководству* программой API. Как и в других упомянутых случаях, наши наблюдения показали, что роль и рычаги управления будут меняться по мере роста вашей экосистемы. Появляются новые задачи, и старые методы становятся неэффективными. На самом деле привязка, особенно на корпоративном уровне, к старым моделям руководства может замедлить достижение успеха вашими API или даже препятствовать этому.

Как и в любой области лидерства, при небольших объеме и масштабе самым эффективным может стать подход, основанный на прямых указаниях. Это работает не только с небольшими, но и с *новыми* командами. Когда нет большого опыта, быстрее всего его можно получить с помощью подробных руководств и/или технологических документов.

Например, мы обнаружили, что руководство API-программой на ранней стадии часто принимает форму длинного документа по процессу, объясняющего конкретные задачи: как создать URL для API, какие названия подходят для URL или где в HTTP должен быть номер версии. Четкие указания по выполнению небольшого количества опций позволяют разработчикам не отклоняться от утвержденного способа реализации API.

Но с ростом программы, при добавлении новых команд и поддержке большего количества бизнес-доменов размер и охват сообщества начинают усложнять поддержку одного руководящего документа с указаниями для всех команд. И даже порекомендовать кому-то работу по написанию и ведению подробных документов, регламентирующих процессы на предприятии, — не лучшая идея. Как мы уже говорили, разнообразие технологий становится сильной стороной большой экосистемы, и попытка обуздать его на уровне управления предприятием может замедлить ход вашей программы.

Вот почему по мере расширения вашего API-ландшафта ваши руководящие документы должны меняться по тональности: вместо прямых инструкций по процессу в них должны быть общие принципы. Например, вместо детального описания URL, подходящих для вашей компании, можно указать разработчикам

на руководство Инженерного совета интернета по дизайну и владению URI (RFC 7320) и привести общие принципы использования этого стандарта в организации.

Другой прекрасный пример *принципиальных указаний* можно найти в большинстве руководств по UI/UX, например «10 удобных в применении эвристик для разработки пользовательского интерфейса» от Якоба Нильсена (<https://oreil.ly/qU66X>). В таких документах есть множество вариантов и обоснований для использования одного шаблона пользовательского интерфейса вместо другого. Они разъясняют, почему и когда что-либо применять, а не просто диктуют требования, которым нужно подчиняться.

Наконец, в крупных организациях, особенно в тех, офисы которых расположены по всему миру, надо изменить способ управления с раздачи указаний на выработку рекомендаций. Это существенно меняет типичную модель централизованного управления. Основной задачей центрального штаба руководства становится не строгое регламентирование действий для команд, а сбор рабочей информации в полевых условиях, поиск совпадений и передача указаний, отражающих избранные методики всей организации.

Так, при росте ландшафта API модель руководства должна измениться с директивных указаний через общие принципы на сбор и передачу методик, используемых опытными командами, внутри компании. В главе 2 вы увидите, что можно задействовать много разных принципов и методик для создания модели руководства, подходящей именно вам.

Итоги главы

В этой вводной главе мы коснулись ряда важных аспектов управления API, описанных в книге. Рассказали, что API продолжают оставаться движущей силой, но лишь около 50 % опрошенных компаний уверены, что правильно ими управляют. Мы также пояснили, что у термина API много значений и это может усложнить создание стабильной модели управления программой.

А важнее всего то, что управление одним API очень отличается от управления ландшафтом API. В первом случае вы можете полагаться на модели AaaP, «жизненный цикл API» и «развитие API». Управление изменениями в API также сосредоточено на понятии одного API. Но это далеко не всё.

Затем мы обсудили управление ландшафтом API — целой экосистемой внутри организации. Управление растущим ландшафтом API требует другого набора навыков и параметров — навыков работы с разнообразием, объемом, изменяемо-

стью, уязвимостью и другими характеристиками. Все они влияют на жизненный цикл API, и мы подробнее рассмотрим их позже.

Наконец, мы подчеркнули, что даже способ принятия решений по поводу вашей программы со временем должен измениться. По мере роста вашей системы вам нужно распределять принятие решений так же, как вы распределяете IT-элементы, такие как репозиторий, вычислительная мощность, безопасность и другие части инфраструктуры вашей компании.

Опираясь на это введение, начнем с рассмотрения понятия *руководства* и того, как вы можете использовать принятие решений и их распределение в качестве основной составляющей вашего общего подхода к управлению API.

ГЛАВА 2

Руководство API

Эй, правила есть правила, и давайте признаем: без них наступает хаос.

Космо Крамер

Мало кто хочет, чтобы им руководили, — у большинства был отрицательный опыт работы под руководством тех, кто плохо планировал свои действия и устанавливал бессмысленные правила. Плохое руководство, как и плохое планирование, усложняет жизнь. Но по нашему опыту, трудно говорить об управлении API, не затрагивая его.

Мы даже позволим себе сказать, что управлять API без руководства *невозможно*.

В компаниях редко применяют термин «*руководство*». И это совершенно нормально. Термины важны, а в некоторых организациях под руководством понимают стремление к чрезмерной централизации и авторитарности. Это может противоречить корпоративной культуре, ценящей независимость сотрудников. Поэтому логично, что тогда слово «руководство» не подойдет. Но вне зависимости от названия так или иначе всегда есть определенная форма управления принятием решений.

Вопрос о необходимости в руководстве API не вызывает у нас интереса, ведь, по нашему мнению, ответ на него всегда утвердительный. Вместо этого спросите себя: «Какими решениями нужно руководить и где?» В поиске ответов на такие вопросы и заключается работа по созданию системы руководства.

От стиля руководства зависят продуктивность, качество продукции и стратегическая ценность. Вы должны будете разработать систему, подходящую именно вам. Наша цель в данной главе — дать вам структурные элементы для этого.

Начнем с обзора трех базовых составляющих хорошего руководства API: решений, управления ими и сложности. Разобравшись с этим, рассмотрим поближе, как распределять решения в организации и как это повлияет на ее работу. Другими словами, мы рассмотрим централизацию, децентрализацию и составляющие решения. Наконец, поговорим о построении системы руководства и исследуем три стиля руководства.

Руководство — центральная часть управления API. Понятия из этой главы будут всплывать на протяжении всей книги. Поэтому стоит потратить время, чтобы понять, что на самом деле означает «руководство API» и как оно может помочь вам создать эффективную систему управления API.

Что такое руководство API

Работа с технологиями заключается в принятии решений, множества решений. Некоторые из них жизненно важны, а некоторые тривиальны. Именно из-за этого можно сказать, что работа технологической команды — это *умственный труд*. Ключевой навык для работника такого труда — своевременное принятие многих решений одного за другим. Когда это происходит, продукты доставляются, вносить изменения становится легче, а команды достигают поставленных целей.

Независимо от того, какие технологии вы внедряете, как проектируете свою архитектуру или с какими компаниями сотрудничаете, именно способность каждого участника принимать решения определяет судьбу вашего бизнеса. Поэтому руководство имеет большое значение. Всем этим решениям нужно придать форму, которая поможет вам достичь целей, поставленных перед организацией.

Конечно, это проще сказать, чем сделать. Чтобы получить больше шансов на успех, вам нужно разобраться в фундаментальных понятиях руководства и их соотношении. Давайте начнем с краткого обзора решений API.

Решения

Чем качественнее будут ваши решения, тем лучших результатов вы добьетесь. Интерфейсы API — это в первую очередь технологический продукт, но для создания лучших API вам нужно принимать решения, которые выходят далеко за рамки написания хорошего кода.

Рассмотрим список решений, которые часто принимают команды разработчиков API.

- Какой URL будет у нашего API — `/payments` или `/PaymentCollection`?
- В каком облачном сервисе разместить наш API?
- У нас есть два API по клиентской информации, от которого из них отказаться?
- Кто войдет в команду по разработке?
- Как назвать эту переменную в Java?

На основании этого краткого списка решений можно сделать несколько выводов. Во-первых, варианты управления API охватывают широкий спектр проблем и людей, поэтому для принятия таких решений нужна тесная координация между командами. Во-вторых, индивидуальные решения сотрудников могут оказать разное влияние — выбор облачного сервиса повлияет на вашу стратегию управления API сильнее, чем название переменной в Java. В-третьих, небольшой выбор может сильно повлиять на масштабирование — если 10 000 переменных в Java будут названы плохо, поддерживать реализации API будет гораздо сложнее.

Все эти решения, принимаемые сообща и с учетом масштаба, нужно собрать воедино для достижения лучшего результата. Это тяжелая работа. Позже мы отдельно рассмотрим эту проблему и дадим несколько советов по изменению системы решений. Но сначала поближе познакомимся с тем, что представляет собой *руководство* этими решениями и почему оно так важно.

Управление принятием решений

Если вы когда-либо работали над небольшим проектом самостоятельно, вы знаете, что успех или провал зависит только от вас. Постоянно принимая верные решения, вы обеспечиваете себе хорошие результаты. Один опытный программист может создавать потрясающие вещи.

Но такой способ работы плохо масштабируется. Когда вашу продукцию начинают использовать, растет потребность в изменениях и новых характеристиках. Вам понадобится принимать гораздо больше решений за меньший промежуток времени. Иначе говоря, нужно больше сотрудников, которые будут этим заниматься. Увеличивать число людей, принимающих решения, нужно с осторожностью. Нельзя рисковать потерей качества решений только из-за того, что ими занимается больше людей. Здесь нам и пригодится руководство.

Руководство — это процесс управления принятием решений и их реализацией. Заметьте, мы не говорим, что руководство связано с контролем или властью. Это

не так. Оно связано с улучшением качества принятия решений вашими сотрудниками. В сфере API качественное руководство означает создание API, которые помогут вашей организации добиться успеха. Да, вам может понадобиться определенный уровень власти, чтобы достичь этого, но это не самоцель.

Помните, что у руководства есть своя цена. Ограничения нужно донести до сотрудников, внедрить и поддерживать. Поощрения, влияющие на принятие решений, должны казаться привлекательными. Стандарты и процессы должны быть документированы, мотивированы и актуальны. Кроме того, нужно постоянно собирать информацию, чтобы наблюдать, как эти стимулы влияют на систему. Возможно, потребуется даже нанять больше сотрудников.

Кроме этих общих расходов есть и скрытые убытки от применения руководства в вашей системе. Существуют начальные расходы, которые появляются, как только вы начинаете руководить системой. Например, если вы назначаете технологическую платформу, которую должны использовать все разработчики, каковы организационные затраты с точки зрения технологических инноваций? А сколько будет стоить недовольство сотрудников? Станет ли после этого сложнее привлекать новые таланты?

Такие убытки трудно предсказать, потому что в реальности вы руководите сложной системой, включающей в себя людей, процессы и технологии. Чтобы руководить ландшафтом API, сначала нужно научиться управлять сложной системой в целом.

Руководство сложными системами

Хорошая новость: для получения хороших результатов не обязательно контролировать каждое решение в организации. Плохая новость: нужно понять, какие решения все же *требуют* контроля. Это сложный вопрос, потому что ответ на него: «Все зависит от обстоятельств».

Если бы вы хотели испечь бисквитный торт, мы могли бы дать вам четкий рецепт. Рассказали бы, сколько нужно муки и яиц, какую температуру устанавливать в духовке. Мы даже могли бы объяснить, как проверить готовность бисквита. Выпечка достаточно маловариативна. Ингредиенты всегда одинаковы, вне зависимости от того, где вы их купите. Духовки запрограммированы на выпекание при определенных стандартных температурах. Самое главное, цель одна и та же — торт определенного вида.

Но вы не печете торты, а эта книга — не кулинарная. Так что придется разбираться с невероятным количеством вариантов. Например, у вас будут сотрудники с разным уровнем способностей к принятию решений. Регулирующие

ограничения будут уникальными для вашей отрасли и местоположения. Вам предстоит обслуживать собственный динамично меняющийся клиентский рынок с его особой потребительской культурой. И в придачу ко всему ваши организационные цели и стратегии будут полностью уникальны.

Из-за всей этой изменчивости сложно прописать единственный правильный «рецепт» руководства API. Осложняющим фактором является эффект неожиданных последствий. Каждый раз, когда вы вводите правило, создаете новый стандарт или применяете какую-то форму руководства, вам придется иметь дело с такими последствиями. Так происходит потому, что все части организации взаимосвязаны.

Например, для улучшения стабильности и качества кода своего API вы вводите стандартную технологическую платформу. Это может привести к увеличению пакетов кода, потому что программисты начнут добавлять новые библиотеки и шаблоны, что, в свою очередь, способно вызвать изменения в развертывании, ведь существующая система не сможет поддерживать увеличившиеся пакеты кода.

Зная это, вы могли бы предсказать и предотвратить такие последствия. Но это невозможно сделать для каждой отдельной ситуации, особенно за короткое время. Вместо этого примите тот факт, что вы работаете со сложной адаптивной системой. И это не проблема, а просто особенность. Следует понять, как воспользоваться этим с выгодой для себя.

Сложная адаптивная система

Говоря о сложной адаптивной системе, мы подразумеваем следующее:

- в ней много взаимозависимых элементов, например люди, технологии, процессы, культура производства;
- составляющие могут динамически менять свое поведение и адаптироваться к системным изменениям (например, команды меняют методы развертывания при внедрении контейнеризации).

Вселенная полна таких систем, и изучение их сложности стало устоявшейся научной дисциплиной. Даже вы сами — сложная адаптивная система. Вы можете воспринимать себя цельной единицей — собой, но это лишь абстракция. На самом деле вы — скопление органических клеток, пусть даже способных на удивительные подвиги: мышление, движение, ощущения и реакцию на внешние события в качестве цельного существа.

На молекулярном уровне отдельные клетки специализированы. Старые клетки отмирают и заменяются новыми, а группы клеток функционируют вместе, чтобы сильнее влиять на ваш организм. Сложность биологической системы, которой

вы являетесь, делает ваше тело очень устойчивым и адаптируемым. Вы не бес- смертны, но, скорее всего, выдержите глобальные перемены в окружающей среде и даже некоторые физические повреждения.

Обычно, говоря о системах в технологиях, подразумевают программные системы и сетевую архитектуру. Такие системы могут усложняться со временем. Интернет — идеальный пример сложности и эмерджентности¹. Отдельные серверы в сети работают независимо друг от друга, но благодаря их взаимосвязям появляется нечто целое, называемое интернетом. Но бо́льшая часть ПО *неадаптивна*.

Интерфейсы API — не исключение. API, которые мы пишем сегодня, не очень адаптивны. Если они хороши, они делают именно то, на что их запрограмми- ровали. После выпуска API вряд ли что-то изменится в его работе, если только кто-то не исправит его. Основная истина о руководстве API в том, что толь- ко оно не продвинет вас далеко. Вместо этого вам нужно управлять людьми в своей организации и решениями, которые они принимают в отношении API. Единственный способ получить лучшие API — это помочь вашим сотрудникам принимать полезные решения по API.

Люди хорошо приспосабливаются, особенно в сравнении с программным обес- печением. Ваша организация API — сложная адаптивная система. Все люди в ней принимают множество *локальных* решений: какие-то из них коллективно, а какие-то индивидуально. Когда таких решений много и принимаются они уже долго, система прогрессирует. Она, как и ваше тело, может адаптироваться ко многим изменениям.

Но управление решениями людей требует особого подхода. Влияние изменений на сложную систему трудно предсказать — изменения в одной части вашей организации могут привести к неожиданным последствиям в другой. Так про- исходит потому, что сотрудники постоянно адаптируются к изменениям среды. Например, введение правила, запрещающего развертывание программного обеспечения в контейнерах, имело бы далеко идущие последствия, влияющие на разработку ПО, наем, процессы развертывания и культуру.

Все это означает, что масштабный подход к управлению API с предварительным планированием и выполнением вряд ли работает. Вместо этого вам нужно «под- толкнуть» систему, внося небольшие изменения и оценив их влияние. Для этого нужны постоянные корректировки и улучшения, так же как при уходе за садом приходится обрезать ветки, сеять семена, поливать, постоянно наблюдая и под- страивая свои нынешние действия под результат предыдущих. В главе 5 мы детальнее разберем концепцию постоянного улучшения.

¹ Эмерджентность — появление у системы свойств, не присущих ее элементам в отдель- ности. — *Примеч. пер.*

Управление решениями

В предыдущем разделе мы ввели понятие руководства решениями в сложной системе. Это помогло вам понять его важную роль в руководстве API: если вы хотите повысить эффективность своей системы руководства, лучше влияйте на решения, которые принимают люди. Мы считаем, что один из лучших способов — сосредоточиться на том, где и кем принимаются решения. Оказывается, что одного оптимального способа распределения решений не существует.

Рассмотрим вымышленный пример руководства дизайном API в двух разных компаниях.

- *Компания Pendant Software.* Всем API-командам дается доступ к электронной книге *Pendant Guidelines for API Design* («Указания Pendant по разработке API»). Эти указания публикуются ежеквартально Центром по высокому качеству и передаче опыта в API компании Pendant — небольшой командой экспертов по API, работающих в этой компании. В них можно найти очень четкие правила разработки API. От всех команд ожидается неукоснительное выполнение указаний, а API перед публикацией автоматически тестируются на соответствие им.

В результате такой политики компания Pendant смогла опубликовать ряд ведущих в своей области и согласованных API, которые высоко оценили другие разработчики. Эти API помогли Pendant выделиться среди конкурентов на рынке.

- *Компания Vandelay Insurance.* Командам API сообщаются бизнес-цели компании и ожидаемые результаты для их продуктов. Эти цели и результаты определяются исполнительными командами и регулярно обновляются. У каждой команды API есть возможность достигать общей бизнес-цели любым выбранным способом. Несколько команд могут выбрать одну и ту же цель. API-команды могут разрабатывать и реализовывать API как им угодно, но каждый продукт должен соответствовать параметрам и стандартам отслеживания компании Vandelay. Стандарты определяются Системным обществом Vandelay — группой, состоящей из участников каждой команды API, которые вступают в нее добровольно и определяют общепринятый набор стандартов.

В результате этой политики Vandelay смогла создать инновационную и адаптивную архитектуру API. Этот ландшафт API позволил компании опередить конкурентов с помощью инновационных бизнес-методик, которые создаются очень быстро на этой технологической платформе.

В наших вымышленных исследованиях и Pendant, и Vandelay очень эффективно управляли принятием решений. Но руководили они этой работой по-разному.

Компания Pendant пришла к успеху с помощью высокоцентрализованного авторитарного подхода, а Vandelay предпочла метод ориентирования на результат. Здесь нет единственно верного подхода, у обоих стилей руководства есть свои преимущества.

Для эффективного руководства принятием решений ответьте на три важных вопроса.

- Какими решениями управлять?
- Кто и где должен принимать эти решения?
- Как стратегия управления решениями повлияет на систему?



В системе с поддержкой API необходимо принять множество решений, как на уровне API, так и на коллективном, «ландшафтном» уровне. Мы рассмотрим весь спектр решений, которые нужно принимать и которыми необходимо управлять, в главах 4 и 9.

А пока сосредоточимся на втором вопросе: где в системе должны приниматься самые важные решения? Чтобы помочь вам в этом разобраться, углубимся в тему управления решениями. Мы займемся поиском компромисса между централизованным и децентрализованным принятием решений и подробнее рассмотрим, что значит распределять решения.

Централизация и децентрализация

Ранее мы ввели понятие сложной адаптивной системы, используя как пример человеческое тело. Природа изобилует такими системами, вы буквально окружены ими. Например, об экосистеме какого-нибудь маленького пруда можно говорить как о сложной адаптивной системе. Он выживает благодаря действиям и взаимосвязям его обитателей. Экосистема адаптируется к меняющимся условиям благодаря локализованному принятию решения каждым из живых организмов.

Но у пруда нет руководителя, и вряд ли лягушки, змеи и рыбы проводят ежеквартальные совещания по управлению. Вместо этого каждый участник системы принимает свои решения и демонстрирует индивидуальное поведение. Эти решения и действия формируют коллективное сложившееся целое, способное выжить, даже если отдельные части системы меняются или исчезают и вновь появляются со временем. Как и большая часть природных сообществ, система пруда выживает, потому что решения в ней *децентрализованы* и *распределены между участниками*.

Как мы уже говорили, ваша организация — это тоже сложная адаптивная система. Она — продукт всех индивидуальных решений сотрудников. Как и в случае с человеческим телом или экосистемой пруда, если бы вы предоставили работникам полную автономию, организация в целом стала бы более устойчивой и адаптивной. Вы получите децентрализованную компанию без начальника, способную выжить благодаря индивидуальным решениям сотрудников (рис. 2.1).

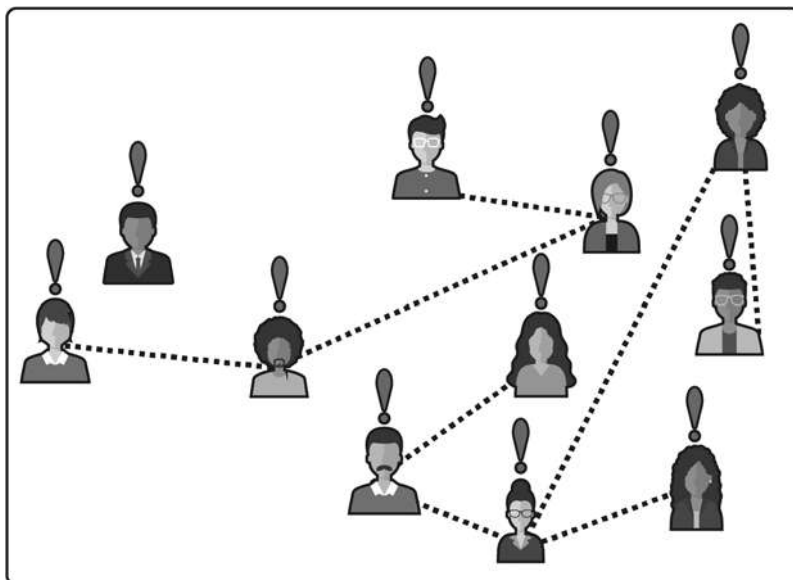


Рис. 2.1. Децентрализованная организация

Вы можете поступить так, но будьте готовы к проблемам, связанным с тем, что в свободных рыночных отношениях тяжело преуспеть так же, как преуспевают сложные системы в природе. Биосистемой пруда руководит естественный отбор. Каждое звено в системе оптимизировано для выживания своего вида, но у него нет цели на системном уровне. Кроме того, в природе системы могут гибнуть. Например, если пустить в пруд новый агрессивный вид живых организмов, вся прежняя система вымрет. И это будет нормально, ведь ее место займет что-то другое, и система в целом остается жизнеспособной.

Но руководители предприятий плохо реагируют на такой уровень неопределенности и отсутствия контроля. Скорее всего, вам нужно будет направить свою систему на конкретные цели, выходящие за рамки выживания. Кроме того, вы вряд ли захотите допустить ликвидацию компании ради того, чтобы ее место заняла другая, более успешная. И практически точно вы хотите уменьшить риск

разрушения всей компании из-за неудачного решения одного сотрудника. Поэтому нужно урезать свободу принятия индивидуальных решений и ввести какую-то отчетность. Один из способов сделать это — *централизация* решений (рис. 2.2).

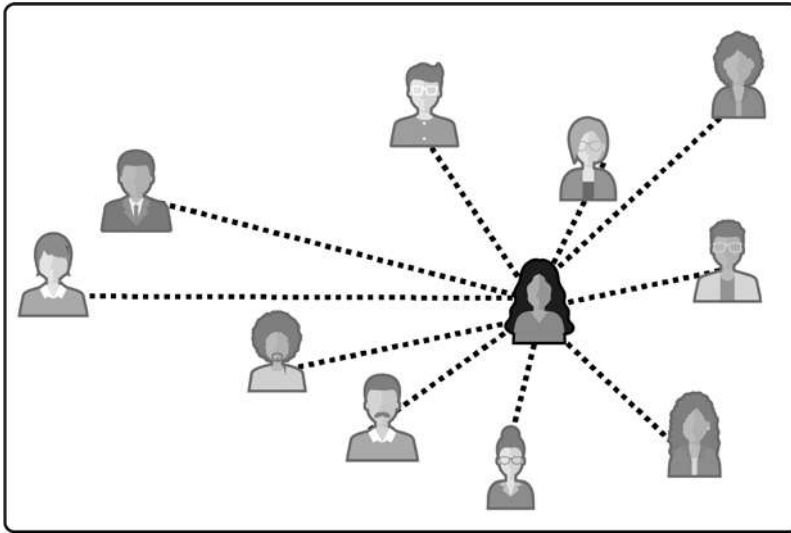


Рис. 2.2. Централизованная организация

Здесь подразумевается, что в организации принятие решений поручено конкретному человеку или команде. Эта централизованная группа принимает решение, которое должны исполнять остальные. Децентрализация означает обратное: отдельные команды могут принимать решения, которые обязаны выполнять только они.

В реальности только централизованных или децентрализованных организаций нет. Разные типы решений распределяются в организации по-разному: одни более централизованно, другие менее. Вам придется позаботиться о том, как распределять решения, влияющие на систему сильнее всего. Так какие из них должны быть более централизованными?

Помните, что главная цель руководства решениями — помочь вашей организации добиться успеха и выжить. Что именно под этим подразумевается, зависит от специфики вашего бизнеса. Но обычно это значит, что решения должны быть своевременными для гибкости бизнеса и качественными для его улучшения. На способность принимать решения влияют три фактора.

- *Доступность и точность информации.* Принять правильное решение очень трудно, если оно основано на неточной информации или ее вовсе нет.

Вы можете не знать цели решения или сопутствующих факторов и быть не в курсе его будущего влияния на систему. Как правило, ответственность за сбор важной для принятия решения информации лежит на том, кто это решение принимает. Но для распределения решений мы должны учесть, как их централизация или децентрализация повлияет на доступность информации.

- *Талант в сфере принятия решений.* Обычно качество решений улучшается, если человек, принимающий их, делает это хорошо. Проще говоря, талантливые и опытные люди принимают решения лучше, чем менее талантливые и неопытные. Распределять решения нужно так, чтобы все таланты раскрылись в полной мере.
- *Затраты на координацию.* Нельзя своевременно принять сложное решение, если процесс не будет коллективным. Но в этом случае придется учесть затраты на координацию. Если они слишком вырастут, вы не сможете быстро принимать решения. Централизация и децентрализация решений могут очень серьезно влиять на эти затраты.

Принимая во внимание эти факторы, вы сможете определить, когда решение должно быть централизовано, а когда — нет. Чтобы разобраться с этим, рассмотрим решение с двух точек зрения: масштаба оптимизации и объема работы. И начнем мы с изучения масштаба и его связи с информацией для принятия решений.

Масштаб оптимизации

Централизованное и децентрализованное решения сильно различаются по своему масштабу. Централизованное решение принимается для целой организации. Объем оптимизации в этом случае включает в себя всю систему, и ваша цель — принять решение, которое ее улучшит. Иначе говоря, это решение должно *оптимизировать объем системы*. Например, централизованная команда может выбрать методологию разработки, которой будет следовать вся компания. Та же команда может выбрать API, который нужно удалить. Оба решения должны быть приняты с учетом выгоды для всей системы.

И наоборот, суть децентрализованного решения в том, что оно *оптимизировано для локального масштаба*. В этом случае вы принимаете решение, которое улучшит частности — информацию, которая относится только к конкретной ситуации. Хотя ваше решение может повлиять на систему в целом, ваша цель — улучшить свои локальные результаты. Например, команда API может принять решение использовать каскадный процесс разработки, потому что она работает совместно с внешней компанией, которая на этом настаивает.

Плюс децентрализованного принятия решений в том, что оно помогает вам добиться больших успехов в эффективности, инновациях и гибкости вашего бизнеса в целом. Это связано с тем, что люди, принимающие решения децентрализованно, могут ограничить объем информации локальным контекстом, который они понимают. Поэтому они могут сформировать решение, опираясь на точные сведения о своей предметной области. Для любого современного бизнеса, пытающегося применять стратегию гибкости и инноваций, подход децентрализованного принятия решений должен быть использован по умолчанию.

Но принятие решений, направленных только на оптимизацию локального масштаба, может вызвать проблемы, особенно если эти решения способны негативно и необратимо повлиять на систему. Бывший генеральный директор Amazon Джефф Безос (<https://oreil.ly/v5Ois>) разделил решения на два типа: если оказались неправильными решения типа 1, их легко отменить, а после реализации неверных решений типа 2 восстановиться почти невозможно. Так, многие крупные компании предпочитают централизованно принимать решения о конфигурации безопасности API, чтобы предотвратить локальную оптимизацию, порождающую уязвимость системы.

Кроме опасности для системы есть и другие причины, по которым единство на системном уровне может быть ценнее локальной оптимизации. Например, отдельная команда API может выбрать самый удобный для их предметной области стиль API. Но если у каждой группы будет свой стиль, станет сложнее использовать API из-за недостатка единообразия, особенно когда для выполнения одного задания приходится применять много API. В этом случае стоит оптимизировать решение по поводу стиля API на системном уровне.

Понадобится как следует обдумать объем оптимизации, планируя, где будет принято решение. Если оно может необратимо повлиять на систему, начните с централизации, чтобы оптимизировать его для масштабов системы. В случае, если качество решения улучшится благодаря локальному контексту, начните с его децентрализации. Но если децентрализация решения может привести к неприемлемой несогласованности на системном уровне, рассмотрите возможность его централизации.

Объем работы

Если бы ресурсы для принятия качественных решений были неограниченными, вам нужно было бы думать только об их масштабе. Но это не так. Поэтому кроме масштаба придется думать еще и об объеме принимаемых решений. Если их нужно много, вырастет нагрузка на людей, принимающих решения, и затраты

на координацию. Чтобы API увеличивались вместе с ростом организации, внимательно распланируйте схему распределения решений.

При масштабной работе децентрализация принятия решений создает большой спрос на тех, кто способен эти решения принимать. Проводя децентрализацию, вы распределяете возможность принятия решений между несколькими командами. Если хотите при этом не потерять в качестве, следует внедрить во все команды людей, предрасположенных к этому, иначе число неудачных результатов будет велико. Поэтому наймите лучших специалистов по принятию решений на каждую руководящую должность в вашей компании.

К сожалению, экспертов в данной области немного — это не секрет. За право нанять талантливых и опытных людей соревнуются многие компании. Некоторые готовы потратить любые деньги, чтобы получить лучших из лучших. Если вам повезло, вы сможете децентрализовать больше решений, потому что ими будут заниматься талантливые специалисты. Но если нет, придется подходить к распределению более прагматично.

В случае, когда первоклассных специалистов немного, вы можете собрать их вместе и отдать самые важные решения на откуп этой группе людей. Так у вас будет больше шансов принимать качественные решения в более короткие сроки. Но эта модель перестанет работать, когда вырастет спрос на решения, потому что централизованной команде придется расти вместе с ним. С увеличением команды возрастут и затраты на распределение принятия этих решений. Независимо от таланта сотрудников затраты на координацию растут с приходом новых людей, и рано или поздно вы не сможете себе этого позволить.

Все это значит, что при распределении решений придется идти на компромиссы. Если решение так же важно, как описанные Джеффом Безосом решения типа 1, вам придется его централизовать, заплатив за это снижением производительности. Напротив, если важнее скорость и локальная оптимизация, можно децентрализовать решение и либо заплатить за прием на работу лучших специалистов, либо смириться со снижением качества решений.

Но есть тонкий подход, позволяющий справиться с этой задачей более гибко, — распределение и передача не всего решения целиком, а лишь его *частей*. Именно его мы и рассмотрим в следующем разделе.

Элементы решения

Непросто распределять решения описанным ранее способом, потому что это отчасти сводится к подходу «все или ничего». Позволить командам самим решить, какой метод разработки они будут использовать, или выбрать самостоятельно и заставить всех его применять? Позволить людям решать, какой API устарел

и подлежит удалению, или лишить их права выбора? На самом деле у руководства гораздо больше нюансов. Далее мы исследуем способ более гибкого распределения решений путем разбиения их на части.

Вместо распределения целого решения можно передавать командам его части. Так вы сможете получить преимущества оптимизации как на системном уровне, так и на локальном высококонтекстном. Часть решения может быть централизованной, а часть — наоборот. Чтобы помочь вам распределять их, мы разбили решения по API на шесть *элементов* (рис. 2.3):

- зарождение идеи;
- поиск вариантов;
- выбор;
- одобрение;
- применение;
- проверка.



Рис. 2.3. Элементы решения

Это не универсальная модель принятия решений. Мы разработали ее, чтобы выделить части решения, сильнее всего влияющие на систему вне зависимости от степени их централизации. Она основана на распространенных в сфере управления бизнесом моделях принятия решений из пяти, шести или семи этапов. Несмотря на то что описанные нами шаги могут применяться к решению, принимаемому одним человеком, полезнее всего они при групповом принятии решений.

Для начала рассмотрим, как влияет на систему распределение идеи в зачатке.

Зарождение идеи

За принятием каждого решения стоит кто-то, кто считает, что это нужно сделать. Это значит, что этот человек определил наличие проблемы и увидел несколько способов ее решения. Иногда это очевидно, но зачастую определение

возможности принятия решения требует таланта и опыта. Необходимо обдумать, какие решения возникают естественно, а какие требуют особого обращения.

Решения по поводу работы API принимаются в процессе ежедневного решения проблем. Например, выбор базы для хранения постоянных данных разработчику было бы непросто игнорировать. Решение принимают, потому что без него нельзя продолжать работу. Но есть ситуации, когда нужно инициировать зарождение идеи. Обычно это происходит по одной из двух причин.

- *Принятие повседневных решений.* Если команда принимает одно решение раз за разом, то со временем необходимость делать это может отпасть. То есть другие возможности больше не рассматриваются, а вместо этого делается предположение, что работа будет продолжаться как обычно. Например, если все реализации API написаны на Java, никому может не прийти в голову выбрать другой язык.
- *Невнимательность к принятию решений.* Иногда команды упускают возможность принять важные решения. Обычно это вызвано привычкой, но иногда — нехваткой информации, опыта или таланта. Например, команда может сосредоточиться на выборе базы для хранения данных и не понять, что API можно разработать так, что постоянное хранение не понадобится.

Не все решения нужно принимать, и это нормально, если вы упустили возможность или считаете какое-то решение действующим по умолчанию. Проблемой это становится, только когда непринятие решения отрицательно сказывается на результатах работы API. Беспочинное требование увеличения числа решений может отрицательно повлиять на продуктивность. Роль руководства API в том, чтобы генерировать больше решений, дающих нужные результаты, и меньше тех, которые ни к чему не приведут.

Поиск вариантов

Трудно сделать выбор, не зная вариантов. Их поиск — это работа по определению возможностей, из которых вы будете выбирать.

Если вы принимаете решение в сфере, где хорошо разбираетесь, найти варианты будет несложно. Но если неизвестных слишком много, придется потратить больше времени на определение возможностей. Например, у опытного программиста на C всегда есть хорошие варианты развития событий, когда принимается решение по циклической структуре, но новичку придется разобраться, где можно использовать оператор цикла `for` или `while` и какая между ними разница.

Даже если вам хорошо знакома предметная область, вы, вероятно, потратите больше времени на формирование выбора, если стоимость и влияние решения

очень высоки. Например, вы можете досконально знать разные облачные хостинги, но все равно предпочтете перепроверить информацию, когда придет время подписывать договор с одним из них. Не появились ли новые предложения, о которых вы не знали? Точно ли вы запомнили цены и условия?

С точки зрения руководства формирование выбора важно, потому что именно здесь устанавливаются *границы* принятия решений. Это особенно полезно, когда список вариантов разрабатывают одни, а выбор делают другие. Например, можно стандартизировать перечень возможных форматов описания API, но позволить отдельным командам решать, какой из них им больше нравится. При этом подходе важно быть уверенным в качестве списка, который вы предлагаете. Если он слишком ограничен и не очень качественен, возникнут проблемы.

Выбор

Выбор производится из нескольких возможных вариантов. Это сердце принятия решений, и многие фокусируются именно на этом шаге. Но его важность зависит от числа доступных вариантов. Если оно велико, то процесс выбора сильно влияет на качество решения. Но если выбор был ограничен безопасными вариантами, различия между которыми невелики, этот шаг может быть быстрым и почти ни на что не влиять.

Рассмотрим этот пример в действии. Предположим, вы отвечаете за настройку безопасности транспортного уровня (TLS) для вашего API на HTTP. Часть этой работы включает в себя решение о том, какие наборы шифров (наборы криптографических алгоритмов) должен поддерживать сервер. Это важное решение, потому что некоторые наборы со временем стали уязвимыми и неправильный выбор может снизить безопасность вашего API. Также если вы выберете наборы шифров, которые не поддерживаются ПО пользователей, то никто не сможет воспользоваться вашим API.

В одном сценарии вам может быть предоставлен список всех известных комплектов шифров и предложено выбрать те, которые должен поддерживать сервер. В этом случае к выбору нужно отнестись внимательно. Вы, вероятно, будете долго разбираться и почувствуете уверенность, только если соберете всю возможную информацию. Если у вас нет большого опыта в области безопасности серверов, вы, вероятно, найдете того, у кого он есть, и попросите выбрать за вас.

А если вместо списка всех возможных наборов шифров вам дадут заранее отобранные? Список вариантов может содержать и актуальную информацию о качестве поддержки каждого набора и его известных уязвимостях. Эта информация поможет вам быстрее сделать выбор, который к тому же не сможет навредить дальнейшей работе, ведь ваше решение ограничено вариантами,

которые уже сочли достаточно безопасными для использования. В этом случае ваше решение будет основано на знаниях о клиентах, работающих с вашим API, и его важности для бизнеса.

Наконец, вам могут дать лишь один вариант — единственный набор шифров, который вы должны применять. Здесь выбор становится лишь формальностью, ведь решение уже было принято за вас. В таком случае качество этого решения полностью зависит от людей, которые нашли этот вариант. В идеале он должен полностью подходить под ваши основные требования.

Итак, важность выбора во многом зависит от числа предложенных вариантов. Здесь появляется возможность для компромисса. Чем больше вы вкладываете в поиск вариантов, тем меньше времени потратите на выбор, и наоборот. Это влияет на распределение элементов решения и ответственность за них. Элемент, который становится более важным, потребует подходящего талантливого сотрудника для принятия решения.

Это значит, что можно сочетать системный и локальный масштаб, распределяя *поиск* и *выбор* вариантов. Например, можно централизовать поиск вариантов метода разработки на основе системного контекста, в то же время позволяя отдельным командам выбирать предпочтительный метод, используя их локальный контекст. Это очень полезная схема для управления большими ландшафтами API и сохранения безопасности и скорости изменений.

Одобрение

Выбор варианта еще не означает принятие решения. Для начала этот выбор нужно одобрить. Одобрение — это работа по определению правильности выбранного варианта. Верно ли сделан выбор? Осуществим ли он? Безопасен ли? Имеет ли он смысл в контексте принятых ранее решений?

Одобрение может быть скрытым или явным. Явным оно называется, когда какой-то человек или команда должны прямо утвердить решение, прежде чем запустить его в работу. Это шаг утверждения в процессе принятия решения. Наверняка вам уже доводилось принимать решения, которые нужно было одобрить. Например, во многих компаниях работники могут выбрать даты своего отпуска из списка рабочих дней, но окончательное решение по утверждению расписания принимает руководство.

Скрытое одобрение происходит автоматически, когда выбранный вариант соответствует определенному набору критериев. Примеры таких критериев: роль человека, делающего выбор, цена выбранного варианта или соблюдение определенной политики. Одобрение может стать скрытым, когда на одном чело-

веке лежат обязанности по совершению выбора и его утверждению. То есть он одобряет сам себя.

Явное одобрение полезно, так как может дополнительно повысить безопасность решения. Но если принимается много решений и все они утверждаются *централизованно*, скорость этого процесса заметно снизится. В итоге слишком много решений будут ждать одобрения. Скрытое одобрение увеличивает скорость принятия решений, расширяя возможность выбора, но в то же время оно сопряжено с большим риском.

Управление этапом одобрения — важная составляющая вашей схемы руководства. Необходимо учитывать качества сотрудников, принимающих решения, влияние неудачных решений на бизнес и степень риска, связанного с предлагаемыми вариантами. Самые важные решения нужно одобрять явно. Для срочных и масштабных лучше ввести скрытую систему одобрения.

Применение

Принятие решения не заканчивается на его утверждении. Решение не реализовано, пока кто-то не начнет работу по выполнению выбранного варианта. Применение — тоже важная часть управления API. Если решения воплощаются слишком медленно или некачественно, значит, весь процесс их принятия пошел насмарку.

Обычно решение не реализуется людьми, которые сделали выбор. Здесь важно понимать, как при этом собирать точные данные. Например, вы можете внедрить в свой ландшафт API в стиле гипермедиа, но если их реализация окажется слишком сложной для дизайнеров и разработчиков, решение придется пересмотреть. Хорошая схема руководства должна учитывать эти практические аспекты. Управлять решениями, улучшая их только в *теории*, — плохая идея. При оценке качества решения важно учитывать такой критерий, как его применимость на практике.

Тестирование

Любое решение, которое вы принимаете в системе управления API, должно быть готово к тестированию. Как правило, мы не учитываем, что позднее наши решения могут быть пересмотрены, изменены или полностью отменены. Проверка позволяет нам планировать непрерывные изменения на уровне принятия решений.

Например, если вы определили «меню» выбора для команд API, разумно определить и процесс, который будет «выходить из меню». Так вы сможете

поддерживать высокий уровень инноваций и предотвратить принятие неверных решений. Но если каждый может оспорить решение ограничить этот выбор, то на самом деле никаких ограничений нет. Поэтому нужно определить, кто может проверять решение и при каких условиях.

Важно также позволять оспаривать решения с течением времени, ведь бизнес-стратегии и окружение могут меняться. Поэтому решения в вашей системе тоже должны пересматриваться. Чтобы спланировать такую адаптируемость, вам нужно встроить в систему функцию тестирования. Здесь важно выбрать того, кто в вашей организации сможет проверить уже существующее решение.

Планирование решений

Теперь мы знаем, что решения состоят из набора элементов. Это позволяет нам передавать отдельным командам части решения, а не весь процесс. Благодаря этой важной характеристике организационной схемы вы сможете сильнее влиять на баланс эффективности и основательности.

Очень важно решить, какой стиль должен иметь новый API. В неуклюжей бинарной схеме «централизация против децентрализации» разработчик должен понять: доверить решение по стилю членам команды (децентрализовать) или же его должно контролировать руководство (централизовать). Плюс распределения полномочий по принятию решений между командами API в том, что каждая может принимать решения в локальном контексте. Преимущества централизации принятия решения в одной стратегической команде — в уменьшении разнообразия в стилях API и контроле качества выбора стиля.

Компромисс не из легких. Но если вместо этого вы распределите *элементы* решения, можно разработать систему управления API, которая находится где-то посередине между этими двумя вариантами. Например, можно решить, что при выборе стиля API за элементы *исследования и поиска вариантов* будет отвечать централизованная группа стратегического управления API, а за элементы *выбора вариантов, одобрения и применения* — сами команды API. Так вы частично пожертвуете новизной, основанной на децентрализации выбора, в пользу преимуществ известного вам набора стилей API, используемого в компании. В то же время распределение элементов выбора стиля и одобрения позволяет командам по API работать максимально быстро, так как не нужно просить разрешения на выбор подходящего стиля.

Чтобы получить максимум преимуществ от планирования решений, необходимо распределять их, основываясь на общей ситуации и целях. Рассмотрим два пространственных сценария решений, чтобы понять, как их планирование может стать полезным инструментом.

Пример планирования решения: выбор языка программирования

Вы осознали важность решения по выбору языка программирования для реализации API и хотели бы им управлять. Ваша организация использует стиль архитектуры «микросервисы», и обязательным требованием была свобода выбора языка программирования. Но после нескольких тестов вы заметили, что разнообразие в языках программирования усложняет для разработчиков переход из команды в команду, а группам по безопасности и разработке — поддержку приложений.

В итоге вы решили попробовать воспользоваться для выбора языка программирования распределением решений из табл. 2.1.

Таблица 2.1. План решения по языку программирования

Идея	Поиск вариантов	Выбор	Одобрение	Применение	Тестирование
Централизовать	Централизовать	Децентрализовать	Децентрализовать	Децентрализовать	Децентрализовать

Так вы сужаете выбор языка программирования до набора вариантов, оптимизированных под вашу систему в целом, но позволяете отдельным командам изменить его под конкретную ситуацию. Вы также позволяете командам по API тестировать решение для проверки новых языков.

Пример планирования решения: выбор инструмента

Главный технический директор вашей компании пытается повысить уровень гибкости и инноваций вашей платформы программного обеспечения. В рамках этой инициативы он разрешил командам API выбирать собственные программные стеки для реализации, включая использование ПО с открытым исходным кодом. Но юридический отдел и отдел закупки обеспокоены этим из-за риска нарушения закона и ухудшения отношений с поставщиками. Чтобы начать вводить эти изменения, вы решили использовать для пробного выбора платформы ПО планирование решения из табл. 2.2.

Таблица 2.2. Планирование решения по выбору инструмента

Идея	Поиск вариантов	Выбор	Одобрение	Применение	Тестирование
Децентрализовать	Децентрализовать	Децентрализовать	Централизовать	Децентрализовать	Централизовать

Локальная оптимизация — один из ключей к стратегии вашего технического директора, поэтому вы решили полностью децентрализовать идею, поиск вариантов и выбор, но для уменьшения риска на уровне системы передали элемент одобрения юридическому отделу и отделу закупки. Сейчас это должно сработать, но вас беспокоит, что с увеличением объема работ этот момент может стать слабым местом системы. Поэтому вы отметили, что нужно наблюдать за ним и при необходимости его изменить.

Проектирование решений на практике

В своей работе по управлению API мы редко документировали решения с помощью карты решений из этого раздела. Причина в том, что с помощью карты решений не всегда удобно рассказать о том, как должна выполняться работа или как команда может достичь своей цели. Рассматривайте ее как полезную модель, которую вы можете держать в своем «наборе инструментов» для управления API.

На практике использование карты решений — не лучший способ описания принципа работы и взаимодействия команд и людей друг с другом для успешного выполнения работы. Это связано с тем, что карта решений — это своего рода высокоуровневая абстракция. Она сосредоточена только на элементах решения. Вместо этого вам нужно изложить свой замысел на языке, соответствующем вашему контексту.

Например, в области управления предприятием вы можете проектировать решения через создание целевой операционной модели (target operating model, ТОМ). ТОМ описывает организационные структуры, модели процессов и инструменты, которые понадобятся командам для достижения успеха. Если вы работаете в сфере технологий и архитектуры, то с помощью *топологии команд* можете нарисовать модель координации, которая может быть воплощена в архитектуре ПО. В итоге вам нужно будет выразить ваше целевое состояние на языке работы, решений и концепций, который будет понятен вашим сотрудникам.



Топологии команд

«Топологии команд» (<https://oreil.ly/wmJZY>) — это книга, где описывается одноименный подход к проектированию, созданный Мэтью Скелтоном и Мануэлем Пайсом. Он предоставляет полезную модель и язык для разработки ПО, фокусируясь на командах и способах их совместной работы.

Важно понимать, что процесс принятия решений можно разбить на распределяемые части. Это способствует большей точности в вашем подходе к руководству и поможет вам добиться более высоких результатов, когда придет время разрабатывать важные части вашей системы руководства. Решив, какие подразделения в организации должны владеть отдельными элементами ключевых решений, вы сможете начать применять решения, направленные на введение ограничений и изменение поведения этих команд и отдельных сотрудников.

Разработка системы руководства

Мы потратили много времени на детали распределения решений, потому что считаем этот момент основным в системе руководства. Но это не единственное, на что нужно будет обратить внимание, если вы хотите эффективно руководить API. Хорошая система руководства API должна иметь следующие характеристики:

- распределение решений, основанное на влиянии, объеме и масштабе;
- усиление системных ограничений и проверка качества (для централизованных решений);
- поощрение формирования решений (для децентрализованных решений);
- адаптивность за счет измерения уровня влияния и постоянных улучшений.

Трудно воспользоваться преимуществами централизации решений, если вся остальная организация не выполняет принятое решение. Вот почему принудительное применение и тестирование должны быть частью системы управления API. До сих пор мы намеренно избегали авторитарных частей управления, но в итоге вам все-таки нужно будет встроить в свою систему хотя бы некоторые ограничения. Даже у самых децентрализованных организаций есть правила, которым нужно следовать. Конечно, проверка качества и усиление ограничений потребуют внедрения некоторого уровня подчинения. Если у централизованной команды, ответственной за принятие решений, нет власти, ее решения не будут иметь веса.

Если у вас не авторитарный стиль управления, вы можете использовать поощрение вместо принуждения. Это особенно полезно, если вы решили децентрализовать принятие решений, но при этом хотите контролировать его. Например, команда по архитектуре может изменить процесс развертывания неизменяемых контейнеров так, что оно станет намного дешевле и проще, чем развертывание контейнеров другого типа. Цель здесь в том, чтобы побудить команды API,

которые имеют право принимать собственные решения по внедрению, чаще выбирать контейнеризацию.

На самом деле ни пряника поощрения, ни кнута ограничений недостаточно, чтобы полностью руководить вашей системой: нужно эффективно задействовать и то, и другое. В общем, если элемент одобрения решения был децентрализован и вы хотите его контролировать, надо использовать поощрение. Если выбор и одобрение были централизованы, а применение — децентрализовано, важно убедиться, что введен определенный уровень ограничений или проверка качества. Таблица 2.3 показывает, когда нужно ограничивать или поощрять решение, основываясь на схеме его планирования.

Таблица 2.3. Когда ограничивать и поощрять решение?

Ограничивать или поощрять?	Идея	Поиск вариантов	Выбор	Одобрение	Применение	Тестирование
Ограничивать		Централизован	Централизован или децентрализован	Централизованно или децентрализовано		
Поощрять		Децентрализован	Децентрализован	Децентрализовано		

Независимо от вашего способа распределения решений и стиля их принятия очень важно определить, как вы влияете на саму систему. В идеале у компании должны быть какие-то индикаторы и параметры процесса, которые можно использовать для измерения влияния изменений. Если их нет, то их обязательно нужно внедрить на уровне организации. Позже мы поговорим о схемах параметров для продуктов API (см. главу 7). Хотя мы сосредоточимся конкретно на параметрах API-продуктов, этот раздел можно применять как пособие по разработке параметров руководства для вашей системы.

Чтобы связать все это вместе, рассмотрим три шаблона руководства API. Они охватывают разные подходы к руководству, но все придерживаются главных принципов: распределение решений, ограничение, поощрение и измерение. Имейте в виду, что мы не предлагаем вам меню — вы не обязаны выбирать для себя одну из этих систем. Мы предлагаем вам эти шаблоны, чтобы показать, как система управления API может быть реализована на концептуальном уровне.

Для каждой из описанных схем мы определим несколько ключевых решений и способов планирования, а также способы обеспечения и стимулирования желаемого поведения, способы распределения талантов, затраты, доходы и параметры каждого подхода.

Шаблон руководства 1: ответственные разработчики

Ответственные разработчики действуют как гейткиперы, обеспечивая соответствие результатов работы команд API минимальному уровню качества. Это централизованные группы, которые обеспечивают качество принятия решений в организации. Они могут быть представлены как официальные советы по проверке, организующие регулярные собрания, или как служба проверки по запросу. Опытные ответственные разработчики могут даже предоставлять инструменты самообслуживания, чтобы упростить работы по тестированию и снизить их стоимость.

ЦЕНТРАЛЬНАЯ КОМАНДА РАЗРАБОТЧИКОВ PAYPAL

В компании PayPal центральная группа, отвечающая за разработку, проверяет все новые проекты API с помощью четырехэтапного процесса¹. Они начинают с изучения предложений по новым API, чтобы убедиться, что они хорошо подходят для бизнеса и других таких еще нет. Затем они тестируют проекты API на их соответствие опубликованным стандартам PayPal. После разработки API команда разработчиков проводит ряд тестов для подтверждения их соответствия контракту на проектирование. Наконец, опубликованный API проверяется на соответствие требованиям безопасности PayPal.

- *Ограничения и поощрение.* Ответственные разработчики наиболее эффективны, когда у них есть полномочия предотвращать принятие низкокачественных и рискованных решений. Обычно это значит, что они уполномочены остановить внедрение изменений, если их требования к качеству не выполняются. В некоторых компаниях у ответственных разработчиков нет полномочий. Вместо этого такие команды публикуют аудиторские записки с указанными рисками. В этих случаях качество принятия решений зависит от желания команды *учесть эти замечания*. Будет это работать или нет, во многом зависит от культуры организации и ее сотрудников. Для эффективного функционирования ответственные разработчики должны быть чем-то большим, чем просто дежурной командой. Эта команда должна как подтверждать правильность принятых решений, так и сообщать командам о том, как удовлетворить их требования к качеству. Это означает, что они должны предоставлять доступную информацию и рекомендации, чтобы помочь командам избежать бесконечного цикла подтверждения соответствия.
- *Распределение талантов.* В этой модели несколько экспертов, принимающих решения, сосредоточено в команде ответственных разработчиков. Но они

¹ Bush T. PayPal's Four-Step Process for Building Governance-Friendly APIs («Четырехэтапный процесс PayPal для создания удобных в управлении API») // Nordic APIs (блог), 9 июня 2020 года, <https://oreil.ly/H6Ahj>.

должны поддерживаться командами API, где есть компетентные лица, способные принимать решения, соответствующие их требованиям. В противном случае система увязнет в низкокачественных проектах, которые нуждаются в постоянной помощи. В этой схеме уровень квалификации команд API обычно повышается со временем по мере прохождения процесса проектирования и анализа.

- *Расходы и доходы.* Основное преимущество этой модели в том, что все API выполняются одной командой контроля качества. Это дает организации максимальную уверенность в том, что были приняты верные решения и учтены риски. Такая цепетильность очень важна для решений, которые могут негативно повлиять на бизнес. Например, в крупных компаниях проекты API почти всегда проверяются, чтобы убедиться в том, что они правильно реализуют средства управления безопасностью и доступом. Но сильная сторона команды ответственных разработчиков — это также и ее большая слабость. Выполнение всех изменений API через единую централизованную команду — это ахиллесова пята производства, которая рано или поздно проявит себя. На ранних стадиях развития компании, использующей API, ответственные разработчики могут оказать огромную помощь. Но со временем это может стать огромной проблемой, заставляя проекты API приостанавливаться в ожидании работы с командой.

Шаблон руководства 2: внедренные централизованные эксперты

В этой схеме, вместо того чтобы проверять результаты работы команды API, в нее внедряются эксперты, чтобы помочь в принятии решений. Типичная реализация этой схемы — модель внутреннего консультирования, где центральная группа экспертов API распределяется между командами API. Эти эксперты либо принимают ключевые решения, либо наделяются полномочиями принимать решения от имени команды. Но главная особенность этой модели в том, что эксперты становятся частью команды API, вкладывая свое время и помогая добиться лучших результатов.

Модель внедренных экспертов опирается на центральную команду экспертов, которые могут быть распределены для работы в командах API. Как ни странно, это означает, что части решений, связанные с исследованиями и выбором, централизованы (даже если они выполняются в объединенных командах). Это работает, когда у центральной группы экспертов есть общее понимание «правильных» решений для компании. Думайте об этом как о распределенной версии команды ответственных разработчиков. Но фактическое согласование

и реализация этих решений по-прежнему принадлежат самим командам, оставляя эти элементы децентрализованными. Мы обсудим этот тип централизованной структуры команды позже, в разделе «Центр поддержки» на с. 290.

ЧЕМПИОНЫ HSBC В ОБЛАСТИ API

HSBC — это глобально разнообразная и децентрализованная организация, где много разных команд создают интерфейсы API для своих клиентов. Чтобы помочь своим командам создавать лучшие API, они создали сеть *чемпионов API* (<https://oreil.ly/Gezig>), которые понимают и применяют стандарты API HSBC для местных проектных команд. Это помогает им распространять знания об API в масштабах всей организации.

- *Ограничения и поощрение.* Внедрение экспертов в проектную команду — это высшая форма принуждения. Потому что внедренные эксперты либо сами принимают решения, либо напрямую информируют о процессе их принятия. Если ваши эксперты принимают решения, соответствующие вашим основным целям, то же самое сделают и команды, в которые они включены.
- *Распределение талантов.* Одна из главных задач координационной команды — поиск и поддержка группы экспертов. Чтобы эта схема работала, вам понадобится группа экспертов API, которые могут быть распределены между командами разработчиков проекта и продукта. Эти специалисты могут финансироваться централизованно, но распределяться децентрализованно между командами API, и их будет нужно масштабировать, чтобы удовлетворить спрос на работу API в системе.
- *Расходы и доходы.* Нахождение на «передовой» работы над API имеет ряд преимуществ. Во-первых, это гарантирует раннее принятие лучших решений благодаря привлечению к работе экспертов центральной команды. Во-вторых, эксперты могут использовать опыт и знания, полученные в ходе работы, для постоянного улучшения централизованного руководства в соответствии с потребностями команд по разработке продуктов и проектов. Но у этого шаблона есть серьезные эксплуатационные проблемы. Он требует наличия достаточного числа экспертов для оказания помощи каждой команде. В зависимости от масштаба вашей организации это может оказаться непростой задачей. Наконец, нужно согласовать усилия для поддержания общего взгляда на оптимизацию на системном уровне среди экспертов, которые сталкиваются с повседневными проблемами при предоставлении продуктов API. Со временем это может привести к созданию полностью децентрализованной модели принятия решений с недостаточной согласованностью или управлением.

Шаблон руководства 3: влияние на самоуправление

Мы заметили тенденцию в современных организациях к уменьшению централизованного контроля и увеличению автономии команд. По мере того как мир бизнеса и технологий продолжает стремиться к увеличению инноваций и ускорению темпов изменений, уменьшается интерес к централизованным командам, авторитарно контролирующим процесс принятия решений. Это привело к появлению третьего шаблона управления, который в большей степени полагается на влияние, а не на контроль.

В этой схеме команды API обладают автономией в принятии решений и владеют всеми элементами этого процесса. Их решения «регулируются», влияя на решения, которые они принимают. Обычно это формулируется так: затруднить выполнение неправильных действий и упростить выполнение правильных.

ЗОЛОТОЙ ПУТЬ КОМПАНИИ SPOTIFY

Spotify использует платформенный подход под названием «Золотой путь» (<https://oreil.ly/chT9y>), который предоставляет каталог инструментов и услуг для инженерных групп Spotify. Это рекомендуемые инструменты в системе Spotify. Командам проще использовать эти инструменты, ведь все знают, что они «благословлены» платформенной командой и получают поддержку. Но при необходимости команда может *отойти от каталога* и использовать инструмент по своему выбору.

- *Ограничения и поощрение.* Эта модель полностью полагается на стимулирование для влияния на принятие решений. Она воплощает принцип Netflix «Свобода и ответственность» (<https://oreil.ly/DdTx1>). Сотрудникам предоставляется «Золотой путь» рекомендуемых решений, который определяется центральной командой. Они могут принять и другое решение, но они все также несут ответственность за успех своих продуктов. В идеале этот баланс побуждает команды принимать решения, соответствующие предложениям центральной команды.
- *Распределение талантов.* Чтобы этот тип шаблона работал, команды должны быть способны принимать правильные решения независимо друг от друга. Это значит, что компетенции должны быть распределены так, чтобы в каждой команде был хотя бы один эксперт, способный принимать правильные решения по API. В организациях, где такой масштаб распределения талантов невозможен, эта схема часто сочетается со схемой команды ответственных разработчиков («Шаблон руководства 1: ответственные разработчики» на с. 63) в качестве меры предосторожности.
- *Затраты и доходы.* Главное преимущество этой схемы — скорость. Команды могут продвигаться очень быстро, когда у них есть полномочия принимать

решения. Но такая скорость сопряжена с риском того, что такие решения будут непоследовательными и/или неадекватными. Кроме того, локальные команды могут чрезмерно оптимизировать свои решения в соответствии с местными условиями в ущерб системе. На практике самоуправление часто сочетается с централизованной схемой управления, чтобы сбалансировать эти факторы.

Внедрение моделей руководства

Как мы уже упоминали, если ваша организация API представляет собой сложную адаптивную систему, то для достижения хороших результатов ей нужно много «подталкиваний». Мы также представили набор шаблонов, которые помогут вам распределять экспертов и инструменты, чтобы выбрать верное направление для принятия решений. В этом разделе мы подробно остановимся на некоторых стратегиях практического применения и внедрения этих шаблонов.

Мы расскажем о высокоуровневых составляющих решения по руководству, которые необходимо будет рассмотреть, включая то, как начать работу, как получить информацию и создавать инструменты и активы. Позже в этой книге мы подробнее остановимся на конкретных аспектах управления и руководства. Например, в главе 9 мы рассмотрим соображения при формировании центральной команды и концепцию «платформы».

Разработка решения

Главный постулат этой книги — чтобы добиться успеха, нужно *непрерывно* управлять API. Это значит, что вам нужно постоянно адаптировать и совершенствовать свою систему управления по мере роста и изменения организации. По правде говоря, вам не удастся создать идеальную систему управления в первый же день.

Но мы все равно должны стремиться начать с принятия решения, которое сразу сработает как можно лучше. Нам также важно убедиться, что мы не внедряем решение, изменение которого потребует больших затрат. Учитывая это, мы предлагаем следующие рекомендации по внедрению нового решения для управления.

- *Внедрять на ранней стадии.* Приступая к созданию нового решения по управлению, постарайтесь начать с реализации модели, описанной в разделе «Шаблон руководства 2: внедренные централизованные эксперты» на с. 64, с небольшим набором предварительных продуктов или проектов. Это не всегда возможно, особенно при наличии множества невыполненных работ по API,

которые нужно быстро проверить. Но если вы можете себе это позволить, то, начиная с внедрения, вы получаете возможность протестировать и изучить ваши стандарты, прежде чем вы утвердите их и сообщите о них организации.

Обычно легче изменить проектное решение в ходе проекта, чем изменить опубликованный стандарт, который приняла компания в полном составе. Еще одно преимущество такого подхода — то, что ваша команда экспертов может создать сеть взаимоотношений с командами разработчиков продукта и получить опыт с передовых позиций. Если вы планируете перейти к централизованному управлению разработкой (как описано в разделе «Шаблон руководства 1: ответственные разработчики» на с. 63), это может помочь противостоять синдрому «оторванности от реальности», синдрому отсутствия связи, который часто развивается в высокоцентрализованных командах.

- *Внедряйте возможность наблюдения на ранних этапах.* Вам никогда не удастся улучшить свою систему, если вы не сможете наблюдать за тем, что в ней происходит. По нашему опыту, стоит инвестировать в наблюдаемость и наглядность на ранних этапах разработки вашего решения по управлению. Чем больше информации вы сможете получить, тем лучше. С практической точки зрения имеет смысл начать со сбора данных. По мере развития вашего решения вы можете улучшать свои функции анализа и наблюдения.
- *Затем автоматизируйте.* Когда дело доходит до управления API, автоматизация дает огромное преимущество. Инструменты и автоматизация снижают эксплуатационные расходы, дают больше возможностей для сбора данных и облегчают всем соблюдение ваших стандартов качества. Но все имеет свою цену. Для внедрения решения по автоматизации нужны усилия и инвестиции. Изменения в решении тоже могут быть сопряжены с большими затратами. Это может привести к тому, что организации не захотят менять свои рекомендации по управлению, так как они ограничены выбранными инструментами. По нашему опыту, автоматизация и инструментальное оснащение идеально подходят для более устоявшихся областей принятия решений в вашем ландшафте API. Мы рекомендуем сначала запустить процесс проверки проектов API человеком, прежде чем создавать инструмент *статистического анализа кода* для автоматизации проверки проектов. Так у вас будет возможность гибко устанавливать правильные проверки, прежде чем вы остановитесь на инструментальном решении.
- *С осторожностью создавайте централизованные команды.* По мере роста числа API (и команд API) в вашей компании неизбежно возникнет необходимость в создании централизованных групп для помощи в управлении работой по принятию решений. Такие команды — неотъемлемая часть масштабирования всех моделей руководства, описанных ранее. Но будьте осторожны при создании новой централизованной команды. В отличие от стандартов

и инструментов, команды часто трудно сократить или расформировать. Иногда централизованная команда может начать делать своей целью собственное существование. Выпустив джинна из бутылки, его трудно вернуть обратно. При создании таких групп начните с временного *заимствования ресурсов* у других команд или сохраните небольшую и работающую с минимальными затратами команду, пока спрос не потребует ее роста. Другой подход в том, чтобы сделать расформирование центральной команды основной целью. В конечном счете центральная команда служит важной цели: оптимизации API для системных задач. Для вас важно найти баланс между удовлетворением этой потребности и созданием ненужных расходов.

Наблюдаемость и наглядность

Мы уже упоминали, что получение данных на ранних этапах — важная составляющая при реализации эффективного решения для руководства. Для начала сосредоточьтесь на сборе информации по этим базовым пунктам:

- все API, которые были выпущены и работают в производственных системах;
- владение и финансирование API в организации;
- динамический трафик для каждого API;
- уровень принятия (или соответствия) ваших стандартов и инструментов для каждого API.

Сложность сбора этой информации будет во многом зависеть от размера вашей организации и ее инвестиций в API. На крупных предприятиях сбор таких данных может и вовсе стать самостоятельным проектом. Подробнее мы рассмотрим этот вопрос в разделе «Понимание ландшафта» на с. 270.

Но для достижения адекватной наблюдаемости потребуется нечто большее, чем просто проект по сбору данных. Вам нужно будет повлиять на решения API, чтобы команды давали вам необходимые данные. Вы можете использовать инфраструктуру и инструменты для автоматизации сбора данных во время выполнения. Постарайтесь сосредоточиться на этих аспектах вашей системы как можно раньше.

Рабочие модели

Если вы внедряете полномочия по проектированию (см. раздел «Шаблон руководства 1: ответственные разработчики» на с. 63) для некоторых областей вашего пространства принятия решений, вам нужно организовать официальные встречи, контрольные точки или форумы для анализа.

Но даже если вы не примените этот шаблон, вам все равно придется придумать, как вы будете собирать информацию и обмениваться ею в организации. Для этого подумайте, как ваши команды будут ежедневно работать и как они будут координировать свои действия друг с другом.

Это важная часть разработки и внедрения системы руководства. Ваша операционная или координационная модель будет иметь сильное влияние на автономность и скорость работы ваших API-команд и на качество принимаемых ими решений. Способ обмена информацией и принятия решений будет во многом зависеть от вашей организационной культуры. Мы подробнее поговорим о командных проектах и моделях координации позже (см. главу 8). Сейчас имейте в виду, что это базовая составляющая разрабатываемого вами решения.

Разработка стратегии управления стандартами

Мы еще не встречали команду, которая не работала бы с какими-то стандартами для разработки своих API. Иногда эти стандарты не записываются и передаются устно. Но в большинстве случаев они все же сохраняются в письменной форме.

Стандарты полезны — они документируют ограниченный набор вариантов для пространства принятия решений. Поэтому тщательно обдумайте, как вы будете собирать, организовывать и распространять стандарты для своей системы. Но помните, что каждый ваш стандарт влечет за собой расходы на управление и эксплуатацию. К сожалению, мы знаем много компаний, которые начинали с небольшого набора полезных стандартов, а затем превращали их в неуправляемый, сложный в использовании (и часто устаревший) беспорядок документации.

В идеале стандартами нужно управлять так же, как продуктом или платформой. Как минимум вы должны определить, как они должны создаваться, как они будут редактироваться, распространяться и поддерживаться.

Например, вы можете открыть доступ к разработке стандартов для всех сотрудников, но уполномоченный орган, отвечающий за разработку, должен определить, какие именно будут опубликованы. Можно также следовать процессу, подобному IETF (<https://oreil.ly/UA97h>), который обеспечивает прозрачность рассмотрения для локального принятия стандартов.

Управление стандартами и процессы не уникальны для сферы API. Но из-за разнообразия решений в этой области стандарты неизбежны. Убедитесь, что работа, которую вы выполняете по развитию процесса стандартизации, имеет смысл для ваших команд и целей.

Итоги главы

В этой главе мы изложили наше определение руководства — управление принятием и реализацией решений. После мы подробнее остановились на том, что значит принять решение и руководить им. Вы узнали, что решения в API могут быть незначительными («Какой будет следующая строчка моего кода?») или важными («С каким поставщиком нам сотрудничать?») и сильно различаются по объему. И главное, вы узнали, что система, которой вам предстоит руководить, относится к *сложным адаптивным системам*. Поэтому трудно заранее предсказать результаты применения любой стратегии управления решениями.

Далее мы подробно рассмотрели распределение решений и сравнили централизацию с децентрализацией. Чтобы помочь вам понять разницу между ними, мы выявили их различия по *масштабу оптимизации* и *объему работы*. Затем мы обсудили, как разбить решения на основные составляющие: идею, поиск вариантов, выбор, одобрение, применение и тестирование. Объединив все эти понятия и используя ограничения и поощрения, можно построить эффективную систему руководства API.

Руководство — это сердце управления API, поэтому неудивительно, что это основное понятие всей книги. Нашей целью в этой главе было ввести основные понятия и рассказать о рычагах управления. Далее мы углубимся в сферу руководства API, затрагивая специфические проблемы, где решения имеют наибольшее значение. Позже мы расскажем, как управлять сотрудниками и что делать, когда API развивается. Следующую главу мы начнем с исследования того, как мышление с точки зрения продукта поможет вам определить важнейшие решения в работе с API.

ГЛАВА 3

API как продукт

Все, что создают люди — будь то продукт, коммуникация или система, — результат воплощения вдохновения в жизнь.

Мэгги Макнаб

Фразу «API как продукт» (AaaS) часто упоминают в разговоре о компаниях, создавших и поддерживающих успешные программы API. Это перефразированное выражение «...как услуга», которое часто используется в технических кругах («ПО как услуга», «платформа как услуга» и т. д.). Как правило, оно отображает важную роль разработки, применения и реализации API, ведь API — это *продукт*, полностью заслуживающий продуманного дизайна, создания прототипов, исследования клиентской базы, тестирования, длительного контроля и технической поддержки. Обычно эта фраза означает: «Мы относимся к нашим API так же, как к любым другим нашим продуктам».

В этой главе мы рассмотрим подход AaaS и то, как его использовать для лучшего проектирования, развертывания ваших API и управления ими. Этот подход включает в себя осознание того, какие решения важны для успеха ваших API и в каком отделе их должны принимать (см. главу 2). Он поможет вам обдумать, какую работу надо централизовать, а какую можно успешно децентрализовать, где лучше задействовать ограничения и поощрения и как измерить степень влияния этих решений для быстрой адаптации ваших продуктов (API).

При создании новой продукции для клиентов приходится принимать множество решений. Независимо, разрабатываете ли вы плееры, ноутбуки или API для очереди сообщений. Во всех этих случаях вам нужно: 1) знать свою целевую аудиторию, 2) понимать и решать ее первостепенные проблемы и 3) обращать

внимание на предложения клиентов об улучшении продукции. Эти важнейшие задачи можно сформулировать в виде трех ключевых уроков, на которых мы сосредоточимся в этой главе:

- дизайн-мышление как способ убедиться в том, что вы знаете свою целевую аудиторию и понимаете ее проблемы;
- поддержка новых клиентов как способ быстро показать им, как успешно пользоваться вашим продуктом;
- опыт разработчика как способ управления жизненным циклом продукта после его выпуска и для выработки идей будущих модификаций.

Вскоре мы узнаем о силе дизайн-мышления и клиентской поддержки на примере таких компаний, как Apple. Увидим, как Джефф Безос помог подразделению Amazon Web Services (AWS) создать устав реализации, обеспечивающий получение четкого и предсказуемого опыта разработчика. Мы общались с представителями многих компаний. Большинство из них понимают, что такое AaaS, но не могут применить этот подход на практике. Но во всех организациях, на счету которых немало успешных продуктов API, поняли, как эффективно использовать три упомянутых ключевых урока, первый из которых связан с тем, как ваши команды *обдумывают* то, что создают.

Программируемая экономика, основанная на API

API — это интерфейсы для создания программируемой экономики. Но они должны быть разработаны так, чтобы их можно было найти, масштабировать и они выполняли заявленные функции для решения проблем разработчиков. Для этого компаниям нужно правильно подходить к управлению ожиданиями и стремлениями своих сообществ разработчиков. Именно здесь в игру вступают отношения с разработчиками. Они устанавливают связь между тем, что может предоставить каждый из ваших API, и квалифицированными специалистами, которые будут внедрять их в другие приложения, — разработчиками. Чуть позже мы рассмотрим, как API меняют правила игры в программируемой экономике, обеспечивая больший охват, масштабируемость и повсеместность, и рассмотрим роль отношений с разработчиками в контексте управления API, пропаганды и евангелизации.

Чтобы говорить о значительности API, нужно понять, почему они так важны для бизнес-стратегии. В 2011 году один из самых известных инвесторов Кремниевой

долины и основатель Netscape Марк Андрессен заметил: «Программное обеспечение поглощает мир».

До этого возможности информационных технологий (ИТ) были интегрированы в организации для поддержки бизнеса. Но с появлением сетевых и инфраструктурных технологий, позволивших реализовать модель «как услуга», когда ПО можно запускать из чужой инфраструктуры (SaaS, PaaS, IaaS), ИТ *стали* бизнесом, где третьи стороны могли предлагать самообслуживание, автоматизированные, программируемые и отслеживаемые функции, позволяя бизнесу сохранять полный контроль. А что стало ключом, позволившим этим сторонним организациям предоставлять возможности «как услуга»? Это открытые API.

Эти программируемые интерфейсы позволяют компаниям и их приложениям открывать свой бизнес для третьих сторон и выходить за пределы собственных возможностей развития. Вне организации находится больше разработчиков, ресурсов, идей, знаний рынка, энтузиазма, инноваций и капитала, чем внутри. Так почему бы не открыть активы и возможности для большего числа экономических и общественных субъектов, которые могут быть вовлечены в создание ценности?

Раньше конкуренция шла по принципу «продукт против продукта», но теперь он сменился на принцип «платформа против платформы», а затем перерос в принцип «экосистемы против экосистем». API — это программируемые интерфейсы, которые ведут от одной формы к другой.

Цена, продвижение, продукт, место → везде

Классические менеджеры знают 4P от гурзу маркетинга Майкла Портера: Price, Promotion, Product, Place (цена, продвижение, продукт, место). Это четыре переменные, которыми вы управляете в маркетинговой стратегии продукта.

Но в цифровом мире, где API «съедают» ПО, по словам Эндрю Босворта, главы отдела развития Facebook, «побеждает не лучший продукт, а тот, который используют все»¹. Это относится и к вашим API. Иногда опыт использования предоставляемого продукта может иметь большее значение для клиента по сравнению с некоторыми дополнительными функциями, которые вы захотите добавить. Итак, опыт разработчика, который вы предоставляете пользователям своего API, будет иметь конкурентное преимущество.

В цифровом мире, где ИТ-возможности предоставляются как услуга через API, цель не в том, чтобы быть в нужном месте, а в том, чтобы быть везде. Речь идет не о том месте, которое вы можете контролировать, а обо всех возможных местах,

¹ Balfour B. Growth Wins («Пост побеждает») // Reforge (запись в блоге), последнее изменение от 25 июля 2018. <https://oreil.ly/UJDIU>.

в любых приложениях. Например, банковская и финансовая индустрия претерпевает изменения из-за API, позволяющих внедрять встроенные финансы.

Пол Рохан, автор и эксперт по Open Banking API, объясняет, что будущее банковского дела не «в банке», а везде, где нужно банковское обслуживание: встраивание в сторонние клиентские сервисы, на платформах недвижимости, в приложениях для планирования свадеб, в виджетах на сайтах автодилеров, на сайтах электронной коммерции — везде¹. Благодаря API банки могут иметь доступ к любой клиентской базе, которой они не владеют, получая при этом возможность предоставлять выгодные предложения.

В сообщении блога, опубликованном в 2020 году, идейный вдохновитель платформы и отраслевой консультант Саймон Торранс подсчитал, что в течение пяти лет возможности встроенного финансирования составят 7,2 миллиарда долларов. Это вдвое превышает общую рыночную стоимость нынешних банковских и финансовых услуг². Когда вы повсюду, вы получаете новый, более широкий охват.

Давайте посмотрим, какие изменения произошли в этой области за последние 20 лет. Для распространения своего продукта в цифровом формате в 2000 году вам нужен был веб-сайт. В 2010 году для этой цели вам понадобилось мобильное приложение. В 2020-м — API. Реалии таковы, что новый способ завоевания цифрового пространства — это интеграция в сторонние сайты и приложения. Речь идет уже не о контроле над одним-двумя каналами, а о внедрении в максимально возможное число тех каналов, где находятся ваши пользователи.

Как описывает Крис Андерсон в своей книге «Длинный хвост» (Nachette), если первые каналы распространения (веб- и мобильные) составляли значительную часть общего трафика, то «длинный хвост» остальных самых маленьких ниш и каналов фактически представлял собой растущий со временем трафик, который иногда становился даже больше, чем традиционные каналы.

Некоторые приложения, такие как Salesforce или eBay, получают основную часть своего трафика через сторонние платформы, а не через собственный сайт или мобильные приложения. Более 50 % их трафика поступает через API. Единственная проблема для компаний была в том, что слишком сложно и дорого появляться во всех этих каналах одновременно. Но теперь можно обращаться к нескольким каналам с помощью одних и тех же API.

¹ Rohan P. Driving Business Growth and Brand Strategy in the Api-Powered Age of Assistance («Стимулирование роста бизнеса и стратегия бренда в эпоху помощи, основанной на API») // APIdays, Лондон, 2019. <https://oreil.ly/IcxbV>.

² Torrance S. Embedded Finance: A Game-Changing Opportunity For Incumbents («Встроенные финансы: возможность изменить правила игры для сотрудников»). 10 августа 2020 г. <https://oreil.ly/L6I3n>.

Теперь API — продукт нашего программируемого мира. В следующей главе вы научитесь обращаться с ними как с продуктами, начиная с ознакомления с проектом и начальных шагов и заканчивая опытом разработчиков и успешными интеграциями.

Дизайн-мышление

Среди разработчиков продукта компания Apple известна, в частности, благодаря своей способности задействовать *дизайн-мышление*. Например, описывая работу по созданию Apple Mac OS X, один из ключевых разработчиков ПО Корделл Ратцлафф сказал: «Мы сосредоточились на том, что, по нашему мнению, было нужно людям, и на том, как они взаимодействуют с компьютером»¹.

Этот подход затем отразился на практике. «Для создания идеи продукта нужны три отчета: о требованиях маркетинга, о технических требованиях и об опыте пользователей», — однажды объяснил бывший вице-президент Apple и один из первопроходцев в области разработки взаимодействия человека и компьютера Дональд Норман².

Такое внимание к потребностям пользователей привело Apple к созданию стабильного бизнеса. Непрерывная цепочка продуктов, выпускаемых в течение нескольких десятилетий, создала им репутацию компании, определяющей новые тенденции в технологиях, и помогла неоднократно занимать большой сегмент рынка.

Тим Браун, генеральный директор IDEO, расположенной в Калифорнии фирмы по разработке и консультированию, определяет термин «*дизайн-мышление*» так³:

Раздел дизайна, где используются чутье и методы разработчика, чтобы продукт соответствовал требованиям пользователей и был технологически осуществимым, а конкурентная бизнес-стратегия могла извлечь из него потребительскую ценность и возможность для рынка.

¹ Meade S. Steve Jobs: Mac OS, Designed by a Bunch of Amateurs («Стив Джобс: Mac OS, разработанная кучкой дилетантов») // Synap Software, LLC (блог), 16 июня 2007 г. <https://oreil.ly/jRURa>.

² Turner D. The Secret of Apple Design («Секрет дизайна Apple») // MIT Technology Review, 1 мая 2007 г. <https://oreil.ly/ehrgv>.

³ Brown T. Design Thinking («Дизайн-мышление») // Harvard Business Review, июнь 2008 г. <https://oreil.ly/VRA7Y>.

Здесь важно пояснить несколько пунктов. Для наших целей нам нужно сосредоточиться на понятиях «соответствие требованиям пользователей» и «конкурентная бизнес-стратегия».

Соответствие требованиям пользователей

Одна из ключевых целей разработки API — решение проблем. Поиск проблем, требующих решения, и определение их приоритетности — часть задачи подхода АааР. Это ответ на вопрос «что делать?» в области API. Еще более важная составляющая — ответ на вопрос «для кого?». Кому вы облегчите жизнь с помощью этого API? Правильное определение целевой аудитории (ЦА) и ее проблем — первый шаг к тому, чтобы убедиться, что вы создаете нужный продукт: он эффективно работает и часто используется ЦА.

Клейтон Кристенсен из Гарвардской бизнес-школы называет процесс осознания потребностей вашей ЦА теорией «работы, которые нужно выполнить». Он говорит: «Люди не просто покупают товары или услуги, они “нанимают” их, чтобы продвинуться в определенных обстоятельствах»¹. Люди (потребители) хотят продвинуться (решить проблемы), и они будут применять (или арендовать) те продукты и услуги, которые, по их мнению, помогут им в этом.

МОЖНО ЛИ ПРИМЕНЯТЬ АААР КАК К ВНУТРЕННИМ, ТАК И К ВНЕШНИМ API?

Да. Но, возможно, с разными затратами времени и ресурсов — это мы обсудим в следующем разделе. Это один из тех уроков, которым Джефф Безос поделился в «Уставе Безоса», позволившем Amazon открыть изначально внутреннюю платформу AWS для использования в качестве приносящего доход внешнего API. Поскольку Amazon с самого начала применяла подход АааР, предложить внешним пользователям внутренний API было не только *возможно* (то есть безопасно), но и *выгодно*.

В большинстве компаний IT-отдел помогает решать проблемы. Обычно их клиенты — сотрудники той же компании (внутренние разработчики). Иногда это важные бизнес-партнеры или даже анонимные разработчики сторонних приложений (внешние разработчики).

Каждая из этих групп разработчиков (внутренняя, партнерская и внешняя) сталкивается со своим набором проблем и по-своему обдумывает и решает их.

¹ Jobs to Be Done («Работы, которые нужно выполнить») // Институт Кристиансена, последнее изменение от 13 октября 2017 г. <https://oreil.ly/11s63>.

Дизайн-мышление побуждает команды лучше узнать свою аудиторию, прежде чем приступить к созданию API в качестве решения. Мы исследуем эту тему в разделе «Познакомьтесь с целевой аудиторией» на с. 87.

Конкурентная бизнес-стратегия

Другая важная часть планирования дизайна — определение *конкурентной бизнес-стратегии* вашего API. Бессмысленно тратить много денег и времени на продукт, который не окупится. Даже когда вы стараетесь разработать правильный продукт для правильной ЦА, важно убедиться, что ресурсы потрачены не впустую и у вас есть четкое представление о доходе после запуска API.

У большинства компаний ограничено количество ресурсов, которое можно выделить на создание API для решения проблем. Поэтому огромное значение приобретает выборка этих проблем. Иногда мы сталкиваемся с компаниями, где API не решают важных бизнес-задач, разбираясь вместо этого с хорошо знакомыми проблемами IT-отдела: открытием таблиц данных или автоматизацией внутренних процессов отдела. Все это, конечно, важно, но такие решения не сильно влияют на ежедневные бизнес-операции и не приближают компанию к достижению ежегодных целей в сфере продаж или производства.

Непросто определить значимые для вашего бизнеса проблемы. Руководству может быть сложно сообщить о целях компании так, чтобы IT-отдел мог легко их понять. И даже когда IT-команда понимает, какие проблемы важны для компании, у нее может не оказаться подходящих методик измерения для подтверждения их предположений. Поэтому важно иметь стандартизированный способ сообщения ключевых бизнес-целей и соответствующих показателей эффективности. Этот аспект пути к успеху вашего API мы обсудим подробнее в разделе «Измерения и ключевые этапы» на с. 193.

Устав Безоса

Запускать успешную программу с API, способную преобразить вашу компанию, непросто независимо от возраста организации. Одна из самых уважаемых корпораций, проработавших этот процесс и не прекращающих преобразования даже десять лет спустя, — это Amazon, создавшая платформу AWS.

Разработанная в начале 2000-х, эта платформа считается шедевром, виртуозно исполненным изобретательной командой специалистов в области IT и бизнеса. Хотя платформа AWS добилась огромного успеха, она возникла из-за внутренней потребности — большого недовольства тем, что IT-программы Amazon тратили слишком много времени на обработку и доставку запросов

бизнес-команды. Команда AWS работала слишком медленно, и вначале продукт не соответствовал требованиям и на техническом (объем работы), и на деловом (качество продукта) уровнях.

Как рассказывает нынешний генеральный директор AWS Энди Джесси, команда AWS (вместе с генеральным директором Amazon Джеффом Безосом и другими сотрудниками) потратила немало времени на определение того, что у них получается удачно и что нужно для разработки и создания базового набора общедоступных сервисов на межоперационной платформе¹.

На разработку плана ушло более трех лет, но в итоге он лег в основу знаменитой новой платформы, действующей по принципу «инфраструктура как услуга» (infrastructure-as-a-service, IaaS). Бизнес, который принес своим разработчикам 45 млрд долларов (прибыль в размере 13,5 млрд долларов США за февраль 2021 года), был создан только благодаря вниманию к деталям и неустанным попыткам улучшить изначальную идею. Примерно так же, как Apple изменила представление покупателей о портативных устройствах, AWS изменила представление бизнеса о серверах и другой инфраструктуре.

AWS смогла изменить мнение об API *внутри* компании во многом благодаря тому, что сейчас известно как «Устав Безоса». Стив Йегги, бывший руководитель разработки ПО в Amazon, описал этот устав в статье «Тирада о Google-платформах»². Один из важнейших пунктов этой записи из блога в том, что Безос выпустил устав, по которому все команды должны раскрывать свой функционал с помощью API и пользоваться функционалом других команд тоже через них. Иными словами, что-то делать можно только через API. Также он потребовал, чтобы все API были созданы и разработаны *так, словно они будут доступны за пределами компании*.

Мысль о том, что «API должны быть внешними», была еще одним важным требованием, повлиявшим на создание, разработку и управление API.

Итак, дизайн-мышление заключается в том, чтобы соответствовать потребностям вашей аудитории и поддерживать жизнеспособные бизнес-стратегии при принятии решения о том, какие API достойны ваших ограниченных ресурсов и внимания. Как это выглядит на практике? Как можно применить знания о продукте к процессу управления API, чтобы он соответствовал подходу «API как продукт»?

¹ Miller R. How AWS Came to Be («Как возникла AWS») // TechCrunch, 2 июля 2016 г. <https://oreil.ly/OtRyN>.

² Stevey's Google Platforms Rant («Тирада Стива о Google-платформах») // GitHub Gist, 11 октября 2011 г. <https://oreil.ly/jxohc>.

Применение дизайн-мышления к API

Вы можете поднять статус API с утилит до продуктов, применяя дизайн-мышление в процессе их создания и разработки. Некоторые компании, с представителями которых мы общались, поступают именно так.

Они решили, что их API, в том числе те, что предназначены для внутреннего пользования, заслуживают того же уровня внимания, изучения и разумного дизайна, что и другие их продукты. Для некоторых компаний это значит, что они должны обучить своих разработчиков API и других сотрудников IT-отдела принципам дизайн-мышления. Для других это означает создание внутри организации мостика между группами, занимающимися дизайном продукта и API. В нескольких компаниях, с которыми мы работали, наблюдались сразу оба этих процесса: обучение разработчиков дизайн-мышлению и усиление связи между командой продукта и разработчиками.

Полное содержание программы по дизайн-мышлению не уместится в эту книгу. Но бóльшая часть курсов предлагает комбинацию тем, уже упомянутых в этой главе, например:

- навыки дизайн-мышления;
- понимание нужд потребителя;
- разработка услуги/потока работ;
- создание прототипов и тестирование;
- факторы, которые нужно учитывать в бизнесе;
- измерение и оценка.

Если у вас в компании уже есть сотрудники, занимающиеся дизайном продукта, они могут обучить команды разработчиков тому, как начать мыслить и действовать, как они. Если же таких нет, то можно посетить курсы по дизайну продукта в местном колледже или университете. Многие из этих учреждений могут предложить адаптировать лекции для вашей компании. Даже если вы просто работник небольшой организации, интересующийся этой темой, можете найти онлайн-курсы по дизайн-мышлению сами.

Одна компания, с которой мы разговаривали (крупный потребительский банк), решила создать свой курс по внутреннему дизайн-мышлению, где специалисты по дизайну продукта будут проводить занятия с командами API в офисах компании. Впоследствии команды API могли обращаться к своим наставникам за советом по улучшению дизайна API. Их целью не было превращение всех раз-

работчиков и архитекторов ПО в профессиональных дизайнеров. Они стремились к тому, чтобы команды API лучше понимали процесс дизайна и научились применять эти навыки в работе.

Важно помнить, что результаты дизайн-мышления — это больше, чем просто улучшение удобства использования или эстетической привлекательности ваших API. Это может привести к лучшему пониманию целевой аудитории (клиентов), созданию API, соответствующих конкурентным бизнес-целям, и более надежному процессу измерения успеха API в экосистеме.

Как бы ни был важен дизайн в подходе АaaP, это лишь начало. Важно также обращать внимание на первоначальный опыт работы с клиентами, после того как API будет выпущен и доступен для использования. Об этом мы поговорим в следующем разделе.

Поддержка новых клиентов

Любой, кто покупал что-либо у Apple, знает, что распаковка их продуктов может стать незабываемым опытом. И это не случайно. Многие годы у Apple существует специальная команда, занимающаяся только улучшением впечатления от распаковки.

Адам Лашински, автор *Inside Apple* («Внутри Apple»), пишет: «Несколько месяцев дизайнер упаковки сидел в своей комнате и занимался самым прозаическим делом — открывал коробки»¹. Он продолжает:

Apple всегда стремится использовать коробки, вызывающие идеальный эмоциональный отклик при открытии... Дизайнер создавал и тестировал бесконечную серию стрелок, цветов и лент для крошечной вкладки, предназначенной, чтобы показать потребителю, в какую сторону тянуть невидимую наклейку без рамок на верхней части коробки нового iPod. Он был просто одержим поиском идеального варианта.

И это внимание к деталям выходит далеко за рамки простого открытия коробки и извлечения устройства. Apple позаботилась о том, чтобы батарея была полностью заряжена, чтобы клиенты могли «запуститься и работать» в течение нескольких секунд, а общее впечатление было приятным и плавным. Команды

¹ *Condcliffe J.* Apple's Packaging Is So Good Because It Employs a Dedicated Box Opener («Упаковка Apple настолько хороша, потому что в ней используется специальный открыватель для коробок») // Gizmodo, 25 января 2012 г. <https://oreil.ly/JrY6S>.

Apple по разработке продукта хотят, чтобы клиенты *полюбили* этот продукт с самого начала.

Стефан Томке и Барбара Фейнберг написали в «Случае из Гарвардской школы бизнеса»: «Стремление вызвать у людей любовь к нашим устройствам и их использованию всегда вдохновляло — и продолжает по сей день — разработку продуктов Apple»¹.

КОГДА API — ВАШ ЕДИНСТВЕННЫЙ ПРОДУКТ

Stripe — это успешная платежная система на основе отличного API, который высоко ценят разработчики. Оценка стартапа на 2021 год составила около 95 миллиардов долларов США при численности сотрудников менее 4000 человек². Вся бизнес-стратегия его основателей заключалась в осуществлении платежных услуг через API. Поэтому они решили с самого начала инвестировать в дизайн-мышление и подход «API как продукт». Для Stripe API — *единственный* продукт. Отношение к API как к продукту помогло им достичь как технических, так и бизнес-целей.

Такое внимание к первому опыту использования продукта применимо и к API. Кажется, почти невозможно добиться того, чтобы разработчики их *полюбили*, но исполнение этой идеи влечет за собой далеко идущие последствия. Если ваши API сложны для понимания, разработчикам будет тяжело с ними работать, и если это займет «слишком много времени», они просто рассердятся и все бросят. В мире API время до того, как все заработает, часто называют «время до первого Hello, World» (Time to first Hello, World). В сфере онлайн-приложений его иногда называют «время до “wow!”» (Time to Wow!) — TTW.

Время до «wow!»

В статье «Ускорение роста: создание “wow-момента”» Дэвид Скок, сотрудник инвестиционной компании Matrix Partners, описывает важность «wow-момента», которого надо добиться в любых отношениях с клиентом: «“Wow!” — это момент... когда ваш покупатель вдруг видит выгоду, которую получает от использования продукта, говоря себе: “Wow! Это круто!”»³. И хотя Скок обра-

¹ Thomke S. H., Feinberg B. Design Thinking and Innovation at Apple («Дизайн-мышление и инновации в Apple»), рассмотрено в мае 2012 г. <https://oreil.ly/wY4IA>.

² Lunden I. Stripe Closes \$600M Round at a \$95B Valuation («Stripe закрывает раунд на сумму 600 миллионов долларов при оценке в 95 миллиардов долларов») // TechCrunch, 14 марта 2021 г. <https://oreil.ly/60fbh>.

³ Skok D. Growth Hacking: Creating a Wow Moment («Ускорение роста: Создание wow-момента») // For Entrepreneurs (блог), 2013. <https://oreil.ly/YyVII>.

щается напрямую к создателям и разработчикам онлайн-сервисов для клиентов, теми же принципами должны руководствоваться и те, кто разрабатывает и развертывает API.

Ключевой элемент подхода TTW — это понимание не только проблемы, которую нужно решить (см. раздел «Дизайн-мышление» на с. 76), но и того, сколько ресурсов требуется для достижения вау-эффекта. Усилия, необходимые для достижения момента, когда пользователь API поймет, как их применять, и узнает, что они могут решить его важные проблемы, — вот препятствие, которое должен преодолеть каждый API для завоевания клиента. Подход Скока заключается в планировании шагов, необходимых, чтобы испытать «вау-момент», и уменьшении препятствий по пути.

Рассмотрим процесс использования API, который присылает список перспективных клиентов для главного продукта вашей компании, `WidgetA`. Типичный ход процесса выглядит примерно так.

1. Отправить запрос `login`, чтобы получить `access_token`.
2. Получить `access_token` и сохранить его.
3. Составить и отправить заявку на `product_list` с применением `access_token`.
4. В полученном списке найти пункт с `name="WidgetA"` и получить `sales_lead_url` этой записи.
5. Использовать `sales_lead_url`, чтобы отправить запрос на все контакты по продажам, где `status="hot"` (с применением `access_token`).
6. Теперь у вас есть список перспективных контактов по продукту `WidgetA`.

Здесь много этапов, но мы видели рабочие процессы с куда большим количеством шагов. И на каждом этапе пользователь может совершить ошибку (например, отправить неправильно составленный запрос). Провайдер API тоже может выдать ошибку (например, сообщение об окончании времени ожидания запроса). Здесь могут появиться три проблемы с ответом на запрос (`login`, `product_list` и `sales_leads`). TTW будет ограничено временем, которое нужно новому разработчику для выяснения принципа функционирования конкретного API и его запуска. Чем больше времени это займет, тем меньше вероятность того, что он испытает «вау-момент» или продолжит пользоваться этим API.

Есть несколько способов улучшить TTW в этом примере. Можно изменить дизайн, предложив прямой запрос для списка перспективных контактов (например, `GET /hotleads-by-product-name?name=WidgetA`). Можно потратить время и написать документацию по сценарию, демонстрирующую новым пользователям,

как решить конкретную проблему. Мы даже можем предложить песочницу для тестирования таких примеров, позволяющую пользователям пропускать работу по аутентификации, пока они изучают API.



Базовые принципы API

Дизайн, документация и тестирование — это *базовые принципы API*. Мы рассмотрим их подробнее в главе 4.

Любые меры по уменьшению времени до «вау!» улучшат мнение потребителя API о вашем продукте и увеличат вероятность того, что им будут пользоваться множество разработчиков как в вашей организации, так и за ее пределами.

Поддержка новых клиентов API

Точно так же, как Apple тратит время на «распаковку», компании, успешно применяющие АaaP, тратят время на то, чтобы «адаптация» их API была максимально простой и полезной. И точно так же, как Apple следит за тем, чтобы батарея уже была заряжена, когда вы открываете свой новый телефон, API-интерфейсы тоже могут быть «заряжены» при первом использовании, что позволит разработчикам создать что-то уже в первые минуты знакомства с новым API.

Ранее, работая над управлением API, мы говорили клиентам, что они должны провести нового пользователя от первого взгляда на стартовую страницу API до работающего примера за 30 минут. Если процесс затягивался, возникал риск потерять потенциального пользователя, а вместе с ним — все ресурсы, вложенные в разработку и развертывание этого API.

Но после того, как один из нас завершил презентацию по адаптации API, к нам подошел представитель Twilio, компании, занимающейся API SMS, и сообщил, что они стремятся провести первоначальную адаптацию за 15 минут или меньше.

Сфера деятельности Twilio (API для SMS) невероятно сложная и запутанная. Представьте, каково это — пытаться создать единый API, который работает с десятками различных SMS-шлюзов и компаний *и к тому же* прост в использовании и понимании. Задача не из легких. Один из ключей к достижению их 15-минутной цели — частое использование разных метрик в обучающих курсах для нахождения слабых мест (моментов, где пользователи API «выпадают») и определение времени, которое потребуется на выполнение задания.

МОМЕНТ НЕО ДЛЯ TWILIO

В 2011 году евангелист API из Twilio Роб Спектр опубликовал запись в блоге по поводу обучения использованию API Twilio для SMS. Он рассказал историю о том, как помогал разработчику применять этот API впервые:

«За 15 минут мы проработали руководство Twilio по исходящим звонкам для быстрого старта, и после того, как мы обошли несколько препятствий, код был исполнен и его трубка Nokia засветилась. Он посмотрел на меня, а потом на экран, ответил на звонок и услышал, как его код говорит: Hello, world.

— Вау, чувак, — ошарашенно сказал он, — я это сделал.

И это — чистая магия».

Спектр называет это «моментом Нео» (отсылка к персонажу Нео из фильмов «Матрица») и утверждает, что он может стать «мощным источником вдохновения» для разработчиков.

В Twilio неустанно работают над API и первым опытом их использования, чтобы максимально увеличить число таких моментов.

Итак, удачный первый опыт — это не просто результат успешной разработки. Он обусловлен пройденными обучающими курсами, посвященными началу работ, и тщательным отслеживанием того, как пользователи API *применяют* эти курсы. Сбор данных помогает получить информацию для улучшения этого опыта. Над первым опытом применения API нужно работать так же, как над самим API, получая для его улучшения обратную связь (лично и автоматически) от пользователей.

Но подход АaaР не ограничивается первым опытом. Допустим, вы завоевали сообщество пользователей-энтузиастов, которые остались с вами надолго. Теперь сосредоточимся на общем *опыте разработчика* ваших API. Он может включать регистрацию, заполнение форм, согласие с условиями предоставления услуг, настройку среды, получение учетных данных приложения, загрузку вспомогательных библиотек, перенаправление на нужный раздел «Начало работы», чтение документации. Все эти шаги должны быть максимально просты и понятны. Если вы работаете в компании, где нужно много времени на проверки по юридическим или нормативным требованиям, вы можете улучшить ознакомление с проектом, предоставив «песочницу», которая копирует реальную среду API, пока пользователи ожидают завершения проверки. Другой прием: спросить, какой стек использует разработчик, чтобы направить его напрямую к нужному SDK на предпочитаемом им языке. В целом старайтесь применять все советы и приемы, которые встречаются вам на пути, чтобы сделать процесс внедрения как можно более приятным.

Опыт разработчика

Обычно клиент еще долго взаимодействует с продуктом, после того как распаковал его. Да, очень важно убедиться, что извлеченный продукт работает, но также важно помнить, что покупатель (в идеале) станет использовать его и дальше. А со временем ожидания пользователей могут меняться. Им хочется чего-то новенького и надоедает то, что нравилось изначально. Они начинают исследовать возможности и даже изобретают уникальные способы применения продукта для решения новых проблем, изначально не заложенных в реализации. Эти непрерывные отношения пользователя с продуктом называются *опытом пользователя* (user experience, UX).

Компания Apple обращает особое внимание на эти отношения. Тай Тран, генеральный директор, основатель социального приложения Blue (<https://oreil.ly/3bkkZ>) и бывший сотрудник Apple, сформулировал это так: «Как только возникает вопрос, делать нам что-то или нет, мы всегда возвращаемся к другому вопросу: как это повлияет на пользовательский опыт?»¹.

Как и любая успешная компания-производитель, Apple утверждает, что клиенты превыше всего и сотрудники должны обращать особое внимание на их взаимодействие с продукцией Apple. И им несложно вносить множество изменений, если они улучшают продукт. Например, с 1992 по 1997 год Apple создала более 70 моделей компьютера Performa (некоторые из них так и не были выпущены) и при создании каждой из них использовала информацию, полученную из отзывов пользователей прошлых версий.

Но, наверное, лучшим примером управления опытом пользователя их продуктов служит подход Apple к технической поддержке — Genius Bar. По словам Вана Бейкера из компании Gartner Research, «Genius Bar — это отличительная черта магазинов Apple, и то, что он бесплатный, выделяет его среди всех прочих предложений в этой сфере»². Предлагая клиентам место, куда они могут прийти с любыми вопросами и проблемами, Apple подчеркивает важность непрерывных отношений между пользователем и продуктом.

Все эти элементы опыта пользователя — подтверждение длительных отношений, стремление вносить небольшие улучшения и обеспечение возможности быстро получить техподдержку — очень важны для создания успешных продуктов API.

¹ Lebowitz S. Apple Employees Take on Any Projects That Will Improve User Experience («Сотрудники Apple берутся за любые проекты, которые улучшат пользовательский опыт») // Business Insider, 5 июля 2018 г. <https://read.bi/2JbmgDb>.

² Forrest C. Decoding the Genius Bar: A Former Employee Shares Insider Secrets for Getting Help at the Apple Store («Расшифровка Genius Bar: Бывший сотрудник делится внутренними секретами получения помощи в Apple Store») // TechRepublic, 3 апреля 2014 г. <https://tek.io/2ykZrJl>.

Познакомьтесь с целевой аудиторией

Для создания успешного АaaР важно убедиться, что вы выбрали правильную аудиторию. Для этого нужно знать, *кто* использует ваш API и *какие* проблемы они пытаются решить (см. раздел «Дизайн-мышление» на с. 76), это важно и для длительных отношений с разработчиками. Сосредоточившись на пользователях и их проблемах, вы начинаете не только понимать, что важно для создания API, но и более творчески подходить к задачам, которые он должен выполнять, чтобы помочь вашей ЦА в решении ее проблем.

Мы уже говорили о необходимости соответствовать требованиям пользователей при разработке продукта. Не стоит об этом забывать и после выпуска API. Сбор отзывов, ознакомление с историями пользователей и пристальное внимание к тому, как используются (или не используются) API, — все это входит в непрерывный опыт разработчика, состоящий из трех важных элементов:

- обнаружение API;
- отчеты об ошибках;
- отслеживание применения API.

Эти и другие составляющие будут подробно освещены в главе 4. Сейчас мы рассмотрим лишь некоторые из аспектов, важные для общего опыта разработчика (developer experience, DX) в вашей стратегии АaaР.

Обнаружение API

То, как разработчики обнаруживают ваши API и определяют их ценность, имеет ключевое значение для начала вашего сотрудничества с этими специалистами. Как разработчики находят ваши API? Поскольку поисковых систем для API пока нет, механизм обнаружения API часто описывается, как говорил Бруно Педро, соучредитель HithHQ, как «сарафанное радио и немного удачи».

Конечно, в случае внешних API вашему общению поможет поисковая оптимизация (SEO) и участие в конференциях разработчиков, маркетинг контента и реклама в интернете, а также корпоративные мероприятия. Но обнаружение все еще слабо развито и не может быть спланировано так, чтобы всегда побеждал лучший. Вам придется развивать свою сеть влияния, и именно здесь сарафанное радио будет действительно мощным инструментом.

Когда технический директор или разработчик спрашивает на форумах, в списках рассылки или соцсетях: «Какой API лучше всего подходит для чего-либо?», в своих ответах пользователи должны упоминать ваш API. Когда разработчики найдут вас, им все равно нужно будет понять, насколько ценен ваш API. Вам нужно будет создать портал для разработчиков, который четко описывает это.

Например, Twilio рекламировали свой SMS API под лозунгом «Мы заставляем ваше приложение говорить». На своем первом сайте компания Stripe заявила: «Обработка платежей. Все сделано правильно». Как ни странно, их исходным сайтом был devpayments.com.

Для внутренних API в крупных компаниях одна из проблем программы API — то, что даже при наличии соответствующего API разработчики в итоге создают собственные, иногда многократно по всей организации. Это может восприниматься как своего рода бунт внутри компании («Они не хотят использовать наши API!»), этот взрыв дублирующих функциональных возможностей чаще всего просто свидетельствует о том, что разработчики не могут найти нужный API при необходимости. Поиск API — это, как правило, смесь сарафанного радио, опроса опытных коллег и изучение каталога или портала для разработчиков. В отрасли таких осведомленных сотрудников называют библиотеками API — они знают местоположение API в системе, кто его владелец и где находится документация.



Обнаружение API

Мы расскажем о роли этого важного элемента поддержки API в разделе «Обнаружение» на с. 138, а о роли изменений в нем в больших системах API — в разделе «Обнаружение» на с. 352.

В решении этой проблемы может помочь центральный каталог API. Создание центра поиска API или портала, где можно получить доступ к документации, примерам и другой важной информации, — еще один хороший способ улучшить обнаружение существующих API.

ПОИСК API

На момент выхода этой книги нет единой общедоступной поисковой системы для API. Одна из причин в том, что интернет-сервисы сложно проиндексировать, ведь большинство из них не имеют открытых ссылок и редко включают в себя ссылки на связанные с ними сервисы. Другая причина в том, что большая часть используемых сегодня API закрыты частными шлюзами и системами защиты доступа, что делает их невидимыми для любых общедоступных поисковых систем.

Есть проекты и форматы с открытым доступом, работающие над созданием поисковиков API: {API}Search (<https://apis.io/>) в формате описания API и сервис ALPS (семантика профиля на уровне приложений) в формате описания сервиса (<http://alps.io/>). Эти и другие проекты — отличная база для создания в будущем общедоступной поисковой системы API. Пока этого не произошло, отдельные организации могут применять эти стандарты для внутреннего пользования, чтобы начать процесс создания ландшафта API, доступного для поиска.

Как минимум одна компания из тех, с сотрудниками которых мы говорили, сделала обязательной публикацию API в центральном каталоге. Так, разработчики API не могли запустить его в производство, не добавив в каталог компании и не убедившись, что все важные API этой организации могут быть найдены в одном месте. Это большой шаг к увеличению возможности обнаружить их программы API.

Отчеты об ошибках

Ошибки происходят постоянно — это часть ландшафта API. Нельзя избавиться от всех, даже применяя удачный дизайн для уменьшения числа клиентских ошибок и используя тесты для удаления багов разработки из кода. Вместо того чтобы пытаться сотворить невозможное (удалить все ошибки), лучше внимательно следить за своими API и документировать все новые ошибки. Это позволит получить представление о том, как целевая аудитория задействует ваши API, и обогатить опыт разработчиков.



Мониторинг API

Отчеты об ошибках и отслеживание использования API (будет рассмотрено далее) относятся к базовому принципу их контроля. Мы обсудим все это в разделе «Мониторинг» на с. 136. А изменения в контроле при росте программы рассмотрим в разделе «Мониторинг» на с. 350.

При создании материальных продуктов, будь то одежда, мебель, канцтовары и т. п., сложно предвидеть, какие ошибки могут возникнуть во время их применения. Если вы не стоите рядом с человеком, пользующимся вашей продукцией, то, скорее всего, упустите детали ее эксплуатации.

Поэтому большинство компаний-производителей выпускают множество прототипов, лично контролируя все тесты. Но плюс эры электроники и виртуальных продуктов в том, что можно встроить в них систему отчетов об ошибках и получать важную информацию, даже когда продукт уже был выпущен и находится в руках пользователей.

Вы можете внедрить отчеты об ошибках в ряде ключевых точек взаимодействия с вашими API.

- *Отчеты пользователя.* Можно добавить в приложение функцию отчетов об ошибках. Она запрашивает у пользователя разрешение на отправку информации в случае появления ошибки. Так вы можете зафиксировать неожиданные условия, возникшие на стороне пользователя.

- *Отчеты шлюза.* Можно добавить отчеты об ошибках на маршрутизаторе или шлюзе API. Это позволит узнать о состоянии запроса, когда он впервые попадает к вам, и поможет найти плохо сформулированные запросы API или другие проблемы, связанные с сетью.
- *Сервисные отчеты.* Можно добавить систему отчетов в сервисы, к которым обращается ваш API. Это поможет найти ошибки в коде сервисов и неполадки на уровне компонентов (проблемы с взаимозависимостями или внутренние проблемы, связанные с изменениями экосистемы организации).

Отчеты об ошибках — отличный способ получить важные отзывы о том, как используется ваш API и где возникают проблемы. Но это только половина процесса контроля. Важно отслеживать и *успешное* применение API.

Отслеживание использования API

Этот процесс относится не только к ошибкам. Важно отслеживать все запросы и затем анализировать эту информацию для поиска шаблонов. Как мы уже говорили, API создаются и применяются во многом ради поддержки вашего бизнеса (см. раздел «Конкурентная бизнес-стратегия» на с. 78). Известный специалист по API Ким Лейн сказал: «Понимание того, как API помогут (или не помогут) вашей организации лучше взаимодействовать с целевой аудиторией, — это и есть их суть»¹.

Данные, помогающие определить, способствует ли ваш API лучшему охвату вашей целевой аудитории, обычно выражаются в OKR (objectives and key results — цели и ключевые результаты) и KPI (key performance indicators — ключевые показатели работы). Мы рассмотрим их подробнее в разделе «OKR и KPI» на с. 193, а сейчас важно отметить, что для достижения своих целей следует знать, как ведут себя API в данный момент. Для этого нужно отслеживать не только ошибки, но и успешные моменты.

Допустим, вы хотите собрать данные о том, какие приложения запрашивают те или иные API, соответствуют ли эти приложения требованиям своих пользователей и целям вашего бизнеса. У отслеживания есть дополнительное преимущество: оно помогает вам увидеть закономерности для широкого круга пользователей — те, что отдельные пользователи могут не заметить. Вы можете обнаружить, что приложения все время отправляют одни и те же запросы API:

¹ Lane K. Your API Should Reflect A Business Objective Not A Backend System («Ваш API должен отражать бизнес-цель, а не внутреннюю систему») // API Evangelist (блог), 17 апреля 2017 г. <https://oreil.ly/XCOTT>.

```
GET http://api.mycompany.org/customers/?last-order=90days
GET http://api.mycompany.org/promotions/?flyer=90daypromo
POST http://api.mycompany.org/mailing/
customerid=1&content="It has been more than 90 days since...."
POST http://api.mycompany.org/mailing/
customerid=2&content="It has been more than 90 days since...."
...
POST http://api.mycompany.org/mailing/
customerid=99&content="It has been more than 90 days since...."
```

Этот шаблон может указывать на потребность в новом, более эффективном для вашей ЦА способе рассылки для ключевых групп клиентов — один вызов из приложения, который объединит целевую группу клиентов с выбранным рекламным контентом:

```
POST http://api.mycompany.org/bulk-mailing/
customer-filter=last-order-90days&content-flyer=90daypromo
```

Этот вызов создает меньше трафика между клиентом и сервером, снижает число возможных сетевых сбоев и упрощает использование для потребителей API. И он предложен не пользователем, а вами благодаря вниманию к информации по отслеживанию применения API.

ПИТЬ СВОЕ ШАМПАНСКОЕ

В 2017 году одна европейская национальная железнодорожная компания решила организовать форумы для сообществ своих разработчиков, один для внешних, другой для внутренних.

Внешнее мероприятие координировалось руководством по коммуникациям и управлению продуктами. Они заказали IT-отделу сбор статических данных, доступных для внешнего использования, и помогли IT-командам разработать набор простых, сфокусированных на задачах API для получения сведений о расположении станций, расписании поездов и т. д. Они были реализованы быстро и рассматривались IT-отделом как «менее мощные», чем его собственные «полнофункциональные» внутренние API. Мероприятие прошло довольно успешно.

Через полгода IT-отдел организовал свой форум, задействуя официальные внутренние API. Вскоре организаторы поняли, что команды разработчиков перешли с внутренних API с полным функционалом на более простые внешние, что сделало работу API-групп эффективнее.

Здесь можно сделать несколько выводов. Во-первых, все разработчики предпочли ориентированные на задачи API. Во-вторых, создание этих «упрощенных» API не требовало много времени и ресурсов. И в-третьих, IT-отделам всегда нужно обращать внимание на то, какие API более популярны и чаще применяются. Последний можно выразить так: «Пить свое шампанское» (использовать свою продукцию). Как и в любом другом случае, в области API всегда лучше, чтобы внутренние и внешние команды задействовали один продукт.

Это подводит нас к еще одной важной области опыта разработчика (DX): как сделать для разработчиков пользование вашим API проще и безопаснее.

Проще и безопаснее

Помимо облегчения прямого обнаружения API и тщательного отслеживания ошибок и общего их использования важно предоставлять потребителям быстрый доступ к постоянным техподдержке и обучению. Ведь именно опыт, который возникает *после* успешного подключения разработчиков, использующих ваши API, обеспечит долгосрочные позитивные отношения с ними. Мы уже видели пример такого внимания, когда Apple использовали Genius Bar как источник поддержки для существующих клиентов. Вашим API тоже нужен свой Genius Bar.

Еще один важный аспект поддержки разработчиков — сделать ваш продукт *безопасным* в использовании. Другими словами, должно быть сложно применить его неправильно, нанеся тем самым какой-либо ущерб. Например, нужно затруднить удаление важных данных или единственного аккаунта администратора и т. д. Если вы обращаете внимание на то, как именно пользователи вашего API (то есть разработчики) *задействуют* его, то сможете определить зоны, где нужно усилить безопасность.

Для создания мощной и длительной связи с разработчиками, использующими ваши API, важно сочетание простоты и безопасности.

Как сделать API безопасным для использования

Есть ряд элементов API, которые могут представлять *риск* для разработчика. Иногда определенные запросы к API могут сделать что-то опасное: удалять все записи о клиентах, обнулять все цены на услуги и т. п. Иногда риск заключается даже в *подключении* к серверу API. Например, настройка команды подключения к API с данными может рассекретить логины и пароли в URL или незашифрованные сообщения. Мы сталкивались со множеством таких проблем в наших обзорах API.

Часто можно избавиться от подобных рисков с помощью *дизайна*. То есть изменить его так, что будет сложнее столкнуться с конкретным риском. Например, для API, удаляющего важные данные, можно разработать запрос *Отмена*. Так, если кто-то удалит их по ошибке, он сможет активировать этот запрос и восстановить их. Или можно требовать повышенных прав доступа для выполнения определенных операций — добавить поле данных (например, пароль), которое нужно заполнить, если запрос обновляет важную информацию.

Но иногда бывает сложно снизить риск с помощью элементов дизайна API. Некоторые вызовы API в принципе рискованны. Например, любой вызов API, удаляющий данные, сопряжен с риском, независимо от количества внесенных

изменений. Некоторые запросы к API всегда будут занимать много времени, потребляя множество ресурсов со стороны сервера. Другие API работают быстро и присылают очень много данных. Например, фильтрующий запрос может прислать сотни тысяч записей.

В тех случаях, когда вызовы API представляют неизбежный риск, вы можете уменьшить негативные последствия, добавив предупреждения в саму документацию по API. Так пользователям будет проще заранее распознать потенциальные опасности и избежать критических ошибок.

Есть много способов отформатировать документацию так, чтобы указать на возможные риски: выделить текст, сообщаящий пользователю о проблеме («Предупреждение: в зависимости от настроек фильтра этот API может прислать более миллиона записей»), или использовать ярлыки с символами. Так вам не придется добавлять в документацию много текста, вместо этого читатели просто узнают ярлык с предупреждением.

Материальные товары часто маркируют информационными и предупреждающими символами (рис. 3.1).

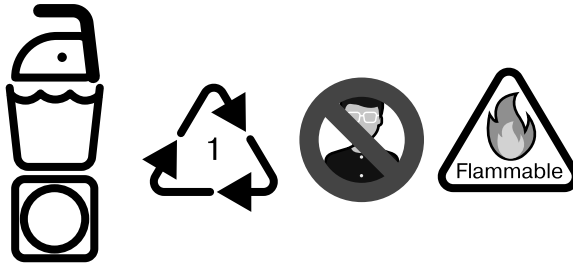


Рис. 3.1. Примеры ярлыков на товарах для дома

Можете использовать такой подход и для своих API (рис. 3.2).



Рис. 3.2. Примеры ярлыков API

Простые предупреждающие символы в сочетании с изменениями дизайна уменьшают вероятность совершения пользователями грубых ошибок. Это значительно увеличит безопасность продукта API.

Как сделать API простым в использовании

Также важно сделать ваш API простым в использовании для потребителей. Если выполнение задачи включает в себя слишком много этапов, названия и номера функций сложные или запутанные, а названия запросов к API кажутся пользователям бессмысленными, ваш API столкнется с проблемами. Разработчики не только будут недовольны его использованием, но и могут начать чаще ошибаться.

Вы можете упростить применение API с помощью дизайна. Например, задействуя шаблоны названий, подходящие к задачам, которые выполняют ваши разработчики. Это отсылает нас к пониманию целевой аудитории (см. раздел «Соответствие требованиям пользователей» на с. 77) и способам решения ее проблем (см. раздел «Конкурентная бизнес-стратегия» на с. 78).

Но даже так, если ваш API объемный (то есть содержит много URL или действий) или сложный (в нем много опций), не всегда можно положиться на дизайн. Вместо этого его нужно упростить для пользователей, задав им нужные вопросы и получив подходящие ответы. Вашему API нужно подобие Genius Bar для разработчиков.

Наверное, простейший способ предоставить им Genius Bar для API — документация. Если поместить в нее не только простую справочную информацию (название API, методы, функции, возвращаемые значения), можно поднять ее до уровня Genius. Например, можно добавить раздел «Часто задаваемые вопросы» и привести в нем ответы (и подсказки) на самые распространенные вопросы пользователей. Расширить его, добавив раздел «Как это делается?», поместив там короткие пошаговые примеры выполнения распространенных задач. Можно даже дать функционирующие примеры, которые разработчики смогут использовать в качестве стартового материала для собственных проектов.



Документация

Мы подробнее поговорим о составлении документации для API в разделе «Документация» на с. 119 и обсудим, как изменятся ваши потребности в этой сфере с ростом ландшафта API, в разделе «Документация» на с. 329.

Следующий шаг после улучшения документации — активная онлайн-форма поддержки или канал чата. Форумы поддержки позволяют разработчикам за-

давать вопросы сообществу пользователей и делиться своим решением проблем. Если сообщество API велико, эти форумы могут стать источником важных исправлений багов и запросов на новые функции. А еще важным хранилищем знаний, накопившихся за долгое время, особенно если там есть отлаженный поисковый механизм.

Еще быстрее обеспечить поддержку клиентов API как в Genius Bar можно с помощью чатов. Они обычно ведутся в реальном времени и добавляют новый уровень персонализации опыту ваших разработчиков. Там также можно использовать общие знания о вашем продукте с API и повышать их уровень.

Наконец, для крупных сообществ API и/или больших организаций имеет смысл обеспечить личную поддержку вашего продукта евангелистами API, наставниками или специалистами по устранению неполадок. Ваша компания может организовать семинары или конференции, где пользователи API будут собираться для работы над проектами или испытания новых функций.

Это работает и для внутреннего сообщества API, то есть сотрудников компании, и для внешнего — сотрудников или пользователей общедоступных API. Чем индивидуальнее будет ваш подход к разработчикам, тем большему они смогут вас научить.

Повышение безопасности API и увеличение простоты в их использовании может иметь большое значение для установления позитивных отношений с пользователями API и для повышения общего опыта разработчиков.

Почему разработчики так важны в экономике API

Как ваш продукт превращается в платформу, а затем в экосистему? Вы заставляете людей работать на ваш продукт и инвестировать в него, вместо того чтобы работать на них или вкладываться в разработку их продукта. Именно здесь API — ключевой элемент накопления ценности.

Снижая затраты на использование преимуществ от нашего решения, вы стимулируете людей и компании тратить время и средства на внедрение вашего API. Вместо того чтобы интегрироваться со всеми, все будут интегрироваться с вами, и это уникальный способ накопления ценности.

Когда Apple выпустила уже миллион приложений, это был миллион приложений, которые Apple не нужно было создавать. Они заняли тысячи рыночных ниш, которые не могли занять из-за огромного объема и неспособности Apple нанять всех необходимых менеджеров по продуктам для анализа всех потребностей

рынка. Объединяя работу и инвестиции других, вы превращаете свои продукты в платформы и экосистемы.

Читатели из Кремниевой долины, возможно, помнят рекламу Twilio на билбордах вдоль шоссе 101 и главных улиц Сан-Франциско, где на красном фоне было написано: «Спросите своего разработчика».

Twilio была одной из первых API-компаний, которая стала активно продвигать евангелизацию разработчиков, потому что они раньше других поняли, что создатели программируемой экономики скоро станут краеугольным камнем корпоративного внедрения API. Они поняли, что разработчик — ключевой фактор влияния в экономике приложений.

Из-за навыков разработки приложений разработчики — центральные участники разработки стратегии API. Каждая интеграция и каждое приложение будут проходить через них. В рыночной стратегии они будут первыми, кто будет использовать ваш API и кто будет создавать приложения на вашей платформе.

Разработчики укажут путь другим разработчикам и помогут вам воспользоваться преимуществами развития рынка. Внутри крупных корпораций они будут вашими внутренними чемпионами по API. Они порекомендуют вам использовать один API вместо другого, потому что знают его лучше и потому что он безопаснее в использовании, лучше разработан и/или документирован по сравнению с другим. В этом контексте мы переходим от моделей «бизнес для потребителя» (B2C) и «бизнес для бизнеса» (B2B) к модели «бизнес для разработчика» (B2D).

Другими словами, в XXI веке API — это новые товары: продукты, которые создаются и хранятся на серверах, распространяются по информационным каналам через сеть, переносятся разработчиками в приложения и продвигаются в цифровых супермаркетах (они же магазины приложений), куда конечные пользователи будут приходить, чтобы загрузить (приложения) или использовать API.

Чтобы АaaS могли создавать программируемые бизнес-модели и накапливали ценность в масштабах интеграции, нужно выделить команду для поддержки этого роста и всегда прислушиваться к потребностям разработчиков, предоставляя им лучший возможный опыт как с технической, так и с человеческой стороны ИТ. В этом и состоит общая роль отношений с разработчиками для API.

В результате некоторые представители отрасли утверждают, что в программируемой экономике, где каждая компания будет предоставлять свои ключевые компетенции другим и использовать их компетенции через API, роль отношений с разработчиками будет становиться все более важной. Может дойти до того, что всем компаниям понадобится отдел по связям с разработчиками, как сейчас в компаниях есть отдел маркетинга.

Отношения с разработчиками API

Как пишет Меджуи в третьем издании книги *Developer Marketing and Relations: The Essential Guide* («Маркетинг и отношения с разработчиками: практическое руководство»), есть четкая и важная взаимосвязь между стратегиями API и вовлечением разработчиков. Она заключается в понимании связи сообщества, кода и контента; понимании разницы между AaaS и продуктами API; принятии правильных показателей для измерения вовлеченности разработчиков. Важно также уделить некоторое время обсуждению внутренних или внешних стратегий монетизации API. Все эти вопросы мы рассмотрим в текущем разделе.

Сообщество, код, контент

Отношения с разработчиками при обсуждении API должны строиться вокруг трех блоков, которые группа по связям с разработчиками SendGrid называет «три К» (three Cs): community, code, content (сообщество, код и контент).

Эти отношения в первую очередь связаны с сообществом. Пока люди будут внедрять API (по крайней мере, пока машины не будут делать это за нас), концепция сообщества будет оставаться важной частью отношений с разработчиками. Быть рядом с разработчиками, взаимодействовать с ними, прислушиваться к их отзывам, вдохновлять их и привлекать их внимание к API — все это часть миссии сообщества в отношениях с разработчиками.

Аспект сообщества важен как «мягкая сила», позволяющая распространять больше информации из уст в уста. Как говорил Тим Фоллс из SendGrid, «личная связь стоит больше, чем клик». Иногда он обнаруживал, что разработчики рекомендуют использовать SendGrid, даже если они никогда им не пользовались, потому что они знали, что команда SendGrid заботится о них.

Сообщество также предполагает посещение мероприятий для разработчиков или конференций по API для поддержания контактов и участие в выступлениях, которые не связаны напрямую с тем, что делает ваш API. Иногда темы могут касаться крутой хакерской атаки, которая стала возможна благодаря вашему API, пакету с открытым исходным кодом, выпущенному для сообщества, а иногда и другим более социальным темам.

Второй блок — это код. Внедрение API связано с кодом, а задача разработчика — создавать полезный код. Если им предоставить готовый код, они смогут быстрее сосредоточиться на реализации бизнес-логики. Затем роль группы по связям с разработчиками состоит в том, чтобы предоставить это в виде образцов кода, SDK, прототипов приложений или определений API (спецификаций), которые разработчики смогут использовать напрямую.

Для членов группы по связям с разработчиками это означает написание кода для поддержки платформы разработчика и API с положительным опытом разработки, о котором мы поговорим в следующем разделе.

Третий блок — это контент. Разработчики любят открытое, честное общение и полезный контент. Это один из лучших способов привлечь их и сохранить в качестве лояльной аудитории вашего блога и экосистемы.

Есть много разных форм контента. Это может быть просто техническое обновление после недавних изменений, блог или электронное письмо с примером положительного опыта использования клиентом, пост специалиста о конкретном способе создания некоторых функций или детально описанный передовой опыт.

Он может быть и шире, как, например, недавний буклет Stripe и серия постов в блоге о том, как сделать компанию и ее приложения углеродно-нейтральными.

Контент — важная часть ваших отношений с разработчиками. Он делает вашу компанию и ее API доступными для поиска с помощью SEO или обмена информацией в соцсетях.

Сообщество, код и контент — это три столпа отношений с разработчиками, к выполнению которых вы должны стремиться.

AaaP в сравнении с API продукта

Говоря о взаимоотношениях разработчиков и API, нужно провести четкую границу. Важно определить, является ли ваш API продуктом или он обеспечивает и поддерживает продукт. Затем вы можете классифицировать его как AaaP или продукт API.

Например, Stripe, Twilio, Mailjet и Avalara относятся к категории AaaP. Они предлагают автономные возможности для конкретных целей — платежи, SMS, электронная почта и проверка налогов.

С другой стороны, интерфейсы API Salesforce, API Facebook, API eBay, API YouTube и API Twitter — это API для продукта. Они предназначены для поддержки и настройки существующей платформы и часто составляют более 50 % от общего трафика платформы и продукта. Они очень важны для бизнеса, но их часто можно использовать бесплатно, потому что их применение увеличивает стоимость основного бизнеса.

Роль отношений с разработчиками различна для AaaP и API продукта.

Конечная цель отношений с разработчиками для AaaP — евангелизация, пропаганда и построение отношений, способствующих увеличению объема бизнеса

с использованием API. Поскольку API — это продукт, который нужно внедрять и продавать, целью будет максимальное увеличение числа ценных интеграций в соответствии с бизнес-моделью.

Если же разработчики не принимают, а только предлагают решения, целью отношений будет обучить их и рассказать о преимуществах API. Они смогут предлагать их в своей организации на уровне предприятия, что приведет к корпоративным интеграциям и высоким доходам.

С другой стороны, отношения с разработчиками для продуктовых API в основном направлены на то, чтобы вдохновлять их на создание приложений, которые будут увеличивать ценность платформы, но не обязательно ее прямые доходы. Когда Facebook открыл API своей платформы, разработчики могли бесплатно создавать приложения или игры, и богатый портфель приложений, который появился в результате, показал, что платформа Facebook никуда не денется, постоянно собирая все больше приложений от разработчиков.

В итоге пользователи останутся не только из-за социальной сети, но и из-за полной экосистемы приложений, окружающей и поддерживающей ее. Точно так же обстоит дело и с приложением Salesforce AppExchange, где по данным на 2021 год насчитывается более 5000 бизнес-приложений.

Здесь Salesforce — это уже не просто ПО для управления взаимоотношениями с клиентами (CRM), а полноценная экосистема бизнес-приложений, работающих на базе CRM и вокруг нее, которая подходит для многих отраслей. Для продуктовых API роль отношений с разработчиками заключается в развитии этой экосистемы, что впоследствии увеличивает ценность и продажи продукта среди конечных пользователей.

Twitter API против Slack API

Согласование ключевых показателей эффективности (KPI) с API очень важно и может полностью изменить будущее вашей платформы. Как отметил Джейсон Коста из GGv Capital в своей статье «Рассказ о двух платформах API», и Twitter, и Slack были популярны среди разработчиков благодаря пользовательской базе и их открытости для создания ценных приложений¹. В итоге Twitter решил, что его бизнес-модель основана не на том, чтобы быть монетизированной экосистемой приложений, а должна быть медиаплатформой, получающей доход за счет рекламы.

¹ Costa J. A Tale of 2 API Platforms («Рассказ о двух платформах API») // Medium, 25 октября 2016 г. <https://oreil.ly/ZzAlj>.

В результате этой идентификации все прошлые опубликованные API внезапно стали отображать противоположный образ мышления платформы. Это стало причиной того, что Twitter ограничил доступ к своим API для третьих лиц, поспособствовав нанесению ущерба экосистеме разработчиков. Спустя годы компания приложила немало усилий для восстановления отношений с разработчиками, опубликовав манифест самого Джека Дорси и наняв таких замечательных сторонников интересов разработчиков, как Ромен Хуэт. Но доверие к использованию API со стороны разработчиков так и не было полностью восстановлено.

Модель Slack, напротив, была основана на создании экосистемы приложений для повышения ценности основного их продукта. Увеличение числа бизнес-приложений повысило ценность коммуникационной платформы Slack, поэтому ключевые показатели эффективности бизнеса были согласованы с API. По сей день API Slack никогда не страдал от напряженности в сообществе разработчиков, что отчасти объясняет, почему разработчики любят создавать ботов на этой платформе.

Эти две истории идеально отражают то, как согласование ваших ключевых показателей эффективности с API и ваших API с вашими ключевыми показателями эффективности влияет на то, как вы управляете своей стратегией взаимодействия с разработчиками и API в долгосрочной перспективе.

Шпаргалка по ROI от DevRel: отслеживание успеха в отношениях с разработчиками

Оценка качества и потенциала вашего сообщества разработчиков — ключевой элемент вашей стратегии взаимоотношений с ними. Многие компании пытаются разработать внутренний инструмент для лучшего понимания своего сообщества разработчиков. Некоторые поставщики систем управления API создали так называемое внутреннее *ПО для управления взаимоотношениями с разработчиками*. Напоминает CRM-решение, но для разработчиков.

Это позволяет лучше общаться с разработчиками, отслеживать и выделять тех, чей потенциал ROI (рентабельность инвестиций) выше, основываясь на целях вашей стратегии API, которые могут включать охват, экосистему приложений или доходы. Кроме того, выявление и привлечение разработчиков, которые находятся на вашей платформе, — это способ активизировать их и вдохновить на создание новых проектов с помощью вашего API.

Для этого вам понадобится то, что Майк Свифт, основатель и генеральный директор Major League Hacking, называет «гайками и болтами» отношений

с разработчиками. Это сочетание практических методов взаимодействия с разработчиками и метрик для эффективного инвестирования и отслеживания. Они состоят из двух частей: отслеживание использования API и отслеживание разработчиков.

«Если вы не можете что-то измерить, вы не можете это улучшить», — говорил адмирал лорд Нельсон. С другой стороны, как гласит закон Гудхарта: «Когда мера становится целью, она перестает быть хорошей мерой». Как найти баланс между метриками и целями вашей стратегии взаимодействия с разработчиками? Вы просто должны сопоставить свои API с KPI.

Ключевых показателей эффективности для API огромное множество, и мы познакомили вас с некоторыми, чтобы помочь вам начать работать с ними. Для получения максимальной отдачи от каждого из них вам следует сочетать их с «пиратской воронкой», созданной Дейвом МакКлюром, основателем известного ускорителя стартапов 500 Startups, более известной как модель AARRR: awareness, acquisition, activation, retention, revenue, referrals (осведомленность, приобретение, активация, удержание, доход и рефералы)¹.

Осведомленность об API. Это показатель того, как люди узнают о вашем API — как они открывают для себя ваш продукт.

- *Количество посещений домашней страницы портала для разработчиков и документации API.* Есть много способов, как платных, так и органически входящих в систему, привлечь разработчиков на вашу домашнюю страницу. Чтобы разработчики захотели использовать ваш API, сначала они должны узнать о вашем ценностном предложении и предлагаемых вами возможностях. Привлечение максимального количества разработчиков — ваша основная метрика осведомленности.
- *Число просмотров и прочтений статей в блоге.* Контент очень важен при построении стратегии взаимодействия с разработчиками. Поэтому все, что вы публикуете, должно отслеживаться. Всегда размещайте ссылку на страницу вашего портала для разработчиков, которая может отслеживать переходы из ваших статей и контролировать вашу аналитику вовлеченности.
- *Количество разработчиков, зарегистрированных на каналах связи.* Попросите читателей подписаться на вашу новостную рассылку, чтобы получать уведомления о свежих статьях и обновлениях API. Это число — ключевой элемент для отслеживания того, сколько участников сообщества хотят продолжать

¹ AARRR Pirate Metrics Framework («Пиратская система метрик AARRR») // ProductPlan, <https://oreil.ly/GiDgb>.

получать от вас новости, и для сравнения их с текущими разработчиками, зарегистрированными в API.

- *Число публичных выступлений.* Осведомленность приходит из офлайн-дискуссий и событий в реальной жизни (IRL). Конференции, встречи и все публичные или частные мероприятия, на которых вы можете повысить осведомленность о своем API, важны для активации ключевого элемента, который нельзя измерить, но который хорошо работает: сарафанное радио. Оно служит основой для запуска фазы вирусного привлечения, о которой мы поговорим позже. Для этого вы можете отслеживать число выступлений, средний размер аудитории и т. д., чтобы рассчитать охват. Если у вас есть стенд на мероприятии, вы можете добавить количество людей, с которыми вы общались.
- *Звезды с открытым исходным кодом и вклады в инструментарий API.* Предоставление полезных инструментов или выпуск ценного ПО под лицензией с открытым исходным кодом могут обеспечить большую известность вашей компании и API. Недавно это принесло успех разработчикам Strapi, которая выпустила инструмент для создания CMS с API на базе GraphQL, и Hugging Face API, которая выпустила свою технологию обработки естественного языка с открытым исходным кодом. Благодаря открытому исходному коду эти компании привлекли разработчиков, расширили свой бизнес и получили много денег от инвесторов — 10 и 15 млн долларов США соответственно. В основе их успеха лежали отношения с разработчиками в управлении сообществом разработчиков вокруг проекта компании с открытым исходным кодом.

Приобретение API. Это метрика, которая показывает нам, как разработчики участвуют в процессе внедрения нашего API.

- *Количество зарегистрированных разработчиков.* Это важный показатель в данной нише, но полезен он только на начальном этапе! Он теряет свою эффективность по мере развития зрелости вашей программы взаимоотношений с разработчиками. Эта метрика позволяет узнать, есть ли соответствие между сообществом разработчиков и воспринимаемой ценностью вашего API и связанных с ним возможностей.
- *Число приложений разработчика.* В большинстве случаев одна учетная запись разработчика связана с одним приложением. Но когда вы набираете популярность или когда ваш API обладает внутренней ценностью, которую можно легко использовать в других приложениях (то есть это API транзакций или бизнес-процесс-как-услуга API), вы увидите два или более приложений в одной учетной записи разработчика. Этот показатель важно отслеживать, ведь такие разработчики станут вашими лучшими посланниками «из уст в уста».

Они понимают ценность вашего продукта настолько, что использовали его многократно. Отслеживание общего числа приложений и среднего значения выборки разработчиков двух и более приложений может стать хорошим показателем на этапе приобретения.

- *Общее количество вызовов API.* Поначалу общее количество вызовов API может быть хорошим показателем, по которому ваша команда по связям с разработчиками может сосредоточиться на расширении использования API и внедрении инноваций в свою маркетинговую стратегию. Они могут сделать упор на том, чтобы вдохновить разработчиков на разные варианты использования в соответствии с распространенными сценариями. Этот показатель важно отслеживать, так как он быстро перестает быть хорошей метрикой, если только ваша стратегия и/или бизнес-модель не привязана к числу вызовов API, например присоединение, оплата по мере использования или косвенные модели, вроде рекламы сторонних страниц.
- *Число сторонних интеграций на другие платформы.* Еще один способ увеличить охват и укрепить отношения с разработчиками — сотрудничать с существующими компаниями, у которых уже есть сообщества разработчиков, создавая подключаемый модуль, надстройку или интеграцию на их рынке для масштабирования. Например, Typeform, платформа для создания анкет с помощью API. Она была основана на внедрении сценария использования в сторонние торговые площадки для доступа к их существующим сообществам разработчиков. Теперь, когда она выросла, Typeform может привлекать приложения на свою платформу и изменять схему интеграции API с «Я трачу время и деньги на интеграцию с вами» на «Вы тратите время и деньги на интеграцию со мной».

Активация API. Показатели активации API помогают нам понять уровень вовлеченности, создаваемый нашими API, особенно на ранних этапах жизненного цикла внедрения.

- *Время до первого «Hello, World» (Time to first Hello World, TTFHW).* Важный показатель конверсии — то, как превратить заинтересованного разработчика в активного. Для этого нужно отслеживать TTFHW — время между регистрацией разработчика на вашей платформе и успешным обращением к вашему API. Согласно рекомендациям Twilio для разработчиков, не более 15 минут — идеальное время DX (опыта разработчика) для успешного использования вашего API. Конечно, для достижения такого временного показателя не все внутренние проверки и процессы возможны в каждой организации. Но его сокращение сильно повлияет на ваш коэффициент активации разработчиков.

- *Количество активных приложений/разработчиков.* Даже если вы уже отслеживаете количество приложений и разработчиков, определение разницы между опытными разработчиками и теми, кто просто использует ваш API для небольших проектов, может помочь вам определить, куда вложить больше ресурсов или когда вам нужно оперативнее реагировать на запрос в службу поддержки. Границу между этими двумя понятиями должен определить менеджер продукта API. Ее важно отслеживать, чтобы понимать разницу. Эта разница также поможет вам определить ваши тарифные планы и установить справедливые ограничения на бесплатные, когда разработчики достаточно расширили свои приложения, чтобы стать «активированными» в качестве клиента.

Удержание API. Показатели удержания API говорят нам о том, как мы поддерживаем активные отношения с разработчиками, с которыми уже начали сотрудничать.

- *Количество «ценных» приложений.* Разница между активированными и ценными приложениями должна быть определена менеджером продукта API в соответствии со стратегией. Ценным приложением может быть то, которое обеспечивает большую видимость вашей экосистемы приложений, привлекает множество пользователей или приносит значительный и растущий доход.
- *Число активных токенов конечных пользователей.* Более конкретная метрика на этапе привлечения — отслеживание срока хранения токенов конечных пользователей в качестве пользователей ваших приложений — потребителей API. Приложения, которые имеют тенденцию к расширению своей пользовательской базы, менее склонны менять свой стек и поставщиков API, чтобы сосредоточиться на клиентах. Именно поэтому компании вроде Stripe по-прежнему могут взимать высокую плату за свои API, ведь платежные возможности — это последнее, что вы хотите изменить, когда растете. Этот показатель может быть действительно полезным, если вы нацелены на экосистему приложений в своей стратегии, как это делают API Facebook и Slack.

Доход от API. Эта метрика позволяет нам отслеживать фактический доход, полученный от деятельности разработчиков в наших API.

- *Прямые доходы от API.* Этот вид метрики прост, если ваша бизнес-модель напрямую связана с платежами. Отслеживание доходов может помочь повлиять на внутренних лиц, принимающих решения в организации, и высшее руководство, указав им на необходимость продолжения инвестиций в отношения с разработчиками для монетизации API.

- *Косвенные доходы от API.* Эту метрику сложнее определить. Она требует субъективного подхода, но попытка связать косвенные метрики API с основными показателями эффективности бизнеса будет стимулировать внутреннюю поддержку отношений с разработчиками. Отношения с разработчиками окупаются в среднесрочной и долгосрочной перспективах, и некоторые менеджеры могут захотеть продемонстрировать руководству более быстрые результаты внутри компании. Предоставление им информации о ценности отношений с разработчиками через преобразование метрик API в базовые показатели эффективности бизнеса поможет команде по связям с разработчиками продолжать получать поддержку. Например, если ваш API способствует росту экосистемы приложений и она увеличивает оценку компании на 100 % для инвесторов и рынка, ценность отношений с разработчиками должна быть привязана к рыночной стоимости капитала компании.

Рефералы API. Идея здесь в том, чтобы сделать существующих довольных пользователей API представителями, помогающими повысить интерес к использованию ваших API в своих сетях. Вот набор метрик для анализа этого показателя.

- *Разговорная активность.* Активность в диалогах важно отслеживать, потому что ваша команда по связям с разработчиками может привлечь разработчиков и менеджеров по продуктам, которые на самом деле обсуждают тему «Какой API лучше всего подходит для этого?» или где найти возможности и рабочие процессы бизнеса, которые были воплощены через API. Эти дискуссии могут происходить там, где находятся разработчики, например, на Discourse, Twitter, Medium, Hacker News, Reddit (<https://oreil.ly/DwVx3>) и публичных форумах Slack.
- *Упоминания от других лиц.* Вы можете найти спикеров и разработчиков, которые ссылаются на ценность вашего API в своих выступлениях или статьях, и превратить их в ваших представителей. Для этого нужно отслеживать эти упоминания на конференциях или в блогах разработчиков и начать их привлекать. Именно так поступили такие компании, как Auth0 со своей программой амбассадоров или Docker с программой Docker Captain, в рамках которой были выявлены лучшие сторонники сообщества.
- *Использование API в успешных хакерских атаках и их присутствие на хакатонах.* Вы можете отслеживать это только в соцсетях или искать упоминания и оповещения поисковых систем. Но знание того, кем ваш API используется, в каких целях и где это происходит — важная часть вашей стратегии взаимодействия с разработчиками. Ваша цель — быть уверенным, что они обратятся к вам в следующий раз, прежде чем начнут создавать свой инструмент.

ФИНАНСИРОВАНИЕ ПОЛЬЗОВАТЕЛЕЙ API С ПОМОЩЬЮ КАПИТАЛА

В настоящее время есть оригинальная стратегия создания инвестиционного фонда для потребителей и разработчиков API. Ее использовали такие крупные компании, как Mailchimp, Twilio, SendGrid, Slack и Stripe. В какой-то момент все они создали инвестиционные фонды специально для компаний-разработчиков, использующих их API. С помощью такого фонда они могут напрямую владеть долей в своих компаниях — потребителях API и согласовывать свои интересы с экосистемой собственных приложений.

У такого подхода много преимуществ, но главное — это возможность для разработчиков получать прибыль от создания приложений на вашей платформе. Даже если инвестиций немного, это позволит вам сохранить лояльность разработчиков к вашей платформе, а не к вашим конкурентам, показав путь к монетизации и/или финансированию.

В рамках другой венчурной стратегии компания Salesforce поощряла разработчиков создавать приложения на Salesforce AppExchange вместо iOS, так как средний доход от приложения на AppExchange тогда составлял 450 000 долларов США вместо 3000 для приложения на iOS (2015 год). Даже французский банк Credit Agricole предложил платить разработчикам в зависимости от популярности их приложения, оставив ежемесячный доход, основанный на активных пользователях их приложений, использующих их общедоступные API.

Монетизация и ценообразование API как продукта

Многие компании хотят монетизировать API, получать доход и демонстрировать ценность для клиентов и экосистемы. Часто трудно добиться максимального сохранения ценности, расширив при этом влияние в экосистеме. Мы поможем вам определить все переменные в стратегии монетизации и ценообразования AaaS.

Ценообразование инфраструктуры и SaaS для API

API — это доступ к вашим возможностям в виде сервиса. Но вам придется позиционировать себя в соответствии с тем, как вы хотите, чтобы клиенты полагались на вас, и как вы хотите предоставлять эти API. В отрасли есть два основных подхода: инфраструктурный и SaaS.

Первый подход часто устанавливает одинаковые цены на одни и те же услуги, без гейткипера, как мы видим в случае с Amazon Web Services и другими поставщиками облачных услуг. Ценообразование всегда публично, соотносится с использованием и не коррелирует с ценностью, созданной пользователем. Независимо от того, можете ли вы заработать 1 доллар или 1 миллион долларов с помощью Amazon Bucket, цена будет одинаковой. Масштаб, в котором разворачивают свою деятельность разработчики AWS, не позволяет разграничивать клиентов, поэтому цены открыты, прозрачны и соответствуют уровням использования.

В рамках подхода SaaS вы можете попытаться разделить клиентов API по уровням в соответствии с ценностью, которую ожидается получить, чтобы извлечь максимальную выгоду, когда это возможно. Например, когда пользователь переходит от нескольких тысяч вызовов API в день к десяткам тысяч, может показаться, что теперь он включен в производственную среду (и имеет свой жизнеспособный бизнес с собственной клиентской базой), поэтому может платить гораздо больше, чем в начале своего пути. Или когда от вас требуют соглашение об уровне обслуживания (service level agreement, SLA), становится ясно, что для них это «денежное» время, и вы можете заставить их платить гораздо больше за тот же доступ к вашим возможностям (со штрафами, если вы не поддерживаете производительность обслуживания и влияете на цепочку создания стоимости их бизнеса).

Некоторые компании даже используют управление API для определения порога выбора ценовых уровней для потребителей API. Они смотрят на среднее количество вызовов API для производственных пользователей и устанавливают корпоративный тарифный план в соответствии с ним. Это позволяет соотнести цену с переходом от тестирования к производству и обеспечить максимальную потенциальную ценность API для потребителей.

Вам придется найти компромисс между рентабельностью и привлечением пользователей. Если ваши клиенты обладают эффектом маховика, повышающим ценность экосистемы, вы можете упростить модель доходов, чтобы максимизировать внедрение. Например, бизнес-модель Facebook основана на рекламе, поэтому API Facebook должен максимально использовать сторонние приложения, побуждающие пользователей проводить больше времени на этой платформе. API бесплатный (до 100 миллионов запросов в день).

Мы собрали список параметров ценообразования API, которые нужно учитывать при назначении цен на них.

- *Свежесть: старое по сравнению с новым.* Если API предоставляет доступ к ресурсам, которые со временем устаревают (например, данные о компании), вы можете установить многоуровневое ценообразование в зависимости от актуальности данных. Некоторые финансовые API дают бесплатный доступ к однодневным данным, но за доступ к свежим нужно платить.
- *Точность: размытость по сравнению с четкостью.* Если API предоставляет доступ к ресурсам с разными уровнями ценности на разных степенях точности, вы можете применить многоуровневое ценообразование в зависимости от степени точности. API для прогнозирования погоды может установить низкую цену для однодневного прогноза, а доступ к прогнозам на три-пять дней предоставить по более высокой цене. API кредитного рейтинга может предоставлять точный кредитный рейтинг за более высокую плату, но

обобщенное (размытое) представление о нем (например, с применением светофорной шкалы «красный, желтый, зеленый») — за меньшую плату.

- *Потребляемость: транзакционная по сравнению с процессной.* Предоставляете ли вы детализированные API, которые клиенты внедряют один за другим, производя оплату по отдельности, недорого или вы собираете сложные бизнес-процессы и заключаете их в один API, продавая их по высокой цене? Например, API Checkr проверяет биографические данные за один вызов API, что помогает компаниям вроде Netflix ответить на один вопрос: можем ли мы нанять этого человека? API Checkr собирает много вызовов API из разных госслужб и юридических источников, выдавая более ценный результат для потребителей, чем если бы они сами создавали и собирали все вызовы API.
- *Сфера применения: сокращенная по сравнению с общей.* API могут предоставлять доступ ко всем вашим внутренним ресурсам или к меньшему набору функций. Многоуровневое ценообразование API может быть основано на способе определения объема доступа к ним: за год, за географический регион, за тип данных, — решать вам, если вы хорошо знаете своих клиентов и понимаете, что они действительно ценят в вашем предложении.
- *Количество: малое по сравнению с большим.* Еще один способ разграничения тарифных планов API — это определение количества данных или допустимого количества запросов. Чем больше вызовов API вы хотите выполнить, тем больше вы платите. Некоторые компании сильно зависят от данных, когда имеют дело с важными клиентами. Поэтому зная, сколько данных им нужно, можно установить уровень ценообразования на уровне объема.
- *Производительность: быстрая по сравнению с медленной.* Соглашения об уровне обслуживания (SLA) — важная часть ценности предоставления API. Гарантия быстрого и надежного доступа по сравнению с его отсутствием может стать сильным фактором, определяющим различия для тарифных планов API.
- *Сопровождение: управляемое по сравнению с делегированным.* Интерфейсы API важно поддерживать во всех версиях. Многие компании версионизируют API, чтобы со временем они могли развиваться. Потребителям API нужно поддерживать свои приложения и обновлять их. Заставив компании платить за поддержку старых версий, вы можете установить другой уровень платы за обслуживание API.
- *Поддержка: полная по сравнению с ограниченной.* Поддержка клиентов тоже может быть фактором, определяющим уровень тарифных планов API. Некоторые клиенты готовы платить за стабильность круглосуточной, мультирегиональной и гарантированной поддержки в течение часа или менее при возникновении технических проблем. Такие услуги можно

предоставить по повышенной цене. При пониженной стоимости или бесплатных планах поддержка может быть предложена путем перенаправления клиентов на общедоступные форумы или связь со специалистом только по электронной почте.

- *Лицензия: все права защищены по сравнению с открытой.* Ваш API может предоставлять доступ к ресурсам, недоступным для всех видов использования. Вы можете ограничить потенциальное использование API для низкооплачиваемых клиентов и открыть расширенный доступ для более высокооплачиваемых.
- *Брендинг: реализация под брендом продавца по сравнению с «основано на технологии».* Некоторые поставщики API предпочитают признание и осведомленность среди сообществ разработчиков, а не меньшие суммы дохода, поскольку они стремятся сохранить ориентацию на крупные предприятия. Одно из решений — предоставление доступа к API бесплатно или по очень низкой цене с требованиями аккредитации для упоминания типа «Услуга предоставлена...» в своих продуктах и приложениях. За повышенную цену клиенты могут снять с себя это обязательство по маркировке использования. Некоторые компании с балльной системой оценивания платежеспособности обязывают вас упоминать в любых онлайн- или мобильных публикациях, откуда получена оценка, и запрещают вам создавать новую, включающую их алгоритм подсчета баллов в качестве переменной, если только вы не приобретаете премиум-план на реализацию под брендом продавца.

Конечно, есть и другие переменные, которые можно учитывать для определения стратегий ценообразования API, но это наиболее распространенные.

Важно знать, что API и экономика «как услуга» предпочитают внедрять простые ценовые и бизнес-модели. Сложные модели, ориентированные на максимальную ценность, меньше направлены на самообслуживание и требуют большей поддержки продаж для привлечения клиентов, чем единое, открытое и прозрачное ценообразование, облегчающее процесс самообслуживания и дающее более точную оценку итоговой цены, даже если средняя отдача на одного клиента окажется меньше.

Итоги главы

В этой главе мы познакомили вас с подходом AaaS и обсудили, как его задействовать для улучшения дизайна, развертывания API и управления ими. Для применения этого подхода нужно знакомиться с целевой аудиторией, понимать и решать ее проблемы, обрабатывать отзывы от пользователей API.

Мы рассмотрели три основных понятия для AaaP:

- дизайн-мышление, которое необходимо, чтобы убедиться, что вы понимаете проблемы своей целевой аудитории;
- поддержку новых клиентов с целью показать им, как успешно пользоваться вашим продуктом;
- опыт разработчика, используемый для управления жизненным циклом вашего продукта и выработки идей для будущих улучшений.

Попутно мы узнали, как приверженность принципам AaaP помогла таким компаниям, как Apple, Amazon, Twilio и другим, не только создавать успешные продукты, но и привлечь преданных клиентов. Независимо от цели вашей программы API сообщество преданных клиентов очень важно для обеспечения ее работоспособности и успеха API в дальнейшем.

Вы получили представление о базовых принципах подхода AaaP, и теперь мы можем перейти к набору навыков, необходимых для взращивания любой успешной API-программы. Мы называем их базовыми принципами API, о них и пойдет речь в следующей главе.

Основы API-продукта

Когда что-то выполнено хорошо, кажется, что это было легко. Никто не понимает, насколько это было сложно. Думая о фильме, большинство представляет двухчасовую законченную идеальную версию. Но ради нее сотни и тысячи людей работали целыми днями многие месяцы.

Джордж Кеннеди

В прошлой главе мы рассмотрели подход к API как к продукту. Теперь давайте взглянем на работу, которую нужно будет проделать, чтобы создать и поддерживать свой продукт. Для разработки качественного API приходится много и тяжело работать.

Из главы 1 вы узнали, что у API есть три составляющие: *интерфейс*, *реализация* и *экземпляр*. Для хорошего API придется затратить время и силы на каждый из этих трех аспектов. Кроме того, нужно следить за своевременным обновлением вашего продукта, когда тот начнет развиваться. Чтобы помочь вам разобраться в этих сложностях, мы разделили всю работу на десять частей — *столпов*.

Мы называем их так, потому что они поддерживают ваш продукт. Если эти опоры не укреплять, ваш продукт обречен на провал. Но это не значит, что для успешного API в них надо вкладываться одинаково. В этой главе мы определили десять столпов. Плюс наличия у вас всех десяти в том, что они не должны поддерживать одинаковый вес. Какие-то из них могут быть крепче, а в некоторые вообще почти не стоит вкладываться. Важно, что общая сила этих столпов поддерживает ваш API, даже когда он начнет развиваться.

Знакомство со столпами

У каждого из столпов API свое поле деятельности. Другими словами, каждый очерчивает ряд *решений*, связанных с API. На самом деле работу по созданию API нельзя четко разделить, и некоторые из столпов будут пересекаться. Но это нормально. Наша цель не в том, чтобы установить неопровержимую истину о работе API, а напротив, разработать полезную модель для исследования и обсуждения работы по созданию продуктов с ними. Мы будем надстраивать эти основные понятия в течение всей книги, рассматривая более сложные модели организации команды, развития продукта и аспектов системы в следующих главах.

Вот десять определенных нами столпов работы с API:

- стратегия;
- дизайн;
- документация;
- разработка;
- тестирование;
- развертывание;
- безопасность;
- мониторинг;
- обнаружение;
- управление изменениями.

В этой главе мы познакомим вас с каждым из этих столпов, исследуем их особенности и важность для успеха API. Мы также опишем область принятия решений для каждого из них и дадим общие указания для их укрепления. Мы не будем давать вам каких-либо конкретных указаний о том, как реализовать какой-либо из столпов этой главы. Полное обсуждение каждой из этих областей работы могло бы занять отдельную книгу, а нам нужно затронуть еще много других тем. Но назовем некоторые из важнейших решений в каждой сфере с точки зрения руководства. Начнем с первого столпа — стратегии.

Стратегия

Выдающиеся продукты начинаются с разработки выдающейся стратегии, и продукты с API — не исключение. Столп «Стратегия» содержит две основные области решений: причина создания этого API (цель) и то, как он поможет вам достичь этой цели (тактика). Важно понимать, что стратегическая цель для API не может существовать в вакууме. Любая цель, которую вы выбираете для API-продукта,

должна приносить пользу и всей организации. Это значит, что прежде всего вам следует понимать стратегию или бизнес-модель своей организации. Если вы не в курсе ее целей, уточните их до создания новых API.

Поддержка вашего бизнеса с помощью API

Влияние вашего продукта API на стратегию организации зависит от направленности бизнеса. Если основной источник дохода компании — продажа доступа к API сторонним разработчикам (например, API для связи от Twilio или для платежей от Stripe), тогда стратегия вашего продукта будет крепко связана со стратегией всей компании. Если API работает эффективно, компания получает доход, а если нет — ее ждет крах. Продукт API становится основным каналом создания ценности для вашей компании, а его архитектура и бизнес-модель будут неявно согласовываться с ее целями.

Но у большинства организаций уже есть «традиционный» бизнес, который API просто должны поддерживать. Тогда API не станет новым основным источником дохода, если компания кардинально не изменит свою стратегию. Например, банк, работающий уже сотни лет, может открыть API для внешних разработчиков, чтобы поддержать инициативу открытого банковского обслуживания.

Если фокусироваться только на бизнес-модели API, можно использовать схему получения дохода от него, согласно которой разработчики платят за доступ к функциям банковского обслуживания, основанным на API. Но если держать в голове общую картину работы банка, эта стратегия API может навредить, ведь она создает барьер для пользователей. Вместо этого можно предложить банковский API бесплатно (в убыток себе) в надежде, что улучшение удаленного доступа к банку приведет к росту продаж самих услуг.

Стратегия API предоставляется не только продуктам для внешнего использования, это важный столп и для внутренних API. Важное различие между стратегиями внутреннего и внешнего API лишь в том, для каких пользователей они предназначены. Но в обоих случаях нужно понять, зачем нужен этот API и как он может помочь в решении проблемы. Вне зависимости от причины появления API стоит разработать для него стратегическую цель, выгодную организации.

После определения цели следует разработать тактику для ее достижения.

Определение тактики

Достичь стратегической цели API поможет план, составленный для вашего продукта. Важно разработать тактику для повышения вероятности успеха. По сути, ваша стратегия направляет всю предстоящую работу, основанную на принятии решений. Задача в том, чтобы понять связь между каждой из этих областей работы по принятию решений и вашей целью.

Рассмотрим несколько примеров.

- *Цель: расширить бизнес-возможности вашей платформы.* Если основное внимание уделяется созданию большего набора API, ориентированных на бизнес, ваша тактика должна в первую очередь быть ориентирована на изменения в разработке и создании API. Например, вы, вероятно, захотите привлечь заинтересованные стороны бизнеса на ранней стадии разработки вашего API, чтобы убедиться, что через ваш интерфейс предоставляются нужные возможности. Доверьтесь им и тогда, когда речь пойдет об определении первостепенных пользователей и ограничений для API.
- *Цель: монетизировать внутренние активы.* Для монетизации нужна тактика, помогающая представить продукт сообществу пользователей, которое сочтет его полезным. Это значит, что вам предстоит работа на конкурентном рынке, поэтому большое значение приобретает опыт разработчика (DX) в использовании вашего API. Это влечет за собой увеличение вложений в такие столпы, как дизайн, документация и обнаружение. Вам понадобятся и маркетинговые исследования для определения нужной целевой аудитории API. Важно убедиться, что ваш продукт ей подходит.
- *Цель: собрать бизнес-идеи.* Если вы задались целью найти свежие идеи за пределами вашей компании, необходимо разработать набор тактик, способствующих использованию вашего API инновационными способами. Нужно выставить его на внешний рынок, сделав привлекательным для сообщества пользователей, которое принесет наибольшую потенциальную инновационную ценность. Приложите максимум усилий к столпу исследования, чтобы использовать все его преимущества для сбора максимального количества идей. Вам также понадобится четкая тактика для определения лучших идей и путей их дальнейшего развития.

Итак, чтобы разработать сильную тактику для API, вам понадобится:

- определить необходимые для успеха столпы;
- выяснить, какое сообщество пользователей приведет вас к успеху;
- собрать данные об общей обстановке, чтобы управлять работой по принятию решений.

Адаптация стратегии

Когда вы начинаете создавать API, важно разработать правильную тактику, но не менее важно, чтобы стратегия оставалась гибкой и готовой к изменениям. Устанавливать стратегическую цель и тактический план раз и навсегда — неудачное решение. Важно корректировать стратегию, основываясь на результатах, полученных от продукта. Если вы никуда не продвигаетесь, следует изменить тактику и цель или даже стереть этот API и начать заново.

Для отслеживания прогресса нужен способ измерения результатов работы API. Очень важно иметь набор метрик для измерения прогресса в достижении ваших стратегических целей, иначе вы никогда не узнаете, насколько хорошо вы справляетесь. Мы подробнее обсудим цели и измерения API в главе 6, когда познакомим вас с OKR и KPI для API. Также измерения зависят от способа сбора данных об API, поэтому придется серьезно отнестись к контролю.

Будьте готовы изменить стратегию в случае изменения контекста вашего API, например, если организация ставит иную стратегическую цель, на рынке появляется новый конкурент или правительство вводит дополнительные ограничения. В каждом из этих случаев быстрая адаптация может значительно повысить ценность вашего API. Но стратегические изменения ограничены изменчивостью вашего API, поэтому столп управления изменениями (его мы обсудим далее) будет важным вложением.

Ключевые решения по управлению стратегией

- *В чем цель и тактический план API?* Определение цели и плана ее достижения — основа работы над стратегией. Важно тщательно продумать, как распределить работу по принятию решений. Вы можете позволить отдельным командам определить свои цели и тактику, чтобы получить преимущество локальной оптимизации, или централизовать планирование целей для оптимизации системы. Если вы решите децентрализовать работу над стратегией API, создайте стимулы и средства контроля для предотвращения непоправимого ущерба от любого отдельного API. Например, централизация этапа одобрения решения о постановке цели может замедлить процесс, но предотвратить непредвиденные проблемы.
- *Как измеряется влияние стратегии?* Определение цели API — локальная задача, но важно убедиться, что ваша цель совпадает с интересами компании. Это измерение может быть децентрализовано и передано командам API, а может быть и централизованным, и стандартизированным. Например, вы можете внедрить согласованный процесс со стандартизированными показателями, которым команды должны следовать для отчетов API. Его преимущество в том, что у вас будет постоянный объем данных для анализа на системном уровне.
- *Когда менять стратегию?* Иногда цели приходится менять, но кто должен принимать такое решение? Проблема изменения цели API в том, что она сильно влияет как на сам API, так и на зависящих от него людей. Вы можете позволить командам самостоятельно определять цель нового API, но при этом необходимо больше контролировать изменения цели, особенно когда API уже активно используется.

Дизайн

Принятие решений о том, как создаваемый продукт будет выглядеть, ощущаться и применяться, — это и есть дизайн. Все, что вы создаете или изменяете, требует решений по дизайну. На самом деле всю работу по принятию решений, которую мы описываем в этой главе, тоже можно рассматривать как работу над дизайном. Но суть столпа «Дизайн API» не только в этом. Речь пойдет о конкретном типе дизайна — решениях, принимаемых при разработке *интерфейса API*.

Мы выделили дизайн интерфейса в отдельный столп, потому что он очень сильно влияет на API. Интерфейс — это лишь одна из составляющих, но пользователи, работающие с вашим продуктом, видят только его. Для них это и есть API. Поэтому дизайн любого интерфейса, над которым вы работаете, значительно воздействует на принимаемые вами решения. Например, решение о том, что API должен иметь событийно управляемый интерфейс, кардинально меняет работу по реализации, развертыванию, контролю и документации API.

При дизайне интерфейса придется принимать много решений. Вот несколько важных вопросов, которые нужно рассмотреть.

- *Терминология.* Какие слова и термины должны понимать ваши пользователи? О каких специальных символах им следует знать?
- *Стили.* Какие протоколы, шаблоны сообщений и стили интерфейса будут применяться? (Будет ли ваш API задействовать шаблон CRUD? Или событийно-ориентированный стиль? Или стиль, похожий на запросы GraphQL?)
- *Взаимодействие.* Как API будет помогать пользователям удовлетворять их потребности? (Какие запросы они должны сделать для достижения цели? Как им будут сообщать о статусе запроса? Какие настройки, фильтры и подсказки по использованию они получат?)
- *Безопасность.* Какие особенности дизайна помогут пользователям избежать ошибок? Как вы будете сообщать им об ошибках и проблемах?
- *Единообразие.* Насколько пользователям будет знаком интерфейс? Будут ли термины этого API совпадать с терминами других API в вашей организации? Будет ли дизайн интерфейса похож на API, которые использовали раньше, или он удивит новизной? Будет ли дизайн отвечать стандартам, принятым в вашей сфере?

Это не полный список, но, как видите, вам нужно будет принять очень много решений. Спроектировать хороший интерфейс — сложная работа. Но давайте уточним нашу цель. Что такое *хороший дизайн* интерфейса API и как улучшить решения в этой сфере?

Что такое хороший дизайн

Если вы определили стратегию своего API, значит, уже установили для него стратегическую цель. Надо понять, как дизайн интерфейса поможет вам приблизиться к ней. Как говорилось в разделе «Стратегия» на с. 112, у вас может быть много разных целей и тактик. Но в целом все они сводятся к выбору между двумя основными направлениями.

1. Привлечь больше пользователей API.
2. Уменьшить затраты на разработку API.

На практике для обеспечения эффективности нужна гораздо более подробная стратегия. Но обобщая цели так, мы можем сделать важное наблюдение: хороший дизайн интерфейса поможет вам работать в обоих направлениях.

Если общее впечатление от использования API приятное, то работать с ним захочет больше пользователей. Дизайн интерфейса важен при получении опыта разработчика, и хороший дизайн привлечет больше пользователей. Также качественный интерфейс уменьшает вероятность совершения ошибки и сокращает число действий по решению проблем пользователя. Это упрощает разработку ПО, которое вы пишете, применяя API с хорошим дизайном.

Поэтому очень важно постараться над разработкой хорошего дизайна интерфейса. Но конкретного набора решений для качественного дизайна нет. Качество интерфейса зависит только от целей его пользователей. К счастью, если вы уже поняли, *зачем* создаете API, то выяснить, *для кого*, — несложная задача. Сделайте это сообщество пользователей своей целевой аудиторией и принимайте решения по дизайну, которые улучшат их впечатления от работы с вашим API.

ОПЫТ РАЗРАБОТЧИКА

В этой главе и на протяжении всей книги мы будем упоминать об *опыте разработчика*. DX — это опыт пользователя вашего API, с той поправкой, что имеется в виду конкретный тип пользователей — разработчики ПО. Иначе говоря, это сумма всех взаимодействий разработчика с вашим API. Дизайн интерфейса очень важен при получении этого опыта, наряду с документацией, маркетингом и техподдержкой. В итоге DX — отражение того, насколько довольны или недовольны ваши пользователи.

Использование шаблонов

Чтобы дизайн интерфейса приносил максимальные результаты, лучше использовать готовые методы или процессы. Большая часть работы над дизайном — это составление предположений о том, что, по вашему мнению, будет работать.

С чего-то надо начинать, поэтому можно скопировать понравившийся интерфейс API или последовать указаниям из какой-нибудь записи в блоге. Это совершенно нормально. Но если вы хотите максимизировать ценность вашего интерфейса, нужно протестировать эти предположения, чтобы убедиться, что вы принимаете оптимальные решения — лучшие из возможных.

Например, мы можем решить выбрать следующий упрощенный процесс.

1. Найти прототип интерфейса.
2. Написать свое приложение-клиент, использующее прототип.
3. Дополнить прототип, опираясь на полученные данные, и попробовать еще раз.

Более сложный процесс может выглядеть так.

1. Устроить совещание по дизайну со всеми заинтересованными лицами — пользователями, специалистами службы техподдержки, разработчиками.
2. Совместно разработать терминологию для интерфейса.
3. Провести опросы в сообществе пользователей.
4. Создать прототип.
5. Протестировать прототип на целевых сообществах пользователей.
6. Утвердить прототип с помощью разработчиков.
7. При необходимости повторить.

Основное различие между этими двумя примерами — в количестве вложенных средств и объеме полученной информации. Решение о том, насколько сильно нужно вложиться в процесс дизайна, стратегическое. Например, интерфейс внешнего API, который будет продаваться на конкурентном рынке, вы наверняка проработаете тщательнее, чем интерфейс внутреннего API для команды разработчиков. Но помните, что даже упрощенный процесс дизайна может принести крупные дивиденды.

ФОРМАТЫ ОПИСАНИЯ API

Вы можете облегчить себе работу по созданию дизайна с помощью машинно-распознаваемого описания интерфейса. Не у всех стилей API есть установленный стандартный формат, но у самых распространенных он существует.

Например, если вы разрабатываете API на SOAP, можно использовать формат WADL (<http://bit.ly/2yt4ebG>), для API на HTTP со стилем CRUD применяется спецификация OpenAPI (<http://bit.ly/2Pn1Z3m>), а для разработки API на gRPC можно взять буферы протоколов (<http://bit.ly/2PdNxdR>). Все эти форматы описания упрощают создание прототипов с помощью инструментальных средств и могут передавать описание интерфейса в виде файла.

Ключевые решения по управлению дизайном

- *Каковы ограничения по дизайну?* Команда API, не ограниченная в области дизайна, может создать модель интерфейса, максимально облегчающую работу и пользовательский опыт. Это удобно для конкретного пользователя, но мы жертвуем гибкостью интерфейса для пользователей в целом. Такая локальная оптимизация влияет на всю систему. Если вы производите несколько API и пользователям нужно задействовать больше одного, придется ввести некоторые ограничения по дизайну. Таким образом, нужно будет централизовать либо дизайн, либо выбор вариантов от дизайнеров. Некоторые централизованные команды публикуют «гид по стилям», чтобы документировать такие ограничения. Он может включать в себя официально разрешенные термины, стили и взаимодействия.
- *Как поделиться моделью интерфейса?* Иными словами, как поддерживать непрерывную работу по дизайну API. Например, если вы полностью централизуете это решение, то можете решить, что все команды API должны создавать дизайн в формате описания OpenAPI. Это ограничивает варианты дизайна опциями спецификации OpenAPI, зато упрощает разделение работы между командами и использование одинакового инструментария и автоматизации в системе.

Документация

Независимо от того, насколько хорошо вы спроектируете интерфейс своего API, ваши пользователи не смогут начать работу без небольших подсказок. Возможно, им потребуется показать, где именно API расположен в сети, научить их терминологии сообщений и интерфейса и сообщить, в каком порядке создавать запросы. Столп «Документация» включает в себе работу по созданию материалов для такого обучения. Мы называем его не «Обучение», потому что чаще всего человек обучается, читая документацию.

Предоставление качественной документации полезно по той же причине, что и разработка качественного интерфейса: положительный опыт разработчика увеличивает стратегическую ценность API. Если документация неудачная, понимать API будет сложнее. Если научиться его использовать слишком трудно, пользователей станет меньше. Если они будут вынуждены его применять, на разработку ПО уйдет больше времени и в нем будет больше проблем.

Методы создания документации

Документацию по API можно предоставлять по-разному: в виде отсылок к ресурсам вашего API, похожим на энциклопедию, обучающих курсов и содержательных материалов и даже в форме четко задокументированных сложных

образцов приложения, которые пользователи могут скопировать. Стилей, форматов и стратегий технической документации невероятное множество. Для упрощения разделим документацию по API в соответствии с двумя базовыми принципами: *«обучай, но не рассказывай»* и *«рассказывай, но не обучай»*. По своему опыту можем сказать, что для качественного обучения пользователей пригодятся оба подхода.

Суть подхода *«рассказывай, но не обучай»* в том, чтобы сообщать пользователям полезные факты о вашем API. Сюда относится, например, составление перечня кодов ошибок API или списка схем тела сообщения. Такой тип документации дает пользователям справочник по интерфейсу. Он состоит в основном из фактов, поэтому разработать его несложно. Обычно такую документацию можно быстро создавать с помощью инструментария, особенно если дизайн интерфейса упорядочен в машинно-распознаваемом формате (см. врезку «Форматы описания API» на с. 118).

В целом такой тип фактического описания деталей и поведения интерфейса требует от вашей команды меньше усилий по проектированию и принятию решений. Основные решения связаны с выбором частей API, подлежащих документации, а не с выбором удобного способа для передачи этой информации.

В противоположность прошлому подходу *«обучай, но не рассказывай»* сосредоточен на разработке обучения. Вместо того чтобы просто излагать факты для читателей, этот подход предоставляет им персонализированное обучение. Он предназначен помочь пользователям достичь целей применения API в процессе его изучения с помощью целенаправленного подхода.

Например, если вы создали API для навигации, можете написать шестиступенчатый обучающий курс, показывающий пользователям, как получить из адреса информацию по GPS. Так вы поможете им выполнить типичное задание, прилагая минимум усилий.

Документация не должна быть пассивной. Читать справочники, руководства и обучающие материалы полезно, но вы можете добавить и немного интерактивности в них. Например, предоставить инструмент для исследования API с веб-интерфейсом, позволяющий отсылать запросы к API в онлайн-режиме.

Хороший инструмент для исследования API — это не просто «тупая» программа, отправляющая запросы в Сеть. Он направляет обучение, предоставляя пользователю список действий, терминов, предложений и исправлений. Большой плюс интерактивных инструментов в том, что они сокращают для пользователей время получения обратной связи. Это цикл изучения, попыток применить

полученную информацию на практике и обучения на результатах. Без таких инструментов пользователям придется тратить время на написание кода или поиск внешних инструментов.



Портал разработчиков

В мире API портал разработчиков — это место (обычно сайт), где есть все ресурсы для поддержки API. Создавать его необязательно, но он может сильно помочь в повышении опыта разработчика для API, давая пользователям удобный способ изучения вашего продукта и взаимодействия с ним.

Инвестиции в документацию

Одного типа документации по API недостаточно для сообщества. У разных пользователей разные потребности, и если вам небезразличен пользовательский опыт, нужно удовлетворить их все. Например, для новых пользователей может быть удобен подход *«обучай, но не рассказывай»* — он директивный и ему легко следовать. Но опытным понравится документация в стиле *«рассказывай, но не обучай»*, ведь они могут быстро сориентироваться в нужных фактах. Интерактивные инструменты придутся по душе тем, кому нравится практический опыт, но они не подходят для тех, кто любит разбираться и планировать.

На практике создание всей этой документации может оказаться затратным занятием. В конце концов, кому-то придется ее разрабатывать и писать — и не один раз, а на протяжении всего существования API. Одна из трудностей работы с документацией по API в том, чтобы синхронизировать ее с изменениями интерфейса и реализации. Если документы некорректны, пользователи быстро начнут проявлять недовольство. Поэтому нужно сразу решить, сколько документации вы сможете поддерживать.

При принятии решения о размере вложений в документацию важно учитывать выгоду для компании от улучшения обучения API. Например, хорошая документация может помочь выделить внешний продукт API среди конкурентов или помочь разработчикам, использующим внутренний API, не знакомым с системой, бизнесом или сферой деятельности его владельца. Уровень вложений в документацию для внешнего API, работающего на конкурентный рынок, обычно выше, чем для внутреннего, и это нормально. В конце концов, вам придется решить, сколько документации нужно вашему API. К тому же эти вложения всегда можно увеличить.

Ключевые решения по управлению документацией

- *Как разработать процесс обучения?* Решения по разработке процесса обучения обычно принимают отдельно от решений по дизайну, реализации и развертыванию API. Если у вас много API, пользователи оценят наличие единого процесса обучения по ним. Но это может привести к потерям: инноваций становится меньше, появляется больше ограничений для команд API, из-за централизации технического описания в нем могут возникнуть слабые места. Нужно сбалансировать необходимость единообразия, с одной стороны, и те разнообразие и инновации, которые следует поддерживать, — с другой. Один из способов сделать это — применение гибридной модели, где для большей части API документация централизована, но команды также имеют право создавать процессы обучения, если пробуют что-то новое.
- *Когда писать документацию?* Нет единственно верного ответа на этот вопрос. Писать ее заранее получится дороже из-за большой вероятности изменений в дизайне и реализации, но плюс в том, что можно на ранних этапах обнаружить проблемы в использовании API. Необходимо выяснить, можно ли безопасно децентрализовать это решение или ему нужно более централизованное управление. Например, нужна ли каждому API независимо от его предназначения документация, написанная заранее? Или лучше позволить каждой команде решать самостоятельно, обдумав все «за» и «против»?

Разработка

Этот столп включает в себя все решения, принимаемые в ходе работы над API. Сложно разработать *реализацию*, полностью соответствующую дизайну *интерфейса*. У столпа «Разработка» невероятно большое пространство для принятия решений. Вам понадобится выбрать, какие технологические продукты и платформы использовать для реализации API, как будет выглядеть архитектура этой реализации, какие языки программирования и конфигурации применять и как API должен вести себя во время запуска. Другими словами, вам нужно разработать ПО для API.

Пользователям API неважно, как он реализован. Все ваши решения о реализации языков программирования, инструментов, баз данных и дизайна ПО для них бессмысленны — важен лишь конечный продукт. Если API делает то, что от него требуется, и делает это как следует, пользователи будут довольны. А то, что вы задействовали какую-то конкретную базу данных (БД) или структуру, для них просто полезная информация.

Но то, что пользователям безразличен ваш выбор, не значит, что ваши решения по разработке не важны. Они очень значимы, в особенности для тех, кто будет создавать, поддерживать и изменять API. Если вы выберете сложные технологии или языки, не понятные никому в компании, поддерживать API будет гораздо сложнее. А если применить слишком неудобный инструментарий или тяжелые для программирования языки, будет сложно вносить изменения в API.

Думая о разработке API, люди часто фокусируются только на решениях, которые надо принимать для первой версии продукта. Они важны, но это лишь небольшая часть работы. Лучше сделать упор на принятии решений по разработке, которые улучшат качество, расширяемость, изменяемость и удобство в сопровождении вашего API на весь срок его работы.

Разработка ПО с учетом всего вышеперечисленного требует опыта и навыков, поэтому нужно вкладываться в специалистов и инструментарий. В конце концов, написать простую программу может кто угодно после нескольких занятий по программированию. Но нужен настоящий профессионал, чтобы создать программы, работающие при увеличении масштаба или одновременном использовании множеством потребителей и успешно справляющиеся со всеми возникающими сложностями, но при этом остающиеся доступными для поддержки и внесения изменений.

Нет никаких жестких правил по разработке и архитектуре ПО для API, так же как их нет для разработки ПО в целом. Конечно, есть много указаний, принципов и советов по его разработке. В целом в сфере разработки API можно применять любые качественные методы разработки ПО, реализуемого на сервере. Единственное, что характерно для API, — это здоровая экосистема специфичных для него фреймворков и инструментов для разработки экземпляров. Теперь рассмотрим возможности, открывающиеся при использовании этих полезных средств.

Применение фреймворков и инструментов

В любом типичном процессе разработки используется множество инструментов, но в случае разработки API нас интересует особая категория — инструменты, помогающие уменьшить усилия по принятию решений и разработке новой реализации API. Сюда входят структуры, облегчающие работу по написанию кода для API, и отдельные инструменты, предлагающие реализацию с небольшим количеством кода или вовсе без него.

Один из самых популярных инструментов в сфере управления API — *шлюз для API*. Это серверный инструмент для снижения стоимости развертывания API

в сети. Обычно шлюзы разрабатываются очень расширяемыми, надежными и безопасными — как правило, основным мотивом для их применения становится именно укрепление безопасности реализации API. Это полезный инструмент — он заметно снижает затраты на разработку API-программы.

Стоимость разработки снижается при использовании таких инструментов, как шлюз, потому что они предназначены для решения большинства ваших проблем и не требуют много сил на программирование. Например, обычный шлюз для API на HTTP должен быть готов принять запросы с порта HTTPS, проанализировать тела запросов JSON, начать взаимодействовать с базой данных сразу после начала работы и потребовать немного конфигурации перед запуском. Все это происходит не по волшебству — кто-то должен запрограммировать этот инструмент на исполнение таких задач. В итоге вы передаете затраты по разработке API внешней компании.

Качественная работа инструментария — это подарок небес. Но проблема с такими инструментами, как шлюзы для API, в том, что они могут исполнять только то, для чего разработаны. Как и в случае с любым механизмом, здесь гибкость ограничена техническим функционалом. Поэтому выбор правильного инструмента важен для разработки. Если ваша стратегия API и дизайн интерфейса требуют совершения нестандартных действий, вам придется заниматься разработкой самостоятельно.

Отношения между интерфейсом и реализацией

Первоначальная цель вашей разработки — поддержка стратегии и дизайна интерфейса. Неважно, насколько прекрасна ваша архитектура и удобен для поддержки код. Если API не выполняет того, что должен по замыслу разработчика интерфейса, это провал. Отсюда можно сделать два вывода.

1. Соответствие *опубликованному* дизайну интерфейса — важный параметр качества реализации.
2. Нужно обновлять реализацию каждый раз после изменения интерфейса.

Это значит, что без дизайна интерфейса нельзя разрабатывать API. А еще, что вашим разработчикам нужно знать, как выглядит интерфейс и когда он меняется. Ваш API-продукт находится в зоне риска, если команда по дизайну выбирает непрактичный интерфейс, не поддающийся реализации. Если дизайнер интерфейса и разработчик реализации являются одним человеком или входят в одну команду, это не страшно, но если это не так, лучше убедиться, что в ваш метод разработки API входит перепроверка интерфейса группой внедрения.

ИСПОЛЬЗОВАНИЕ ОПИСАНИЙ API, ПРИБЛИЖЕННЫХ К КОДУ

Один из способов улучшить ваши шансы на синхронизацию реализации и интерфейса — внедрение описания интерфейса в саму реализацию. Например, если ваш формат описания API передает дизайн интерфейса, вы можете сохранить этот файл в своем репозитории кода или даже разработать автоматизированный тест, проверяющий ваше соответствие интерфейсу. Можно даже создать скелет программы, основываясь на формате описания (хотя на самом деле это эффективно только для первой версии).

Можно применить и противоположный подход: вместо получения описания интерфейса и использования его вместе с кодом внедрить его вручную прямо в код. Некоторые структуры позволяют задействовать аннотации с описанием интерфейса API. Сочетание кода и аннотации способно стать «источником истины» для интерфейса, который можно даже применять для создания документации по API. Любой из этих подходов поможет вам убедиться, что реализация API не нарушает обещаний, данных дизайном интерфейса.

Ключевые решения по управлению разработкой

Что можно использовать для реализации? Это основной вопрос в рамках руководства реализацией, включающий множество решений: из каких баз данных, языков программирования и компонентов системы выбирать? Какие библиотеки, структуры и инструменты задействовать для поддержки работы? С какими поставщиками сотрудничать? Можно ли использовать код из открытых источников?

На момент написания этой книги был большой интерес к децентрализации таких решений для улучшения локальной оптимизации. Многие компании сообщали нам, что их API стало проще создавать и поддерживать после того, как командам дали больше свободы при разработке реализации.

Но у децентрализации есть своя цена: меньше согласованности и возможностей для оптимизации системы. Это значит, что сотрудникам станет сложнее переходить из одной команды в другую, к тому же будет меньше возможностей экономить при увеличении масштаба API и контролировать всю реализацию в целом.

По нашему опыту, лучше предоставить больше свободы реализации, но нужно учитывать, какую свободу решений может дать ваша система. Один из способов упростить поддержку децентрализованных решений по реализации — централизация элемента выбора. Вы централизуете поиск вариантов технологий, но децентрализуете выбор каждой команды.

Тестирование

Если вам небезразлично качество вашего API, вам нужно вложиться в его тестирование. Столп «Тестирование» содержит решения о том, *что* нужно тестировать и *как*. В целом тестирование API не сильно отличается от обычного тестирования любого проекта программного обеспечения. Вам нужно будет применять методики проверки качества ПО к *реализации, интерфейсу и экземпляру* с API так же, как к традиционному приложению ПО. Но, как и разработка, тестирование в сфере API проходит несколько иначе благодаря системе инструментов, библиотек и вспомогательных приложений.

Что нужно для тестирования

Главная цель тестирования API — убедиться, что он может достичь стратегической цели. Но как мы уже знаем, эта цель появляется благодаря работе, основанной на решениях, во всех десяти столпах. Таким образом, второстепенная цель тестирования API — убедиться в качестве проделанной работы во всех столпах для поддержания вашей стратегии. Например, сложность API в применении и обучении может повлиять на стратегическую цель «привлечь больше пользователей». Также это означает необходимость определить конкретные тесты для достижения высокого качества интерфейса. Нужно проверить проделанную работу и на внутреннее единообразие. Например, может понадобиться проверить, что разработанная вами реализация совпадает с интерфейсом.

Вот список типичных категорий тестов, применяемых владельцами API.

- *Юзабилити и UX-тестирование (тестирование опыта пользователя)*. Поиск проблем с применением в интерфейсе, документации и на этапе обнаружения. Например, предоставить разработчикам документацию по API и наблюдать, как они ищут по ней клиентский код.
- *Модульное тестирование*. Поиск ошибок в реализации на детальном уровне. Например, запустить тест JUnit на каждом уровне кода реализации API, написанной на Java.
- *Интеграционное тестирование*. Поиск ошибок в реализации и интерфейсе с помощью запросов к экземпляру API. Например, запустить тестовый скрипт, выполняющий вызовы API для работающего экземпляра API в среде разработки.
- *Тестирование производительности и нагрузки*. Поиск нефункциональных ошибок в уже опубликованных программах и их окружении. Например, запустить скрипт, тестирующий производительность, который имитирует производственную нагрузку для работающего экземпляра API в тестовой среде, приближенной к реальной.

- *Тестирование безопасности.* Поиск уязвимостей в интерфейсе, реализации и экземпляре API. Например, нанять ударную группу для поиска уязвимостей в запущенной программе API в безопасной тестовой среде.
- *Производственное тестирование.* Поиск ошибок в использовании, функциональности и работе программы после публикации продукта API в производственной среде. Например, выполнить многомерный тест с применением документации по API, где разным пользователям даются несколько различающиеся версии содержимого, и улучшить документацию, основываясь на результатах.

Это не исчерпывающий список. Есть и другие тесты. И даже описанные здесь можно разбить на много подкатегорий. Главное стратегическое решение в области тестирования — определить, сколько тестов будет достаточно. В идеале ваша стратегия по API может помочь принять это решение.

Если у вас в приоритете качество и единообразие, то нужно потратить много времени и денег на тестирование API перед публикацией. Но если ваш API экспериментальный, можете рискнуть и выполнить минимум тестов. Например, совершенно нормально, если у API для платежей солидного банка и API новой соцсети будут совершенно разные по масштабам подходы к тестированию.

Инструменты для тестирования API

Тестирование может оказаться дорогим, поэтому полезно найти инструменты, упрощающие улучшение качества продукта. Например, некоторые организации успешно применяют метод управляемого тестирования, когда тесты пишутся до создания реализации или интерфейса. Цель такого подхода — в изменении производственной культуры команды на такую, где тесты будут центральным элементом, чтобы все решения по дизайну отражались в реализации, лучше адаптированной под тесты. Как результат — API более высокого качества, так как в него заранее встроено соответствие всем тестам.

Для уменьшения затрат на тестирование можно применять инструментальные средства и автоматизацию. Самые полезные инструменты в арсенале для тестирования API — это симуляторы и тестовые двойники или моки. Поскольку взаимосвязанная природа ПО API затрудняет тестирование отдельных частей, придется как-то имитировать другие компоненты. Могут понадобиться инструменты для имитации каждого из следующих.

- *Клиент.* В ходе тестирования API вам придется имитировать запросы от клиентов. Есть много инструментов для этого. Лучшие из них можно конфигурировать так, чтобы приблизиться к реальным типам сообщений и количеству трафика.

- *Серверная часть.* Скорее всего, у ваших API будут свои зависимости — набор внутренних API, база данных или какой-либо внешний ресурс. Для проведения тестов на интеграцию, безопасность и проверки работы программы вам наверняка понадобится как-то симулировать эти взаимосвязи.
- *Среда.* Нужно будет симулировать и производственную среду. Много лет назад это могло означать поддержку и работу запланированной среды только для этой цели. Но сейчас многие организации используют инструменты виртуализации для снижения расходов на воссоздание среды для тестирования.
- *API.* Иногда вам может понадобиться симулировать даже свой API. Например, когда нужно протестировать один из поддерживающих компонентов — скажем, программу для просмотра API на портале разработки, отправляющую запросы. К тому же симуляция вашего API — это ценный ресурс, который можно предоставить пользователям. Такую симуляцию часто называют *песочницей*, потому что это место, где разработчики могут поиграть с API и данными без последствий. Она требует некоторых вложений, но может значительно повысить опыт разработчика для вашего API.



Как приблизить песочницу к производственным условиям

Выпуская песочницу для пользователей API, убедитесь, что она максимально точно воспроизводит производственную среду. Пользователи будут гораздо довольнее, если единственным изменением, которое от них потребуется, будет перенос их кода в программу, запущенную в производство. Нет ничего более удручающего, чем потратить много времени и сил на выявление неполадок в интеграции API и обнаружить, что экземпляр выглядит и ведет себя иначе.

Ключевые решения по управлению тестированием

- *Где проводить тестирование?* В последнее время процесс тестирования становится все более децентрализованным. Важно определить, насколько вы хотите централизовать каждый из описанных нами тестов для API. Централизация процесса тестирования позволит вам лучше контролировать его, но появится риск создания слабых мест. Частично его можно уменьшить с помощью автоматизации, но тогда нужно решить, кто займется конфигурацией и поддержкой автоматизированной системы. Большинство организаций используют как централизованные, так и распределенные системы. Мы советуем децентрализовать ранние стадии тестирования для ускорения процесса и централизовать поздние из соображений безопасности.
- *Сколько тестов необходимо?* Даже позволив отдельным командам создавать свои тесты, вы можете централизованно принять решение по поводу минимально необходимого уровня тестирования. Некоторые компании используют

инструменты покрытия кода, предоставляющие отчеты о том, какой объем кода был протестирован. С точки зрения метрик и качества охват не идеален, но он поддается количественному измерению и позволяет установить минимальный порог, которому должны соответствовать все команды. Если у вас есть подходящие сотрудники, можете децентрализовать и это решение, позволив отдельным командам решать, что будет правильно для их API.

Развертывание

Реализация API — это то, что воплощает *интерфейс*. Но чтобы она приносила пользу, ее нужно правильно развернуть. Столп «Развертывание» содержит всю работу по перемещению реализации API в среду, где ее сможет использовать ваша целевая аудитория. Развернутый API — это работающая *программа*, и вам может понадобиться управлять несколькими ее копиями, чтобы API работал правильно. Сложность в том, чтобы убедиться, что все копии программы ведут себя единообразно, доступны пользователям и их можно максимально легко изменить.

Развертывать программное обеспечение сейчас гораздо сложнее, чем в прошлом. Это связано с усложнением наших программных архитектур и с большим количеством взаимосвязанных зависимостей, чем раньше. Кроме того, компании сейчас многого ожидают от систем в плане доступности и надежности — они хотят, чтобы все работало постоянно и при каждом запросе. И не забудьте: они требуют, чтобы изменения выполнялись незамедлительно. Для соответствия этим ожиданиям вам придется принимать качественные решения по поводу системы развертывания API.

Работа с неопределенностью

Улучшение качества развертывания API подразумевает, что поведение копий ваших программ API отвечает ожиданиям пользователей. Очевидно, большая часть работы над этим не связана с развертыванием. Нужно написать качественный, четкий код реализации и тщательно его протестировать, если вы хотите снизить число проблем при производстве. Но иногда даже после принятия всех этих мер сложности все равно появляются. Это связано с высоким уровнем неопределенности и непредсказуемости для опубликованного API.

Например, что произойдет в случае внезапного резкого роста спроса на ваш API-продукт? Справятся ли программы с возросшей нагрузкой? Что случится, если оператор случайно опубликует старую версию программы или внешняя услуга, от которой зависит API, станет недоступной? Неопределенность может проявиться по многим причинам: из-за пользователей, человеческой ошибки, аппаратной составляющей, внешних взаимозависимостей. Для усиления без-

опасности развертывания вам нужно одновременно использовать два противоположных подхода: устранение неопределенности и ее принятие.

Распространенный метод сокращения неопределенности при развертывании API — применение принципа *неизменности*. Неизменность — это невозможность измениться. Другими словами, состояние «только для чтения».

Ее можно задействовать по-разному. Например, если вы не разрешаете своим операторам изменять переменные среды сервера или устанавливать пакеты ПО вручную. Точно так же вы можете создавать неизменяемые пакеты развертывания API, то есть пакет, который можно только заменить. Принцип неизменности укрепляет безопасность, ведь благодаря ему уменьшается неопределенность, вызванная действиями человека.

Но у вас никогда не получится полностью избавиться от неопределенности. Невозможно предсказать все случайности и проверить все возможности. Поэтому важная составляющая вашей работы по принятию решений — понять, как сохранить систему в безопасности, даже в случае возникновения неожиданностей.

Часть этой работы происходит на уровне реализации API (написание защитного кода), а часть — на уровне системы (разработка устойчивой системной архитектуры). Но очень много предстоит работы и на уровне развертывания и работы с программой. Например, постоянно наблюдая за состоянием готовых программ с API и системных ресурсов, можно найти проблемы и исправить их до того, как они повлияют на пользователей.



Разработка устойчивого программного обеспечения

Один из наших любимых источников информации по безопасности развернутого API — книга Майкла Найгарда *Release It!*¹. Если вы ее еще не читали, обязательно прочтите. Это кладезь шаблонов реализации и развертывания для повышения безопасности и отказоустойчивости вашего продукта API.

Одна из неопределенностей, которую вам придется принять, — это изменения API. Хотя было бы неплохо заморозить все изменения, после того как ваш API заработает как следует, изменения — это неизбежность, к которой вы должны быть готовы. Фактически максимально быстрое развертывание изменений должно быть целью вашей работы по развертыванию.

¹ *Нейгард М. Release it! Проектирование и дизайн ПО для тех, кому не все равно.* — Питер, 2016.

Автоматизация развертывания

Есть только два способа ускорить развертывание: выполнять меньше работы и делать ее быстрее. Иногда этого можно добиться, изменив процесс работы — например, выбрав другой подход к ней или другую практику. Это сложно, но может помочь. Мы углубимся в эту тему в главе 8.

Другой способ ускориться — автоматизировать задания по развертыванию, ранее выполняемые людьми. Например, автоматизируя процесс тестирования, создания и развертывания кода API, можно будет выпускать программы одним нажатием кнопки.

Использование инструментов развертывания и автоматизации может показаться быстрым и выигрышным путем, но помните о долгосрочных затратах. Автоматизировать работу — это как проводить механизацию на фабрике: эффективность улучшается, но процесс становится менее гибким. Также автоматизация приносит с собой расходы на установку и поддержку. Вряд ли она начнет работать «сразу после распаковки» и сама приспособится к меняющимся требованиям и окружению. Поэтому, автоматизируя систему, будьте готовы к затратам на поддержку этого процесса и его удаление.



APIOps: DevOps для API

Многое из описанного в этом разделе совпадает с принципами культуры DevOps. Есть даже новый термин для применения методов DevOps к API — APIOps. Нам кажется, что DevOps прекрасно подходит к сфере API и у этого подхода есть чему поучиться.

Ключевые решения по управлению развертыванием

- *Кто решает, когда пора публиковать API?* Это один из важнейших вопросов. Если у вас есть талантливые сотрудники, которым вы доверяете, устойчивая к сбоям архитектура и сфера бизнеса, где простительна случайная неудача, можете полностью децентрализовать это решение. В ином случае надо определить, какие его части централизовать. Распределение этого решения обычно очень детальное. Например, вы можете доверить «кнопку публикации» участникам команд, которым доверяете, или публиковать программу в тестовой среде, где централизованная команда сможет определить, подходит или не подходит решение. Распределяйте решение так, чтобы это соответствовало вашим ограничениям и обеспечивало максимальную скорость с надлежащим уровнем безопасности в масштабе.

- *С помощью чего происходит развертывание?* В последние годы приобрел особую важность вопрос о том, как ПО развертывается и доставляется. Оказывается, это решение может постепенно изменить направление всей системы. Например, растущая популярность развертывания в контейнерах удешевила и упростила создание неизменяемых облачных развертываний. Подумайте, кто будет принимать это важное для вашей организации решение. Если оно будет децентрализовано и оптимизировано локально, тот, кто его принимает, может не осознавать его влияние на безопасность, совместимость и масштаб. Но у централизованного лица может не найтись решения, подходящего для всех реализаций и всего ПО, которые нужно разместить. Как обычно, в этом случае пригодятся ограничение вариантов и распределение элементов решения.

Помимо вопроса о централизации, нужно также обдумать, какая команда лучше подходит для принятия самых важных решений. Должны ли группы эксплуатации и межплатформенного ПО определять варианты формата развертывания? Или это может сделать команда по архитектуре? А может, по реализации? Важность здесь в распределении талантов: участники каких команд смогут лучше оценить ситуацию?

Безопасность

API упрощают связь между ПО, но также приносят новые проблемы. Открытость API делает их потенциальной мишенью, создавая новое пространство для атаки. Появилось больше дверей, которые можно открыть, а за ними — больше соковок! Поэтому важно уделить особое внимание укреплению безопасности своего API. Столп «Безопасность» включает в себя решения, которые нужно принять, чтобы:

- защитить систему, клиентов API и конечных пользователей от возможных угроз;
- поддержать законное использование API;
- защитить личные данные и ресурсы.

Эти три простые цели скрывают невероятно сложную предметную область. Одна из главных ошибок владельцев продуктов API — считать, что безопасность держится на нескольких технологических решениях. Это не значит, что технологические решения в сфере безопасности не важны! Но если вы действительно хотите укрепить столп «Безопасность», нужно расширить контекст принятия решений, основанных на безопасности.

Комплексный подход

Для укрепления безопасности API сделайте ее частью процесса принятия решений по всем десяти столпам, применяя характеристики безопасности во время выполнения программы. Вначале нужно найти идентификаторы, распознать клиентов и конечных пользователей, авторизовать использование и ограничить скорость. Многие из этого можно выполнить самостоятельно. Также можно задействовать безопасный и ускоренный подход — применить инструменты и библиотеки, которые сделают все это за вас.

В обеспечение безопасности API входит и множество действий за пределами работы клиента с ПО API. Ее могут укрепить изменения в производственной культуре, например поощрение подхода «безопасность прежде всего» у инженеров и дизайнеров. Можно ввести процессы, предотвращающие внесение вредоносных изменений в экземпляр, и пересмотреть документацию, чтобы убедиться, что информация не просачивается за ее пределы. Можно обучить сотрудников отдела продаж и техподдержки не допускать случайного обнародования личных данных и не поддаваться психологическим атакам.

Конечно, все сказанное выходит за рамки традиционной сферы безопасности API. Но ведь она не может существовать сама по себе. Это часть взаимосвязанной системы, и ее нужно считать одним из элементов целостного подхода к безопасности компании. Бессмысленно делать вид, что она не связана с общей стратегией безопасности. Те, кто захочет воспользоваться вашей системой в своих целях, явно так считать не будут.

Итак, вам понадобится принимать решения о том, как внедрить работу с API в общую стратегию безопасности компании. Иногда это довольно легко, иногда — нет. Одна из больших сложностей API — найти баланс между желаниями сделать его открытым и простым в использовании и закрыть к нему доступ. Как далеко вы пойдете, в любом случае должно зависеть от вашей организационной стратегии и стратегических целей вашего API.

Ключевые решения по управлению безопасностью

- *Какие решения нужно одобрять?* Все решения, принимаемые в вашей организации, могут сделать систему уязвимой. Но тщательно проверять каждое невозможно. Важно определить, какие из них делают API безопаснее, и убедиться в их качестве. Это во многом будет зависеть от общей обстановки. Есть ли зоны в вашей архитектуре, которым вы доверяете, но которым нужны безопасные границы? Дизайнеры и разработчики уже имели дело с системами безопасности? Происходит ли вся работа внутри или вы сотрудничаете

с внешними разработчиками? Все это может изменить выбор решений, требующих одобрения.

- *Сколько безопасности нужно API?* Бóльшая часть аппарата безопасности — это стандартизация рабочих решений для защиты системы и ее пользователей. Например, у вас могут быть правила о том, где хранить файлы или какие алгоритмы шифрования использовать. Повышенная стандартизация ограничивает свободу команд и людей на использование инноваций. В контексте безопасности это обычно оправдывается последствиями одного неудачного решения, но не все API обязательно нуждаются в одинаковом уровне проверки и защиты. Например, API, который внешние разработчики используют для финансовых операций, потребует больше вложений в безопасность, чем API для сохранения данных о работе приложения. Но кто должен принимать это решение?

Как обычно, основные критерии — контекст, талант и стратегия. Централизация этого решения позволяет команде по безопасности примерно оценить ситуацию, основываясь на своем понимании системного окружения. Но иногда из-за таких общих правил что-то может просочиться, особенно когда команды API применяют инновации, не учтенные централизованной командой. Если принятие этого решения — прерогатива команд, они сами могут оценить ситуацию, но для этого нужен высокий уровень знаний по безопасности. И наконец, если вы работаете в сфере, где приоритет отдается безопасности и снижению рисков, вы можете в конечном итоге навязать самый высокий уровень безопасности для всего, независимо от локального контекста и влияния на скорость.

ПРОЕКТ OWASP ПО БЕЗОПАСНОСТИ API

Проект OWASP по безопасности API (<https://oreil.ly/Vn5YA>) — отличный ресурс для проверки того, что вы предприняли все необходимые меры для защиты своего API. Открытый проект по безопасности веб-приложений (OWASP) — это некоммерческая общественная организация, предоставляющая рекомендации по обеспечению безопасности веб-приложений. В последние годы они поддерживают сообщество материалами по API для помощи в устранении наиболее распространенных угроз, с которыми сталкиваются владельцы API. Если вы хотите принять более взвешенное решение о безопасности своего API, убедитесь, что ваша команда прочитала и поняла рекомендации по безопасности API OWASP до начала проектирования и разработки.

Двенадцать принципов безопасности API

Ниже приводится контрольный список 12 основных принципов безопасности API, составленный на основе контрольного списка безопасности Юрия Субача. Вы можете использовать его, чтобы направлять свою команду к безопасным и защищенным API.

- *Конфиденциальность API.* Ограничение доступа к информации — первое правило безопасности API. Ресурс должен быть доступен через API только авторизованным пользователям и защищен от нежелательных получателей во время передачи, обработки или в состоянии покоя.
- *Целостность API.* Информация от API всегда должна быть достоверной и точной. Ресурс должен быть защищен от любых модификаций или удалений, а нежелательные изменения должны обнаруживаться автоматически.
- *Доступность API.* Доступность — это гарантия надежного доступа к информации уполномоченных лиц. Она зависит от требований, предъявляемых на уровне инфраструктуры и приложений, в сочетании с соответствующими инженерными процессами в организации.
- *Экономичность алгоритма.* Разработка и реализация API должны быть максимально просты. Сложные API трудно проверять и улучшать, они более подвержены ошибкам. С точки зрения безопасности и удобства использования минимализм — это хорошо.
- *Отказоустойчивые API по умолчанию.* Доступ к любой конечной точке/ресурсу API должен быть запрещен по умолчанию. Он должен предоставляться только при наличии специального разрешения. Качественная безопасность API следует схеме защиты «когда доступ должен быть предоставлен», а не «когда доступ должен быть ограничен».
- *Полное посредничество.* Доступ ко всем ресурсам API всегда должен быть подтвержден. Каждая конечная точка должна быть оснащена механизмом авторизации. Этот принцип выводит соображения безопасности на общесистемный уровень.
- *Открытый дизайн API.* Хороший дизайн безопасности API не должен быть секретом. Он должен быть задокументирован и основан на определенных стандартах безопасности и открытых протоколах. Безопасность API затрагивает все заинтересованные стороны организации, включая партнеров или потребителей.
- *Наименьшие привилегии API.* Каждый потребитель в системе должен работать с минимальными разрешениями API, необходимыми для выполнения работы. Это ограничивает ущерб, принесенный случайностью или ошибкой, связанной с конкретным потребителем API.
- *Психологическая приемлемость.* Эффективная реализация безопасности API должна защищать систему, но не препятствовать ее правильному применению пользователями и выполнению ими всех требований безопасности. Уровень безопасности API должен соответствовать уровню угрозы. Мощный механизм защиты API для неконфиденциальных ресурсов может быть непропорциональным с точки зрения усилий для потребителей.

- *Сокращение площади атаки API.* Ограничение площади атаки на API — это минимизация того, что может быть использовано злоумышленниками. Для уменьшения площади атаки на API вы можете предоставлять только то, что необходимо, и снизить возможный ущерб путем ограничения объема и скорости, регулирования числа запросов на вызовы API перед дальнейшей проверкой пользователя и проведением тщательной проверки сценариев использования.
- *Глубинная защита API.* Несколько уровней контроля затрудняют использование API. Вы можете ограничить доступ к серверу несколькими известными IP-адресам (белая маркировка), применить двухфакторную аутентификацию и множество других методов, повышающих уровень обеспечения безопасности вашего API.
- *Политика нулевого доверия.* Политика нулевого доверия означает, что любые сторонние поставщики API и сторонние потребители API по умолчанию считаются небезопасными. Это означает реализацию всех соответствующих мер безопасности для внутренних и внешних API, как если бы все они были внешними и ненадежными по умолчанию.
- *Безопасный отказ API.* Все API часто не справляются с обработкой транзакций из-за неправильного ввода, перегрузки запросов или по другим причинам. Любой сбой внутри экземпляра API не должен отменять механизмы безопасности и обязан запрещать доступ в случае сбоя.
- *Правильное устранение проблем безопасности API.* Как только проблема безопасности API будет выявлена, сосредоточьтесь на ее правильном устранении и избегайте «быстрых решений», способных помочь в краткосрочной перспективе, но не устраняющих истинный корень проблемы. Разработчики и эксперты по безопасности API должны понять основную причину неполадки, создать для нее тест и устранить с минимальным воздействием на систему. По завершении исправления систему нужно протестировать во всех поддерживаемых средах и на всех платформах. Часто нарушения безопасности API возникают из-за сбоев, выявленных командой API, но исправленных должным образом.

Мониторинг

Важно, чтобы ваш продукт API обладал таким качеством, как наблюдаемость состояния. Нельзя успешно управлять API без актуальной информации о том, как он работает и как его используют.

Столп «Мониторинг» касается того, как сделать эту информацию доступной и полезной. Со временем при расширении системы контроль становится для

управления API таким же важным, как дизайн интерфейса или разработка реализации, ведь если бродить в темноте, можно споткнуться и упасть.

В экземпляре API нужно отслеживать много моментов:

- проблемы (ошибки, системные сбои, предупреждения, неполадки);
- состояние системы (ЦП, память, ввод-вывод, состояние жесткого диска);
- состояние API (время с начала работы, общее состояние, число выполненных заданий);
- архив сообщений (тела сообщений с запросами и ответами, заголовки сообщений, метаданные);
- данные об использовании (общее число запросов, применение устройства / ресурсов, количество запросов на одного пользователя).



Как узнать больше о мониторинге

За исключением пунктов об API и мониторинге использования, описанные параметры не уникальны для программных компонентов на основе API. Если вам нужно качественное руководство по контролю сетевых компонентов ПО, советуем прочитать книгу *Site Reliability Engineering* от Google¹ (<https://oreil.ly/Pl6lu>). Эта книга знакомит с разработкой систем ПО и содержит очень понятный список пунктов, требующих контроля. Еще один хороший ресурс — «Метод RED» от компании Weaveworks (<https://oreil.ly/qJKtI>). Он определяет три категории параметров микросервиса: оценку (rate), количество ошибок (errors) и продолжительность (duration).

Каждая из этих групп параметров по-своему полезна для вашего API. Данные о состоянии и проблемах помогут вам найти ошибки и разобраться с ними, что может уменьшить последствия возникающих проблем. Данные об обработке сообщений могут помочь в устранении неполадок API и проблем на системном уровне. Данные об использовании помогают улучшить продукт, показывая, как пользователи работают с API. Но прежде всего нужно будет обеспечить доступ к этим данным и убедиться, что ваш метод их сбора и представления надежен.

Чем больше данных вы соберете, тем больше у вас будет возможностей для улучшения продукта. Поэтому в идеале вы должны создавать бесконечный поток данных для своего API. Затраты на производство, сбор, хранение и анализ этих данных неизбежны, но иногда они становятся просто неподъемными. Например, если время полного цикла обработки запросов API удваивается из-за того, что ему нужно скопировать данные, следует либо сократить число журналов,

¹ Бейер Б., Джоунс К., Петофф Дж. *Site Reliability Engineering*. Надежность и безотказность как в Google. — Питер, 2021.

либо найти лучший способ делать это. Прежде всего вам нужно решить, что вы можете позволить себе отслеживать, зная, каковы будут затраты на мониторинг.

Еще одно важное решение — насколько последовательным будет ваш мониторинг API. Если способ, которым ваш API предоставляет данные, совместим с другими инструментами, отраслевыми стандартами или организационными соглашениями, применять его будет легче. Разработка системы мониторинга во многом похожа на разработку интерфейса. Если интерфейс для мониторинга уникален, будет сложнее научиться использовать его и собирать данные. Это не страшно, если у вас один API и вы сами его отслеживаете, но если их десятки или сотни, придется пожертвовать уникальностью. Мы подробнее рассмотрим эту тему в главе 7.

Ключевые решения по управлению мониторингом

- *Что нужно отслеживать?* Это важное решение. Поначалу можете оставить его на усмотрение отдельных команд, но при расширении API выгоднее будет единообразие в контроле. Согласованные данные улучшат вашу способность наблюдать за влиянием и поведением системы. Это значит, что вам понадобится централизовать какие-то элементы данного решения.
- *Как собирать, анализировать и передавать данные?* Централизация решения по осуществлению контроля облегчит работу с данными API, но может ограничить свободу действий отдельных команд. Важно решить, какие из элементов этого решения централизовать, а какие распределить между командами. И еще важнее оно становится, когда речь идет о данных, требующих защиты.

Обнаружение

API ценен только тогда, когда им пользуются, но для этого его сначала надо найти. Столп «Обнаружение» посвящен тому, как облегчить вашей целевой аудитории обнаружение API. Для этого нужно помочь пользователям быстро понять, что делает API, чем он может быть полезен, как начать работу с ним и как в нем разобраться. Обнаружение — важная составляющая опыта разработчика. Чтобы ваш продукт было действительно легко найти, понадобятся качественные решения по дизайну, документации и реализации, а также дополнительная работа по обнаружению.

В сфере API есть два основных типа обнаружения: в период разработки и в период работы приложения. Первое направлено на то, чтобы пользователям было проще узнать о продукте с API. Оно включает в себя информирование о его существовании, функциональности и проблемах, которые он может решить.

Обнаружение в период работы приложения происходит уже после развертывания API. Оно помогает ПО обнаруживать местонахождение API в сети, основываясь на каких-либо фильтрах или параметрах. Этот тип обнаружения направлен на людей, и для него в основном требуется работа по продвижению и маркетингу. Обнаружение в период работы приложения направлено на автоматические клиенты и основывается на инструментарии и автоматизации. Теперь подробнее рассмотрим каждый вариант.

Обнаружение во время работы приложения

Это способ улучшения изменяемости вашего ландшафта API. Если у вас хорошая система обнаружения в период работы, то вы можете менять местонахождение программ с API с минимальным влиянием на клиентов. Это особенно полезно, если запущено много копий программы одновременно. Например, архитектура на основе микросервисов часто поддерживает обнаружение в период работы, чтобы сервисы было проще находить. Большая часть работы, которую вам нужно будет сделать, чтобы это произошло, связана с основами разработки и развертывания API.

Обнаружение в период работы — это полезный механизм, и его стоит ввести, если вы управляете сложным ландшафтом API. У нас недостаточно времени, чтобы углубляться в детали, но для его реализации нужны инвестиции в технологии на уровне ландшафта, API и пользователей. Под столпом «Обнаружение» в этой книге подразумевается в основном обнаружение в ходе разработки.

Обнаружение во время разработки

Чтобы помочь людям узнать о своем API в период разработки, нужно убедиться, что им доступен нужный тип документации. Пользователи должны быстро получать доступ к документации о том, что делает API и какие проблемы решает. Такая копия маркетинга продукта важная, но не единственная часть обнаружения. Главная составляющая этого столпа — помощь пользователям, которые ищут информацию. Для достижения успеха вам нужно сотрудничать с сообществом пользователей и рекламировать им свой продукт. Как вы это делаете, зависит от контекста вашей пользовательской базы.

- *Внешние API.* Если API предназначен в основном для людей, не работающих в вашей команде или организации, понадобятся инвестиции, чтобы донести до них свой послыл. Нужно будет представить им продукт API примерно так же, как вы представили бы ПО: используя оптимизацию поисковых механизмов, спонсирование мероприятий, взаимодействие с сообществом и рекламу.

Ваша цель — убедиться, что все потенциальные пользователи API понимают, как этот продукт поможет им в решении их проблем. Конечно, ваши конкретные маркетинговые приемы будут во многом зависеть от контекста, характера API и целевой аудитории.

Например, если вы разрабатываете продукт API для SMS и конкурируете за внимание пользователей-разработчиков, то нужно рекламировать его там, где могут оказаться потенциальные клиенты: на конференциях веб-разработчиков, в блогах на тему двухфакторной аутентификации или на конференциях по телекоммуникациям.

Если ЦА — независимые разработчики, можно положиться на рекламу в Сети. Но чтобы привлечь крупные предприятия, придется вложить деньги в команду менеджеров по продажам, создающих сеть из людей, с которыми они общаются. Если вы конкурируете на переполненном рынке, вам, вероятно, придется приложить много усилий, чтобы выделиться, но если ваш продукт уникален, вам может понадобиться лишь немного магии поисковой оптимизации, чтобы привлечь людей. Когда дело доходит до маркетинга продукта, контекст имеет решающее значение.

- *Внутренние API.* Если API используют ваши разработчики, то аудитория вам обеспечена. Но это не значит, что можно игнорировать возможность обнаружения вашего API. Внутренний API тоже надо найти, чтобы применить. И если этот процесс будет усложняться, возникнут проблемы. Не зная о нем, внутренние разработчики не будут им пользоваться и даже могут потратить время на создание другого точно такого же API. Конкуренция на внутреннем рынке API — обычно хороший признак, а возможность повторного использования часто переоценивается на предприятиях. Но дублирование качественных API из-за неосведомленности — это пустая трата ресурсов.

Внутренние API могут продаваться почти так же, как и внешние. Разница лишь в маркетинговой экосистеме. Если внешний API направлен, например, на поисковик Google, то внутреннему может быть достаточно просто попасть в корпоративную электронную таблицу со всеми API компании. Для рекламы внешнего API эффективно будет спонсировать конференцию разработчиков, а для внутреннего стоит просто пообщаться со всеми командами в организации и научить им пользоваться.

Большой проблемой в рекламе внутренних API может быть отсутствие стандартизации внутри компании. Может оказаться даже, что в большой организации есть несколько списков API. Чтобы выполнить свою работу качественно, нужно убедиться, что API числится во всех важных реестрах. Также сложно может быть узнать обо всех ваших командах разработчиков и уделить им время. Но если для вас важно, чтобы API использовался, постарайтесь сделать это.

Ключевые решения по управлению обнаружением

- *Каким будет опыт обнаружения?* Важно обеспечить качественный опыт обнаружения API для пользователей. Это означает выбор инструментов обнаружения, взаимодействия с пользователем и целевой аудиторией. С ростом вашего API понадобится также решить, насколько согласованным должен быть этот опыт. Если вам нужна высокая согласованность, может потребоваться централизовать проектные решения.
- *Где и как будут рекламироваться API?* На продвижение API уходит много сил и времени, поэтому нужно выбрать того, кто определит время начала его рекламы. Это решение можно оставить на усмотрение отдельных команд по API или централизовать. Передавая решение командам, убедитесь, что никакие централизованные инструменты и процессы для обнаружения не мешают им.
- *Как поддерживается качество опыта обнаружения?* Со временем API меняются, и информация в любой системе обнаружения становится более размытой. Кто должен проверять ее и следить за показателями качества пользовательского опыта с момента публикации API?

Управление изменениями

Если бы API не нужно было изменять, управлять ими стало бы гораздо проще. Но их нужно исправлять, обновлять и улучшать. Вы должны быть всегда готовы к этому. Столп «Управление изменениями» содержит все важные решения по планированию и управлению при изменениях. Это очень важная и сложная сфера — на самом деле вся наша книга посвящена рассмотрению именно этой темы.

Управление изменениями направлено на три основные цели:

- выбор лучших вариантов изменений;
- максимально быстрая реализация этих изменений;
- внесение изменений без нарушений в работе программы.

Выбор лучших вариантов вызывает изменения, позволяющие достичь стратегических целей API. Но ради этого нужно научиться расставлять приоритеты и планировать, основываясь на затратах, внешних факторах и ограничениях. Это и есть работа по управлению продуктом, и это одна из причин, по которым мы ввели в главе 3 концепцию «API как продукт».

Определение четких стратегических целей и сообщества пользователей, на которое они направлены, позволит более ясно определять полезность вносимых изменений. Чем больше вы узнаете о работе в каждом из остальных девяти столпов, тем лучше вы будете оценивать затраты. Вооружившись надежной

информацией о стоимости и ценности, вы сможете принимать разумные решения о продукте для своего API.

Балансировать между безопасностью и скоростью изменений непросто, но необходимо. Каждое из ваших решений в столпах продукта API на что-то влияет. Фокус в том, чтобы стараться принимать решения, максимально увеличивающие одно из этих качеств с минимальными потерями для другого.

В главах 5, 7 и 8 мы исследуем эту мысль подробнее с точки зрения затрат, временных изменений и влияния на изменения организации и производственной культуры. В последних главах мы введем дополнительную сложность — масштаб. Главы 9, 10 и 11 относятся к управлению изменениями не в одном API, а в их ландшафте.

Внедрение изменений — это только половина работы по управлению ими. Другая половина — сообщить пользователям, что у них прибавилось работы из-за этих изменений. Например, если меняете модель интерфейса, следует сообщить командам по дизайну, разработке и операциям о новом задании.

В зависимости от характера изменений вам, скорее всего, нужно будет сообщить и пользователям, что им придется обновить клиентское ПО. Обычно это делается с помощью контроля версий API. То, как вы используете версию, во многом зависит от стиля вашего API и применяемых протоколов, форматов и технологий. Мы подробнее поговорим об этом в подразделе «Контроль версий» на с. 329.

Ключевые решения по управлению изменениями

Какие изменения надо вносить быстро, а какие безопасно? Решение о том, как обращаться с разными типами изменений, очень важно. Централизуя его, вы можете создать согласованное правило, допускающее разные процессы выпуска в зависимости от их влияния. Некоторые называют этот подход бимодальным или двухскоростным, но нам кажется, что в сложной организации больше двух скоростей. Также вы можете децентрализовать это решение, позволив отдельным командам самостоятельно оценивать влияние. Опасность в том, что группы могут оценить его неточно, поэтому важно убедиться в надежности своей системной архитектуры.

Совместное использование столпов

Столпы из этой главы сосредоточивают в себе множество решений и усилий. Но мы не пронумеровали их, не расставили приоритеты, не расположили их в какой-то последовательности, и сделали это преднамеренно. Методы реализации проектов и продуктов в разных организациях различаются. Поэтому мы

хотели дать вам структурированный способ определения ключевых решений и работ, так, чтобы это было полезно для любого метода разработки ПО, который вы предпочитаете.

Но одна из проблем, связанных с разделением решений и работы на столпы, в том, что они могут начать восприниматься как отдельные и независимые категории. Но в реальности так не бывает. В этом разделе мы опишем некоторые популярные способы совместного использования основных компонентов API для достижения целей разработки продуктов. Мы рассмотрим способы взаимовлияния отдельных столпов и целостную перспективу, которую вам нужно будет принять при их использовании. В главе 7 мы увидим, как меняются инвестиции в разные компоненты в течение срока службы API.

Для начала рассмотрим, как компоненты API используются вместе для решения задач планирования и разработки API.

Применение столпов во время планирования

В последние годы этапы планирования и проектирования получили несколько дурную славу. Многие специалисты по внедрению опасаются попасть в ловушку «Большого проектирования наперед» (Big Design Up Front, BDUF) (<https://oreil.ly/0tDY1>), когда команда тратит непропорционально много времени на проектирование, становясь полностью оторванной от практических реалий реализации. Поскольку наша отрасль продолжает внедрять принципы Agile (гибкость), методы Scrum и управление Kanban, все больше внимания уделяется «действию» и подходу «тестируй и учись». И это здорово.

Но мы также знаем о большой ценности наличия четкой цели, последовательной стратегии и плана действий, способствующего достижению результатов. Необходимость планирования и проектирования особенно важна для продукта API, ведь изменения любого интерфейса всегда сопряжены с определенными затратами. Все мы сталкивались с разочарованием, когда приходилось заново учиться что-то делать при изменении пользовательского интерфейса приложения. Модернизация интерфейса API может стоить очень дорого, поскольку она может легко повлиять на чужой код или на всю бизнес-модель.

Вот почему, независимо от методологии доставки, стоит начать с четкого плана для вашего API. По нашему опыту, даже Agile-ориентированные API-команды могут извлечь пользу из небольшого предварительного планирования. Для продуктов API важно начать в правильном направлении и согласовать работу по основным столпам. Мы выделили три области, на которых важно сосредоточиться: согласование дизайна, создание прототипов и определение границ.

Проверьте согласованность стратегии и дизайна

Как мы уже упоминали в разделе «Дизайн» на с. 116, очень важно согласовать работу над стратегией и дизайном. Работа по реализации API в рамках проектирования (см. раздел «Дизайн» на с. 116) и разработки (см. раздел «Разработка» на с. 122) включает невероятно широкий круг решений. Мы видели, как многие специалисты-практики испытывают трудности в отсутствие четко определенной цели или задачи.

Для повышения согласованности важно постоянно проверять свой дизайн на соответствие стратегии. Принимая решения по дизайну и реализации, легко потерять стратегическую перспективу своей работы. Вам следует провести повторную калибровку, проверив решения на соответствие вашим стратегическим целям. Если удалось определить ключевые показатели эффективности (KPI) или цели и ключевые результаты (OKR), тестирование решений станет намного проще.

СРАВНЕНИЕ ДВУХ СТРАТЕГИЙ: TWITTER И SLACK

Сравнение приложений Twitter и Slack поможет понять, как решения, которые мы принимаем в стратегии, могут повлиять на все решения, принимаемые в других столпах. В статье «История о двух платформах API» Джейсон Коста подчеркивает влияние, которое стратегическое направление (или его отсутствие) может оказать на разработку продукта API. Он подчеркивает, как Slack целенаправленно принимала решения по проектированию, разработке и управлению изменениями для достижения своих стратегических целей¹. Коста противопоставляет этот подход более органичному, изменяющемуся стратегическому направлению Twitter, что, по его словам, привело к потенциально невозможному разрыву с разработчиками, использующими их API.

Его тематическое исследование подчеркивает важное соображение для любого создателя API: решения, принимаемые нами в рамках одного стратегического столпа, могут оказать сокрушительное воздействие на все остальные столпы, с которыми мы работаем.

Прототип на ранней стадии

Современные методы доставки ПО подчеркивают важность итераций, спринтов и небольших пакетов изменений. Это важный элемент для успешного использования продукта API. Каждый раз старайтесь реализовать свою стратегию как можно раньше, чтобы проверить ее реализуемость. Это означает выполнение работ по тестированию (см. раздел «Тестирование» на с. 126) и разработке (см. раздел «Разработка» на с. 122) даже в процессе создания вашей стратегии.

¹ Costa J. A Tale of 2 API Platforms («История о двух платформах API») Medium, 25 октября 2016 г. <https://oreil.ly/ZzAlj>.

Для такого рода деятельности есть много названий: проверка концепций, пилотные проекты, «стальные нити», MVP (наиболее ценный специалист) и т. д. Ценность ранней реализации огромна вне зависимости от того, как вы ее называете. Фактически идея непрерывного совершенствования — одна из ключевых тем этой книги.

ИНСТРУМЕНТЫ ДЛЯ СОЗДАНИЯ ПРОТОТИПОВ API

Несколько лет назад создание прототипа API означало привлечение инженера для написания пользовательского кода. Но сегодня есть много инструментов, способных в этом помочь. К ним относятся такие фреймворки, как Spring Boot, которые ускоряют работу, обсуждаемую в разделе «Разработка» на с. 122, позволяя командам быстро создавать прототипы API, пригодные для тестирования. Группы могут использовать и инструменты проектирования интерфейсов для быстрого воплощения столпа документации («Документация» на с. 119).

Существует растущая экосистема веб-плагинов и модулей интегрированных сред разработки, которые могут помочь вам быстро создать спецификацию API. Некоторые инструменты даже позволяют командам быстро создавать API на основе существующего набора или базы данных. Рекомендуем найти инструмент, который поможет вам сосредоточиться на аспектах разработки и использования API как можно раньше — на этапах проектирования и планирования.

Определение границ

Ваш продукт API может состоять из набора отдельных компонентов. Это особенно заметно в архитектурном стиле «микросервис», который становится все более популярным для реализации API. В связи с этим большую важность приобретает заблаговременное планирование границ компонентов для реализации стратегии развития вашего продукта API.

ЧТО ТАКОЕ МИКРОСЕРВИС

Официального, согласованного определения микросервиса нет. Вместо этого он отражает стиль архитектуры на основе API, который развивался в 2010-х годах по мере изменения технологий и организаций. Почти все реализации микросервисов можно охарактеризовать расслоением приложений на набор компонентов с поддержкой API, имеющих «правильный размер» для обеспечения ценности для бизнеса.

Это означает, что частью вашей работы по проектированию одного API стало определение того, как его можно разделить на несколько частей. Здесь самое сложное — определить правильный набор границ для ваших компонентов. Команды все чаще выполняют первоначальную работу по определению границ на ранних этапах, чтобы иметь возможность создавать компоненты, которые лучше соответствуют их стратегии.

Использование столпов для реализации

Для воплощения стратегии API нам нужно реализовать продукт. В главе 7 мы рассмотрим, что значит реализовать продукт API с точки зрения жизненного цикла. Но прежде чем мы это сделаем, стоит рассмотреть, как будет выполняться фактическая работа. Мы уже разобрали четыре очень важных столпа для создания API: проектирование, документация, разработка и тестирование. Теперь нам нужно изучить, как их правильно использовать.

Если у вас есть опыт разработки ПО, вы знаете, что основные принципы, которые мы определили для создания API, не совсем новые. Все программное обеспечение, которое мы пишем сегодня — как API, так и нет — проходит через классические этапы проектирования, разработки, документирования и тестирования.

Есть и много методологий разработки ПО, которые можно использовать для управления работой на этих этапах. Например, Kanban, Scrum и Scaled Agile Framework пользуются популярностью среди специалистов как способ применения принципов гибкости Agile (<https://oreil.ly/ITAEJ>) при разработке продукта. Мы уверены, что у вашей организации есть устоявшийся способ создания программного обеспечения, который вы сможете применить в разработке API.

Но одна из особенностей создания API в том, что они объединяют много разных концепций в один конечный продукт. Мы уже затрагивали эту тему, когда освещали проблему понимания интерфейсов, реализации и экземпляров (см. пункт «Интерфейс, реализация и экземпляр» на с. 26). Вам нужно выяснить, как применить творческие принципы работы API так, чтобы объединить эти части. Как проектировать, разрабатывать, документировать и тестировать API так, чтобы интерфейс и реализация обеспечивали наибольшую стратегическую ценность?

К сожалению, нет единого подхода, позволяющего раскрыть эту ценность. Но хорошая новость в том, что нам удалось выявить три подхода, которые практикующие специалисты используют для достижения успеха: «документация — в первую очередь», «код — в первую очередь» и «тестирование — в первую очередь». Давайте рассмотрим каждый из них.

Документация — в первую очередь

Переходя к инженерным аспектам API, мы часто фокусируемся на их технологических составляющих: коде и конфигурации, которые управляют поведением при реализации. Но ни одно из этих действий во время выполнения не про-

исходит без участия человека, работающего над клиентским кодом, который использует API. Именно поэтому некоторые команды применяют подход «документация — в первую очередь» к методу исполнения API.

Этот подход означает, что команда отдает приоритет разработке пользовательского интерфейса API — документации. Вместо того чтобы начинать с обдумывания технических особенностей реализации на Go, Java, NodeJS или другом языке, они сосредотачиваются на документировании API еще до его появления. Например, при создании API для платежей мы можем начать с принятия решений и работы в столпе документации (см. раздел «Документация» на с. 119), прежде чем писать код.

Одно из преимуществ такого подхода — возможность протестировать пользовательский интерфейс API до вложения средств в любую сопутствующую работу по его внедрению. Например, составив руководство по использованию и примеры для нашего нового платежного API, мы сможем протестировать его с группой потенциальных разработчиков. Изменить документацию на основе их отзывов будет гораздо проще, чем изменить фактическую реализацию API.

Но «в первую очередь документация» не означает «только документация». На практике лучше разворачивать деятельность с разделов «Разработка» на с. 122 и «Тестирование» на с. 126 в период завершения работы над спецификацией. Вы можете сделать еще один шаг вперед, разработав прототипы или «макеты» документированного API, чтобы предложить живой, вызываемый продукт для раннего тестирования.

Ключ к подходу, ориентированному на документацию, в том, что мы определяем наши решения по реализации на обучаемости, пригодности и понятности API. Это хороший метод, который можно использовать, чтобы убедиться, что разработчик-потребитель — на первом месте для вашей команды на этапе создания продукта. А еще это хороший способ предоставить ранние результаты и активы нетехническим заинтересованным сторонам и спонсорам.

Одна из проблем такого подхода в том, что он может привести к появлению продуктов, которые обещают слишком много, но не способны это выполнить. Важно убедиться, что интерфейсы, которые документируются на бумаге, могут быть реализованы инженерными командами. Иногда вы можете опираться на сложный набор последующих возможностей, которые не могут быть изменены. Когда желаемое целевое состояние, которое документируется, сильно отличается от реальности, реализуемой пользователем, стоимость создания API может сильно вырасти.

Код — в первую очередь

Подход, ориентированный на код, в первую очередь фокусируется на сложности реализации внутренних компонентов API. Это значит, что команда расставляет приоритеты в действиях «Разработка» на с. 122 и «Тестирование» на с. 126 так, чтобы они могли быстро и эффективно реализовать первый выпуск продукта API. Это не значит, что команда не будет предоставлять никакой документации по API, но это означает, что документированный дизайн будет соответствовать решениям, принятым во время внедрения, а не наоборот.

Подход «в первую очередь — код» наиболее полезен, когда скорость выпуска перевешивает удобство и простоту использования API. Например, у команд, создающих микросервисы, обычно в приоритете инженерная работа, ведь они не планируют делиться своим микросервисом с другими командами. В этом случае их внимание может быть сосредоточено на упрощении изменения и выпуска кода, а не на потреблении.

Этот подход может быть полезен для быстрого исследования и тестирования возможности реализации продукта API в качестве «доказательства концепции» или «технологического всплеска», чтобы проверить, были ли выявлены или устранены все элементы с высоким риском. Например, команда API может начать писать код для гипермедийного или GraphQL API в качестве первого шага, потому что они незнакомы с этим конкретным стилем API и должны оценить практические аспекты дизайна.

Команды, ориентированные на код, могут (и должны) создавать документацию для своего интерфейса. В зависимости от характера проекта она может быть проще, чем у команды, ориентированной на документацию. Например, команды разработчиков кода обычно документируют свои API с помощью машиночитаемых языков описания API, таких как OpenAPI, а не создают удобочитаемые руководства. Иногда команда заявляет, что код и есть документация. Но основной аспект этого подхода в том, что работа над документацией сосредоточена только на передаче проектных решений, принятых на этапе кодирования.

Как и следовало ожидать, подход «в первую очередь — код» позволяет легко спроецировать технические аспекты и аспекты реализации на дизайн интерфейса. Чтобы сделать API с кодовым подходом удобнее в использовании для сторонних потребителей, часто нужно внести изменения в код или создать другой API, который будет находиться поверх него. Важно, что если вы выходите за рамки команды или организации в контролируемой среде, такой подход трудно сохранить надолго.

Тестирование — в первую очередь

Современная разновидность подходов, ориентированных на код и документацию, — реализация «тестирование в первую очередь». В этом типе разработки API тестируемость API в приоритете над его документируемостью или реализуемостью. Это применение разработки на основе тестов (TDD) к домену API.

РАЗРАБОТКА НА ОСНОВЕ ТЕСТОВ

Эта концепция была популяризирована Кентом Бекон в его книге «Разработка через тестирование, с примерами» (<https://oreil.ly/ZP66W>). На практике TDD реализуется как формально, так и неформально, разными способами. Но основная мысль в том, что код, предназначенный для производства, разрабатывается и проектируется так, чтобы его можно было протестировать.

Подход, основанный на тестировании, означает, что команда API начинает с создания тестовых случаев, соответствующих желаемому целевому состоянию, после чего следует работа по реализации для обеспечения прохождения тестового примера. Для разработки (см. раздел «Разработка» на с. 122) это обычно означает написание тестов, вызывающих API, еще до его написания. Например, при разработке платежного API мы напишем тестовый пример для совершения платежа через HTTP-запрос прежде, чем писать код для обработки и выполнения запроса.

Все становится еще интереснее, когда мы рассматриваем столп документации (см. раздел «Документация» на с. 119). По крайней мере, подход к разработке, основанный на тестировании, означает, что документация должна отражать тестовые примеры, которые мы разрабатываем. По сути, создание тестовых примеров — это деятельность по проектированию интерфейса. Но можно пойти дальше и поэкспериментировать со способами автоматического создания документации и примеров кода на основе написанных тестовых примеров.

Начать с тестов — отличный способ улучшить тестируемость продукта API. Он приводит к более безопасным и предсказуемым изменениям в будущем, ведь команда уверена в своей способности тестировать собственные разворачиваемые приложения. Но тестирование в первую очередь связано с дополнительными затратами на разработку и может затянуть время до первого выпуска. На практике многие команды применяют подход, основанный на документации, к своему продукту API и подход, основанный на тестировании, — к разработке.

Использование столпов для управления и запуска

За последние двадцать лет возросла потребность в создании ПО, которое можно быстро поставлять и изменять, обеспечивая при этом стабильную, безопасную и надежную работу. Это привело к изменениям в способах разработки и эксплуатации ПО. Организации все больше внедряют культуру DevOps, проектирование надежности сайтов и подход DevSecOps, внедряющий безопасность в процесс разработки.

Сдвиг Ops влево

Раньше разработчики писали приложение, а после передавали его команде эксплуатации, чтобы она запустила и поддерживала его. Но сегодня растет интерес к другому способу разработки приложений. Команды, воплощающие культуру DevOps, объединяют процессы разработки и эксплуатации, чтобы приложения были работоспособными с самого начала. Во многих современных командах разработчиков операционная деятельность стала полноправным участником процесса разработки, а не чем-то второстепенным.

Применение этого подхода к разработке API означает, что нужно согласовать столпы проектирования, разработки, тестирования, развертывания и мониторинга. Значит, командам необходимо будет совместно участвовать в принятии решений и работе по созданию и эксплуатации. Команда API, принимающая решения об инструментах и фреймворках, должна учитывать как проблемы написания кода, так и проблемы развертывания и запуска.

На практике большинство организаций в итоге внедряют платформу инструментов автоматизации DevOps, отвечающих потребностям команд API. Цель этих инструментов — ускорить время для создания и изменения API и улучшить их работоспособность. Обычно эти инструменты позволяют стандартизировать и автоматизировать задачи тестирования, развертывания и мониторинга.

На момент написания этой книги корпоративная организация могла бы развернуть платформу DevOps со следующими инструментами.

- *Конвейеры CI/CD.* Инструмент непрерывного улучшения (CI) и непрерывной доставки¹ (CD) автоматизирует процесс тестирования и развертывания программного компонента. В пространстве API конвейеры CI/CD часто настраиваются для проверки того, что API не внесет критических измене-

¹ Или непрерывного внедрения.

ний до его развертывания в производственной среде. Такая автоматизация тестирования и внедрения может применяться ко всем компонентам вашего продукта API, включая проектные ресурсы, документацию и реализацию.

- *Контейнеризация.* Сегодня API часто реализуются в виде контейнеров, способных работать как автономные единицы развертывания. Внедрение контейнеризации может кардинально изменить способ реализации API. Многие организации, которые начинают контейнеризировать свои API, в итоге внедряют подход «микросервисов», при котором они разбивают продукт на более мелкие, независимо развертываемые части.
- *Инструменты наблюдаемости.* Для облегчения мониторинга и устранения неполадок команды DevOps все чаще внедряют инструменты, которые помогают агрегировать, визуализировать и внедрить данные журналов и мониторинга. Для команд API это значит, что деятельность по проектированию и разработке должна соответствовать стандартизированным интерфейсам и форматам, определенным командами DevOps. Особенность в случае с API в том, что инструменты обеспечения наблюдаемости часто распространяются на внешних потребителей. Например, портал для разработчиков может предлагать аналитику использования и устранения неполадок третьим лицам, применяющим продукты компании.

Сдвиг безопасности влево

Как мы уже упоминали, API-интерфейсы представляют собой особую поверхность атаки для потенциальных злоумышленников. Разработчикам API нужно продумать меры по снижению угроз на всех порталах документации, при проектировании интерфейса, хранении данных и реализации кода. В последние годы появились три модели безопасности, которые меняют подход к работе с API.

- *Автоматизация DevSecOps.* Инструментарий, ориентированный на безопасность, имеет то же влияние на способ разработки API, что и инструменты, ориентированные на эксплуатацию. Организации все чаще добавляют сканеры уязвимостей в свои конвейеры CI/CD, чтобы все изменения проверялись быстро и эффективно, прежде чем будут внедрены в производство. Например, многие корпоративные команды API используют сканеры, проверяющие реализацию API на предмет угроз OWASP. Если сделать такое сканирование уязвимостей частью процесса кодирования, команды будут устранять уязвимости безопасности на ранних этапах процесса разработки.
- *Автоматизированное обнаружение угроз.* Сегодня компании не только пассивно проверяют код при срабатывании сканеров, но и активно сканируют

ресурсы для поиска уязвимостей. В настоящее время существует зрелая экосистема инструментов, помогающих командам непрерывно отслеживать журналы, репозитории кода, базы данных и даже живые API. Такой непрерывный мониторинг угроз помогает изменить поведение команд разработчиков API так, чтобы проблемы безопасности учитывались на ранних этапах процесса разработки.

- *Модели безопасности с нулевым доверием.* Раньше большинство моделей безопасности зависели от создания защищенного периметра, чтобы можно было доверять системам внутри него. Но в последние годы компания Google популяризировала модель «нулевого доверия», где ни одной системе нельзя доверять только на основании ее местоположения. Этот сдвиг — результат все более децентрализованных моделей организации и развертывания, связанных с современной разработкой ПО. Для команд API «нулевое доверие» означает, что создателям нужно учитывать, как будут применяться средства контроля доступа в рамках их деятельности по проектированию и разработке.

Платформы времени выполнения

Внедрение операций и работ по обеспечению безопасности в рамках столпов проектирования, разработки и тестирования требует определенных затрат. Команды, сосредоточенные на написании кода, теперь должны детально разбираться в аспектах ОС, инфраструктуре и безопасности. Но появляющийся набор инструментов и платформ помогает снизить некоторые из этих затрат.

Команды разработчиков API все более зависимы от сред выполнения, где будет работать их ПО. Это связано с тем, что современные платформы могут справиться со многими сложностями из-за применения принципов работы и запуска. Инструменты, технологии и платформы постоянно меняются, но на момент написания этой статьи выделяются три технологии: Kubernetes, сервисные сетки и бессерверные технологии.

Kubernetes. Стандартизация единицы развертывания (например, в виде «отгружаемого» контейнера) помогла многим командам улучшить работу и запуск своих API. Но безопасный и устойчивый запуск контейнера в производственной среде все еще требует тщательного планирования и управления. Именно поэтому многие команды используют платформу оркестровки (управления) контейнерами Kubernetes (<https://kubernetes.io>).

Kubernetes обеспечивает стандартизированный способ развертывания, масштабирования, запуска и мониторинга рабочей нагрузки контейнера. Это значит, что вам не нужно тратить время на поиск оптимального решения для выполнения

этих задач — все уже сделано за вас. Но с точки зрения столпов важно понимать, как это решение влияет на работу всех остальных компонентов API. При разворачивании их в Kubernetes нужно описать их конфигурации развертывания, маршрутизации и масштабирования. Это означает, что команда, принимающая решения по проектированию, разработке и тестированию, тоже должна хорошо разбираться в Kubernetes для создания рабочего ПО.

Сервисная сетка. Сейчас, с приходом популярности стиля микросервисов, появляется все больше программных продуктов, состоящих из более мелких, структурированных программных модулей и API. Но разбивая API на более мелкие части, вы усложняете работу по соединению этих частей в работоспособном виде. Ведь появляется больше вещей, которыми нужно управлять. Чтобы справиться с этими расходами, некоторые команды используют концепцию платформы, называемую *сервисной сеткой*.

Она помогает снизить эксплуатационные расходы на связь между программными компонентами по сети. Внедрение сервисной сетки часто сопряжено с высокими первоначальными затратами из-за сложности работы и запуска, так как большинство инструментов сервисной сетки сложно настраивать, устанавливать и обслуживать. Но сервисная сетка может принести большие дивиденды по всем столпам проектирования и разработки — она дает вашим API-командам свободу в создании небольших единиц развертывания так, чтобы их можно было безопасно запускать.

Бессерверные малокодовые технологии и будущее. Общая черта инновационных платформ среды выполнения для API в том, что они помогают нам быстрее и проще создавать масштабируемое и отказоустойчивое ПО. Большинство новшеств, которые мы видели, делают это как за счет снижения затрат на сложность, так и за счет внедрения стандартизации.

Эта тенденция продолжается с растущей популярностью «бессерверных» архитектур, скрывающих всю сложность работы платформы за стандартизированным, управляемым событиями интерфейсом. Точно так же тенденция к архитектуре с «малым кодом» скрывает сложность архитектуры, поддерживаемой API, за стандартизированным интерфейсом разработки.

Рассмотрение деталей бессерверной, низкокодовой архитектуры, сервисных сеток и Kubernetes выходит за рамки этой книги. Но для управления API важно, чтобы вы понимали, как эти инновации платформы влияют на решения и работу, которой вы управляете по всем направлениям. Например, внедрение бессерверной платформы сильно снижает стоимость и объем вашего столпа разработки, но тогда возникает необходимость в экспертных знаниях в столпах бессерверной разработки, эксплуатации и запуска.

Итоги главы

В этой главе мы рассмотрели десять столпов, поддерживающих работу по созданию API. Каждый из них содержит решения, влияющие на конечный продукт. Не все они требуют одинакового количества усилий, и вам нужно определить для себя важность каждого столпа, основываясь на общей обстановке и целях API.

При росте ландшафта API нужно будет обдумать, как распределить решения по каждому из этих столпов. Подробнее об этом вы узнаете в главе 11. Мы также заметили, что некоторые столпы работают в группах. Это влечет за собой дополнительные последствия и запутывает работу по созданию API в целом. Настало время понять, как управлять затратами на изменения в процессе жизненного цикла взаимодействий и каким должен быть уровень инвестиций с правильным уровнем зрелости для вашего API.

Но сначала нужно детальнее исследовать десятый столп — управление изменениями. Какова цена изменений в API? Подробно рассмотрим это в следующей главе.

Непрерывное улучшение API

Нет необходимости меняться. Выживание обязательно.

У. Эдвардс Деминг

В прошлой главе мы ознакомились с жизненным циклом API и определили основные направления работы, на которых нужно сосредоточиться. Этот жизненный цикл определяет, что именно сделать для первоначального выпуска вашего API. Столпы важны, когда речь идет об изменениях, которые вы будете вносить в течение всего периода жизненного цикла уже опубликованного API. Управление изменениями в API — это важнейшая часть стратегии управления им в целом.

Изменения в API могут серьезно повлиять на ваше ПО, саму программу и пользовательский опыт. Внесение в код изменений, способных навредить существующему API, может создать катастрофическую цепную реакцию, влияющую на все компоненты, использующие этот код. Даже изменения, не нарушающие внешний интерфейс API, могут вызвать большие проблемы, если случайно поменяют *поведение* этого API. Более того, один популярный API внутри вашей организации может создать длинный список взаимозависимостей, которые будет трудно задокументировать или даже увидеть. Поэтому управление изменениями — важный раздел в управлении API.

Если бы в опубликованные API не надо было вносить изменения, управлять ими было бы довольно легко. Но это неотъемлемая часть активно применяемых API. В какой-то момент нужно будет исправить ошибку, улучшить опыт разработчика или оптимизировать код реализации. Для этого нужно вносить коррективы в используемый API.

Управление изменениями API усложняется большим объемом работы. API-продукт — это не только интерфейс. Он состоит из многих частей: интерфейсов, кода, данных, документации, инструментов и процессов. Все эти части продукта можно изменять, и управлять ими надо осторожно.

Делать это непросто, но необходимо. К тому же возможность внесения изменений дает больше свободы. Невозможность изменять опубликованный API сильно бы усложнила процесс выпуска изначального релиза. Приходилось бы применять к разработке API правила конструирования и запуска космических ракет. Понадобился бы подход BDUF (Big Design Up Front) для предварительного планирования и серьезные вложения в разработку, чтобы убедиться в долговечности этого API. Пришлось бы сначала учесть все возможные неполадки и предотвратить их.

К счастью, необязательно работать по такой схеме. Добавление в API возможности вносить изменения может принести большую выгоду. Более дешевые и простые изменения означают, что их можно делать чаще. Это покрывает некоторые риски, потому что появляется возможность быстрее исправлять неполадки. Так вы можете чаще улучшать API.

В этой главе мы познакомим вас с философией непрерывного улучшения API, учитывающей изменения. Вы узнаете, как серия небольших поэтапных изменений может стать лучшим способом модификации вашего API, почему его трудно изменить и что вы можете сделать, чтобы улучшить эту изменчивость. Для начала давайте разберемся, что означает «непрерывное управление изменениями» применительно к миру API.

Непрерывное управление изменениями

Хотя идея поддержки непрерывных изменений для вашего API может показаться привлекательной, важно помнить о причине этой поддержки. Применяя подход АааР из главы 3, мы можем рассматривать изменения как попытку *улучшить* API, а не просто изменения ради изменений. Значит, любое время, затраченное на изменение API, должно быть оправдано. Два ключевых способа оценить наше улучшение — сосредоточиться на (1) повышении опыта разработчиков и (2) снижении стоимости техобслуживания для спонсоров продукта.

Конечно, не каждое изменение улучшит ваш API сразу. Например, вы можете модифицировать способ масштабирования вашего API, чтобы удовлетворить будущий спрос — изменение, которое не окупится, пока не возрастет объем использования, не приведет к немедленному ощутимому улучшению опыта разработчиков, но может предотвратить его ухудшение в будущем. Суть в том,

что любое изменение нужно рассматривать с точки зрения его способности улучшить продукт, даже если вознаграждение за эти инвестиции будет отсроченным.

В этом разделе мы сосредоточимся на двух базовых элементах управления изменениями со временем: 1) принятие модели постепенного улучшения в вашей организации и 2) увеличение скорости изменений в целом. Для начала давайте рассмотрим понятие инкрементализма как метода управления изменениями.

Постепенное улучшение

Если изменение — это путь к улучшению продукта, то разумная цель управления — максимально упростить изменение вашего API. Лучшая версия вашего API появится благодаря непрерывному циклу изменений или улучшений. Некоторые из них практически не будут улучшать продукт немедленно — некоторые ваши попытки могут даже временно ухудшить опыт разработчика. В таком случае необходимо применить другое улучшение для снижения последствий неудачного эксперимента. Со временем эти постоянные попытки постепенно улучшить API выгодно повлияют на ваш продукт и опыт разработчика.

Постепенное улучшение означает, что вы представляете, куда хотите двигаться, но решили делать маленькие шаги на пути к цели, а не выпускать большие изменения в попытке соответствовать всем будущим требованиям. Серия небольших изменений позволяет команде по API отреагировать на результаты каждого из них, эффективно проводя серию небольших экспериментов, чтобы найти лучший путь к меняющейся цели.

Есть много способов приблизиться к постепенному улучшению, и мы остановимся на трех: модель Деминга «Планируй — делай — изучай — действуй», метод Бойда «Петля OODA» и теория ограничений Голдратта. У каждого своя точка зрения на способ создания процесса непрерывного совершенствования.

Планируй — делай — изучай — действуй

Концепция непрерывного внесения небольших изменений — устоявшаяся схема, корнями уходящая в промышленность. В 1980-е годы первопроходец в сфере контроля качества У. Эдвард Деминг сформулировал версию этой идеи в виде подхода, который он назвал системой углубленного знания. Система Деминга работает со сложной природой больших организаций и применяет научный подход для улучшения способа производства продукции. Один из ее краеугольных камней — это цикл «Планируй — делай — изучай — действуй» (Plan — do — study — act (PDSA)), определяющий постепенный и экспериментальный метод улучшения процесса (рис. 5.1).

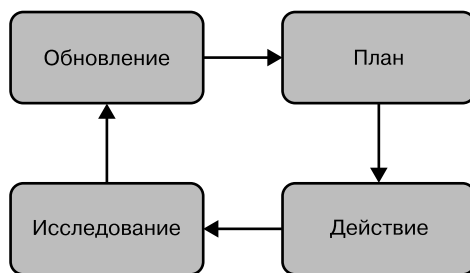


Рис. 5.1. Цикл PDSA Деминга

Процесс PDSA определяет четыре этапа применения улучшений. Сначала вы составляете *план* — теорию о том, как можно улучшить систему в соответствии с целью и какие изменения внести для проверки этой теории. Затем вы *действуете* — реализуете намеченные в плане изменения. После этого *изучаете* — отслеживаете и измеряете эффект от этих изменений, сравнивая результаты с планом. И наконец, вооружившись новой информацией, можете *действовать*, обновляя цель, теорию или действие, реализующие изменения, чтобы улучшить систему.

Если вы хотите улучшить взаимодействие разработчиков с вашим API, можете начать с сокращения времени, которое нужно разработчикам для изучения принципа работы интерфейса. Возможно, вы планируете обновить документацию, чтобы сделать ее удобнее для разработчиков. Затем вы можете «воплотить» план — обновить ресурсы и документацию, исследовать число ошибок разработчиков, изучивших ее. С помощью этих измерений вы можете пересмотреть необходимые изменения в документации или даже внести более значительные изменения в саму модель интерфейса.

Колесо PDSA описывает постепенный, экспериментальный процесс улучшения системы: вы вносите небольшие изменения, измеряете их эффект и используете полученные знания для дальнейшего улучшения. Это очень эффективный способ работы со сложными системами — такими, где тяжело предугадать результаты небольших изменений.



PDSA Деминга хорошо работает для компаний с высокой терпимостью к экспериментам и устоявшейся культурой проверки и анализа после внедрения.

Идеи Деминга и его колесо PDSA изначально были созданы для улучшения контроля качества на фабриках и сборочных производствах, но эта модель ока-

залась полезной везде, где нужно улучшить сложную систему, включая системы ПО. Такие методики производства ПО, как бережливая разработка (Lean), непрерывное улучшение (Kaizen) или гибкая методология (Agile), объединяет один принцип непрерывного улучшения определенного объекта. Иногда объект улучшения — это процесс, иногда — продукт, но в любом случае постоянные изменения, ориентированные на цель, обеспечивают гибкость и успех.

Наблюдай — ориентируйся — решай — действуй

Еще одна популярная модель для создания непрерывного потока качественных решений — петля OODA Джона Бойда (<https://oreil.ly/M5oeN>). OODA означает «Наблюдай — ориентируйся — решай — действуй» (Observe, orient, decide, act). Как и модель PDSA, OODA интерактивна — каждый раз вы проходите через одинаковые этапы, пытаетесь непрерывно совершенствовать свою работу. В 1950-х годах Бойд заметил, что американские летчики-истребители во время Корейской войны неизменно побеждали в воздушных боях, несмотря на то что пилоты противника летали на более совершенных самолетах. Бойд утверждал, что его исследование объясняет это противоречие, показывая, что американские летчики применяют более эффективные стратегии в воздушных боях — отсюда и метод петли OODA.



У петли OODA Бойда красочная предыстория, и она опирается на очень интересную информацию о том, как ведут себя американские пилоты в бою, принимая решения «в режиме реального времени». На самом деле, большая часть работ Бойда посвящена войне и конфликтам. Конечно, военные действия — не всегда подходящая аналогия для IT-организаций, но OODA продолжает оставаться актуальной в некоторых кругах. Подробнее о петлях OODA в организациях читайте в статье Роберта Грина «OODA и вы» (<https://oreil.ly/XqZn8>).

В двух словах применение петли OODA к вашей организации выглядит так.

- *Наблюдай.* Выберите целевую проблему (масштабирование, безопасность, набор функций и т. д.) и соберите как можно больше информации с максимально возможным числом доступных точек зрения. Здесь важно добавить много данных без какой-либо фильтрации, редактирования или анализа.
- *Оrientируйся.* На этом этапе вы применяете свой опыт, знания и навыки анализа к собранным данным. Сейчас самое время отфильтровать «неприменимую» информацию и сузить поле до нескольких вероятных вариантов.

- *Решай.* Пришло время взвесить варианты, затраты и выгоды, сделав приблизительную оценку того, какое действие нужно предпринять.
- *Действуй.* Вы выполняете принятое решение с помощью запланированных действий. Но это еще не конец. Ведь мы имеем дело с циклом, поэтому результаты ваших действий становятся предметом наблюдения, что возвращает вас в начало петли.

Важно, что эта модель была разработана для обучения пилотов, принимающих решения за доли секунды. Они повторяют этот цикл постоянно и быстро. Фактически один из ключевых выводов из работы Бойда — скорость имеет значение. Если вы будете действовать быстрее противника, то сможете одержать верх даже в случае его технического превосходства. По этой причине OODA часто используется при высокой конкуренции на рынке, а в случае с IT-компаниями — там, где раннее и частое внедрение имеет явное преимущество.



Поскольку петля OODA основана на принятии важных решений и быстрых действиях, эта модель хорошо работает, когда вам нужно опередить конкурентов и вы готовы к быстрому исполнению на основе богатого набора рыночных данных.

Но есть много ситуаций, когда скорость отходит на второй план. В таких случаях будет полезнее сосредоточиться на одной-двух конкретных проблемах и решить каждую, прежде чем переходить к следующей. Один из подходов, отвечающих этому критерию, — теория ограничений Голдратта.

Теория ограничений

Теория ограничений (Theory of Constraints, ТОС) была описана в книге «Цель»¹ Элияха М. Голдратта и Джеффа Кокса в 1984 году (<https://oreil.ly/t9Xda>). Это вымышленный рассказ о попытке одного менеджера «развернуть» неработающее производственное предприятие за 90 дней, прежде чем его закроют. Благодаря серии дистанционных консультаций с близким другом главный герой приобретает знания и навыки и учится применять ТОС для улучшения работы компании.



Книга Голдратта и Кокса продолжает оставаться бестселлером. В 2014 году было выпущено юбилейное 30-е издание. Несмотря на то что в 1980-х книга была посвящена операциям на физических предприятиях, многие из уроков в этой книге применимы к IT-организациям и сегодня.

¹ Голдратт Э. М., Кокс Дж. Цель. Процесс непрерывного улучшения.

Основная идея ТОС Голдратта (<https://oreil.ly/HI8SS>) состоит в следующем: ключ к успеху — в выявлении сдерживающих факторов (или ограничений) организации и совершении ряда шагов для их устранения. По завершении этого процесса настанет время определить новый сдерживающий фактор и начать все сначала.

Вот шаги, которые Голдратт и Кокс описывают в своей книге.

1. Определите сдерживающие факторы (ограничения) системы и нацельтесь на один из них.
2. Решите, как использовать это ограничение (по сути, взломать систему).
3. Подчините все остальное предыдущему решению (лазерная фокусировка).
4. Уменьшите сдерживающий фактор (исправьте, замените или устраните ограничение).
5. После устранения сдерживающего фактора вернитесь к шагу 1.

В ТОС ограничение — это все, что мешает системе (вашей компании) достичь цели. Для Голдратта и Кокса такая цель — получение прибыли. Стоит отметить, что в ТОС сдерживающим фактором может быть вовсе не то, что «идет не так». Это может быть просто какая-то неэффективная, дорогая или ненадежная практика. Даже когда все идет гладко, где-то может быть сдерживающий фактор, заслуживающий внимания.

В мире API ТОС можно применить, например, для повышения опыта разработчиков, ускорения времени выхода на рынок, создания более совершенного дизайна API, соответствия требованиям к функциям продукта, внедрения более эффективных и действенных внутренних сервисов, повышения надежности и т. д.



Благодаря своему принципу лазерной фокусировки на ключевой проблеме, мешающей организации достичь своей цели, подход ТОС может хорошо подойти компаниям, которые не находятся под прямой угрозой на рынке и хотят применить постепенные улучшения.

Итак, к чему все это приведет?

Мы выделили несколько устоявшихся подходов к поддержке постепенных улучшений в вашей организации. Но они могут не соответствовать культуре и ценностям вашей компании. Тем не менее это не единственные способы внедрения непрерывного совершенствования. Мы привели три этих метода в качестве примеров и для поддержки, пока вы разрабатываете собственные стратегии — те, что будут соответствовать вашему организационному стилю. Мы оставляем

на ваше усмотрение разработку механики, но внедрение непрерывного цикла улучшений для вашего API — ключевое требование для создания продукта, способного поддерживать неизменно высокое качество.

Скорость изменения API

Если вы хотите вносить много небольших изменений в свой продукт, важно убедиться, что это можно делать быстро. Иначе стоимость таких изменений станет заметной проблемой. Независимо от вашего *текущего* темпа ускоренное внедрение улучшений позволит спонсорам вашего API сократить путь к инновациям и получить конкурентное преимущество на рынке. При этом качество ваших изменений должно быть на определенном уровне, иначе вы рискуете нанести ущерб надежности вашего продукта API.

Повышение скорости изменений и улучшение их качества важно для любого типа API. Если вы не будете успевать быстро вносить эти изменения в интерфейс, то в итоге замедлите свою способность запускать новые продукты, улучшать пользовательский интерфейс и бизнес-возможности. Оптимизация скорости и безопасности жизненного цикла изменений API сильно влияет на общую скорость изменений в вашей компании.

Но как вносить изменения в API так, чтобы *оптимизировать* эти ресурсы, учитывая, что у вас ограниченное количество ресурсов? Когда предложенное вами изменение проходит через все стадии жизненного цикла API, как убедиться, что вы движетесь с максимальной скоростью?

Есть три основных способа увеличить скорость жизненного цикла API, не жертвуя качеством: с помощью инструментов, планирования организационного процесса и уменьшения усилий.

Инструменты и автоматизация

Одно из решений для увеличения скорости и безопасности изменений — автоматизация человеческого труда. Инструменты — это привлекательный вариант. Он снижает вероятность человеческой ошибки и уменьшает время на выполнение задания. Например, инструменты «непрерывная интеграция и доставка» (CI/CD) могут автоматизировать тестирование и выпуск реализации API, значительно снизив затраты на внесение изменений.

Но польза инструмента зависит от его качества и времени, которое вы готовы потратить на его установку и настройку. Всегда будут значительные затраты и риски, связанные с новыми инструментами. Поэтому если продукт уже ста-

билен и активно используется (это фаза API, которую мы далее будем называть *осуществлением*), то их нужно вводить осторожно, на экспериментальной основе.

Все типы изменений API можно автоматизировать с помощью инструментов. На момент написания этой книги рынок инструментов безопасности, документации, развертывания и конфигурирования API процветал. Они обеспечивают более быстрые и надежные процессы изменений.

Планирование организационного процесса и корпоративная культура

Труд по внесению изменений в API может считаться умственным — работой, которая требует скоординированного процесса принятия решений. Если вы создаете один API в небольшой команде, усилия по координации часто очень малы. Но в масштабе крупной организации со множеством API и компонентов ПО большие затраты на скоординированное принятие решений уменьшают возможность отдельной команды изменить продукт.

Эта зависящая от человека часть процесса изменений — обычно самое слабое место на пути к высокой скорости. Причина в том, что его сложнее всего понять и изменить. Нельзя купить организационное планирование или корпоративную культуру так же, как документацию для API или инструмент CI/CD.

В главе 8 мы уделим больше времени организационным аспектам управления API, включая возможности создания платформы для принятия решений, способствующей быстрым и качественным изменениям.

Сокращение напрасных трат ресурсов

Еще один способ повысить скорость и качество улучшений — тратить на них меньше ресурсов. Избавившись от работы над API, которая хуже всего окупается с точки зрения достижения целей продукта, вы сможете сильно увеличить скорость изменений. Сокращение напрасных трат ресурсов уменьшает и вероятность непредвиденных ситуаций, делая процесс изменений надежнее.

Например, API, который создан и применяется одной и той же командой разработчиков, не требует такого же уровня вложений, как общедоступный API, используемый сотнями сторонних разработчиков. Но здесь нужно учитывать множество переменных. В главе 7, когда речь пойдет о жизненном цикле продукта API, мы введем систему переменных, которая станет отправной точкой для размышлений о том, какие инвестиции вы хотите сделать.

Изменения API

В первой главе мы обозначили разницу между составляющими продукта API: интерфейсом, реализацией и экземпляром. После публикации API нужно управлять изменениями всех этих составляющих.

Иногда придется изменять все вместе, но может оказаться, что вы будете изменять какие-то из этих элементов API независимо от других. Здесь мы рассмотрим влияние изменений в каждой из этих частей и даже добавим новый тип элементов API — «ресурсы поддержки». Они включают в себя части API, используемые только для повышения опыта разработчика.

Применение философии непрерывного, постепенного совершенствования к вашему продукту (со скоростью) означает целенаправленную разработку процесса изменений для всех четырех типов изменений API. Эти четыре типа изменений будут взаимно влиять друг на друга. Они формируют комплекс взаимозависимых изменений: изменения в модели интерфейса будут иметь далеко идущие последствия, а ресурсы поддержки можно легко изменять отдельно от остальных. Вы лучше поймете, почему существуют эти зависимости, когда мы исследуем каждый из типов изменений.

Жизненный цикл релиза API

ПО становится другим, когда в него вносятся изменения. То, что вы делаете для внесения правильных изменений в кратчайшие сроки и с лучшим качеством, называется процессом релиза.

Как и у ПО, у API тоже есть процесс релиза — ряд действий для внесения изменений. Мы называем этот процесс жизненным циклом из-за циклического характера этих изменений: пока одно осуществляется, следующее уже готово. Понимать, как протекает жизненный цикл релиза, очень важно, ведь он сильно влияет на возможность улучшения вашего API.

Каждое изменение, вносимое в API, должно быть реализовано. Жизненный цикл релиза — это набор действий, приводящих к его реализации. Он определяет, как изменение из идеи превращается в готовую и поддерживаемую часть системы. Жизненный цикл релиза объединяет все столпы из главы 4 в последовательность согласованной работы.

Если жизненный цикл выпуска будет медленным, скорость изменений вашего API уменьшится. Если в нем не гарантируется качество, изменять его будет более рискованно. Если жизненный цикл отклоняется от требований к изменениям, последние будут менее полезными.

Важно понимать, как протекает жизненный цикл релиза. Плюс в том, что у API он не отличается от жизненного цикла ПО или поставки компонентов системы. Поэтому вы можете применять существующие руководства для релизов программного обеспечения к компонентам API. Рассмотрим самые популярные из них.

Один из самых известных жизненных циклов ПО — традиционный *жизненный цикл разработки систем* (system development lifecycle, SDLC). Он существует с 1960-х годов и определяет набор этапов для разработки и выпуска системы программного обеспечения. Число и название используемых этапов могут меняться, но обычно этот набор выглядит так: подготовка, анализ, разработка, конструирование, тестирование, реализация и техническая поддержка.

Следуя по ступеням SDLC по порядку, можно разработать ПО по *каскадной* модели. На самом деле Уинстон Ройс изобрел не эту модель, но теперь этот тип жизненного цикла называют именно так. Это значит, что каждая фаза SDLC должна быть закончена до начала следующей. Так изменение проходит с верхней ступени через каждую следующую.

Один из недостатков каскадной модели в том, что нужен высокий уровень уверенности в требованиях и вообще в этой области, потому что эта модель не подходит для работы с большим количеством изменений в спецификациях. Если это проблема, вы можете использовать *циклический* процесс разработки ПО. Этот подход позволяет команде разработки производить несколько циклов релизов для одного множества требований. Каждый цикл выполняет подмножество требований, чтобы по завершении всех циклов все требования были выполнены. Эта модель соответствует подходам, рассмотренным нами в разделе «Постепенное улучшение» на с. 157.

Развивая идею циклов, мы приходим к *спиральному* SDLC. В этой модели ПО разрабатывается, конструируется и тестируется в циклических стадиях, где каждый цикл способен выполнить изначальные требования. В спиральном SDLC воплощается дух гибкой методологии разработки и метода SCRUM.

Мы рассмотрели три популярные формы жизненного цикла ПО. У каждой есть свои преимущества и недостатки, и нужно выбрать ту, которая подходит именно вам. В этой книге мы пытаемся дать вам возможность использовать любой стиль. Говоря об изменениях, мы будем иметь в виду жизненный цикл вашего релиза, но не будем говорить вам, в какой последовательности должны выполняться основные действия или какой жизненный цикл ПО вам использовать. Вместо этого сосредоточимся на улучшениях продукта, которые может обеспечить жизненный цикл релиза. Но сначала давайте подробнее поговорим о типах изменений API, которые придется поддерживать этому жизненному циклу.

Изменение модели интерфейса

У каждого API есть модель интерфейса. Это информация, описывающая поведение API с точки зрения *пользователя*. Она рассматривает набор абстрактных понятий, определяющих принцип работы API, и включает в себя сведения о протоколах связи, сообщениях и терминологии. Отличительная черта модели API — то, что она не воплощена в жизнь. Это абстракция, и ее невозможно использовать в компьютерной системе.

Хотя модель интерфейса нельзя задействовать с помощью ПО, ею можно поделиться с пользователями. Для этого модель должна быть сохранена или *выражена*. Например, можно выразить модель интерфейса с помощью квадратиков и линий на маркерной доске. Такую нарисованную модель нельзя *задействовать*, но она поможет вашей команде работать над дизайном API.

Конечно, модели интерфейса — это не только рисунки. Их можно выражать и с помощью языков, управляемых моделями, или даже с помощью кода приложения. Например, OpenAPI Specification — популярный стандартизированный язык для описания моделей интерфейса. Использование стандартизированного языка для моделирования дает вам бонус — систему инструментов, которая может уменьшить затраты на реализацию вашей модели.

Рисовать или составлять модель можно как угодно — нет никаких правил относительно уровня детализации или формата передачи модели. Но помните, что метод, который вы выберете, сильно повлияет на уровень детализации и описания. Маркерные доски и техника свободного рисования дают максимальную свободу мысли, но ограничены физическими размерами и возможностью реализовать модели. Языки описания API обеспечивают более быстрый путь к реализации, но ограничивают свободу жестким синтаксисом и терминологией.

Столп «Дизайн» жизненного цикла нашего API сосредоточен на создании и изменении модели интерфейса, поэтому большая часть описанной нами работы идеально ему подходит. Но она связана не только с дизайном. На самом деле большинство столпов жизненного цикла зависит от модели интерфейса. Это происходит потому, что они — тоже выражения этой модели.

Допустим, вы выразили модель интерфейса в виде рисунка на доске или с помощью языка OpenAPI — и так же будете выражать ее в коде своего приложения, документации API и модели данных. Когда интерфейс будет опубликован и разработчики начнут писать код, который его использует, они также будут выражать вашу модель в своих реализациях. Все эти выражения подразумевают отношения взаимозависимости. Вот почему изменения в модели так важны.



Предметно-ориентированное проектирование

Идея ПО, ориентированного на модель, где реализация — выражение этой модели, пришла к нам из подхода Эрика Эванса — предметно-ориентированного проектирования. Если вы еще не читали его книгу «Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем» (Эддисон-Уэсли), стоит сделать это!

У лучших API-продуктов согласованные модели интерфейса. Поэтому разработчикам не приходится разрешать конфликты между документацией и уже опубликованными API из-за различий выражаемых моделей. Это стремление к единообразию усложняет внесение изменений в модели интерфейса, потому что их нужно синхронизировать во всем продукте.

Использование согласованной модели не означает, что ваш код реализации и внутренняя база данных должны использовать ту же модель, что и интерфейс вашего API. На самом деле не лучшая идея использовать одну модель для вашего интерфейса, кода и данных. То, что лучше всего работает для пользователей вашего интерфейса, не обязательно будет так же хорошо для вашей реализации. Использование согласованной модели означает, что внутренние части реализации вашего API должны быть переведены в эту согласованную модель интерфейса, прежде чем достигнут поверхности API.

Изменения в модели интерфейса сильно влияют на систему, но они неизбежны для любого активно используемого продукта. Возможно, вам потребуется добавить поддержку новой функции, внести изменения для повышения удобства использования API или отказаться от части интерфейса, так как ваша бизнес-модель сильно изменилась. Из-за всех взаимозависимостей изменения модели интерфейса всегда способны повлиять на код в приложениях, задействующих этот API.

Влияние изменения модели интерфейса на потребителей API во многом зависит от уровня связи между их кодом и вашим интерфейсом. Если у ваших API слабая связанность, то даже при крупных изменениях эффект будет меньше. Например, использование API с событийно-ориентированным стилем или стилем гипермедиа имеет преимущество — более слабую связанность между клиентским кодом и API.

В случае событийно-ориентированной системы вы можете менять алгоритм соответствия шаблону, не внося никаких изменений в компонент, отправляющий события. API с гипермедиа может позволить вам совершать манипуляции с обязательными свойствами для инициирования работы, не меняя клиентский код, совершающий запрос.

Выбор подходящего стиля интерфейса может помочь уменьшить затраты на изменения его модели. Но за это тоже придется платить. Чтобы они работали, нужно создать подходящую инфраструктуру и реализации на обеих сторонах — клиентской и серверной. Обычно ограничения и окружение, в котором вы работаете, сужают ваш выбор. Например, разработчики, которые пишут клиентское ПО для вашего API, могут не иметь опыта написания приложений в стиле гипермедиа. В таких случаях придется просто смириться с высокой стоимостью моделей интерфейса.

Лучший способ уменьшить внешнее влияние изменений модели — вносить их *до* публикации интерфейса. Джошуа Блох, разработчик Java Collections API, сказал: «Общедоступные API, как бриллианты — вечны»¹. Как только вы открываете интерфейс для пользователей, вносить изменения становится сложнее. Мудрый владелец продукта API по максимуму загружает изменения в модель интерфейса на стадии дизайна, чтобы не платить за них после публикации.

Изменения в реализации

Реализация API — это модель интерфейса, выраженная с помощью компонентов, воплощающих ее в жизнь. Она позволяет другим составляющим ПО использовать этот интерфейс. Реализация API включает в себя код, конфигурацию, данные, инфраструктуру и даже выбор протокола. Эти компоненты — *скрытые* части продукта, они заставляют API работать, но мы не обязаны делиться их деталями с пользователями.

Невозможно опубликовать API без реализации, и ее придется постоянно менять на протяжении всего существования API. Так как реализация воплощает модель интерфейса, ее нужно менять при каждом изменении этой модели. Но иногда вы сможете менять реализацию независимо от модели. Например, чтобы исправить ошибку в коде реализации, уменьшить время ожидания медленного API или даже полностью переписать код.

Когда изменения в реализации не привязаны к модели, их эффект скрыт за интерфейсом API. Так пользователям не придется ничего менять, чтобы воспользоваться этими обновлениями. Это не значит, что на них это никак не *повлияет*. Например, оптимизация работы приложения может сильно сказаться на восприятии этой работы пользователем. Но так вы можете избежать управления изменениями в клиентском ПО, зависящими от вашего API. В общем,

¹ Bloch J. Joshua Bloch: Bumper-Sticker API Design («Джошуа Блох: Дизайн API для наклеек на бампер») // InfoQ, 22 сентября 2008 г. <https://oreil.ly/ibvwF>.

изменения в реализации могут быть сделаны и после публикации API, без изменений в модели интерфейса.

Риск внесения независимых изменений в реализацию в том, что они могут ухудшить надежность, единообразие и доступность продукта. Например, если изменения в коде портят работу программы API или реализация начинает отличаться от своей документации, ваши клиентские приложения пострадают. Изменения в реализации могут повлиять на экземпляр и ресурсы поддержки API, поэтому каждый из них надо согласованно обновлять, тестировать и одобрять.

Изменения в экземпляре

Мы уже говорили, что реализация API выражает модель в виде интерфейса, который можно использовать. Но это возможно, только когда она запущена на устройстве в сети, доступном для пользовательских приложений. *Экземпляр API* — это управляемое работающее выражение модели интерфейса, доступное для применения вашей целевой аудиторией.

Любые изменения в модели интерфейса или реализации потребуют соответствующих изменений в готовых программах. Изменения в API нельзя считать внесенными, пока вы не обновили экземпляры, используемые его потребителями приложениями. Но программу API можно менять, не затрагивая модель или реализацию. Это может быть простой случай изменения конфигурационного значения или что-то более сложное, например дублирование и удаление работающей программы. Такие изменения влияют только на свойства системы во время работы программы. Прежде всего на доступность, наблюдаемость, надежность и качество работы.

Для уменьшения влияния на систему независимых изменений в экземпляре нужно заранее продумать дизайн системной архитектуры. Мы обсудим системные функции и самые значимые факторы позже, когда будем знакомиться с ландшафтом API.

Изменения в ресурсах поддержки

Если API — это продукт, он должен быть не просто запущенным на сервере кодом, выражающим модель. Из первой главы мы узнали, что поддержка разработчиков, которые должны использовать наши API, — это важная часть философии «API как продукт». Создание позитивного опыта разработчика почти всегда требует каких-либо вспомогательных ресурсов за пределами реализации

интерфейса. В их число могут входить документация по API, регистрация разработчиков, инструменты для выявления неполадок и раздачи важных материалов и даже сотрудники техподдержки, которые будут помогать разработчикам решать проблемы.

В течение всего жизненного цикла API вам нужно будет обновлять и улучшать материалы, процессы и квалификацию сотрудников, поддерживающих продукт. Часто это будет результатом каскадного изменения модели интерфейса, реализации или экземпляра API. Ресурсы поддержки ниже по течению будут затронуты при изменениях любой части API. Так, затраты на изменения вашего API вырастут по мере того, как вы будете создавать больше вспомогательных ресурсов для разработчиков.

Независимые изменения можно вносить и в ресурсы поддержки. Например, изменить внешний вид страницы с документацией во время модернизации. Такие изменения сильно влияют на опыт разработчика вашего продукта API, но не на модель интерфейса, реализацию или экземпляр. Точнее, могут влиять, но не напрямую, а в результате того, что программой начинают чаще пользоваться и больше интересоваться.

Изменения во вспомогательных ресурсах вызывают наименьший каскадный эффект, но они могут привести к наивысшим затратам, потому что больше всех зависят от других элементов API. Снижение стоимости этих изменений может принести большие дивиденды с точки зрения общих затрат на изменения в продукте с API. Поэтому лучше вкладываться в дизайн, инструменты и автоматизацию, чтобы тратить меньше сил на изменение этих ресурсов.

Улучшение изменяемости API

Мы привели несколько веских причин для непрерывного улучшения жизненного цикла API. Вы увидели, что быстрое внесение небольших изменений — идеальный способ улучшить продукт API, и больше узнали о типах изменений и улучшений, которые для этого нужны. Но на практике сложно применить этот подход к API, потому что по мере усложнения интерфейса затраты на изменения возрастают.

Есть три основных типа затрат, связанных с изменением API, которые могут уменьшить степень изменяемости: трата ресурсов на выполнение работы, альтернативные издержки и издержки, связанные с изменением зависимых компонентов. Минимизируя их, вы сможете чаще изменять API. Больше изменений — больше возможностей для постепенного улучшения продукта.

Стоимость ресурсов

Самые очевидные затраты на изменение модели интерфейса, реализации, экземпляра или вспомогательных активов вашего API — это время, энергия и деньги. Их снижение увеличит возможность добавления новых улучшений в продукт.

Ранее (см. раздел «Скорость изменения API» на с. 162) мы говорили о важности быстрых изменений и определили, что сокращение усилий, инструментальных и организационных ресурсов поможет снизить некоторые затраты на эту работу. Но на самом деле увеличение скорости изменений — сложная проблема.

Число ресурсов для изменения API зависит от следующих факторов: сложности проблемы, компетентности сотрудников, разработки процесса изменений, сложности и качества реализации. Это длинный и не исчерпывающий список. К счастью, уменьшение затрат на работу — это основная цель профессиональной разработки программного обеспечения. Вам в помощь выпущены горы исследований, советов и экспертных мнений, а то, что полезно для ПО, обычно полезно и для API.

Определение конкретных методик изменений, процессов контроля качества, архитектур, реализаций и инструментов автоматизации, которые можно использовать для сокращения трат ресурсов, выходит за рамки этой книги. Мы попытались дать вам несколько основных стратегий, схем и методик, которые помогут добиться высокой скорости изменений, но превратить общие советы в конкретные методы для своей организации вам придется самим.

Альтернативные издержки

Другой вид затрат, который может замедлить перемены, — это желание отказаться от изменения API, чтобы сначала собрать больше информации. Том и Мэри Поппендик, создатели «Бережливого подхода к разработке ПО», описывают это как откладывание важного решения до *позднейшего ответственного момента*.

Что еще сложнее, вы должны учитывать цену невыполнения изменений и связанные с этим упущенные возможности для улучшения вашего продукта и для сбора отзывов об изменении. Лучше избегать принципа «последнего ответственного момента», чтобы не отвлекаться на переживания о том, что нужно еще подождать и получить больше информации. Внесение небольших изменений кода в опубликованном компоненте ПО — пример момента, когда решение можно

считать недостаточно важным для того, чтобы беспокоиться об упущенных возможностях. Это особенно верно, если вы получаете немедленную обратную связь об ошибке, а времени на устранение проблемы немного.

Многие типичные изменения в API-продуктах подходят под эти характеристики: они некритичны и легко восстанавливаются. Например, изменение вида документации API для человека: вы быстро узнаете о его успешной реализации, и его довольно просто откатить в случае неполадок. Но после реализации некоторых типов изменений API сложно восстановиться, и обращаться с ними нужно осторожно. Например, изменение модели интерфейса API, которое может иметь далеко идущие последствия. Важно правильно регулировать такие изменения и всегда учитывать потери из-за отсутствия достаточного объема информации.

Один из способов удешевить альтернативные издержки внесения изменений — лучше собирать информацию. В главе 9 мы познакомимся с таким качеством системы, как *видимость*, которое может значительно снизить эту стоимость.

Затраты из-за связанности

Когда речь идет об API, в особенности о *модели интерфейса*, основное препятствие на пути к простым изменениям — это *связанность* между API и его пользователями. Есть много стилей API, но какой бы вы ни выбрали, все равно появится зависимость или связанность между отправителями и получателями сообщений. Это сильно влияет на то, что можно изменить в API и когда это сделать.

API — это только канал для связи между модулями программного обеспечения. При общении между людьми используется общее понимание слов, жестов и знаков, облегчающее разговор. То же касается и компонентов ПО. Например, общее знание терминологии сообщений, сигнатур интерфейса и структур данных полезно при построении осмысленного взаимодействия между компонентами. Для изменяемости API важно, сколько таких правил общения жестко запрограммировано в коде опубликованного компонента. Когда семантика API определяется во время разработки, растут затраты на изменение интерфейса.

Связанность неизбежна и может существовать в разных местах и разных формах. На самом деле, когда вы слышите, что конкретный API «сильно» или «слабо» связан, нужно провести практически детективное расследование, чтобы понять, что под этим подразумевалось. Что сетевой адрес API где-то жестко запрограммирован? Или речь идет об изменяемости семантики и терминологии сообщений? Или о том, насколько легко можно создавать новые копии программ с API, не влияя на пользователей?

Например, событийно-ориентированную архитектуру (event-driven architectures, EDA) часто описывают как предлагающую слабую связанность отправителя и получателя событий. Но оказывается, что эта слабая связанность относится только к известным отправителю сведениям о том, какие компоненты получают его сообщения. На самом деле структура, формат или терминология сообщений о событиях могут создавать тесную связь и быть источником поломки для компонентов, потребляющих сообщения. Подробнее о влиянии стиля API на связанность читайте в главе 6.

Некоторые стили API особенно строги в плане того, что определяется во время разработки. Если вы создаете интерфейс в стиле RPC, то наверняка используете какой-либо *язык определения интерфейса*, четко документирующий его модель. Плюс четко определенной модели в том, что становится проще писать код — у API со стилем RPC обычно много инструментов, сильно облегчающих начало работы.

Проблема четко определенной модели интерфейса проявляется, когда нужно внести изменения в этот интерфейс. Пользуясь подходом непрерывного улучшения, вы можете обнаружить много возможностей для небольших улучшений вашего интерфейса. Но поскольку семантика API жестко запрограммирована в опубликованном коде, изменения в модели интерфейса потребуют соответствующих изменений в коде всех потребителей API.

Обычно мы стремимся предотвратить проблемы в приложениях-клиентах, зависящих от наших API. Но может оказаться, что надежность одних клиентов вас заботит меньше, чем других. Например, изменение API, которое повредит редко используемому приложению от сторонней компании, более оправданно, чем то, которое повредит мобильному приложению вашей организации, которым пользуются покупатели.

Нельзя однозначно сказать, какой уровень связанности нужен вашему API и когда вносить изменения. Если бы слабая связанность была бесплатной, ее бы применяли все, но долгосрочная ценность сопряжена с краткосрочными затратами, а создание API, хорошо справляющихся с изменениями, требует затрат на начальном этапе. Придется довольно рано принять решение о стоимости изменений и о том, какой тип API вам понадобится.

Помните, что низкий уровень изменяемости в сочетании с высокой стоимостью изменений в коде означает, что непрерывное улучшение модели API — нереалистичная стратегия. В лучшем случае ваши непрерывные улучшения будут ограничены изменениями модели интерфейса, которые не вредят приложениям-клиентам. Тогда лучше начать с подхода BDUF, прежде чем модель интерфейса начнет использоваться.

Разве все это не просто BDUF?

Если вы знакомы с Agile-манифестом, то можете подумать, что описанное в этом разделе — пример антипаттерна идеального дизайна на начальном этапе (Big Design Up Front, BDUF), которого специалисты по гибкой методологии стараются избегать. И да, и нет. Во-первых, мы понимаем ценность ограничения усилий по планированию (по времени и ресурсам) при разработке ПО. Как отмечается в гибкой методологии разработки (<https://oreil.ly/khE1R>), хотя «следование плану» важно, лучше отдать предпочтение «реагированию на изменения». И это ключевой момент в *управлении изменениями*: ценность следования плану.

Когда речь идет об API, такие изменения может быть сложно вносить из-за эффекта домино, который изменения оказывают на код приложения, которое их использует, особенно потребительский код API, написанный командами, контролирующими код службы API. Отличный пример — сторонние API, которые использует ваша организация. Вы не контролируете их разработку или реализацию и все же полагаетесь на то, что этот интерфейс есть и будет стабильным и надежным.

Попробуйте рассматривать свои API как «сторонние» для других команд в вашей компании. Когда придет время менять эти API, вы должны быть уверены, что поддерживаете стабильность и надежность. Имеет особую важность и надлежащее планирование. Оно позволит убедиться, что вы понимаете основную аудиторию API, правильно сформулировали ее цель и имеете общее представление о направлении, в котором должен развиваться дизайн для удовлетворения потребностей вашей целевой аудитории. Но как мы писали в разделе «Постепенное улучшение» на с. 157, не нужно прорабатывать *все* детали до начала работы. Держите в уме долгосрочную цель, пока выполняете шаги для ее достижения.

Важно помнить, что, снижая стоимость изменений, вы снижаете и потребность в практике BDUF. Часто расширенная деятельность по «планированию» сосредоточена на количественной оценке и снижении риска самих изменений. Усилия, направленные в это русло, могут легко затмить работу по тщательному изучению, документированию и определению новых функций, в которые и нужно будет внести изменения. Вот почему небольшие доработки ПО так ценны в ваших усилиях по управлению изменениями. Чем меньше изменение, тем меньше рисков и тем легче «отменить» изменение при столкновении с неожиданными проблемами.

По нашему опыту, успешны те компании, у которых есть четкое и последовательное видение конечной цели. Они управляют прогрессом и постоянно находятся в поиске новых доказательств, способных помочь им скорректировать свои краткосрочные ожидания. У организации, успешно занимающейся непрерывными изменениями, есть набор встроенных практик, подобных тем, что были упомянуты в этой главе.

Итоги главы

В этой главе вы познакомились с моделью непрерывного улучшения изменений и узнали, чем этот подход хорош для ваших API. Мы выделили четыре типа изменений в API: изменения модели интерфейса, реализации, экземпляра и ресурсов поддержки. Чтобы все это работало, мы подчеркнули важность скорости изменений и прошли по основным препятствиям изменяемости API, включая связанность клиентского кода с API.

Далее мы познакомим вас с моделью зрелости, которая поможет вам структурировать ваши непрерывные изменения в контексте постоянно развивающегося продукта API.

ГЛАВА 6

Стили API

Дизайн во многом зависит от ограничений.

Чарльз Имс

API — важный элемент дизайна в любой инфраструктуре, соединяющий компоненты в цифровом виде. API позволяют составляющим обмениваться данными, и если посмотреть на это так, можно понять, что на самом деле представляют собой API общего шаблона. Под словом «шаблон» мы подразумеваем общие коммуникационные взаимодействия, поддерживающие API. Заметьте, что это более высокий уровень абстракции, чем конкретные технологии, определяющие конкретные способы реализации.

Поскольку API — это такой общий шаблон, возникает вопрос, есть ли единственный верный способ разработки API. Но все немного сложнее.

Хороший пример — дебаты «REST против GraphQL», которые ведутся уже несколько лет. Если отбросить рассуждения о превосходстве одного подхода к API над другим, можно сразу понять, что в этом вопросе сравниваются вещи на разных уровнях. Давайте кратко рассмотрим эти уровни, потому что они дают нам отличный способ отличить шаблоны (которые мы называем стилями API) от технологий.

REST — это шаблон. Это значит, что нет «технологии» или «протокола» REST. HTTP — полезная основа для реализации этого шаблона. Но чтобы получить REST-архитектуру, которая может быть реализована, нужны типы мультимедиа (термин, используемый в интернете для обозначения полезной нагрузки, которой обмениваются через API).

С другой стороны, GraphQL — это технология, определяющая, как клиенты могут делать запросы к модели данных, управляемой на сервере. Она определяет,

что нужно для использования GraphQL API: форматы обмена и семантику того, как он обеспечивает работу GraphQL API.

GraphQL — не единственный способ превращения шаблона запроса в конкретную технологию. Но на сегодняшний день он один из самых популярных. Другие технологии, основанные на этом шаблоне запросов: OData в пространстве корпоративных IT и SPARQL с уклоном, ориентированным на исследования.

Это показывает нам важность различий между общим шаблоном проектирования API и конкретной технологией, являющейся способом реализации этого шаблона. Так мы можем более четко говорить либо об общем подходе к проектированию API, либо о конкретной технологии, которая используется для конкретного проектирования.

Мы называем эти подходы к проектированию стилями API. В следующем разделе мы рассмотрим пять основных стилей в пространстве API, их свойства и типичные области применения. Если вернуться к прошлому сравнению, то в основе этих стилей лежат два из пяти. Первый сосредоточен на ресурсах как наиболее фундаментальной абстракции API, а второй — на возможностях запросов как основной абстракции API.

API — это языки

Прежде чем мы углубимся в стили, давайте сделаем шаг назад и посмотрим, что же такое API на самом деле. Это язык, определяющий способы взаимодействия приложений. Как и любому другому языку, языкам API для работы нужны две составляющие: способы обмена отдельными сообщениями (можно представить это как предложения) и способы, с помощью которых обмен сообщениями превращается в осмысленный разговор (можно представить как общую цель ведения содержательного разговора) (рис. 6.1).

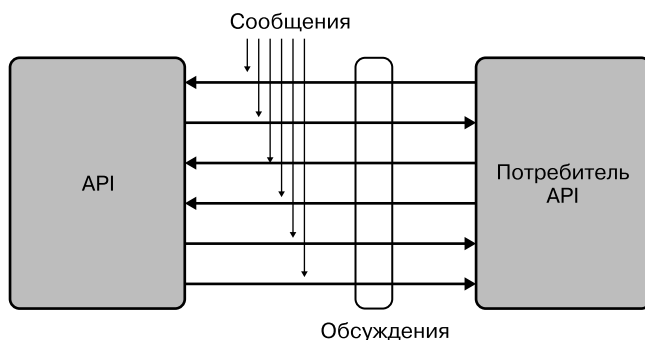


Рис. 6.1. API — это языки: сообщения и разговоры

Поскольку API — это языки в IT-пространстве, важно подумать и об основных абстракциях, на которых они основаны. Они проявляются в моделях и в элементах общения (обмен сообщениями).

Чуть позже мы внимательно рассмотрим основную абстракцию, лежащую в основе стиля API («первый принцип» стиля API), и базовые модели взаимодействия. Во всех этих случаях мы смотрим на то, как это проявляется для потребителя (который «видит» только API, а не реализацию) и для разработчиков (которым приходится разрабатывать код, реализующий API).

Давайте рассмотрим два простых реальных примера того, как проблема может быть важна для определения подходящего решения.

Для API, позволяющих отправлять данные, например заказ, лучше использовать стиль с традиционным потоком управления. Если API поддерживает процесс покупки, возможно, есть рабочий процесс запроса информации о продукте, предоставления информации о покупке, получения подтверждения покупки и предоставления информации о доставке. Все это хорошо работает в традиционных API с запросом/ответом. Стили, использующие этот шаблон, могут особенно хорошо подходить для представления процесса покупки чего-либо в качестве управляемого процесса.

Для API, уведомляющего потребителей о событиях, может быть полезно рассмотреть стиль с другим шаблоном взаимодействия. Например, если API может уведомлять потребителей об изменении адреса клиента, было бы полезно, чтобы он запускал событие, а потребители слушали его и получали уведомление, когда это происходит. Так им не придется проводить какие-либо опросы, а события будут распространяться и обрабатываться быстро и эффективно.

Важно помнить, что все стили могут использоваться для разработки и реализации API для обоих этих сценариев. Просто проблема, решаемая с помощью API, — важное ограничение, если дело доходит до принятия решения о выборе стиля и технологии. Как говорится, если у вас есть только молоток, любая проблема похожа на гвоздь. Если рассматривать стили как инструменты в вашем инструментарии API, то чем с большим количеством API вы работаете, тем вероятнее, что наличие более одного «инструмента стиля» поможет вам решить проблемы качественнее.

Конечно, есть и другие важные ограничения: ландшафт API, предполагаемая аудитория API (частная/партнерская/публичная), знания о предпочтениях потребителей и многое другое. Мы обсудим все это подробнее в разделе «Как выбрать стиль и технологию API» на с. 188, но сначала поговорим об отдельных стилях.

Пять стилей API

Стили API — это *шаблоны взаимодействия API*, основанные на модели взаимодействия и основных абстракциях API. Шаблон взаимодействия означает, что стиль API будет определять, как проектируется API и как этот дизайн будет реализован в конкретной технологии.

Один из наиболее важных аспектов стилей API — то, что в идеале проектные ограничения API, выбор стиля и технологии для реализации должны быть согласованы. Их несогласованность может привести к плохим дизайну (когда проектные ограничения и стиль не совпадают) или реализациям (когда стиль и технология не совпадают).

Пять стилей в этой книге были выбраны на основе моделей взаимодействия и популярных технологий в пространстве API. Конечно, можно было бы составить и другой список, но те стили, что мы здесь упоминаем, хорошо зарекомендовали себя в нашей практике API и послужили основой для лучшего понимания множества существующих технологий API.

Самые важные аспекты для каждого из стилей — модель взаимодействия и основные абстракции. Именно на этих темах мы и сосредоточились. Как мы обсудим чуть позже, ни один из них по сути не «лучше» и не «хуже» других. Все они имеют конкретную историю и мотивацию. Их пригодность зависит от ограничений конкретной задачи проектирования API.

Для каждого из стилей мы приводим рисунок, на котором видны их основные свойства: модель взаимодействия и основные абстракции. Еще мы опишем, как стиль соотносится с технологиями, приведя несколько хорошо известных примеров.

Туннельный стиль

Туннельный стиль уходит своими корнями в размышления о том, как раскрыть существующие возможности с точки зрения IT. Он восходит к таким идеям, как удаленный вызов процедур (remote procedure call, RPC), рассматривающий проектирование распределенных систем так, чтобы они «ощущались» как локальная система. Идея в том, что API определяется для существующих «процедур» (или любое другое название, которое среда программирования использует для вызова именованной единицы кода). После API становится простым расширением того, что в локальном сценарии программирования было бы простым вызовом именованной процедуры.

Туннельный стиль удобен для разработчика, так как на создание API будет затрачено совсем немного усилий. Основные абстракции этого стиля — процедуры, и часто они уже существуют. Можно использовать инструменты для предоставления процедур в виде API, тогда многие задачи по «созданию API» можно автоматизировать. По-прежнему должен быть некоторый уровень управления для обеспечения безопасности API, но это можно решить с помощью шлюза API.

На рис. 6.2 показана эта простая модель: API предоставляются реализациями, каждая из которых имеет свою единственную «конечную точку», где все открытые процедуры доступны в виде API. Вызовы этих процедур «туннелируются» через эту точку, откуда и происходит название стиля. Если потребители используют API в разных реализациях, они должны использовать их отдельные конечные точки.

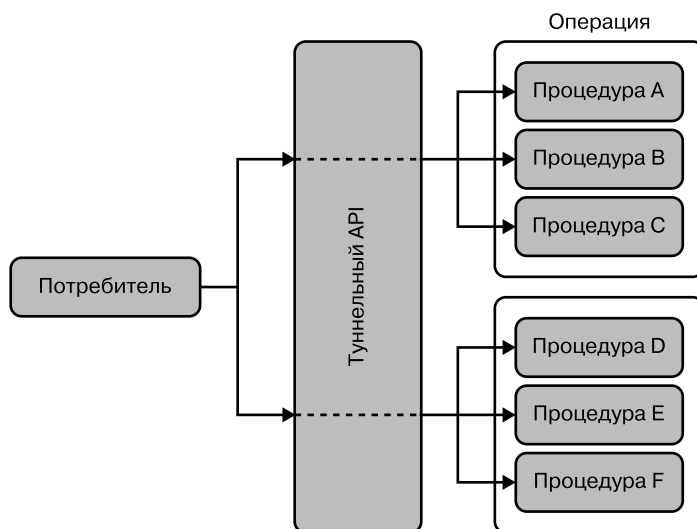


Рис. 6.2. Стили API: туннельный стиль

Одна из проблем в том, что у «конечной точки API» мало общего с реальным API, который она раскрывает. Это просто технический путь доступа («туннель»), через который должны проходить все вызовы, что может усложнить управление безопасностью и другими вопросами на сетевом уровне. Доступ к API за конечной точкой выглядит идентично. Это значит, что сложнее управлять API с компонентами, не встроенными в реализацию.

Хотя проблема управления API может рассматриваться как чисто техническая, в туннельном стиле есть более глубокая проблема: он в основном сосредоточен на

раскрытии реализаций. Это означает отсутствие этапа, на котором API сначала рассматривается с точки зрения потребителя, затем разрабатывается и реализуется так, чтобы API удовлетворял пользовательские потребности.

Туннельный стиль был выбран для первой волны «веб-сервисов» (конец 1990-х — начало 2000-х годов), которые использовали SOAP — протокол, основанный на XML, для удаленного вызова процедур. Нет ни одной причины, по которой SOAP в итоге не оправдал ожиданий. Но это не способствовало повышению уровня внедрения, ведь большинство конечных точек SOAP напрямую раскрывали детали реализации, которые было трудно понять и использовать потенциальным потребителям API.

SOAP (и другие протоколы туннельного типа) используют HTTP в качестве простого транспортного протокола для «туннелирования» к конечной точке. Это стало основной причиной, по которой проект оказался таким, — было несложно добавить эти конечные точки в конфигурации сетевой защиты HTTP, и поэтому предполагалось, что такой дизайн поможет внедрению.

Еще одно преимущество «туннельного» подхода — то, что SOAP и аналогичные ему протоколы могут быть туннелированы через разные «транспортные протоколы». Так отделы IT и службы безопасности могли постепенно переходить от одного транспортного протокола к другому, следя за их надежностью и безопасностью для использования в качестве туннеля.

Но приверженцы второй волны «веб-сервисов» начали смотреть на HTTP иначе. Они утверждали, что HTTP был разработан для взаимодействия с отдельными ресурсами (страницы, изображения и т. д.) и что стиль API, в большей степени соответствующий сети, был бы более подходящим способом разработки и реализации API. Так появился *ресурсный стиль*.

Ресурсный стиль

В отличие от туннельного стиля, ресурсный начинается с ориентации на потребителя. Основное внимание уделяется тому, какие ресурсы предоставлять потребителям, чтобы они могли взаимодействовать с ними. Слово «ресурс» здесь можно трактовать свободно. Можно предположить, что оно схоже по объему с тем, что вы имеете в ресурсах как веб-страницы при разработке сайта.

Могут существовать ресурсы для постоянных понятий (продукты, категории продуктов и информация о клиентах). Но могут быть и ресурсы для концепций, ориентированных на процесс (заказ продукции или выбор варианта доставки). В общем, все, что служит концепцией, которую нужно определить, так как она используется во взаимодействии поставщика и потребителя, превращается в ресурс.

Как показано на рис. 6.3, общая структура не сильно отличается от туннельного стиля. Но только если смотреть на нее с очень высокого уровня. Большая разница в том, как создаются компоненты на диаграмме. Тогда как процедуры в туннельном стиле просто раскрывают то, что определено в реализации, ресурсы создают модель, полученную с точки зрения потребителя.

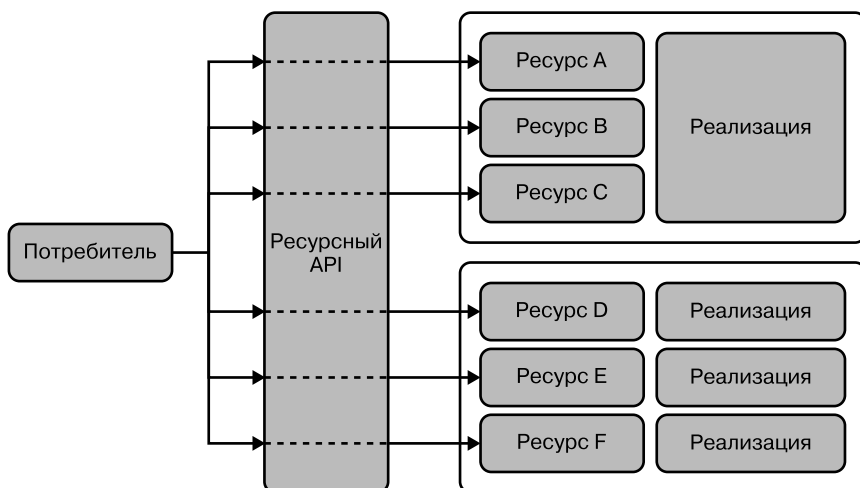


Рис. 6.3. Стили API: ресурсный стиль

Например, реализация процесса оформления заказа может иметь множество ресурсов для работы. Они вполне могут напоминать веб-страницы, которые вы просматриваете на многих сайтах, посвященных покупкам: вы выбираете товары, просматривая продукты и добавляя их в корзину, переходите к оформлению заказа, производите оплату и предоставляете информацию о доставке. Каждый шаг на этом пути — ресурс, с которым вы взаимодействуете, и разработка торгового веб-сайта означает сопоставление разных аспектов общего процесса покупок с ресурсами.

В хорошо разработанном торговом API эти шаги будут представлены отдельными ресурсами. Возможно, им понадобится некоторая информация для связи разных этапов (например, идентификатор корзины покупок и идентификатор заказа). Мы обсудим в разделе «Гипермедийный стиль» на с. 183, как сделать это более изящно.

Но помимо этого, потребитель будет использовать API на основе того, как их функция была разложена на отдельные ресурсы, подобно тому как процессы в реальной жизни отражают последовательность четко определенных взаимодействий.

Идея ресурсов позволяет нам раскрыть соответствующие аспекты функциональности API и в то же время скрывать детали реализации за ресурсами. Но этому стилю недостает возможности лучше отражать то, что часто между этими ресурсами есть рабочие процессы. Если все, что имеет значение, — это раскрытие ресурсов, то это не такая уж проблема. Но часто между ресурсами есть процессы (или другие виды взаимосвязей), и в этом случае гипермедийный стиль добавляет важный элемент к ресурсному для решения этих проблем.

Гипермедийный стиль

Гипермедийный стиль использует ресурсный стиль, добавляя к нему важный компонент сети: ссылки между ресурсами. В интернете можно перемещаться между ресурсами с помощью ссылок (вместо того чтобы знать каждый ресурс отдельно и вводить его URI в адресную строку), и гипермедийный стиль делает то же самое для ресурсов API.

Это значит, что на первый взгляд стиль гипермедиа похож на стиль ресурсов. Основные абстракции гипермедийного API — его связанные ресурсы, а сами ресурсы отображаются так же, как и в ресурсном стиле. Но существенным отличием стиля гипермедиа является еще одна фундаментальная абстракция — это ссылки между ресурсами, как показано на рис. 6.4.

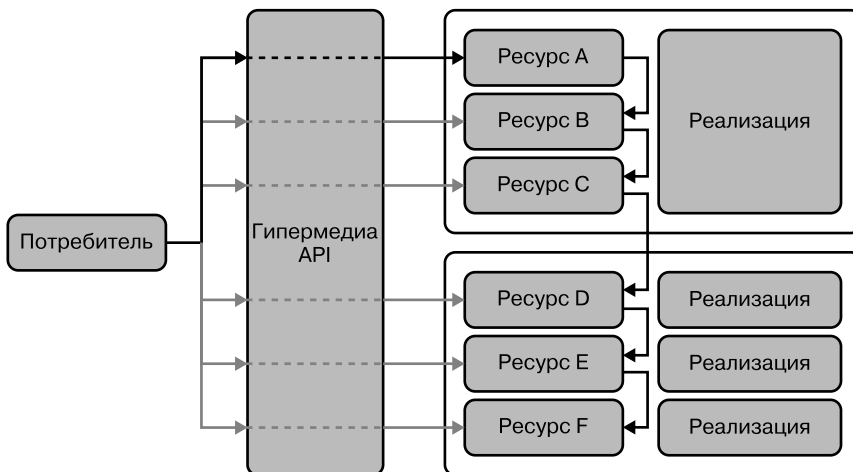


Рис. 6.4. Стили API: гипермедийный стиль

Поскольку мы упомянули интернет как известный пример гипермедийной системы, стоит указать на важное различие с API: в интернете люди читают страницы и решают, по какой ссылке перейти. В гипермедийных API это решение

принимается компьютером. Значит, ссылки должны иметь машиночитаемые метки, чтобы компьютер смог определить доступные варианты и сделать выбор. Эти метки похожи на текст ссылки, по которой люди переходят на веб-страницы, но они представлены в машиночитаемом виде ресурсов, который сейчас во многих случаях будет JSON.

В интернете вы можете «перемещаться» в браузере по ссылкам. То же самое можно делать в гипермедийном API, где вы можете «перемещаться» по ресурсам с помощью ссылок между ними. Чтобы понять принципиальное отличие от ресурсного стиля, просто представьте себе веб-сеть без ссылок: это было бы совсем не то же самое, не так ли?

Есть два основных преимущества гипермедийных API по сравнению с ресурсными, и оба они — следствие добавления ссылок.

Ссылки помогают в сценариях с «основными рабочими процессами», ведь в этом случае использование API становится вопросом перехода по нужным ссылкам для работы. Хорошо продуманный гипермедийный API всегда будет давать все ссылки, нужные для выбора следующего доступного шага.

Некоторые из них могут зависеть от контекста. Например, в API для покупок та часть рабочего процесса, где нужна информация о доставке, может предоставлять разные варианты в зависимости от клиента и других условий (например, заказанный товар и место доставки). Проектирование этих вариантов в API приводит к хорошему опыту разработчика (DX), так как сразу становится ясно, какой следующий шаг обеспечивает рабочий процесс.

Ссылки охватывают ресурсы, и неважно, предоставляются эти ресурсы одним API или несколькими. Это значит, что гипермедиа — отличный способ обеспечить единый и простой в использовании интерфейс для работы со всеми ресурсами, даже если они предоставляются разными API.

Как мы обсудим в главе 9, дизайн API и хороший DX (опыт разработчика) применимы не только к отдельным API. Они важны для всего ландшафта. Поскольку гипермедиа могут связывать разные API, разработчикам становится проще работать с несколькими сразу, когда они предоставляют ссылки, соединяющие ресурсы между ними.

Все это звучит здорово, и, конечно, верно, что успех интернета как очень большой и масштабируемой информационной системы указывает на то, что гипермедиа может быть хорошим образцом для подражания. Некоторые популярные API используют этот стиль, но он по-прежнему менее популярен, чем ресурсный.

Одна из причин в том, что разработчикам поначалу может быть сложно работать с гипермедиа. Для них традиционно считается, что код, который мы пишем, — это поток управления и что мы используем функции (см. раздел «Туннельный стиль» на с. 179) или ресурсы (см. раздел «Ресурсный стиль» на с. 181) на этом пути. Управление получаемыми данными требует изменения мышления и практики программирования, что может быть одной из причин, по которой гипермедийный стиль так медленно набирает обороты.

В стилях API, как и во всем, что касается технологий, нет единого решения, которое бы отлично подходило для всех проблем. Хотя у гипермедиа и есть некоторые полезные свойства, он может привести к «болтливым» API, требующим множества взаимодействий для получения доступа к необходимой информации. Если потребитель API с самого начала знает, чего хочет, более эффективным будет позволить ему сказать об этом. Именно эта идея лежит в основе стиля запросов, описанного далее. Он основан на модели, когда API дает доступ к сложному набору ресурсов и позволяет потребителю написать запрос для получения того, что ему нужно.

Стиль запросов

Стиль запросов сильно отличается от стилей ресурсов и гипермедиа, так как обеспечивает единую точку входа для доступа к большому набору ресурсов. Идея стиля запросов в том, что эти ресурсы структурированно управляются поставщиком API. Эту структуру можно запрашивать, а ответ содержит результаты запроса. На определенном уровне это можно рассматривать как аналог принципа работы базы данных. У них есть основная модель для хранимых данных и язык запросов, который можно использовать для выбора и получения части этих данных (рис. 6.5).

Как и в случае с БД, выбор модели данных и языка запросов может отличаться в зависимости от технологии. Но важно то, что каждый запрос API становится конкретным запросом, который должен быть интерпретирован и разрешен API. Поэтому эта модель сильно отличается от моделей ресурсов и гипермедиа, где ресурсы имеют довольно фиксированные представления, которые могут быть получены с помощью запросов API.

Одно из преимуществ стиля запросов в том, что каждый потребитель может запросить то, что ему нужно. Это означает, что с помощью хорошо составленного запроса можно объединить результаты, для получения которых потребовалось бы множество запросов в ресурсных/гипермедийных API. Но чтобы это работало, потребители должны хорошо понимать базовые модели данных

и запросов и доменную модель API (чтобы знать, что запрашивать в сложной доменной модели API).

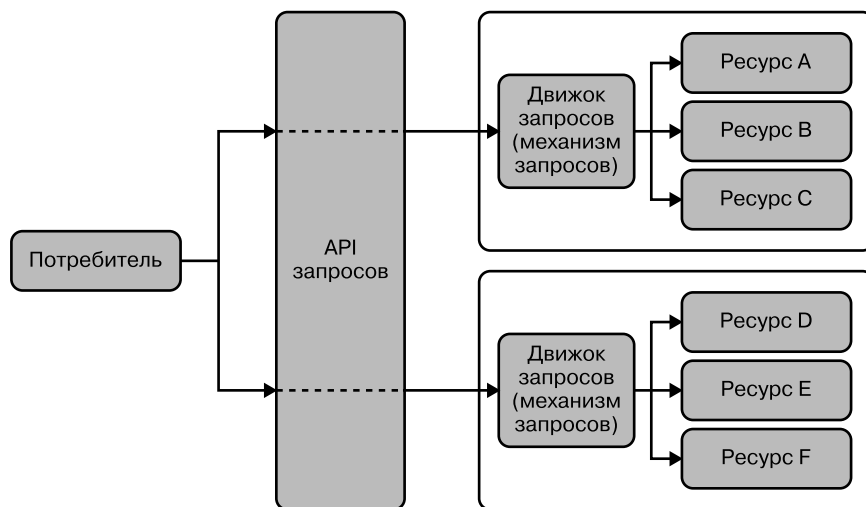


Рис. 6.5. Стили API: стиль запросов

Как мы уже говорили, нет «одного лучшего стиля для API» без учета ограничений, накладываемых на API. Учитывая сегодняшние тенденции в технологиях API, кажется, что API в стиле запросов особенно успешны, когда речь идет о создании одностраничных приложений (single-page application, SPA).

В таких приложениях используются частные API, применимые только в рамках одной организации между командой по разработке серверных приложений и разработчиками пользовательского интерфейса, работающими, например, над мобильными или веб-приложениями. В этом сценарии общие знания предметной области очень хороши, изменения в модели данных могут координироваться между командами, и вообще более высокая эффективность стоит больших усилий по координации.

Все описанные ранее стили (в «Туннельном стиле» на с. 179, «Ресурсном стиле» на с. 181, «Гипермедийном стиле» на с. 183 и «Стиле запросов» на с. 185) разделяют одно основное предположение: API используется по принципу запрос/ответ, когда потребитель посылает запрос и ожидает ответа. Этот шаблон полезен, когда потребитель служит отправной точкой взаимодействия. Но как насчет сценариев, где что-то происходит на стороне сервера, а потребитель хотел бы получить уведомление? В этом случае подойдет пятый и последний стиль, основанный на событиях.

Событийный стиль

В отличие от рассмотренных ранее стилей, основная идея этого в том, чтобы изменить шаблон взаимодействия. Вместо того чтобы потребители запрашивали что-то у поставщика, он создает события, которые доставляются потребителям API. Такая схема взаимодействия сразу же вызывает вопрос: как происходит эта доставка и откуда вообще известно, что потребитель заинтересован в получении определенных типов событий?

Эта проблема может быть решена только путем внедрения той или иной формы инфраструктуры. Это можно сделать по-разному. Иногда инфраструктура принимает форму шаблона «Публикация/Подписка» (PubSub), а иногда это более разделенный уровень, управляющий событиями по типам и позволяющий производить и использовать события на их основе. Общий шаблон показан на рис. 6.6.

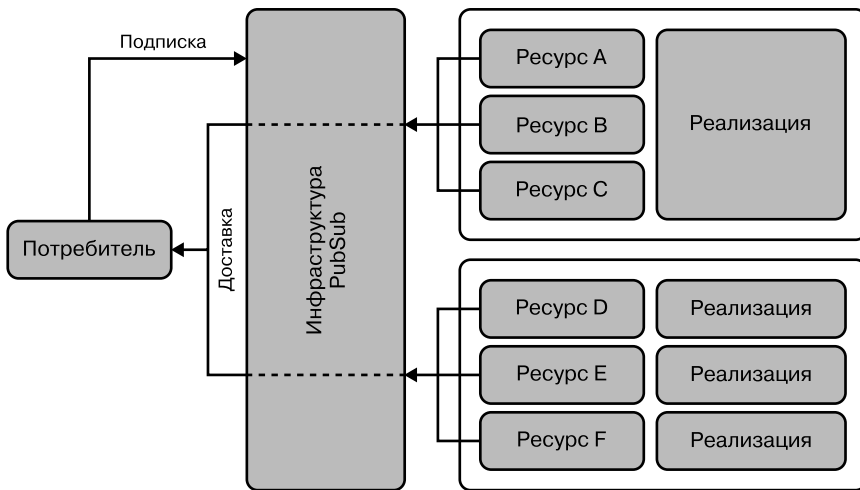


Рис. 6.6. Стили API: стиль, основанный на событиях

В целом идея этого стиля в том, что взаимодействие запускается событиями, поэтому идея API основана на них как основной абстракции. Есть два общих способа достичь этого в конкретных архитектурах.

Первый подход: потребители событий (клиенты в обычной терминологии API) напрямую связаны с производителями, и поток событий, генерируемых ими, доставляется потребителям. Иногда это может быть такой низкоуровневый процесс, как получение потока измерений от некоторого устройства, где каждое событие — это измерение, выполненное устройством. В этом случае подписка означает получение потока событий из конкретного источника.

Второй подход: потребители событий подключаются к структуре доставки (иногда называемой *посредником сообщений*), отделяющей их от производителей. Структура берет на себя управление событиями, а потребители должны подписываться на определенные типы событий, чтобы она могла убедиться, что события этого типа доставляются подписчикам. В этом случае архитектура гораздо больше сосредоточена вокруг структуры доставки, и все производители и потребители событий подключены к ней.

Как и в других стилях, здесь основные абстракции — процедуры (см. раздел «Туннельный стиль» на с. 179), ресурсы (см. разделы «Ресурсный стиль» на с. 181 и «Гипермедийный стиль» на с. 183) и схемы/запросы (см. раздел «Стиль запросов» на с. 185). Это значит, что при использовании событийного стиля все должно управляться событиями. AsyncAPI — это популярный язык описания, ориентированный на события (он называет их сообщениями).

Одно из интересных отличий этого стиля — лежащая в его основе архитектура. Все остальные стили по своей сути децентрализованы, так как предполагают синхронное взаимодействие между потребителями и производителями. В большинстве случаев использования событийного стиля на сегодняшний день он применяет структуру доставки (посредник сообщений), упомянутую ранее, полагаясь на централизованную инфраструктуру, с которой взаимодействуют все. Хотя современные продукты, такие как Kafka, обладают высокой масштабируемостью и отказоустойчивостью, это заметное отличие по сравнению с децентрализованными подходами, на которых основаны другие стили.

Как выбрать стиль и технологию API

Теперь у вас, вероятно, возник вопрос, как выбрать подходящий стиль и технологию, реализующую его. Мы ответим на него в следующих двух разделах.

Выбор стиля

Как обычно бывает, нет «единственного лучшего стиля», который можно было бы выбрать среди пяти рассмотренных нами. Все зависит от ограничений, а их можно разделить на три категории.

- **Проблема.** Как уже говорилось ранее, у каждого стиля есть определенная направленность и свои сильные стороны. Поэтому важно подумать о проблеме, которая решается с помощью API. Связана ли она с предоставлением доступа к структурированным и, возможно, сложным данным? Тогда стиль запросов подойдет. Возможно, она раскрывает процессы, по которым потребители должны иметь возможность перемещаться? В этом случае вам подойдет

стиль гипермедиа. Или это проблема, где происходят события, о которых потребители хотят узнать? Решить ее поможет событийный стиль.

- *Потребители.* Каждый API создается для потребления, поэтому потребители всегда должны быть важным аспектом проектирования. Поскольку API в идеале используются повторно, не всегда можно учесть всех потребителей и их ограничения, но лучше постараться проектировать с учетом хотя бы *некоторых* потребителей, делая предположения о других. Информация для пользователей может быть представлена в виде предпочитаемых стилей или технологий, но это может быть и вопрос о том, насколько простым должно быть понимание или использование API и какие сценарии будут способствовать их внедрению.
- *Контекст.* Большинство API — это часть ландшафта. Он может иметь разную аудиторию и сферу применения, в зависимости от того, предназначены ли API для частного, партнерского или публичного использования. Но во всех этих случаях важно учитывать более широкий контекст. Ведь цель API должна быть в том, чтобы стать хорошим API *в контексте его использования*. Поэтому, если ландшафт API поддерживает определенный стиль, это будет аргументом в пользу применения данного стиля для нового API, разработанного в рамках этого ландшафта.

Важно думать о выборе стиля как об одной из частей процесса разработки API, а «Дизайн-мышление» на с. 76 советует нам всегда помнить о потребителях. Поэтому, прежде чем приступить к проектированию API, подумайте, будет ли стиль соответствовать потребностям пользователя, и выберите соответствующую технологию, которая должна восприниматься так же.

Выбор технологии для стиля

Следующая задача после выбора стиля — выбор подходящей для него технологии. Как уже говорилось, у каждого стиля есть много вариантов, из которых вы можете выбрать.

Например, для стиля ресурсов есть REST как архитектурный шаблон, но это не значит, что он предоставляет вам конкретные технологии. Для REST выбор HTTP в качестве протокола — популярный, а в отношении формата представления можно точно сказать, что JSON намного превосходит другие представления (например, XML, который был популярен до JSON).

Что касается стиля запросов, то справедливо будет сказать, что GraphQL на сегодня — самый популярный выбор. Есть альтернативы, такие как SPARQL, который обычно используется в сценариях, основанных на технологиях, являющихся частью стека технологий Resource Description Framework (RDF, среда

описания ресурсов). Большое преимущество GraphQL в том, что он подключается к экосистеме, основанной на JSON. Хотя GraphQL не использует JSON для запросов, он возвращает результаты в формате JSON, что облегчает их обработку в средах, ориентированных на этот формат.

Что касается событийного стиля, то на сегодняшний день есть определенный импульс в пользу реализации *всех* API организации так. Как мы обсудим далее, это не единственный способ подхода, но идея, которая действительно имеет определенный импульс, и всякий раз, когда обсуждается этот подход, часто упоминается Kafka.

Хотя в этом случае Kafka часто превращается в важнейшую и центральную часть API-стратегии организации, можно также обрабатывать события в большей степени на основе отдельных API. Тогда можно использовать специальные протоколы, такие как Server-Sent Events (SSE, события, посылаемые сервером) или WebSockets, например для отправки событий в приложения, работающие на базе браузера.

Не загоняйте себя в угол стиля

Как и во многих других областях архитектуры, редко есть единственный лучший способ решения всех проблем в области проектирования. То же самое относится и к стилям API. Нет «лучшего» стиля. У всех есть сильные и слабые стороны, которые зависят от решаемой проблемы.

Одна из наших целей в этой книге в том, чтобы не просто рассматривать отдельные проблемы и решения. Мы не хотим фокусироваться на рассмотрении только одного API и рекомендовать, какой стиль (и технологию) использовать для него. Мы всегда хотим «уменьшить масштаб» и посмотреть на общую картину. Об этом мы поговорим в главе 9.

Реальность общей картины такова, что API постоянно развиваются, и проектирование ландшафта с учетом изменений — важный фактор. В прошлом мы видели довольно жесткие подходы в отношении стилей и технологий. Были ландшафты, ориентированные на SOAP (основанные на туннельном стиле), на HTTP (основанные на стиле ресурсов или, реже, на стиле гипермедиа), а в последние годы наметилась тенденция к созданию ландшафтов, ориентированных на GraphQL (основанных на стиле запросов). Самая последняя волна, по-видимому, пришла в виде EDA, часто вместе с Kafka, который использует событийный стиль.

Один из возможных подходов — не «выбирать» один из стилей (и технологий) в качестве единственного дизайна, а вместо этого убедиться, что ландшафт API имеет некоторое разнообразие. Эта тема часто поднимается в главе 2, где об-

суждается поиск баланса между внесением порядка и организации в ландшафт API, но без сильного его ограничения. Это непростое решение, заслуживающее отдельной книги.

Но, возвращаясь к стилям, которые мы обсуждали (и к начальной цитате о том, что «дизайн во многом зависит от ограничений»): для больших организаций ограниченность и попытки решить все проблемы с помощью одного стиля — редко хороший выбор. Вместо этого, если рассматривать стили API (и технологии) как функцию проблемы, которую решает API, будет легче создать ландшафт API с балансом разнообразия и согласованности.

Итоги главы

В целом стили API — это способ рассмотрения дизайна API, при котором меньше внимания уделяется конкретным техническим деталям и больше — общим шаблонам взаимодействия API. Мы рассмотрели пять стилей API вместе с их основными абстракциями и сценариями, для которых они хорошо подходят. Мы также обсудили, как выбрать стиль, соответствующий вашей проблеме, и перейти к выбору технологии, соответствующей этому стилю.

Наконец, мы вкратце обсудили взаимосвязь стилей API и разнообразия в вашем ландшафте. Если из этого раздела и можно сделать какой-то один важный вывод, то он будет такой: нужно взвешенно подходить к иногда разгорающимся спорам вокруг технологий API. API могут использоваться для предоставления самых разных возможностей и могут быть предназначены и разработаны для самых разных потребителей. Не загонять себя в угол стиля — важный момент, роль которого будет лишь расти по мере развития и вашего ландшафта API.

ГЛАВА 7

Жизненный цикл API-продуктов

Старение — обязательно, взросление — нет.

Приписывается Чили Дэвису

В деле управления API очень важно понимать роль изменений. Как говорилось ранее, есть разные типы затрат, связанных с изменениями API: траты ресурсов, альтернативные издержки, затраты из-за связанности. Общая стоимость изменения зависит от того, в какую часть API оно вносится.

Кроме того, стоимость изменений API динамична — с изменением контекста API меняются и затраты на него. Например, стоимость соединения неиспользуемого API близка к нулю, но стоимость соединения того же API с сотнями зависящих от него потребительских приложений была бы огромной по сравнению с этим.

В реальности управление API еще сложнее, чем в этом примере. Что, если у API есть только одно клиентское приложение, которым владеет основной партнер вашего бизнеса? А если зарегистрированных разработчиков сотни, но никто не приносит доход вашим основным продуктам? Что, если API приносит доход, но больше не подходит под вашу бизнес-модель? В каждом из этих случаев затраты будут разные. Есть сотни вариаций контекста, которые нужно учитывать. Из-за этого трудно дать общую оценку развития API для всех продуктов.

Несмотря на сложность этого проекта, неплохо иметь универсально применимую модель зрелости API. Во-первых, мы получили бы общий способ измерения успеха API. Во-вторых, появился бы шаблон управления ими на каждом этапе существования, особенно с точки зрения затрат на изменения. Поэтому мы попытаемся создать такую модель, которая подойдет всем.

В этой главе мы познакомим вас с жизненным циклом API-продукта и представим модель зрелости API. Вы узнаете о пяти стадиях, которые можно отнести

ко всем API. Чтобы они соответствовали контексту, мы представим способ определения ключевых моментов, подходящих вашим бизнес-стратегиям. Наконец, мы изучим, как каждая стадия жизненного цикла влияет на столпы работы с API. Но прежде, чем углубиться в тему жизненного цикла продукта, нужно определить способ измерения API-продуктов.

Измерения и ключевые этапы

Жизненный цикл API-продукта, с которым мы обещали вас познакомить в этой главе, состоит из пяти стадий, разграниченных ключевыми этапами. Эти моменты определяют входной критерий для API. Пока API будет развиваться от стадии создания через появление дохода до удаления, он будет проходить через ключевые этапы. Чтобы определить эти этапы для продукта, вам нужен способ измерения и отслеживания API.

В главе 4 мы описали столп управления API «Мониторинг». Создание системы сбора данных — важный первый шаг к измерению прогресса продукта API. Нельзя отследить прогресс, если вы не знаете, где находитесь. Сбор данных — техническая задача, которую можно решить с помощью качественных дизайнов и инструментов. Но для определения нужных данных для измерений жизненного цикла продукта необходим другой подход.

Вам понадобится выявить ключевые этапы, подходящие для ваших API, стратегии и бизнеса. Если вы делаете это сами, мы можем создать общий набор стадий жизненного цикла, которые можно применять к вашему уникальному контексту. Для создания этих моментов определите цели и параметры, подходящие для продукта. В этом разделе мы познакомим вас с двумя инструментами, которые помогут определиться с этим, — OKR и KPI.

OKR и KPI

В этой книге мы часто используем термин «*ключевой показатель эффективности*» (key performance indicator, KPI), говоря об измерении стоимости или качества чего-либо. KPI — это не магия, а просто красивый термин для описания особого способа сбора данных. Он описывает, насколько хорошо работает измеряемый объект. Сложность в том, чтобы определить как можно меньше измерений, дающих наибольшее количество информации. Поэтому такие параметры и называются *ключевыми* показателями эффективности.

Данные KPI полезны, поскольку, в отличие от общего сбора информации, отбираются тщательно. KPI должны помочь команде по управлению понять,

как обстоят дела у другой команды или продукта. Они обеспечивают ясность в отношении производительности измеряемой величины, чтобы помочь в оптимизации. Например, двумя ключевыми показателями эффективности для колл-центра могут быть число потерянных вызовов и среднее время ожидания для вызывающих абонентов. Часто оценивая эти данные и стремясь их улучшить, вы можете сильно повлиять на решения по управлению.

Если управленческие решения сильно зависят от показателей эффективности, то тщательный отбор данных крайне важен. Неточные метрики приводят к неудачным решениям, поэтому нужно выбрать правильные KPI. Кто-то должен определить главные факторы успеха организации и в соответствии с ними разработать параметры. Но как это происходит?

Некоторые компании используют *OKR*, чтобы определить свои *цели* (objectives) и *ключевые результаты* (key results), необходимые для их достижения. OKR помогают командам по управлению ответить на вопросы «Куда мы хотим двигаться?» и «Что для этого нужно?». В зависимости от того, кого вы слушаете, OKR либо тесно связаны с KPI, либо должны полностью их заменить. В любом случае они тоже полезны, так как представляют собой целенаправленную попытку связать многоуровневые цели в организации с необходимыми для прогресса результатами.



OKR в LinkedIn

Некоторые организации обнаружили, что OKR очень полезны в их стремлении к успеху. Например, генеральный директор LinkedIn Джефф Вайнер считает OKR важным инструментом для согласования командных и индивидуальных стратегий с целями организации. Он уверен, что OKR — это «то, что вы хотите выполнить в конкретный период времени, и это скорее приблизительная цель, чем заявленный план. Это то, что нужно срочно донести до пользователей»¹. Для Вайнера OKR полезны, только когда цели продуманы и непрерывно транслируются, располагаются по нисходящей и закрепляются.

Используя эти термины в книге, мы не стремимся убедить вас в необходимости OKR или KPI. Для успешного управления API вам не нужен консультант по OKR или KPI. Это полезные инструменты, но основное значение имеет корпоративная культура, а также подход к постановке целей и оценке эффективности. Мы решили применять эти термины, потому что знаем, что они — ключ

¹ The Management Framework that Propelled LinkedIn to a \$20 Billion Company («Система управления, которая помогла LinkedIn стать компанией стоимостью 20 миллиардов долларов») // First Round Review, 7 февраля 2015 г. <https://oreil.ly/yDkkd>.

к огромному количеству информации, советов и инструментов для тех, кто захочет углубиться в тему. Но самое главное — иметь четкие цели и измеримые данные, чтобы отследить прогресс вашего продукта.



Дополнительное чтение

Если вы хотите узнать больше о KPI и OKR, мы предлагаем начать с книги Энди Гроува «Высокоэффективный менеджмент» (High Output Management), которая легла в основу движения OKR. Если вам нужно что-то более поучительное, взгляните на книгу «Цели и ключевые результаты» (Objectives and Key Results) Бена Ламорта и Пола Р. Нивена.

Определение цели API

Цель, которую вы ставите для API, должна отражать стратегические цели ваших команды и организации. Она не обязана полностью совпадать с общей целью организации, но должна быть с ней связана. Определение цели API поможет компании продвинуться к своей цели. Если ваш API приносит ожидаемый доход, это выгодно организации. Отношения между целью API и задачами организации должны быть ясными и понятными.

Достижение такого рода согласования целей требует, чтобы вы что-то понимали в стратегии вашей организации. Но это в идеале. Если же нет, это должно стать вашим первым шагом. В мире OKR, где каждая часть компании определяет свои цели, связанные с общей, цели можно каскадировать.

Например, команда генерального директора ставит стратегическую задачу и определяет ключевые результаты, что позволяет бизнес-подразделению создавать цели, способствующие достижению этих результатов. У отделов внутри подразделения тоже могут быть свои цели, связанные с определенными результатами, и т. д. Итак, OKR могут спускаться через разные команды и разных сотрудников компании.

OKR — не единственный путь достижения такой связи между целями. Например, в сбалансированной системе показателей Роберта Каплана и Дэвида Нортона (<https://oreil.ly/HYlns>) есть похожий метод нисходящего расположения целей по показателям. Эта и другие похожие системы (<https://oreil.ly/fp9fA>) используются как минимум с 1960-х годов. Вы можете сами определить, как связать свои цели API с задачами организации. Важно, чтобы цели существовали и приносили доход вашей компании и спонсорам.

Правил относительно постановки целей для API нет, но в табл. 7.1 приведены примеры некоторых распространенных типов целей.

Таблица 7.1. Примеры целей для API

Тип цели	Описание
Использование API	Достичь определенного числа обращений за определенный период
Регистрация API	Достичь определенного числа новых регистраций или общего числа регистраций
Тип клиентов	Привлечь конкретный тип клиентов. Например, банк
Влияние	Позитивно повлиять на бизнес с помощью API. Например, увеличить процент заказов продукции
Идеи	Собрать определенное количество новых идей/моделей для бизнеса от сторонних пользователей API
Доходы	Новые доходы, связанные с бизнес-моделями API
Экосистема приложений	Количество приложений, использующих API и завершающих работу над продуктом
Внутреннее повторное использование	Число отделов или бизнес-подразделений, повторно использующих внутренний API

Их можно сочетать — например, задать цели и по использованию, и по типу клиентов. Но помните, что чем больше целей, тем сложнее оптимизировать дизайн под конкретную. Цель API движет вперед всю работу, которую вы для нее делаете. Но это не значит, что она никогда не изменится. Вам придется переоценивать цели, если поменяются установки всей организации или окажется, что ваша цель не приносит дохода.

Определение измеримых результатов

Цель полезна, только если степень ее выполнения можно точно измерить. Иначе это плохая цель или просто стремление. Управление API означает установление четкого набора измеримых целей и корректировку вашей стратегии в зависимости от вашего прогресса в их достижении. Для этого нужна продуманная разработка параметров или KPI вашего API.

Качественные метрики позволяют добиться качественных целей. Это значит, что наши измеримые результаты позволят достичь целей, которые мы уже определили. Постановка четких целей и целей, поддающихся измерению, сильно упрощает определение ключевых результатов или показателей прогресса. Но для начала нужно определить параметры этих измерений.

Если вы заинтересованы в определении качественных параметров данных, советуем прочесть книгу Дугласа Хаббарда «Как измерить что угодно» (*How to Measure Anything*). Она станет прекрасной отправной точкой для понимания того, почему и как надо измерять. Хаббард рассказывает, что цель измерения — помогать принимать решения в сферах, в которых вы не уверены. Именно это нам и нужно — мы можем знать свою цель, но сомневаться в своем продвижении к ней или в том, как измерить желаемые результаты.

В своей книге Хаббард определяет набор вопросов, которые вы можете задать себе, чтобы выяснить, какой тип измерительного «инструмента» вам нужен. Давайте применим эти вопросы к области измерения API.

- *В чем мы не уверены?* Хаббард рассказывает, что разделять объекты измерения очень полезно. Когда у чего-то, что вы хотите измерить, высокий уровень неопределенности, поищите способы разделить это на меньшие части, которые измерить проще. Например, вы хотите измерить степень удовлетворенности разработчиков вашим API. Здесь неопределенность очень велика. Но как разбить ее на части поменьше? Это возможно: запросы в техподдержку, рекомендации и рейтинги продукции — это измеримые параметры, которые можно использовать для определения уровня удовлетворенности пользователей.
- *Как это (или его части) измеряли другие?* По возможности учитесь на измерениях, сделанных другими. Если вам повезет и проблемная область будет та же, вы сможете воспроизвести эти измерения. Но даже если такой возможности не будет, изучение чужих измерений очень информативно и поможет вам с вашими собственными. Начните с лекции специалиста по стратегии API Джона Массера «KPI для API» (<https://oreil.ly/t70nQ>). Но, к сожалению, в публичном доступе не так уж много примеров измерений API.

Впрочем, многие из измерений, применяемых к API, имеют аналоги в других сферах. Любые измерения опыта разработчика могут быть вдохновлены общей областью измерения пользовательского опыта. Меры воздействия на бизнес могут быть взяты из мира OKR и KPI в целом. Измерения регистрации, использования и активности имеют параллели в сфере продуктового менеджмента. Таким образом, найти похожие примеры для решения своих проблем должно быть несложно.

- *Как наблюдаемые объекты поддаются измерению?* После разделения и определения других источников у вас должно сформироваться более ясное представление о том, что вы хотите измерить. Для ответа на этот вопрос надо определить, как это сделать. Например, для измерения числа запросов к техподдержке нужно отследить все каналы, по которым они приходят: электронную почту,

соцсети, телефон и личное взаимодействие. На этом этапе вы начинаете разрабатывать систему сбора данных.

- *Насколько действительно нужно это измерить?* С неограниченным бюджетом вы могли бы создать идеальную систему сбора данных. Но нужно ли это? Хаббард предлагает обдумать, насколько важна точная информация для принятия решений о наших продуктах API. Здесь решает контекст. Насколько важен этот API для вашего бизнеса? Как решения по управлению повлияют на организацию? Например, если вы разрабатываете API только для себя, вам может не понадобиться внимательно им управлять и вы будете мало вкладывать в точные измерения.
- *Где можно допустить ошибку?* Здесь нужно обдумать возможные погрешности ваших измерений. Есть ли противоречия? Влияет ли на результаты метод наблюдения? Цель — определить, где могут быть проблемы и как их решить. В сфере API они могут возникать из-за технических сложностей (передают ли инструменты точные данные?), отсутствия каких-то данных (отслеживаем ли мы все запросы к техподдержке?) или некорректного разделения (верные ли это измерения степени удовлетворенности разработчиков?).
- *Какой инструмент выбрать?* Под инструментом Хаббард подразумевает процесс или систему для непрерывного сбора данных. В сфере API это должен быть тот вид KPI, который надо измерять параллельно с отслеживанием разработанной для него реализации.

Вооружившись ответами на эти вопросы и примерами из других источников, вы сможете определить нужные измерения. Сделав это и проделав работу из раздела «Мониторинг» главы 4, вы будете готовы разрабатывать KPI для жизненного цикла продукта.

Жизненный цикл API-продукта

Общая модель развития продукта уже существует. Она называется *жизненным циклом продукта* и определяет четыре стадии — разработку, рост, зрелость и упадок, через которые проходят все продукты с точки зрения рыночного спроса. Мы взяли концепцию *жизненного цикла продукта* и применили ее к API, чтобы создать *жизненный цикл продукта API*. Он состоит из пяти стадий: *создания, публикации, окупаемости, поддержки и удаления* (рис. 7.1).

Как мы уже говорили, жизненный цикл продукта API — это модель, которая поможет вам отслеживать прогресс API и менять стиль управления по мере его развития.

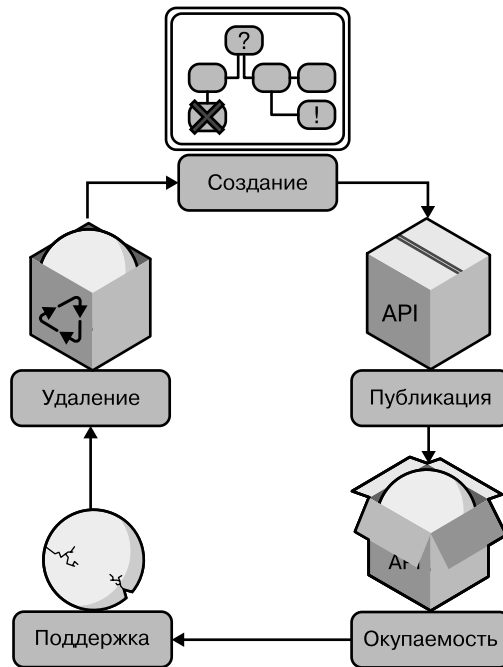


Рис. 7.1. Жизненный цикл API-продукта

В главе 5 мы описали жизненный цикл релиза API. Жизненный цикл продукта — это надмножество этих выпусков. Каждая стадия жизненного цикла API-продукта может содержать много отдельных релизов. Релизы и изменения не приводят к переходу продукта API на следующую стадию зрелости, но могут косвенно этому поспособствовать.

Далее мы подробно разберем все стадии жизненного цикла продукта и расскажем о том, что происходит на каждой из них, раскрывая ключевые моменты, которые нужно определить для их достижения.

Стадия 1: создание

API на стадии *создания* имеет следующие характеристики:

- это новый или заменяющий уже не существующий API;
- он не развернут в работающей среде;
- он недоступен для надежного использования.

У каждого API есть исходная точка — кто-то где-то в организации решает, что надо опубликовать API, а самого API еще нет. Есть много причин для разработки API, и на этой стадии нужно точно определить все движущие факторы. Вы хотите продавать доступ к этому API? Он ускорит разработку приложений? Или это просто канал для доступа к данным? Понимать, для чего компании нужен конкретный API, крайне важно для определения целей, ценности и целевой аудитории.

API на исходной стадии легко изменяются. Как мы узнали из главы 5, модель интерфейса труднее менять, когда она активно используется приложениями. На этапе создания API есть возможность вносить важные корректировки, не волнуясь о затратах из-за связанности. Расход ресурсов в это время также будет минимальным, потому что появление ошибок или дефектов мало на что влияет.

Но скрытые затраты на этапе создания включают в себя растущие альтернативные издержки, связанные с тем, что ваш продукт API не продвигается к следующему этапу зрелости. Это может произойти, если вы не публикуете API для пользователей, потому что хотите больше времени потратить на разные аспекты дизайна, пока можно безопасно вносить изменения. Но если от вашего API зависит работа других команд, организаций или сотрудников, его отсутствие может стать проблемой. Часто оказывается, что публикация хорошего API сегодня полезнее для бизнеса, чем публикация отличного, но завтра.

Продолжительность пребывания API на стадии создания становится важным решением по управлению продуктом. Нужно сравнить важность свободы при создании дизайна и растущую стоимость альтернативных издержек с увеличением затрат на связанность и усилия, приложенные на более поздних стадиях продукта. Проверенное правило — сначала разбираться с частями API, которые труднее всего изменять. Например, если вы создаете API на HTTP в стиле CRUD, нужно по максимуму разработать, протестировать и улучшить модель интерфейса на стадии создания. Ведь если модель не рассчитана на расширяемость, что часто случается, позже затраты из-за связанности резко возрастают.

На этом этапе важно собрать команду, которая станет развивать ваш API. Всегда можно добавить в нее сотрудников, если сложность продукта вырастет, но создание изначальной команды продукта — важный шаг. Размер, качество и культура производства в ней сильно повлияют на создаваемый продукт (см. главу 8). Нужно максимально точно подобрать эти качества на ранней стадии его жизненного цикла.

Ключевые моменты стадии создания

Жизнь любого API начинается с создания, но нужно решить, когда конкретно наступит этот момент. Как определить начало пути продукта API? Конечно, все может произойти само собой. Совершенно нормально децентрализовать решение о создании и позволить отдельным командам создавать конкурентоспособные продукты API. Но вы можете определить какой-то минимальный уровень осмоторности перед началом работы над API-продуктом.

Например, вы можете решить, что перед проектированием и разработкой для каждого продукта API должна быть определена стратегическая цель. Это потребует централизовать какую-то часть решений — скорее всего, одобрение. Или можно ввести правило, по которому любой сотрудник может создать продукт API, но работу можно начинать, только если он найдет еще троих человек, которые захотят потратить на него три месяца.

Определение ключевого момента создания зависит от контекста. Но важно иметь общее понимание того, что нужно для запуска жизненного цикла API. Это поможет избежать напрасных вложений в продукты, которые не стоят ваших усилий.

Методология: создание API с участием разработчиков-любителей

Большинство методологий проектирования API основаны на технологиях, где участвуют только технические специалисты, разработчики и архитекторы. Но в программы и проекты API вовлекается все больше бизнесменов, поэтому в процессе проектирования API появилась новая концепция. Теперь в нее входят не только разработчики, но и заинтересованные стороны бизнеса, которых мы можем назвать *разработчиками-любителями*.

Интересную методику разработал Арно Лоре в своей книге *The Design Of Web APIs* («Проектирование веб-интерфейсов API»).

Этот метод включает в себя поощрение всех заинтересованных сторон в процессе разработки API на этапе создания. Он заключается в том, чтобы вовлекать их в обсуждение. Заинтересованные стороны определяют потребности своего бизнеса простым языком в форме, которую будет легко перевести в технические термины:

Кто	Что	Как	Входные данные (источник)	Выходные данные (использование)	Цель
Кто пользователь?	Что они могут делать?	Как они это делают?	Что им нужно? Откуда они приходят?	Что они получают? Что они используют?	Какова конечная цель?

Ответив на следующие вопросы всей командой, с участием всех деловых и технически заинтересованных сторон, вы получите более глубокое понимание всей цепочки создания ценности API:

Кто	Что	Как	Входные данные (источник)	Выходные данные (использование)	Цель
Пользователь банковского приложения	Купить финансовый продукт	Поиск финансового продукта для подписки	Рынок финансовых продуктов	Продукт (подать заявку на подписку)	Поиск финансового продукта через изучение рынка
Внутренний разработчик	Обновить предложение продукции	Добавить продукт на рынок	Описание продукта, характеристики, значок, название, предоставленные менеджером по продукту	Описание продукта на рынке	Добавить продукт на рынок

Так заинтересованные стороны бизнеса могут участвовать в разработке спецификации API и предоставлять полезную информацию для технически заинтересованных сторон для согласования с машиночитаемыми спецификациями API.

Если подумать, то для REST API:

- «Что» — это ресурс, который будут обрабатывать, и путь к нему;
- «Кто» — это пользователь и функция авторизации и управления доступом API;
- «Как» — это глаголы HTTPs для управления ресурсами (получать/GET, отправлять/POST, помещать/PUT, исправлять/PATCH, удалять/DELETE, связывать/LINK);
- «Входные данные» — это поля API для отправки в качестве параметров или тела;
- «Выходные данные» — это ответ API;
- «Цель» — это история пользователя API, которая должна быть представлена.

С помощью этой методологии вы сможете создавать API и поддерживать постоянную связь между разработчиками и разработчиками-любителями. Члены команды, определяющие API в бизнес-терминах, и те, кто способен перевести их в технические, совместно работают над созданием API, соответствующего всем целям.

Конечно, это не единственная методология. Другие включают в себя разработку API, например APIOps Request and Responses Canvas (канва запросов и ответов на операции) и его API Design with Events Canvas (дизайн с помощью канвы событий), предназначенные для создания событийно-управляемых API.

Стадия 2: публикация

API на стадии *публикации* имеет следующие характеристики:

- API-программа развернута в рабочей среде;
- она доступна одному или нескольким сообществам разработчиков;
- стратегическая ценность API еще не окупилась.

Публикация API — это важный ключевой момент для продукта. Это точка входа во вторую стадию зрелости. API считается опубликованным, когда экземпляр становится доступным пользователям. Это момент, когда вы официально «открываете двери» в API и приветствуете всех заинтересованных.

Публикация невозможна без развертывания — действий по перемещению реализации API в один или несколько экземпляров. Но само по себе развертывание не считается публикацией. Например, можно разместить прототип дизайна API и на стадии *создания*, не объявляя о его готовности. Публикация происходит, когда вы сообщаете пользователям, что API открыт для бизнеса и готов к использованию.

Если вы создаете открытый API для сторонних разработчиков, то на этой стадии убеждаетесь, что они могут его найти и применять. Если ваш API — внутренний, это момент, когда его добавляют в каталог предприятия и доступ к нему открывается для команд других проектов. В случае API, поддерживающего одно приложение, именно на этой стадии вы сообщаете команде по разработке, что он стабилен и готов к использованию.

Открытие доступа для клиентских приложений — первый шаг к получению дохода от API. Но на стадии публикации этот лишь доход *потенциальный*. Если сравнивать с магазином, то вы уже открыли двери, но еще ничего не продали. Нельзя получить доход от API без публикации, но она еще не гарантирует его получение. Создание API еще не означает привлечение целевой аудитории.

Временной промежуток между публикацией API и получением дохода зависит от стратегии. Если цель его создания нереалистична, вы застрянете на стадии публикации. Когда цель проста, она будет достигнута при первом же применении. Окружение тоже играет важную роль. Если вы разрабатываете API для своего приложения, то лучше контролируете его судьбу. А API, созданный для сторонних организаций, потребует терпения и вложений. Но в любом случае

вы должны быть нацелены на то, чтобы как можно скорее перевести API на стадию *окупаемости*.

Но будьте осторожны, ведь изменения воздействуют на опубликованный API-продукт. Хотя на этой стадии вы можете влиять на зависимые от него клиентские приложения, эти клиенты еще не возмещают его стоимость вашему бизнесу. Поэтому, если вы вводите эффективное изменение, краткосрочной потери дохода не будет. Некоторые организации видят в этом возможность ставить больше экспериментов, собирать больше данных и сильнее рисковать.

Важно осторожно вносить изменения на стадии публикации, зная о возможных долгосрочных эффектах. Существующие клиенты могут принести доход, если продолжают использовать API, но избыток изменений способен оттолкнуть их еще до этого момента. Если продукт API существует на конкурентном рынке, это тоже может негативно повлиять на возможность привлечь нужные типы клиентов для реализации вашей стратегии.

Обычно в этот момент API легко изменить, потому что главные пользователи еще не активизировались. Но помните, что у изменения качества API на этой стадии могут быть неожиданные последствия, которые воспрепятствуют получению от целевой базы пользователей желаемых инвестиций. Программа API, которая часто не работает или меняет модель интерфейса, вредя клиентским приложениям, подает явные (и не положительные) сигналы потенциальной пользовательской базе.

Уровень изменений API на стадии публикации должен зависеть от самих API, их изменяемости, объема доступа (общий, частный или партнерский) и типов пользователей, для которых они предназначены.

Ключевые моменты стадии публикации

Ключевые моменты, которые вы выберете для стадии публикации, должны определять, когда API готов к активному использованию. Нужно решить, какой триггер будет обозначать эту готовность. Вот несколько примеров:

- API загружен в рабочую среду;
- сайт API заработал;
- API зарегистрирован в корпоративном реестре;
- открытие доступа к API анонсировано по электронной почте.

Важно и выбрать параметры, определяющие, что API уже используется. Это поможет выяснить, как желаемые изменения на стадии публикации повлияют на пользователей. Полезными параметрами могут быть реестр пользователей, количество обращений и просмотров документации.

Методология: пользовательские истории API для конечных потребителей клиентских приложений

Возможно, вы уже знакомы с концепцией пользовательских историй. В гибких методологиях это небольшая, автономная единица разработки, содержащая простое описание функции, рассказанное с точки зрения человека, который хочет получить новые возможности, обычно клиента системы. Она описывает, как достичь конкретной цели в рамках продукта, имеет следующий формат: «В роли [личность пользователя] я хочу [выполнить это действие], чтобы [я мог достичь этой цели]».

Клиентские истории API могут использовать тот же подход, но сосредоточиться на целях API — на том, чтобы соответствовать более чем одной истории конечного пользователя. Согласно проекту, описанному на этапе создания, цель в том, чтобы минимизировать число конечных точек. Это обеспечивает уровень простоты и согласованности и гарантирует, что API могут поддерживать более одной пользовательской истории, описанной для клиентского приложения. Общее правило в том, что у вас должно быть меньше историй пользователей конечных точек API, чем историй пользователей клиентского приложения.

Внутренние API. Если вы используете собственные API, то, вероятно, знаете истории конечных пользователей клиентских приложений, которые хотите реализовать. В этом случае опубликованные вами пользовательские истории API должны охватывать все потребности и возможности историй конечных пользователей клиентских приложений. Это хороший способ узнать, соответствует ли этап публикации требованиям внутреннего использования.

Одна история пользователя API — одна история пользователя. Простой способ написать свои пользовательские истории API — сопоставить их с каждой функцией, которую вы хотите включить в клиентских приложениях, использующих их:

История пользователя API	История пользователя
Как разработчик, я хочу получить доступ к учетным записям пользователей в LinkedIn, чтобы я мог их зарегистрировать	Как пользователь, я хочу иметь возможность связаться с контактом через данные его профиля LinkedIn
Как разработчик, я хочу позволить пользователям загружать фотографию, чтобы я мог отображать ее в пользовательском интерфейсе профиля	Как пользователь, я хочу иметь возможность выбрать фотографию своего профиля, чтобы меня можно было узнать в приложении

Распространенная проблема, на которую следует обратить внимание, в том, что ваши API будут слишком детализированы и слишком связаны с окончательным

интерфейсом приложений для конечных пользователей. Это не позволяет разбивать простоту и возможность повторного использования API.

Одна история пользователя API — много историй пользователя. Настоящая цель — сохранить простоту и согласованность конечных точек API и максимизировать их повторное использование во всех готовых к работе внутренних клиентских приложениях. Хотя вы можете начать с одной истории API для каждой истории конечного пользователя клиента, когда это возможно и обосновано, стремитесь, чтобы одна история API ссылалась на несколько историй конечного пользователя:

История пользователя API	История пользователя
Как разработчик, я хочу получить доступ к учетной записи пользователя LinkedIn, чтобы я мог запросить авторизацию с помощью OAuth 2.0	Как пользователь, я хочу иметь возможность подключиться к LinkedIn, чтобы я мог зарегистрироваться в два клика
То же самое, что и выше	Как пользователь, я хочу иметь возможность импортировать свои сообщения из LinkedIn в свой профиль
Как разработчик, я хочу позволить пользователям загружать фотографию, чтобы я мог отображать ее в пользовательском интерфейсе профиля	Как пользователь, я хочу иметь возможность выбирать фотографию профиля, чтобы меня можно было узнать в приложении
То же самое, что и выше	Как пользователь, я хочу иметь возможность обновлять фотографию профиля, чтобы люди из моих контактов могли взаимодействовать с моим профилем

Открытый API для третьих сторон. Если у вас уже есть внутренний API, который вы хотите опубликовать для других, или если вы создаете открытый API для доступа партнеров в экосистеме, нужен второй шаг. Определите новый набор историй клиентских приложений.

Действительно, партнеры по экосистеме, внешние потребители вашего API и сторонние разработчики могут захотеть использовать ваш API для создания разных типов функций (и разных приложений), отличных от вашего основного клиентского приложения. Для API потребуется задокументировать новый набор пользовательских историй экосистемы. Только на этом этапе публикации вы сможете по-настоящему углубиться в работу и узнать об этих функциях от внешних пользователей (с помощью опросов и информационно-разъяснительных мероприятий, направленных на изучение и оценку потребностей вашей экосистемы).

Истории пользователей API часто приходится переформулировать, чтобы они соответствовали новым историям внешних клиентских приложений.

Рассмотрим пример.

История пользователя API	История пользователя
Как разработчик, я хочу получить доступ к учетной записи пользователя LinkedIn, чтобы я мог запросить авторизацию с помощью OAuth2.0	Как пользователь, я хочу иметь возможность подключиться к LinkedIn, чтобы я мог зарегистрироваться в два клика
То же самое, что и выше	Как пользователь, я хочу иметь возможность импортировать свои сообщения из LinkedIn в свой профиль
Как разработчик, я хочу позволить пользователям загружать фотографию, чтобы я мог отображать ее в пользовательском интерфейсе профиля	Как пользователь, я хочу иметь возможность выбирать фотографию профиля, чтобы меня можно было узнать в приложении
То же самое, что и выше	Как пользователь, я хочу иметь возможность обновлять фотографию профиля, чтобы люди из моих контактов могли взаимодействовать с моим профилем
Как сторонний разработчик финансово-технологических приложений, я хочу получить доступ к фотографии пользователя для подтверждения личности	Как пользователь сторонних финансово-технологических приложений, я хочу иметь возможность подтвердить свою личность для создания учетной записи

Конечно, это не единственная методология, которую можно использовать. Но это обоснованный, практичный метод, который вы можете использовать для изучения своих пользователей на этапе публикации, что поможет вам перейти на следующий этап зрелости.

Как издатель API, вы сможете дополнять пользовательские истории, документирующие потребности пользователей, открывая новые функции и идеи продуктов из внутренних и внешних источников и постепенно раскрывая их ценность.

Стадия 3: окупаемость

API на стадии *окупаемости* имеет следующие характеристики:

- опубликованная программа с API создана и доступна;
- ее использование выполняет основную бизнес- или техническую задачу;
- доход от нее может возрасть;
- нарушение этого API повлияет на эффективность работы пользователей.

Думать о своем API как о продукте означает постоянно улучшать его для поддержки бизнес-целей. До этого момента ваш продукт API обладал *потенциальной* ценностью. Но когда целевая аудитория начинает его использовать способом,

соответствующим вашим стратегическим целям, вы наконец можете считать его ценностью *реализованной*.

Окупаемость API — это конечная цель повышения эффективности работы. Максимально быстрый переход к стадии *реализации* и долгое получение дохода — отличительные признаки API с высокой ценностью. Задача его владельца — решить, что именно означает окупаемость. Для этой стадии сложно определить параметры, потому что для этого владелец продукта должен прекрасно разбираться в целях API.

Правильное определение целей становится важным шагом к окупаемости — или как минимум к умению измерить свою способность создавать ценные API и управлять ею. Видимость и прозрачность ваших продуктов крайне важны, если вы управляете несколькими API одновременно. Поэтому измерения окупаемости имеют большое значение.

Например, целью окупаемости платежного API, который рекламировали сторонним разработчикам как платный продукт, могут быть 10 000 платных платежей за месяц. По этому параметру владелец продукта может четко понять, что даже если 6000 разработчиков зарегистрировались, чтобы использовать API, 5000 запросов на платежи в месяц означают, что он не окупился.

Но у API, применяемого только внутри одной организации, будет другая цель окупаемости. Например, для платежного API, применяемого внутри банковской архитектуры ПО, целью может быть проведение платежей онлайн-банкинга. В этом примере, как только система онлайн-банкинга начинает использовать этот API для платежей, он считается окупившимся вне зависимости от числа запросов.

Картину усложняет то, что цель окупаемости должна не только отражать контекст API, но и постоянно пересматриваться и проверяться с изменением этого контекста. Платежный API, который вы с выгодой для себя передаете сторонним разработчикам, потребует изменения цели окупаемости при изменении бизнес-стратегии, лежащей в его основе. Скажем, если вы решите, что долгосрочная стабильность требует от вас нацеленности прежде всего на корпоративный рынок, тогда ключевая цель окупаемости может измениться примерно на такую: «Провести 500 запросов на платежи в компании Fortune 500».

Ключевые моменты стадии окупаемости

Чтобы создать KPI, определяющий, достигли ли вы этой стадии, нужно иметь хорошее представление о том, для кого вы создаете API. Целевую аудиторию API будет несложно определить, если вы смогли поставить достаточно четкие цели. Это не значит, что необходимый вам пользователь должен подходить под конкретные параметры. Многие API создаются максимально гибкими, чтобы

подходить всем. Важно быть уверенным в типе пользовательского доступа, который означает, что вы реализовали API.

На этом этапе также полезно измерить уровень взаимодействия пользователей с вашим API. На самом деле основная цель API на этой стадии — обеспечить уровень заинтересованности, ведущий к использованию продукта на законном основании, что бы это ни означало в вашем случае.

API вошел в стадию окупаемости, но ваша работа на этом не закончена. Успех заключается в продолжении получения дохода от продукта. Для этого нужен набор параметров, способный отследить прогресс и в соответствии с полученными данными принимать решения по управлению продуктом. OKR и KPI, которые мы обсуждали ранее, лучше всего применять к API именно на этой стадии.

Методология: полотно интерфейса ценностного предложения

В своей книге *API Product Management* («Управление продуктами API») Андреа Зулиан и Амансио Боуза выдвигают идею думать о дизайне вашего API с точки зрения его ценностного предложения, а не просто как технического интерфейса. Вдохновившись канвой ценностного предложения Остервальдера, они создали канву интерфейса ценностного предложения, который поможет вам понять, достигли ли вы своей реализованной ценности.

Она состоит из метода работы по определению реальной ценности вашего API, того, как он соответствует потребностям пользователей и позволяет им получать прибыль. У этого метода две составляющие: профиль клиента и карта ценностного предложения.

- *Профиль клиента.* Здесь описываются задания, которые клиент хочет выполнить, и вытекающие выгоды и трудности, которые облегчают выполнение работы или препятствуют ее выполнению.
- *Карта интерфейса ценностного предложения.* Это карта соответствующих приложений, продуктов и услуг, данных и бизнес-процессов вашей компании. На ее основе можно определить компоненты, облегчающие боль и создающие прибыль, связанные с болью и выгодой клиента соответственно. Обезболивающие средства устраняют боль клиента, а создатели прибыли способствуют получению им прибыли.

Те, кто снимает боль, и те, кто создает выгоду, формируют ценностное предложение. Интерфейс — это интерфейс ценностного предложения (Value Proposition Interface, VPI), которым является API. VPI описывает интерфейс и ценностное предложение, а также то, как клиент может их использовать.

Пройдя следующие два цикла, вы поставите себя на место пользователя и оцените по очереди боль и выгоду от API.

Сначала вы будете отвечать с точки зрения *боли*.

- *Работа с клиентами*. Опишите работы, которые нужно выполнить клиенту.
- *Боли клиента*. Четко определите, почему они вызывают боль. Подтвердите эти боли с клиентом.
- *Источники ценности*. Перечислите соответствующие источники данных, приложения, бизнес-процессы и другие задействованные продукты и услуги.
- *Средства облегчения боли*. Перечислите функции вашего API-продукта, которые облегчат боль клиента.
- *Интерфейс ценностного предложения*. Переведите характеристики продукта в характеристики API (опишите ресурсы и методы API).

Теперь переделайте то же самое с точки зрения *выгоды*.

- *Работа с клиентами*. Опишите работы, которые нужно выполнить клиенту (как и ранее).
- *Прибыль клиента*. Четко определите, что может принести выгоду. Подтвердите эти выгоды вместе с клиентом.
- *Источники ценности*. Перечислите соответствующие источники данных, приложения, бизнес-процессы и другие задействованные продукты и услуги.
- *Создатели выгоды*. Перечислите особенности вашего API-продукта, которые приведут к увеличению прибыли.
- *Интерфейс ценностного предложения*. Переведите характеристики продукта в характеристики API (опишите ресурсы и методы API).

Как объясняют Зулиан и Боуза, важно убедиться, что выгоды — это не просто положительная сторона боли, а реальные выгоды и возможности, которые дает новый API.

Эта методология поможет вам отточить ценностное предложение API и максимизировать его реализованную ценность.

Стадия 4: поддержка

API на стадии *поддержки* имеет следующие характеристики:

- он активно используется одним или несколькими клиентскими приложениями;
- его окупаемость находится в стагнации или стремится к уменьшению;
- его не улучшают активно.

Пока API приносит доход, он остается на стадии окупаемости. Но однажды темпы роста замедлятся, и API перейдет в стабильную фазу или даже начнет меньше использоваться. Когда это случается, API переходит в стадию *поддержки*.

Сейчас API все еще нужно изменять, но цель изменений на стадии поддержки отличается от цели на стадии окупаемости. Теперь изменения вносят для поддержки стабильности API. Это могут быть исправления ошибок, модернизация и изменения ради соответствия каким-то требованиям, но на привлечение новых пользователей будет направлена очень небольшая их часть.

Вносить изменения на стадии поддержки нужно осторожно: убедитесь, что пользователям API, приносящим доход, эти изменения не повредят. Здесь лучше не рисковать. Для внесения значительного изменения вам может понадобиться перенести API обратно на стадию публикации и попробовать снова получить потерянный доход (иногда для этого выпускают новую версию API).

Ключевые моменты стадии поддержки

Они будут зависеть от ключевых моментов стадии осуществления и основываться на тенденциях. Например, если у вас уже есть параметры прироста числа пользователей для стадии окупаемости, то на этапе поддержки вам помогут соответствующие измерения прироста за последние полгода. Если количество пользователей не меняется, то, возможно, ваш API вошел в стадию поддержки. Нужно будет определить, какие параметры станут ключевыми показателями, за какой период и в какой момент начинается стагнация.

Методология: самообслуживание и автоматизация

На этапе обслуживания API реализовал свою производственную ценность и решает проблемы внутренних и внешних пользователей. Чтобы сохранить эту ценность как можно дольше, цель будет заключаться в снижении необходимых текущих затрат при сохранении ценности продукта API. Она достигается путем расширения возможностей самообслуживания со стороны потребителей и внедрения как можно большего количества автоматизации в бизнес-процессы.

Для потребителя подход к самообслуживанию будет заключаться в максимизации его автономии. Например, опытные разработчики смогут зарегистрироваться, безопасно поделиться своими учетными данными, прочитать документацию, протестировать API, предоставить свою среду и следовать пошаговым руководствам на основе сценариев использования без помощи человека. В компаниях, предоставляющих API с лучшим опытом для разработчиков, более 90 % пользователей успешно интегрируются без необходимости в индивидуальной поддержке.

Для поставщика цель будет заключаться в снижении эксплуатационных затрат на поддержание API в рабочем состоянии. Ее можно достичь путем взаимного обмена и автоматизации. На этом этапе API становится частью портфеля под управлением владельца API или менеджера продукта, который работает с несколькими API (в то время как на этапе реализации чаще всего один менеджер продукта работает с одним API).

Наряду с этим подходом все большее внимание уделяется автоматизации действий. Например, используя подход DevOps для API, или APIOps, вы можете протестировать дизайн, документацию, разработку и развертывание с помощью автоматизированной цепочки инструментов APIOps, снижающей необходимость ручной работы по исправлению ошибок, применению исправлений безопасности и установке обновлений.

На этапе техобслуживания целью будет поддержание работы API при максимальном соотношении стоимости и затрат. Самообслуживание на стороне пользователя и автоматизация рабочих процессов на стороне поставщика могут позволить вам оставаться в этой стадии зрелости как можно дольше, прежде чем затраты на обслуживание API не превысят создаваемую ценность. Тогда наступает время закрывать API.

Стадия 5: удаление

API на стадии *удаления* имеет следующие характеристики:

- опубликованная программа API создана и доступна;
- доход от нее уже не оправдывает поддержку;
- принято решение об окончании ее использования.

Все когда-нибудь заканчивается, и ваш API тоже однажды устареет. Он может войти в эту стадию по многим причинам, включая потерю спроса, изменение затрат на поддержку, появление новых, более подходящих альтернатив и смену целей вашего бизнеса. Все эти сценарии могут стать причинами невозможности поддерживать окупаемость или фундаментального изменения цели продукта API.

Вхождение API в эту стадию не означает, что он исчез, его *нужно удалить*. Команда этого продукта должна спланировать и выполнить удаление API из списка работающих и доступных. Она может определить цель удаления API, но обычно цель — максимальное снижение связанных с ним затрат. Иногда для этого нужно удалить все программы с этим API с производственных серверов, а иногда — просто отметить его как неактивный и перестать вносить изменения или предоставлять для него техподдержку.

Решение о необходимости удаления обычно зависит от затрат на этой стадии, включая лишение пользователей этого API чего-то полезного. Может быть сложно удалить API, от которого зависят другие. Владелец внутренних API может не иметь прав на удаление программы, так как последствия могут быть неожиданными. Если API открытый, организация может беспокоиться за репутацию своей торговой марки и доверие пользователей, которое можно утратить, удалив функцию.

С точки зрения продукта удаление API не нужно считать провалом или ошибкой. Это естественная часть цикла непрерывного улучшения всего ландшафта вашего API.

Ключевые моменты стадии удаления

Как и на других стадиях зрелости API, важно, чтобы команда продукта определила ключевые моменты, которые покажут, что API находится на стадии удаления. Они могут быть связаны с его работой (например, число сообщений, обработанных за какой-то период времени) или затратами на него (например, примерные затраты на улучшение API для достижения бизнес-целей).

Google известна тем, что удаляет продукты и проекты, не соответствующие измеримым параметрам за определенный период времени. У нее таким параметром может быть число активных пользователей (в сотнях тысяч), а ожидаемый прирост пользователей — довольно большим. Такие параметры важны для стратегии, требующей значительного прироста пользователей, но не подойдут внутреннему API для их идентификации.

Ключевые моменты для стадии удаления могут показывать нижнюю или верхнюю границу параметра. Вы можете установить минимальное число запросов, которые должен обработать API на стадии поддержки, или максимальный уровень затрат, по достижении которого продукт войдет в стадию удаления. Затраты на удаление продукта варьируются в зависимости от типов приложений, которые он поддерживает, и числа пользователей-разработчиков. Поэтому нужно самостоятельно установить эти границы, основываясь на уникальном контексте своего API.

Методология: удаление API без нарушения работы приложений с помощью метрик API

Внесение критических изменений, прекращение работы API — все, что означает возвращение разработчика к коду, чтобы приложение работало как следует, — пугает. Так и в ваших пользователей-разработчиков удаление API может вселить страх. Как поставщик, вы можете сделать это несколькими

наиболее безопасными и приемлемыми способами. Можно сделать это без уведомления, по любой причине и без обратной связи, но лучше сохранить доброжелательность и доверие своих пользователей, сделав все уважительно. Если все сделано верно, вы можете даже избежать сбоя любых приложений, работающих на вашем API.

Политика устаревания и отмены. Хорошая практика — заблаговременное оповещение пользователей о вашей политике устаревания. Что вы будете делать, когда API перестанет предоставляться? Пользователи вашего API хотят знать об этом сейчас. *Устаревание* — это объявление API не рекомендуемым к использованию или внедрению. Так часто случается, когда на замену создается новый API и мы объявляем устаревшим предыдущий. *Отмена* — это официальный выход на пенсию и закрытие API и его реализации.

Часто все начинается с объявления о том, что с определенной даты API будет объявлен устаревшим, с указанием веских причин и объяснением того, как заменить функциональность более новой версией. Такое сообщение дает время техническим и бизнес-командам понять, что делать, и составить планы. Часто публикуется план действий, где сообщается о том, как будет происходить прекращение работы.

Первым этапом может быть прекращение выполнения SLA или поддержки клиентов с низким уровнем оплаты. На портале документации API будет размещен предупреждающий баннер, информирующий о том, что «этот API будет устаревшим» и посетители будут перенаправлены по ссылке на ресурс с новой версией или заменяющим решением.

Вторым важным этапом станет прекращение поддержки всех клиентов. Некоторые компании даже помещают предупреждающие сообщения прямо в ответы API в документации, чтобы предупредить разработчиков о том, что произойдет.

Затем наступает этап фактического прекращения поддержки, официальное закрытие API, и он полностью выводится из эксплуатации.

Отслеживание использования с помощью метрик API для политики «написать один раз, запустить навсегда». Некоторые компании, как, например, Stripe или Salesforce, могут заявлять, что они никогда не будут ломать API или выводить их из эксплуатации. Они называют это политикой «написать один раз, запустить навсегда». Она обещает разработчикам, что им придется написать код только один раз, чтобы приложение API работало вечно. Чему мы можем научиться у них в отношении вывода API из эксплуатации? Основной способ управления этой политикой — поддерживать все версии в актуальном состоянии. И они действительно это делают! Но не все компании могут выдержать расходы на поддержку, поэтому есть и другой вариант.

Используя аналитику управления API, вы действительно можете узнать, какой пользователь или приложение использует ту или иную версию ваших API. Зная об этом, когда вы собираетесь вывести из эксплуатации версию API, вы можете предвидеть, на кого это повлияет и как. Затронет ли это ваших крупнейших клиентов? Затронет ли это критически важные внутренние приложения?

После составления карты этих воздействий вы сможете управлять отношениями по-человечески. Поговорите напрямую с заинтересованными сторонами, которые будут затронуты, и обсудите ваш план действий и альтернативы.

Если сообщить людям заранее и предложить новые версии или решения для замены, то вы увидите, как все больше пользователей API, подлежащего удалению, перейдут на вашу новую версию. Если вам повезет и у вас будет достаточно стимулов, то к окончанию срока эксплуатации может не остаться ни одного пользователя устаревшего API. Если у вас все еще есть пользователи API, которые не будут обновляться до новой версии, вам придется с этим разобраться. Ваше первое решение: продлить срок вывода из эксплуатации и смириться с тем, что это приведет к нарушению работы приложений этих потребителей. Но это может быть не самым дипломатичным вариантом.

Для внешних API можно увеличить стоимость поддержки этого API (как Microsoft увеличивала поддержку старых версий Windows для корпоративных клиентов). Это создает финансовые стимулы для компаний переходить на вашу новую версию.

Для внутренних API это может быть сделано с помощью технических средств, таких как прекращение обязательств по SLA, или через управленческое решение навязать обновление внутренним потребителям API.

Применение жизненного цикла продукта к столпам

Описанный ранее жизненный цикл API-продукта помогает выбрать пути развития вашего продукта. Это поможет при планировании затрат на изменения API на каждой стадии. Жизненный цикл продукта поможет управлять и работой, которую нужно проделать для вашего API. В этом разделе мы используем десять столпов разработки API-продукта из главы 4, чтобы показать изменение этой работы в зависимости от стадии жизненного цикла продукта.

Столпы работы по управлению API важны на каждой стадии цикла. У вас не получится проигнорировать ни один из них. Но на определенных стадиях

некоторые столпы важнее остальных (табл. 7.2). Они заслуживают большего внимания и, возможно, больших инвестиций на определенных этапах жизненного цикла продукта API.

Таблица 7.2. Влияние столпов в зависимости от стадии жизненного цикла API

Столп	Создание	Публикация	Окупаемость	Поддержка	Удаление
Стратегия	✓				✓
Дизайн	✓	✓			
Разработка	✓	✓			
Развертывание		✓	✓		
Документация			✓		
Тестирование	✓		✓		
Безопасность	✓				
Мониторинг		✓		✓	
Обнаружение		✓	✓		
Управление изменениями			✓		✓



Работа со столпами

Столпы, которые мы выделяем в этих подразделах, — не единственные аспекты, над которыми нужно работать. Вы будете вносить изменения и улучшения в API постоянно. Возможно, придется работать над всеми столпами на всех стадиях работы API. Наша цель — показать, какие из них сильнее влияют на разных стадиях, чтобы вы смогли грамотно инвестировать в них свои ресурсы.

Создание

На этапе *создания* вы сосредотачиваетесь на разработке лучшей модели API, прежде чем привлечь активных пользователей. Это потребует особого внимания к стратегии, дизайну, разработке, тестированию и безопасности.

Стратегия

На этом этапе прежде всего нужно развивать стратегию. Когда она будет разработана, вы получите еще очень мало отзывов об использовании продукта, потому что большая часть работы в это время посвящена дизайну и реализации. Из-за недостатка данных по стратегии она практически не будет меняться на этой стадии. Иначе будет, только если затраты на реализацию стратегии ока-

жутся слишком высокими. Например, вы можете понять, что создавать дизайн и реализацию, соответствующие вашей стратегической цели, будет непрактично. Тогда придется вносить изменения.

На стадии создания вы:

- разрабатываете изначальную стратегию;
- проверяете, практично ли создавать по ней дизайн и реализацию;
- обновляете свои цели и тактику в зависимости от их осуществимости.

Дизайн

В главе 5 мы обсудили сложность внесения изменений в модели интерфейса API после начала его активного использования. Именно поэтому дизайн модели интерфейса так важен на ранних стадиях жизненного цикла продукта. Если вы разработаете качественный дизайн во время создания API, у вас будет больше возможностей для доработки и улучшения на ранних стадиях.

Одна из сложностей разработки дизайна на стадии создания — большое количество допущений. Вы предполагаете, что принятые решения по дизайну модели интерфейса будут понятны разработчикам, и думаете, что этот дизайн будет практичен в реализации. К сожалению, это не всегда так.

Для получения лучшей версии дизайна интерфейса на такой ранней стадии вам нужно будет оценить модель. Понадобится получить обратную связь от группы внедрения о том, что ваш дизайн осуществим, — в идеале эта оценка включает в себя разработку прототипов, к которым можно делать запросы. Вам нужны и отзывы от разработчиков, представляющих целевую аудиторию.

На стадии создания вы:

- разрабатываете изначальную модель интерфейса;
- тестируете дизайн с точки зрения пользователя;
- оцениваете реализуемость этой модели.

Разработка

На стадии создания разработка заключается в реализации модели интерфейса. Как мы уже сказали, она может включать в себя и разработку прототипов для тестирования дизайна. Ее основная цель на первой стадии существования продукта — создать работающую реализацию, которая предоставляет все функции, описанные в модели интерфейса. Но для получения долгосрочной выгоды реализация должна также сокращать затраты на поддержку кода, данных и инфраструктуры и изменения в них.

На стадии создания вы:

- получаете прототипы;
- тестируете дизайн интерфейса с точки зрения реализации;
- разрабатываете первоначальную реализацию API.

Тестирование

На стадии создания вам нужно протестировать дизайн интерфейса и первоначальную реализацию. Это позволит выявить проблемы с удобством использования и улучшить дизайн API на ранней стадии. Как и при любом обеспечении качества, стоимость юзабилити-тестирования может варьироваться. Дорогостоящая версия может включать в себя лабораторные тесты удобства применения, фокус-группы, опросы и интервью. Бюджетная версия может предусматривать просто написание кода для API.

Нужный уровень инвестиций должен определяться предполагаемой выгодой, улучшая качество. Если вы работаете на высококонкурентном рынке API и у ЦА много вариантов выбора, то лучше инвестировать в удобство использования. Если вы разрабатываете внутренний API, стоит провести достаточное количество тестов, чтобы подтвердить свои предположения по поводу дизайна. Но в любом случае тестировать эти предположения необходимо, чтобы избежать роста затрат на изменения модели интерфейса.

Вы захотите протестировать и реализацию, но на стадии создания ее качество не так важно. API пока не опубликован для пользователей, поэтому можно отложить проверку реализации. Это не значит, что тестировать код на стадии создания не нужно. Внедрение таких методов, как разработка на основе тестирования, может улучшить качество реализации. Мы имеем в виду, что это решение нужно принимать, основываясь на своем рабочем окружении.

На стадии создания вы:

- определяете и осуществляете стратегию тестирования модели интерфейса;
- определяете стратегию тестирования реализации.

Безопасность

Когда дело доходит до безопасности, разумнее всего инвестировать в нее в течение всего срока службы API. Количество необходимых для этого усилий будет зависеть от ограничений, наложенных на вас сферой вашей деятельности,

правительством и конкурентным рынком. Но трудно представить сценарий, где не потребуется вообще ничего делать для обеспечения безопасности. Вам в любом случае придется работать над ней, чтобы защитить себя, свою систему и пользователей.

Большую часть работы нужно сделать *до* публикации API. Открыть двери в свою программу и только потом подумать о безопасности — неудачное решение. Поэтому мы выделили стадию создания как самую важную для столпа «Безопасность». Это может показаться нелогичным, но мы считаем, что работа по обеспечению безопасности на этой стадии важнее всего — она повысит ваши шансы на успех. Безопасность важнее всего для работающей API-программы, но ее фундамент закладывается при разработке дизайна и реализации.

На стадии создания работа по безопасности должна быть сосредоточена на применении политики безопасности к предложенному дизайну. Если в вашей сфере или организации нет четких требований, нужно определить их. Сейчас важно сделать безопасность вопросом первостепенной важности.

Работа по реализации на стадии создания должна включать в себя разработку безопасной инфраструктуры для вашего API. Сюда входят функции контроля доступа и чрезмерного использования, из-за которого ваша услуга может стать недоступной для законных пользователей. Все API достаточно важны и могут быть уязвимы. Компоненты, которые посчитали незначительными и не стоящими вложений в безопасность, позже становятся прекрасными мишенями для применения в незаконных целях.

На стадии создания вы:

- определяете требования к безопасности;
- тестируете модель интерфейса на соответствие этим требованиям;
- определяете стратегию защиты первоначальной реализации и готовых программ.

Публикация

Стадия *публикации* — это момент «открытия дверей» в ваш продукт API, с которого начинается его официальное использование. На этой стадии другие люди начинают зависеть от вашего API и писать код, основываясь на модели интерфейса, которую вы им рекламировали. Столпы, которые наиболее важны на этом этапе, — «Дизайн», «Разработка», «Развертывание», «Документация», «Мониторинг» и «Обнаружение».

Дизайн

Хотя бо́льшая часть работы над дизайном происходит на стадии создания, она важна и на стадии публикации, ведь позволяет улучшить дизайн интерфейса, основываясь на реальном применении. Только после публикации API вы поймете, верны ли были ваши предположения. Что-то обнаружится уже при тестировании на стадии создания, но вы узнаете много нового, когда реальные пользователи применяют ваш API.

Конечно, вы будете вносить изменения в интерфейс на протяжении всего существования продукта. Каждый раз, когда потребуется добавить новую характеристику, улучшить существующую операцию или упростить использование, вы станете менять модель интерфейса. Но такие изменения проще вносить на стадиях создания и публикации. Стадия публикации — это последняя возможность внести важные изменения в дизайн с минимальным вредом для системы или влиянием на пользователей, приносящих доход.

На стадии публикации вы:

- анализируете простоту использования интерфейса;
- проверяете предположения по дизайну, сделанные на стадии создания;
- улучшаете модель интерфейса, основываясь на полученной информации.

Разработка

Если вы меняете интерфейс, в итоге придется менять и реализацию. Но это не самая интересная часть этого столпа на стадии публикации. Мы выделяем его, потому что на этом этапе проще всего оптимизировать реализацию независимо от модели интерфейса. Это ваш шанс улучшить реализацию, чтобы она стала более производительной и ее было легче изменять и масштабировать.

Конечно, можно сделать это и на стадии создания, но публикация дает преимущество — вы можете основать оптимизацию на реальном использовании программы. В отличие от модели интерфейса, реализацию можно менять понемногу и постепенно. Так вы избежите большого объема работы по предварительному написанию кода. Вместо этого вы можете оптимизировать его по частям, узнавая больше о том, что нужно улучшить. Вы станете оптимизировать реализацию в течение всего жизненного цикла API, но на стадии публикации можно сделать максимум с минимальным риском.

На стадии публикации вы оптимизируете реализацию:

- с точки зрения масштабирования и работы программы;
- с точки зрения изменений;
- основываясь на наблюдениях за использованием.

Развертывание

API не может считаться опубликованным, если экземпляр не был развернут. Это ключевой столп стадии публикации. Как минимум нужно убедиться, что программа доступна пользователям, но было бы неплохо начать создавать инфраструктуру развертывания, которая будет поддерживать дальнейший рост. Это особенно важно, если стратегическая цель API включает в себя увеличение объема использования. Например, для достижения цели по доходам или инновациям может потребоваться архитектура развертывания, способная справиться с большим спросом.

Один из аспектов разработки — создание процесса, позволяющего вносить в API изменения (помните о важности поддержания высокой скорости). Разработка этого процесса должна в идеале начинаться на стадии создания продукта, но на стадии публикации он нужен уже более срочно.

Другой аспект разработки — ввод экземпляров API в эксплуатацию. Под ним подразумеваются создание и поддержка системы, соответствующей требованиям по масштабируемости, доступности и изменчивости вашего продукта. Хорошая ОС обеспечит доступность и производительность вашего API даже при росте потребности в системных ресурсах. Поддержка рабочего состояния программ — важнейшая часть формирования качественного опыта разработчика. API, который часто недоступен или медленно работает, вряд ли сможет добраться до стадии окупаемости.

На стадии публикации вы:

- развертываете программу API;
- фокусируетесь на открытии доступа к API;
- планируете и разрабатываете развертывание с учетом будущего спроса.

Документация

Работать с документацией придется в течение всего жизненного цикла API, но особенно важно это на стадиях публикации и окупаемости. На стадии публикации вам придется увеличивать доход от API, привлекая нужных пользователей. Это дает возможность поэкспериментировать с дизайном документации и придумать, что именно поможет их привлечь.

Это значит, что вы начинаете с низкого уровня зрелости документации, наращивая его в процессе изучения пользователей вашего API. Например, вначале можете предложить только техническую справку, но, основываясь на наблюдении за использованием, добавить обучающие курсы и примеры. Это позволяет разоблачать проблемные места или точки развития API в документации. Вы можете

найти их, инвестируя на стадии развития в пользовательское тестирование или изучая вопросы пользователей на стадии публикации.

На стадии публикации вы:

- публикуете документацию;
- улучшаете ее, основываясь на реальном использовании API.

Мониторинг

Отзывы о продукте очень важны на стадиях публикации и окупаемости. На этапе публикации вам нужны качественные параметры, чтобы определить, достигли ли вы ключевого момента реализации. На этапе окупаемости нужны данные, позволяющие убедиться, что спрос на API и доходы от него все еще растут. Мониторинг полезен на протяжении всего жизненного цикла продукта, но на этих конкретных стадиях он очень важен. Обычно на обоих этапах работают с одними и теми же параметрами. Поэтому если здесь вы вложите в качественный контроль, то позже сможете применить это решение еще раз.

На стадии публикации вы:

- разрабатываете и реализуете стратегические параметры API и мониторинг ландшафта API;
- создаете систему контроля, которую можно использовать и на стадии окупаемости.

Обнаружение

Обнаружение — наиболее ситуационно зависимый из всех десяти столпов. Работа по обнаружению — это то, что вы делаете для продвижения продукта API, создания отношений с разработчиками и общего укрепления связи API с ЦА. Если вы разрабатываете внутренний API, для его обнаружения нужно просто отправить e-mail. Если вы создаете его для крупного предприятия, обнаружение может означать отслеживание процесса приема и регистрации новых сервисов. Создание API для широкой публики может означать наем команды из десяти человек для создания и реализации маркетинговой стратегии. Поэтому вложения сил и средств могут очень сильно варьироваться.

Но в любом случае независимо от потраченных сил обнаружение важнее всего на стадии *публикации*. Именно тогда нужно максимально заинтересовать пользователей своим API, потому что у вас уже есть доступные экземпляры

и правильное использование поможет получить выгоду от продукта. Но как происходит обнаружение и сколько вы в него инвестируете, полностью зависит от вашей ситуации.

На стадии публикации вы инвестируете в маркетинг, отношения с пользователями и поисковую доступность API.

Окупаемость

Дойти до стадии *окупаемости* — цель любого продукта. Теперь главная задача — увеличить доход от API, не влияя на пользователей, которые больше всего вам в этом помогают. Самые важные столпы на этой стадии — «Развертывание», «Документация», «Тестирование», «Обнаружение» и «Управление изменениями».

Развертывание

Когда ваш API уже окупился, важно обеспечить его постоянную доступность и поддерживать его рабочее состояние. Поэтому здесь на первый план выходит архитектура развертывания. На стадии публикации вы создавали изначальный дизайн развертывания, а сейчас фокусируетесь на его поддержке и улучшении. Вы должны поддерживать рабочее состояние своей услуги с учетом изменения спроса. Внесение таких изменений может даже потребовать перепроектирования реализации. Это совершенно нормально, если вы можете защитить самых ценных пользователей от негативных последствий.

На стадии окупаемости вы:

- убеждаетесь, что API-программы всегда доступны;
- постоянно улучшаете и оптимизируете архитектуру развертывания;
- при необходимости улучшаете реализацию.

Документация

На стадии окупаемости можно продолжить повышать уровень опыта разработчика вашего продукта, в частности, улучшая документацию и обучение. Хотя на этом этапе становится все труднее изменить модель интерфейса, изменение документации оказывает гораздо меньшее влияние. Люди куда лучше приспосабливаются к изменениям, чем ПО, поэтому у вас есть возможность экспериментировать с новыми форматами, стилями, инструментами и презентацией. Цель — сохранить уровень применения программы, уменьшая пробелы в знаниях новых пользователей.

На стадии окупаемости вы:

- продолжаете улучшать документацию;
- экспериментируете с дополнительными ресурсами поддержки, например программами для анализа API, клиентскими библиотеками, книгами, видео;
- привлекаете новых пользователей, уменьшая пробелы в их знаниях.

Тестирование

На стадии окупаемости тестирование предотвращает негативные последствия для пользователей от изменений любой части API. Здесь использование API напрямую приносит вам доход. Изменения важны, но при этом нужно снизить риск нежелательных последствий. Размер вложений в такое тестирование должен быть основан на уровне негативных последствий.

В идеале тесты на стадии окупаемости должны быть разработаны еще на стадиях публикации и создания API. Но когда ваш продукт начинает входить в стадию окупаемости, нужно оценить стратегию тестирования и убедиться, что она обеспечивает оптимальный уровень снижения риска. Когда API достигнет стадии поддержки и удаления, необходимость в тестировании снизится. На этих стадиях вы сможете полагаться на уже созданные ресурсы.

На стадии окупаемости вы:

- применяете стратегию тестирования к изменениям интерфейса, реализации и экземпляра;
- непрерывно улучшаете способы тестирования;
- создаете решения по тестированию, которые можно использовать на следующих стадиях.

Обнаружение

Обнаружение на стадии окупаемости и на стадии публикации схоже. Отличие лишь в том, что здесь работа должна быть более точной. Вы станете лучше понимать, кто приносит вам больший доход, поэтому можете больше инвестировать в отношения именно с ними.

На стадии окупаемости вы:

- продолжаете инвестировать в маркетинг, отношения с пользователями и поисковую доступность;
- больше инвестируете в сообщества пользователей, приносящие выгоду.

Управление изменениями

В основе жизненного цикла API лежит растущее влияние изменений на него. Фактически на протяжении всего раздела мы описывали управление изменениями в каждом из остальных столпов продукта. Но сам столп «Управление изменениями» становится максимально важен на стадии окупаемости.

В главе 5 мы описали четыре типа изменений, которыми вам нужно управлять: изменения модели интерфейса, реализации, экземпляров и вспомогательных ресурсов. В рамках каждого столпа вы вносите изменения в некоторые из этих частей API, иногда одновременно. Ими нужно управлять, чтобы снизить негативный эффект. Важнее всего это делать, когда продукт активно используется и приносит доход. Здесь проявляется вся ценность системы управления изменениями и стратегии контроля версий.

На стадии окупаемости вы:

- разрабатываете и внедряете систему управления изменениями;
- сообщаете об изменениях пользователям, службе техподдержки и спонсорам;
- поддерживаете изменения для уменьшения влияния на доход.

Поддержка

На стадии поддержки вы уже не получаете нового дохода, но не должны вредить существующим пользователям. Ваша цель в этот момент — поддерживать систему в рабочем состоянии. Для этого нужно много работы, но самая важная задача — это мониторинг.

Мониторинг

Если API находится на стадии поддержки, ваша единственная задача — поддержание статус-кво. Меньше внимания уделяется дизайну, разработке и изменениям и больше — поддержке и доступности. В этот момент вам не обязательно улучшать мониторинг, потому что большая часть работы была проделана на стадиях публикации и окупаемости. Но это самый важный столп для стадии поддержки, поэтому важно потратить время и силы, чтобы убедиться, что вы получаете нужные данные на уровне системы и продукта.

Одна из задач — добиться того, чтобы система сообщала вам, когда случается что-то необычное. Это будет сигналом для того, чтобы предпринять какие-то действия. Другая цель контроля на стадии поддержки — следить за выгодой API. Когда она становится слишком низкой, его пора удалять.

На стадии поддержки вы:

- убеждаетесь, что система контроля работает;
- определяете схемы, которые потребуют особого внимания;
- наблюдаете за параметрами, которые могут показать, что пора принимать решение об удалении.

Удаление

Хоть это и последняя стадия жизненного цикла, помните: API еще существует. Сейчас вы решаете, что его нужно удалить. Самые важные столпы на этой стадии — «Стратегия» и «Управление изменениями».

Стратегия

Когда приходит время удалить API, нужно решить несколько конкретных стратегических вопросов. Как поддержать пользователей, компенсировать им потери и успокоить их? Есть ли новый API, который они могут применять? Когда и как будет удален API? Как сообщить пользовательской базе о запланированном удалении? Независимо от масштаба, контекста и ограничений API нужно будет создать стратегию удаления, хотя бы минимальную и неофициальную.

Для этого следует определить новые цели, тактику и план действий. Изначальная цель API уже не важна. Вместо этого нужна цель удаления продукта. Ею может стать снижение числа потерянных пользователей, если вы хотите перевести их на новый API, или скорейшее уменьшение затрат на поддержку продукта. Каждая из этих целей требует тактического плана и действий для их достижения.

На стадии удаления вы:

- определяете стратегию удаления (или перехода);
- определяете новую цель, тактический план и шаги для ее достижения;
- измеряете прогресс на пути к этой цели.

Управление изменениями

Управление изменениями на стадии удаления означает управление последствиями удаления продукта. Сейчас не время улучшать API, поэтому мы фокусируемся не на контроле версий или выпуске нового варианта. Нам нужно снизить влияние грядущего удаления на пользователей, ваши торговую марку и организацию и эффективно управлять этим изменением. Эта работа должна соответствовать стратегии удаления.

На стадии удаления вы:

- снижаете эффект от удаления API;
- разрабатываете и реализуете план сообщения пользователям и деактивации программы;
- управляете изменениями реализации и экземпляра, необходимыми для деактивации.

Итоги главы

В этой главе мы описали жизненный цикл API, состоящий из пяти стадий работы успешного продукта. Мы рассказали, что для определения уровня развития API нужны проработанные цели и параметры измерения. Описали, как управление одним продуктом API меняется на разных стадиях жизненного цикла. В следующей главе мы рассмотрим жизненный цикл API с точки зрения сотрудников и команд, работающих с ним.

ГЛАВА 8

Команды API

Великие достижения в бизнесе не совершаются одним человеком. Они совершаются командой.

Стив Джобс

Вы могли заметить, что мы откладывали на потом обсуждение создания и набора команд для работы над вашей программой API и управления ею. Эта тема важна, но собрать и обобщить информацию о таком индивидуальном для каждой организации понятии непросто. Каждая компания по-своему управляет сотрудниками, устанавливает границы внутри организации (подразделений, отделов, продуктов, услуг, команд и т. д.) и создает свою иерархию сотрудников. Эти различия усложняют создание единого набора рекомендуемых методик построения успешных команд API.

Но поговорив с представителями нескольких компаний, мы смогли определить общие схемы и методики, которыми можем поделиться. Все организации описывают необходимую работу и ответственных за нее сотрудников с помощью названий каких-либо команд, должностей и рабочих обязанностей. Единой системы в названиях *должностей* сотрудников в разных компаниях мы не нашли, зато обнаружили довольно схожий набор *обязанностей*. Другими словами, не важно, как называется должность, важно, что выполнять надо примерно одинаковую работу.

Идею о том, что нужно сосредоточиться на обязанностях, а не должностях, поддерживает разработчик ПО, писатель и преподаватель Саймон Браун: «Не-

возможно просто стать разработчиком ПО за одну ночь или после повышения. Это должностная обязанность, а не звание»¹.

Это относится ко всем обязанностям в команде API. Поэтому мы начнем главу с того, что мы называем общим набором ролей API. Примерно так же, как мы представили столпы API (см. главу 4), рассмотрим эти роли через призму распределенных задач, которые кто-то должен решать, и обсудим, как столпы API соотносятся с ними.

Мы обнаружили, что в некоторых случаях точный состав команды API может меняться в зависимости от зрелости API, над которым работают сотрудники. Например, на ранней стадии создания (см. раздел «Стадия 1: создание» на с. 199) вам не нужно уделять внимание тестированию или DevOps, а на стадии поддержки (см. раздел «Стадия 4: поддержка» на с. 210) обычно не так уж много работы для создателей клиентских и серверных приложений. Поэтому кратко рассмотрим, сотрудники с какими обязанностями могут понадобиться в течение всего жизненного цикла каждого из ваших API.

Еще один важный аспект — взаимодействие команд между собой. В большинстве компаний, с которыми мы работаем, есть отдел координации, или «команда по командам», которая помогает всем группам, отделам и подразделениям вне зависимости от стадии развития их API обмениваться информацией. Она управляет их взаимодействием и поощряет совместную работу. Дополнительный процесс «работы с сотрудниками» будет подробно рассмотрен позднее, когда мы позначим вас с нашим пониманием ландшафта API.

Наконец, для обеспечения продуктивного сотрудничества команд важна *корпоративная культура*. Мы обнаружили, что успешные компании вкладывают много времени и ресурсов в управление этой сферой. Как мы уже говорили, расширить руководство API можно с помощью распределения элементов принятия решений. Один из способов создать единообразие в этой области — обращать много внимания на корпоративную культуру и умело подталкивать ее в нужном направлении. В последнем разделе этой главы мы опишем несколько ключевых понятий, которые используют организации для определения корпоративной культуры, наблюдения за ней и влияния на нее с целью улучшения общей эффективности программ API.

Но начнем с определения набора распространенных обязанностей, которые мы видим в большинстве команд API, и их применения для выполнения всей работы по столпам API из главы 4.

¹ Brown S. Are You a Software Architect? («Вы архитектор программного обеспечения?») // InfoQ, 9 февраля 2010 г. <https://oreil.ly/GEznF>.

Обязанности в команде API

Так же, как мы показали набор общих *навыков* для работы с вышеупомянутыми столпами API, мы создали и список распространенных *обязанностей* тех, кто работает с API. Эти обязанности представлены в виде списка должностей. Но должности в командах API не очень стандартизированы в рамках компаний. Программный менеджер API в одной организации может называться владельцем API в другой, а архитектор API в компании В будет архитектором продукта в компании Z и т. д.

Поэтому названия должностей, используемые здесь, могут не совпадать с теми, что используют в вашей компании. Но мы почти уверены, что сами обязанности в вашей организации есть или как минимум *должны* быть. Потому что, как столпы API включают в себя все навыки для создания успешной программы, так и описанные здесь обязанности *нужно* кому-то выполнять.

Поэтому, читая список обязанностей в команде API и соотнесенных с ними должностей, вы можете сопоставить их с аналогичными внутри своей организации. Кстати, это очень хорошее упражнение. Если в своем списке должностей и их описаний вы обнаружите, что в нем нет одной или более описанных обязанностей, то у вас есть возможность улучшить описание должностей в организации так, чтобы все эти обязанности были охвачены.



Объем ответственности

Важно помнить, что каждая обязанность связана с определенным объемом ответственности. Выполняя обязанность, сотрудник берет на себя ответственность за все решаемые ею задачи, большая часть которых включает в себя принятие решений с определенным набором навыков (дизайн, разработка, развертывание и т. д.).

Взяв за основу это объяснение, давайте пройдемся по списку обязанностей, чтобы вы поняли, кто за что отвечает при создании работоспособной программы API. Можно сразу заметить, что мы разделили список на две части:

- бизнес-роли;
- технические обязанности.

Такое разделение может показаться предвзятым и не соответствовать разделению внутри вашей организации. Но мы считаем, что это поможет разобраться с тем, сотрудники с какими обязанностями занимаются достижением бизнес-целей (OKR), а с какими — выполнением технических задач (KPI). Мы обсуждали,

как эти два понятия соотносятся друг с другом и используются в управлении API, в разделе «OKR и KPI» главы 6.



Напоминание по поводу обязанностей и должностей

Помните, что должности, которые мы перечисляем, были придуманы, чтобы подчеркнуть связь между обязанностями в команде API и *столпами API* из главы 4. Обязанности и должности внутри вашей организации могут отличаться от них, но столпы API включают в себя все навыки и сферы ответственности по API, которые нужно охватить вашей компании. Как вы соотнесете столпы с должностями и обязанностями — вам решать.

Бизнес-роли

Первая группа обязанностей, о которой мы поговорим, — это так называемые *бизнес-роли*. Исполняющие их сотрудники сосредоточены на деловой стороне API. Обычно их сфера ответственности включает в себя защиту интересов клиента, связь продукта с четкими стратегическими целями (например, продвижение новых продуктов, улучшение продаж и т. д.) и соотнесение вашего API с OKR компании. Иногда такие сотрудники работают в отделах по развитию бизнеса или разработке продукта. Иногда они занимают должности, связанные с IT. Важная разница между этими обязанностями и техническими в том, что бизнес-роли связаны в первую очередь с бизнес-целями.

Мы определили пять бизнес-ролей по принятию решений, которые обычно существуют в командах API.

- *Менеджер API-продукта*. Менеджер продукта (МП) — иногда его называют владельцем продукта — основное контактное лицо API. Это связано с подходом «API как продукт» (АааР) из главы 3. Этот сотрудник несет ответственность за наличие у API четких OKR и KPI и за то, что остальные члены команды могут поддержать необходимые столпы API (глава 4). МП также отвечает за мониторинг API и его успешное сопровождение на протяжении всего жизненного цикла (глава 7). Обязанности менеджера продукта API в том, чтобы дать ответ на вопрос «Что делать?» (для описания работы, которую нужно сделать) и разъяснить его остальным членам команды. За ответ на вопрос «Как это сделать?» будут отвечать участники команды с техническими обязанностями. Также менеджер продукта ответственен за то, чтобы ожидаемый опыт разработчика (дизайн продукта, знакомство с ним и последующие отношения) соответствовал требованиям пользователей API. Обязанность МП — убедиться, что все части совпадают, как должны.

- *Дизайнер API.* Дизайнер API ответственен за все аспекты дизайна. Он должен убедиться, что интерфейс работает, его можно использовать и опыт разработчиков после взаимодействия с ним будет положительным. Дизайнер должен убедиться, что API помогает команде достичь определенных бизнес-OKR. Иногда он сотрудничает с техническими специалистами, чтобы удостовериться, что дизайн помогает команде достичь и технических KPI. Он также является контактным лицом для клиентов и может стать «голосом клиента», помогая команде принимать решения об облике API. Наконец, дизайнера могут попросить убедиться, что общий дизайн соответствует установленным рекомендациям по стилю для всей компании.
- *Технический писатель для API.* Он отвечает за создание документации по API для всех заинтересованных лиц, связанных с продуктом. Это не только пользователи API (например, разработчики, использующие конечный продукт), но и участники внутренней команды, и другие заинтересованные лица в организации (например, директор по информационным технологиям, технический директор и т. д.). У большинства писателей есть техническое образование и опыт программирования, но не у всех, и это не всегда нужно. Для технического писателя важно эффективно взаимодействовать с другими людьми, результативно искать информацию и брать интервью, ведь им нужно понимать точку зрения как производителей, так и пользователей API. Поэтому часто технические писатели плотно сотрудничают с дизайнером и менеджером продукта, чтобы убедиться, что документация точна и соответствует указаниям по стилям и дизайну внутри компании.
- *Евангелист API.* Евангелист API отвечает за продвижение и поддержку практики и культуры API в компании. Это особенно важно в больших организациях, где внутренние пользователи не могут быстро получить доступ к создавшей продукт команде. Евангелисты должны убедиться, что все внутренние разработчики, задействующие API, понимают его и могут достичь своих целей. Они отвечают за сбор отзывов от пользователей API и их передачу остальным членам команды. В некоторых случаях евангелисты могут нести ответственность за создание образцов, демоверсий, материалов для обучения и других ресурсов поддержки, предназначенных для улучшения опыта разработчика.
- *Специалист по отношениям с разработчиками (ОР).* Иногда этого сотрудника называют адвокатом разработчиков или специалистом по ОР. Обычно он сосредоточен на внешнем использовании. Как и евангелисты API, специалисты по ОР отвечают за создание образцов, демоверсий, обучающих материалов и других ресурсов для продвижения продукта. И опять же, как и евангелисты, зачастую именно специалисты по ОР ответственны за получение отзывов от пользователей и преобразование их в исправление ошибок или появление новых характеристик. Но в отличие от внутренних евангелистов, эти специалисты часто получают задание «продать» продукт API

более широкой аудитории. Поэтому они выезжают в организации клиентов, проводят предпродажные мероприятия и осуществляют непрерывную поддержку продукта для важных клиентов. Их дополнительные обязанности могут включать в себя выступления на публичных мероприятиях, ведение блога или написание статей о применении продукта и другие действия, направленные на достижение установленных бизнес-целей.

Хотя в основном эти пять должностей связаны с бизнес-целями и стратегиями, большинство из них по-прежнему полагаются на определенный уровень технических знаний и навыков. Следующий список обязанностей сосредоточен только на технических аспектах создания, развертывания и поддержки продукта API.

Технические обязанности

Второй список обязанностей, которые мы определили, — *технические обязанности*. Они относятся в основном к техническим деталям реализации дизайна API, его тестирования, развертывания и поддержки в рабочем состоянии. Обычно сотрудники, выполняющие эти обязанности, ответственны за озвучивание решений IT-отдела, в том числе касающихся безопасных, масштабируемых и защищенных реализаций, которые можно должным образом поддерживать. Зачастую технические специалисты отвечают за достижение важных KPI и за помощь бизнес-специалистам в достижении их OKR.

Несмотря на то что мы разделили список ролей на две группы, есть кое-что общее между бизнес-ролями и техническими обязанностями. Например, у должности менеджера продукта есть параллель в виде должности ведущего инженера API с технической стороны. Цель обеих групп — создание и развертывание технически стабильного и экономически оправданного API-продукта.

Мы определили шесть технических должностей, принимающих основные решения в области реализации, развертывания и поддержки успешных API.

- *Ведущий инженер API*. Это основное контактное лицо по всем вопросам, связанным с разработкой, тестированием, контролем и развертыванием продукта API. Эта должность — эквивалент должности менеджера продукта, только с технической стороны. Так же, как менеджер продукта отвечает на вопрос «Что делать?», ведущий инженер API отвечает на вопрос «Как это сделать?» — что нужно, чтобы создать, разместить и поддерживать API. Ведущий инженер ответственен за координацию действий других технических специалистов команды.
- *Архитектор API*. Архитектор API отвечает за архитектурные детали дизайна продукта и за то, чтобы API мог легко взаимодействовать с необходимыми системными ресурсами, включая API других команд. Он отвечает за всю

архитектуру ПО и систем всей организации, в том числе ограничения по безопасности, параметры стабильности и надежности, выбор протоколов и форматов и другие нефункциональные элементы, установленные для систем ПО вашей компании.

- *Фронтенд-разработчик приложения (фронтенд-разработчик)*. Он отвечает за обеспечение приложением качественного пользовательского опыта. Для этого нужно внести приложение в реестр API компании, на портал для пользователей и совершить другие действия, связанные с клиентской или пользовательской частью API. Как и дизайнер, фронтенд-разработчик должен защищать интересы пользователей API, но с технической точки зрения.
- *Разработчик серверной части приложения (бэкенд-разработчик)*. Он отвечает за детали реализации фактического интерфейса API, реализации репозитория, подключения его к любым другим службам, необходимым для выполнения его работы, и в целом добросовестно выполняет видение МП и проектировщика API. Он несет ответственность за то, чтобы запущенный в производство API был надежен, стабилен и единообразен.
- *Тестировщик/ОК*. Специалист по тестированию/обеспечению качества (ОК) отвечает за все, что связано с проверкой дизайна API и тестированием его функциональности, безопасности и стабильности. Обычно в его обязанности входят написание самих тестов (или помощь в этом разработчику клиентской/серверной части) и обеспечение их эффективного запуска. Как правило, это не только лабораторные испытания и тестирование поведения, но и тесты на межоперационную совместимость, масштабируемость, безопасность и мощность. Обычно для них используются структуры и инструменты, отобранные отделом тестирования/ОК для всей компании.
- *DevOps-инженер*. Он ответственен за все аспекты создания и развертывания API, в том числе мониторинг производительности API, чтобы убедиться, что API соответствует установленным техническим KPI и вносит свой вклад в OKR компании. Обычно это означает работу с инструментами конвейера доставки, создание сценариев сборки (или обучение этому других), управление графиком выпуска, архивирование артефактов сборки и поддержку любых откатов сломанных выпусков, если это необходимо. Инженер DevOps отвечает и за поддержку сводной панели, показывающей данные отслеживания в реальном времени, и за хранение и поиск файлов регистрации API офлайн, чтобы помочь изучить, продиагностировать и решить любые проблемы, которые могут возникнуть после запуска API в производство. В зависимости от вариантов производственного хостинга компании сотрудникам DevOps потребуется поддерживать несколько сред, включая рабочий стол, сборку, тестирование, подготовку и производство как в локальной среде, так и в облачной системе.

В этом разделе мы познакомили вас с работой, выполняемой на протяжении всего жизненного цикла API, в виде списка обязанностей или сфер ответственности. Чтобы было проще, мы составили два списка обязанностей (деловых и технических) и дали им названия, похожие на распространенные реальные должности.

Как сказано в начале раздела, обязанности просто распределяют области знаний, которые должны быть охвачены при создании программ API. Ниже мы рассмотрим создание самих команд.

Команды API

В прошлом разделе мы определили 11 обязанностей с разными сферами ответственности. Командам нужны сотрудники для выполнения этих обязанностей, чтобы охватить все важные аспекты управления API. Но описанные нами обязанности отличаются от ролей реальных специалистов в команде. Нет необходимости в том, чтобы каждая обязанность относилась к отдельному сотруднику. Например, во многих организациях евангелист API и специалист по отношениям с разработчиками — зачастую один сотрудник. Еще один пример — в некоторых небольших командах одному человеку могут поручить обязанности по тестированию/ОК и по DevOps.



Люди могут относиться к нескольким командам

Хотя в этом разделе мы собираемся описать некоторые конкретные команды, вам решать, как распределить людей между ними. Совершенно нормально укомплектовать каждую команду API сотрудниками, которые будут работать в ней полный день и занимать только эту должность. Но можно разрешить им работать в нескольких командах одновременно. Позже в этой главе мы расскажем вам о модели «отряды, племена, отделения и гильдии», принятой в компании Spotify, где используется матричный подход к членству в командах.

Может оказаться, что команде не нужно охватывать *все* роли на всех стадиях развития API (см. главу 7). Например, в фазе поддержки жизненного цикла API вам обычно не нужна помощь разработчиков клиентской и серверной частей приложения. А в некоторых организациях часть обязанностей исполняют не участники конкретной команды, а «плавающие» сотрудники компании. К примеру, обязанности дизайнера может взять на себя один из сотрудников отдела дизайна продукции, работающий по запросу с любой командой API, где нужна разработка дизайна.

Команды и зрелость API

В главе 7 мы описали изменения API в течение всего срока службы. И важно понять, как вместе с этим будет меняться ваша команда, чтобы грамотно распланировать ее работу. На каждой стадии жизненного цикла API-продукта некоторые обязанности первостепенны, а некоторые — нет. Основные обязанности — те, выполнение которых дает наиболее ощутимый эффект. Например, на стадии создания почти все участники команды несут большую ответственность, но решения дизайнера по дизайну интерфейса очень сильно влияют на всю работу.

Сотрудники с первостепенными обязанностями ответственны и за работу, обязательную к выполнению. Например, на стадии публикации невозможно разместить API, если никто не исполняет обязанности специалиста DevOps и не создает архитектуру развертывания.

Как видите, формирование команд сильно зависит от развития API и от того, какие обязанности важны в конкретный момент. Учитывая это, давайте рассмотрим стадии жизненного цикла API (см. главу 7) и определим на каждой из них первостепенные и второстепенные обязанности и действия, за которые будут отвечать сотрудники.

Стадия 1: создание

- *Первостепенные обязанности:* менеджер продукта, дизайнер, ведущий инженер.
- *Второстепенные обязанности:* евангелист API, DevOps, архитектор API, бэкэнд-разработчик.

Стадия создания — это возможность разработать основную стратегию и качественный дизайн интерфейса, не влияя на пользователей. Для разработки хорошей стратегии API вам нужен тот, кто хорошо понимает общее состояние организации и сферу API-продукта — человек, который сможет выбрать правильный курс действий. Обычно это менеджер продукта. У хорошего менеджера API-продукта будет достаточно опыта, чтобы определить цель API, которая поможет спонсирующей организации, и тактический план, позволяющий это сделать.

Должность дизайнера идеально подходит для разработки дизайна интерфейса. Хороший дизайнер API сможет принимать качественные решения по поводу дизайна модели интерфейса, основываясь на своем опыте. То есть решать, как должна выглядеть модель и как нужно тестировать и одобрять предположения по дизайну. Самое главное, что хороший дизайнер чувствует, сколько нужно *инвестиций* в дизайн, основываясь на общей ситуации.

Наряду с работой по дизайну интерфейса кто-то должен будет заниматься проектированием, архитектурой и конструированием реализации API. Часть этой работы будет экспериментальной. Именно эта реализация в итоге будет опубликована на следующей стадии. Для этой разработки нужна команда с межфункциональными талантами, которой управляют архитектор и ведущий инженер API.

В табл. 8.1 и 8.2 указаны основные и дополнительные действия на этапе создания.

Таблица 8.1. Первостепенные действия на стадии создания

Действие	Сотрудник (-и)
Разработка стратегии	Менеджер продукта
Разработка модели интерфейса	Дизайнер
Инженерное обеспечение реализации	Архитектор API, ведущий инженер, разработчик

Таблица 8.2. Дополнительные действия на стадии создания

Действие	Сотрудник (-и)
Разработка прототипов	Ведущий инженер, бэкэнд-разработчик
Тестирование реализуемости дизайна	Архитектор API, ведущий инженер, бэкэнд-разработчик, технический писатель
Тестирование безопасности дизайна и реализации	Архитектор API, тестировщик / инженер ОК
Тестирование реализуемости дизайна на рынке	Евангелист API, специалист по ОР
Тестирование простоты использования дизайна	Дизайнер
Планирование и выполнение стратегии тестирования реализации	Ведущий инженер, тестировщик / инженер ОК

Стадия 2: публикация

- *Первостепенные обязанности:* менеджер продукта, технический писатель для API, DevOps.
- *Второстепенные обязанности:* фронтенд-разработчик, дизайнер, бэкэнд-разработчик, евангелист API, специалист по ОР.

Достижение стадии публикации означает, что вы готовы предоставить пользователям доступ к вашему продукту. Для этого вам нужны люди с опытом

развертывания, документирования и обнаружения API. Есть еще много дополнительных действий, которые стоит совершить, если API ценен и у вас есть возможность их выполнить.

Важная часть работы на этой стадии — публикация изначального набора документации. Поэтому нужен сотрудник, выполняющий обязанности технического писателя, который будет создавать и публиковать документы. Технический писатель — основная обязанность на стадии публикации. Хороший специалист поможет потенциальным пользователям начать работу, а существующим — ускорить. Это обязательно понадобится вам на данном этапе, потому что так вы быстрее достигнете стадии окупаемости.

Для публикации API нужно развернуть готовые экземпляры. Обычно это обязанность специалиста по DevOps. На этой стадии он отвечает за разработку развертывания, контроль выполнения и развертывание архитектуры экземпляров API.

Менеджер продукта должен запустить процесс публикации. Это будет иметь особое значение для вас и вашей ЦА в зависимости от контекста продукта. Это может означать регистрацию API во внутреннем каталоге услуг, отправку e-mail потенциальным пользователям и т. п. В любом случае именно менеджер продукта отвечает за то, чтобы это произошло.

Помимо этих основных действий, есть ряд дополнительных, которые улучшат качество продукта. Документация и другие ресурсы поддержки должны где-то храниться, поэтому многие организации на этой стадии создают портал для разработчиков. Когда API уже будет активно использоваться, вы сможете улучшить дизайн и реализацию, основываясь на данных об их применении (столп «Мониторинг»). Также рекомендуется продолжать стимулировать использование, проводя маркетинговую и исследовательскую работу.

В табл. 8.3 и 8.4 указаны основные и дополнительные действия на стадии публикации.

Таблица 8.3. Первостепенные действия на стадии публикации

Действие	Сотрудник (-и)
Создание и публикация документации	Технический писатель
Разработка архитектуры развертывания и развертывание готовых программ	DevOps
Публикация API (то есть официальное открытие доступа)	Менеджер продукта

Таблица 8.4. Дополнительные действия на стадии публикации

Действие	Сотрудник (-и)
Разработка и реализация портала	Фронтенд-разработчик
Продвижение API на рынке	Евангелист API, специалист по ОР
Сбор отзывов пользователей о дизайне	Евангелист API, специалист по ОР
Улучшение дизайна интерфейса	Дизайнер
Сбор информации об использовании развернутых программ	Ведущий инженер, DevOps
Улучшение и оптимизация реализации	Ведущий инженер, бэкенд-разработчик
Тестирование безопасности реализации и развертывания	Архитектор API, тестировщик/инженер ОК

Стадия 3: окупаемость

- *Первостепенные обязанности:* DevOps, менеджер продукта.
- *Второстепенные обязанности:* дизайнер, специалист по тестированию/QA, архитектор API, ведущий инженер, бэкенд-разработчик, фронтенд-разработчик, технический писатель, специалист по ОР, евангелист API.

Когда API начинает окупаться, ставки растут. Теперь нужно следить за постоянной доступностью API для важных пользователей. Поэтому первостепенными действиями становятся управление изменениями и улучшение архитектуры развертывания.

Несмотря на то что API приносит доход, вам предстоит внести еще много изменений в интерфейс, реализацию и готовые программы. Хороший менеджер продукта должен уметь управлять всеми этими изменениями без потери дохода и негативного влияния на пользователей. Как именно это делать, зависит от сотрудников, стратегических приоритетов и культуры организации.

Пока менеджер продукта управляет изменениями, инженер DevOps занимается повышением устойчивости, наблюдаемости, масштабируемости и производительности архитектуры развертывания. Хороший специалист сможет применить правильные инструментарий и методы, основываясь на ситуационных аспектах API, чтобы предотвратить снижение качества для постоянных, приносящих доход пользователей.

Чтобы доход увеличивался, имеет смысл продолжить улучшать и продвигать свое предложение на рынке. Вот почему для этого этапа определен такой же набор дополнительных действий по анализу, реализации и обнаружению, как

и для предыдущего. Это не обязательно, но без постоянного улучшения ваш API может быстро перейти в стадию поддержки, не успев окупить вложения в него.

В табл. 8.5 и 8.6 указаны основные и дополнительные действия на стадии окупаемости.

Таблица 8.5. Первостепенные действия на стадии окупаемости

Действие	Сотрудник (-и)
Улучшение и оптимизация архитектуры развертывания	DevOps
Управление изменениями и распределение приоритетов в этой сфере	Менеджер продукта

Таблица 8.6. Дополнительные действия на стадии окупаемости

Действие	Сотрудник (-и)
Улучшение дизайна интерфейса	Дизайнер
Улучшение и оптимизация тестов	Тестировщик/инженер ОК
Улучшение и оптимизация реализации	Архитектор API, ведущий инженер, бэкэнд-разработчик
Тестирование безопасности реализации и развертывания	Архитектор API, тестировщик/инженер ОК
Улучшение и оптимизация начала работы с программой и обучения пользователей	Фронтенд-разработчик, технический писатель, DevOps
Продвижение API на рынке	Евангелист API, специалист по ОР

Стадия 4: поддержка

- *Первостепенные обязанности:* специалист по ОР, DevOps, архитектор API.
- *Второстепенные обязанности:* менеджер продукта, ведущий инженер API, бэкэнд-разработчик.

На стадии поддержки ваша цель — сохранять работоспособность API. Ключевая должность здесь — инженер DevOps, который должен контролировать и поддерживать развернутые программы. Кроме выполнения основной работы важно следить за изменениями системы, которые могут создать новые задачи для команды. Хороший архитектор API будет настроен на изменения, которые могут повлиять на их работу, и сможет отделить самые важные, чтобы внести их, не мешая работе продукта.

Вам также понадобится определенный уровень взаимодействия с пользователями и поддержки для них, даже если API больше не продается активно. Лучше всего эту работу сможет выполнить специалист по ОР. Он проследит, чтобы продукт был полезен новым и уже существующим пользователям, даже если уровень дохода от него не растет.

Чтобы укрепить эту поддержку, менеджер продукта и технические специалисты должны быть готовы внести любые нужные изменения. Хотя число улучшений к тому моменту значительно уменьшается, их все равно придется вносить для поддержания работы архитектора API и специалиста ОР.

В табл. 8.7 и 8.8 указаны основные и дополнительные действия на стадии поддержки.

Таблица 8.7. Первостепенные действия на стадии поддержки

Действие	Сотрудник (-и)
Улучшение и оптимизация системы мониторинга	DevOps
Поддержка существующих пользователей	Специалист по ОР
Определение системных изменений, которые ухудшат качество API	Архитектор API

Таблица 8.8. Дополнительные действия на стадии поддержки

Действие	Сотрудник (-и)
Планирование изменений в реализации	Менеджер продукта
Внесение нужных изменений в реализацию	Ведущий инженер, бэкэнд-разработчик
Внесение нужных изменений в развертывание	DevOps, бэкэнд-разработчик

Стадия 5: удаление

- *Первостепенная обязанность:* менеджер продукта.
- *Второстепенные обязанности:* специалист по ОР, евангелист API, архитектор API, DevOps, ведущий инженер.

Основная работа на стадии удаления — стратегическая, поэтому ключевую роль играет менеджер продукта. Хороший МП сможет определить оптимальную стратегию прекращения поддержки для данных обстоятельств. У него должно быть достаточно опыта для разработки тактического плана не только для создания нового API, но и для удаления старого.

Следование этой стратегии означает удаление развернутого экземпляра из архитектуры развертывания и поддержку пользователей во время перехода. Инженер DevOps отвечает за деактивацию API в сфере развертывания, а специалист по ОР — за его деактивацию для пользователей.

Вам также может понадобиться технический план для выполнения стратегического, составленного менеджером продукта. Например, может быть логичным отправлять ответные сообщения с информацией о будущей деактивации или выбирать специальные заголовки для ответов, показывающие, что API на стадии удаления. Этот план должен разработать сотрудник с техническими знаниями, поэтому обычно это делает архитектор API или ведущий инженер.

В табл. 8.9 и 8.10 указаны основные и дополнительные действия на стадии удаления.

Таблица 8.9. Первостепенные действия на стадии удаления

Действие	Сотрудник (-и)
Разработка стратегии удаления	Менеджер продукта

Таблица 8.10. Дополнительные действия на стадии удаления

Действие	Сотрудник (-и)
Оповещение пользователей о плане удаления и помощь с переходом на новый API	Специалист ОР, евангелист API, технический писатель
Разработка технической стратегии удаления	Архитектор API, ведущий инженер
Обновление архитектуры развертывания и осторожное удаление готовых программ	DevOps, ведущий инженер

В этом разделе мы обсудили, как стадия жизненного цикла отдельного API-продукта влияет на состав команды по его разработке и на первостепенные и второстепенные обязанности ее участников. Еще мы узнали, что с изменением API меняется и комплектация его команды.

Другой важный аспект деятельности команд API — масштабирование нескольких из них. В большинстве компаний с жизнеспособными программами работает больше одной команды API. Как они сотрудничают? Какая тактика нужна, чтобы их цели не пересекались и не противоречили друг другу? Как убедиться в единообразии действий множества команд? Эти вопросы — последнее, что мы рассмотрим в этом разделе.

Масштабирование команд

Понимание того, что обязанности — это важные составляющие команд, а комплектование последних зависит от стадии развития продукта — это только начало сложностей с руководством командами API. Другой важный элемент — это работа с многими командами. Обычно команда есть у каждого API, а их больше одного. Работа со множеством команд переносит вас на новый уровень сложности.

Рекомендуется рассматривать каждую команду API как независимую группу — это значит, что они могут решать свои проблемы с минимальной зависимостью от других команд. Но реальность далека от теории. На самом деле они не могут друг без друга работать! Как же с этим разобраться? Есть постоянный баланс между поддержанием независимости и плодотворным сотрудничеством. Важно разработать больше одной стратегии для команд и хорошо разбираться в том, как разные части (команды) соединяются в одно целое.

В своей книге *Team of Teams: New Rules of Engagement for a Complex World* («Команда команд: правила вовлеченности в сложном мире») генерал Стэнли Маккрystal говорит о новом способе мышления, помогающем большим компаниям добиться успеха: «По мере того как мир становится все быстрее и взаимозависимее, нам важно найти способы масштабировать текучесть команд во всей организации»¹. Это означает — понять, как поощрить совместную работу команд, не заставляя их быть зависимыми друг от друга.

Одна из организаций, известных тем, что умеет масштабировать систему своих команд, — Spotify. Технический документ Spotify 2012 года по этой теме — часто цитируемая ссылка на размышления о способах повышения эффективности как отдельных команд, так и межкомандных коммуникаций. Несмотря на то что он несколько устарел (шесть лет — очень долгий срок для интернета!), мы обнаружили, что многие организации часто используют похожие подходы. Поэтому мы считаем, что стоит понять основные из изложенных там мыслей и исследовать, как вы можете применить их в своей компании.

Команды и обязанности в Spotify

В 2012 году тренеры по Agile Хенрик Книберг и Андерс Иварссон опубликовали статью *Scaling Agile @ Spotify* («Гибкое масштабирование в Spotify»), где первое предложение звучит так: «Работать с несколькими командами в организации, занимающейся разработкой продуктов, всегда сложно!»². Затем Книберг

¹ McChrystal S. et al. *Team of Teams* («Команда команд»). New York: Portfolio, 2015.

² Ivarsson K. and A. *Scaling Agile @ Spotify* («Гибкое масштабирование в Spotify»). Октябрь 2012 г. <https://oreil.ly/TcVDp>.

и Иварссон объясняют, как в Spotify разработали модель управления командами, помогающую делиться информацией по максимуму, не ставя под удар независимость отдельных групп. Такую модель или ее вариации мы сегодня наблюдаем во многих компаниях.

В модели Spotify есть четыре ключевых способа группировки:

- отряд;
- племя;
- отдел;
- гильдия.

Отряды — это небольшие автономные команды из пяти-семи сотрудников, похожие на Scrum-команды. Это базовая рабочая единица Spotify. У членов отряда есть все навыки для выполнения порученной им работы, от дизайна до развертывания, как и у участников команд, о которых мы говорили. В Spotify у каждого отряда есть миссия или задача *внутри* большой группы одного продукта. Например, для разработки проигрывателя на Android один отряд может заниматься воспроизведением, другой — поиском и т. д. Все отряды выполняют отдельные задачи.

Племя появляется в модели Spotify с увеличением масштаба деятельности. К примеру, уже упомянутый проигрыватель на Android, сайт или серверная услуга по хранению информации, используемая остальными клиентскими программами. Здесь племя — это объединение отрядов. В Spotify стараются удерживать общее число сотрудников в племени на уровне 100 человек. Его достаточно, чтобы в группе было нужное разнообразие специалистов для выполнения задачи, но недостаточно, чтобы стало сложно поддерживать нормальные отношения.



Числа Данбара

Максимальный размер отряда (7 человек) и племени (100) основан на работах британского социолога Робина Данбара. Мы поговорим о нем подробнее в разделе «Разумное использование чисел Данбара» далее.

С отрядами и племенами Spotify может выстроить эффективную стратегию создания и поддержки своих продуктов и услуг. Но это только половина задачи. Важно также достичь определенного уровня производительности внутри сообщества. Для этого нужно организовать взаимодействие между отрядами и группами, чтобы делиться знаниями и обеспечивать единообразие в командах и продуктах. Здесь в игру вступают отделы и гильдии Spotify.

Поскольку все команды автономны, скорее всего, в каждой есть дизайнер, бэкэнд-разработчик, менеджер продукта и т. д. У каждого человека, выполня-

ющего эти обязанности, есть свои задачи и определенный опыт обучения. Часто его опыт такой же, как у других сотрудников, исполняющих те же обязанности в других командах.

Например, чтобы быть хорошим менеджером продукта в племени инфраструктуры, нужен набор навыков, который есть у всех менеджеров продукта, даже если у всех будут разные подходы. Поэтому логично, что все менеджеры продукта внутри одного племени периодически собираются, обмениваясь опытом и знаниями. В модели Spotify это называется *отделом* — когда сотрудники с одинаковыми обязанностями в одном племени (то есть группе одного и того же продукта) собираются вместе и делятся знаниями.

Гильдии же — это способ обмена знаниями между несколькими группами продуктов. Например, если несколько менеджеров продукта из всех сфер компании (от продуктов для клиентов до внутренних систем) собираются вместе, появляется дополнительный уровень обмена знаниями. В вашей компании гильдия может быть ежегодным собранием руководителей команд со всего мира, на котором обсуждается, над чем они работают в своих отделениях.

Модель из отрядов, племен, отделов и гильдий обеспечивает сочетание автономных команд и отсутствие изолированных групп сотрудников. Такой подход к масштабированию команд помогает Spotify поддерживать баланс между независимостью и сотрудничеством.

Особенности вашего подхода к масштабированию

После того как Spotify поделилась своей историей, многие крупные организации попытались внедрить их модель в надежде подражать гибкости и культуре продукта компании. Организации, которым, казалось, удалось добиться успеха в повышении гибкости, адаптировали модель к своим условиям и развили ее. На практике простое копирование модели Spotify дает мало пользы, разве что в качестве безопасной отправной точки. Об этом свидетельствует то, что «сама Spotify продолжает совершенствоваться и развиваться» (<https://oreil.ly/dQJUt>) свои методы работы, выходя за рамки времени, описанного в их статье.

Правильный способ масштабирования команд API будет зависеть от контекста и ограничений вашей организации. То, что работает в Spotify, может не работать в Google. То, что работает в розничной сети, может не работать в государственном космическом агентстве. Контекст имеет значение.

Мы определили три фактора, оказывающих наибольшее влияние на модель масштабирования ваших команд: ценность организации, приоритетные цели и распределение ценных кадров.

Ценность организации

Разные организации занимаются разными вещами, что сильно влияет на способ масштабирования их команд. Хотя технология, на которой основан API, будет схожей, ценность, получаемая в результате, часто отличается. Мы рекомендуем определить основные типы API, обеспечивающие максимальную отдачу от дополнительных инвестиций. Это поможет вам понять, как должна работать ваша модель масштабирования.

На первый взгляд, у большинства частных компаний схожие цели: увеличение дохода, снижение затрат и удовлетворение нужд сотрудников. Но под этим общим подходом у большинства компаний скрываются более индивидуализированные стратегии. Важно понять основные приоритеты вашей организации — и где она готова пойти на уступки ради достижения цели.

Например, технологическая компания может сосредоточиться на создании набора API-интерфейсов, которые они предлагают другим разработчикам для покупки и использования. Это может потребовать больших вложений в инфраструктуру и проектирование, чтобы конкурировать с другими технологическими решениями. И наоборот, розничная компания может сосредоточиться на ценности, возникающей в результате более быстрого и частого изменения клиентского опыта — при использовании легкодоступных на рынке технологических платформ.

Понимание того, какой тип работы приносит наибольшую пользу вашей организации, должно помочь вам принять решение о том, какие команды (и API) вам нужно создать. Оно включает в себя определение уровня инвестиций в отношения с разработчиками, управление продуктами и разработчиков для команд в вашей фирме.

Масштаб организации

API и их команды могут расти довольно быстро, как только начинают работать. Поэтому вам важно продумать, как принимаемые вами решения в мире API будут внедряться в более крупную компанию и ее сотрудников. Вам нужно обратить внимание на размер, масштаб и сложность вашей организации. Это крупная, глобально распределенная фирма? Сколько в ней подразделений? Есть ли в ней четкие авторитетные лица, принимающие решения?

Если вы хотите, чтобы ваши команды API быстро продвигались в большой компании, вам нужно выяснить, как они смогут связываться со всеми заинтересованными сторонами, надзорными органами и властями. Если же вы управляете API в небольшом, быстроразвивающемся стартапе, нужно разработать стратегию масштабирования, которая не создаст узких мест в организации.

Распределение опыта

Самая большая опасность, связанная с «копированием и вставкой» подхода к масштабированию команды, в том, что число ценных кадров разное в разных компаниях. Это связано с различиями в ценностях организаций и в их масштабе. В среднем банке вряд ли найдется столько же экспертов по API и технологиям, как в крупной компании по разработке ПО. Так происходит потому, что подобные вложения в сотрудников бессмысленны для их бизнес-модели или размера их организации.

Но это не сильно влияет на то, как вы масштабируете свои команды API. Если у вас меньше людей, которые могут принимать важные решения, нужно централизовать их или найти способ распределить их опыт.

Подход Spotify к масштабированию команд — децентрализованный. Масштабирование встроено в саму рабочую модель. Другой способ, который мы видели в некоторых компаниях, — создание центральной команды, которая собирает информацию у других команд и делится ею через брошюры, документы и семинары, посвященные улучшению методик¹.

Это не единственные факторы, которые нужно учитывать при масштабировании команд, но мы считаем, что они самые важные. Рассмотрение вашей организации с точки зрения ее ценности, масштаба и опыта поможет вам адаптировать модель масштабирования конкретно к вашим условиям.

Это подводит нас к последнему разделу главы, посвященному корпоративной культуре. То, как участники команды сотрудничают друг с другом и с другими командами, сильно зависит от культуры и ценностей внутри организации. Поэтому важно потратить время на изучение и создание своей корпоративной культуры.

Культура и команды

Корпоративная культура может служить скрытой формой руководства в организации. В главе 2 мы рассказали о централизованных и децентрализованных решениях. Вкратце повторим: когда решения централизованы, вы должны использовать власть, чтобы люди правильно с ними работали. Но когда они децентрализованы, применение власти как инструмента контроля и одобрения не работает. Здесь и нужна культура. Она как невидимая рука, формирующая принятие решений в командах и во всей компании, без необходимости во властных механизмах полномочий, таких как процессы, стандарты или общие инструменты.

¹ Мы подробнее поговорим об этой команде в разделе «Центр поддержки» на с. 290.

С правильными культурой и сотрудниками вы можете спокойно децентрализовать бóльшую часть решений без угрозы единообразию результатов. Поэтому вложения в создание корпоративной культуры могут окупиться стократ.

Для принятия верных решений нужно не только знать, что изменить, но и как распределить ответственность за эти изменения. Корпоративная культура — тоже важная составляющая этого процесса. Некоторые IT-специалисты не готовы обсуждать это, но корпоративная культура заслуживает внимания.

Понятие «корпоративная культура» впервые появилось в книге *The Changing Culture of a Factory* («Меняющаяся культура предприятия»). Доктор Эллиот Джекс определил ее так:

Культура предприятия — это общепринятый и традиционный образ мышления и действий, который в той или иной степени разделяют все его работники и которому должны научиться новопривывшие.

Признание в организации культуры приводит к возможности повлиять на нее, направляя в определенную сторону. Так формируется представление о том, какая именно культура в вашей организации и как ее можно изменить.

В 1970–80-е годы появляется много книг и теорий по классификации корпоративной культуры, ее выявлению и управлению ею. Одно из важнейших изданий того периода — «Образы организаций» Гарета Моргана (*Images of Organization*, Sage). Морган выступил с идеей о том, что корпоративную культуру можно охарактеризовать простыми метафорами — «машина», «организм», «мозг» и т. д. Они помогают понять, как работает культура компании и как определить способы, которыми ее можно изменить.

Мы не будем рассматривать здесь последние 70 лет исследований в этой области, но отметим, что многие компании, с которыми мы сотрудничаем, активно работают над тем, чтобы разобраться в культуре своей организации, и тем, как ее можно улучшить и перенаправить. Исходя из этого, назовем три темы, которые часто затрагиваются в разговоре с представителями компаний, разрабатывающих API, управляющих ими и оказывающих услуги в сфере IT:

- наблюдения Мэла Конвея за тем, как *взаимодействие* групп влияет на результат;
- теории Робина Данбара о том, как *размер* команды влияет на коммуникацию;
- наблюдения Кристофера Александра за тем, как *разнообразие* влияет на продуктивность.

Мы также коснемся роли экспериментов в корпоративной культуре и того, как они влияют на команды.

Закон Конвея

В последние несколько лет работа Мэла Конвея «Как комитеты создают новое?», опубликованная в 1967 году, стала почти *обязательной* темой в презентациях, посвященных микросервисам и API. В этой работе представлено то, что Фред Брукс в 1975 году в книге *Mythical Man Month*¹ назвал законом Конвея, который гласит²:

Организация вынуждена проектировать системы, которые неизбежно копируют ее структуру коммуникаций.

Это наблюдение о том, как способ организации групп влияет на результат, который они производят. Эрик С. Реймонд, автор эссе *The Cathedral & the Bazaar* («Собор и базар»), сделал дополнительное наблюдение: «Если у вас над компилятором работают четыре группы, вы получите четырехпроходный компилятор»³. Если кратко, то закон Конвея говорит, что в области производства ПО организационные границы определяют, какие приложения у вас получатся. Это и хорошо, и плохо.

Как сказано в начале главы, ПО, которое мы пишем, «глупое» — оно делает только (и точно) то, что ему говорят делать. Конвей напоминает: то, как мы группируем сотрудников и где проводим границы между группами, определяет полученные результаты.

Поэтому некоторые IT-консультанты говорят о применении «обратного закона Конвея». Они советуют сначала сформировать команды и очертить границы, чтобы получить желаемые результаты. В какой-то степени это может сработать, но тут появляются свои проблемы. В той же работе 1967 года Конвей предупреждает, что не стоит работать организационным «скальпелем» слишком агрессивно:

[Закон Конвея] создает проблемы, потому что необходимость взаимодействия в любой момент зависит от системы, работающей в этот момент. Поскольку первый предложенный дизайн почти никогда не оказывается лучшим, может понадобиться изменить эту систему. Поэтому для эффективного дизайна важна гибкость организации.

¹ Брукс Ф. Мифический человеко-месяц. — Питер, 2021.

² Conway M. E. How Do Committees Invent? («Как комитеты изобретают?») // Datamation, апрель 1968 г. <https://oreil.ly/PXGIIt>.

³ Logan P. Conway's Law: How to Dissolve Communication Barriers in Your API Development Organization («Закон Конвея: как устранить коммуникационные барьеры в вашей организации по разработке API») // Medium, 24 августа 2018 г. <https://oreil.ly/PassA>.

По сути, вы не можете перехитрить закон Конвея! Здесь нужен компромисс. Важно отметить, что понятие организационной структуры имеет ключевое значение для влияния на корпоративную культуру. У компаний, хорошо управляющих своей культурой, как минимум две общие черты: они работают над тем, чтобы сделать границы одновременно четкими и гибкими.

Установить четкие границы между командами на ранней стадии проекта — нормально. Это поможет распределить ответственность *внутри* команд и разделить интерфейсы *между* ними. Важно помнить и о предостережении Конвея: «Первый предложенный дизайн почти никогда не оказывается лучшим». Поэтому, работая над API и его проектами, важно корректировать границы, основываясь на новых обстоятельствах. Это нормальная часть работы.

РАЗРАБОТКА, УПРАВЛЯЕМАЯ МОДЕЛЯМИ

Это полезный способ согласования вашей модели команды и интерфейсов API. Такой подход описан Эриком Эвансом в его книге *Доменно-ориентированное проектирование* (Domain-Driven Design). Это означает, что вы создаете и поддерживаете набор моделей, которые *выражаются* как API в вашей архитектуре и в виде команд в вашей организационной структуре. Обновляя модель, вы обновляете свои команды и архитектуру, и наоборот. Это позволяет вам постоянно улучшать ваши команды и дизайн системы.

Хороший пример такого подхода — *модель командных топологий* Мэтью Скелтона и Мануэля Паиса (<https://oreil.ly/wmJZY>). Она помогает проектировать команды, ограниченные API и программными компонентами. Вы можете использовать командные топологии вместе с доменно-ориентированными моделями проектирования и архитектуры ПО для разработки дизайна, который поддерживает закон Конвея, а не борется с ним.

Как и другие аспекты в сфере управления API, управление культурой *непрерывно*. Конвей подсказывает, как можно повлиять на изменения (например, сфокусироваться на границах между командами), и предупреждает, что первые попытки редко становятся лучшими («вам может понадобиться изменить эту систему»). Поэтому появляется вопрос: как команды и их размер могут повлиять на корпоративную культуру? Ответ мы можем найти в работах Робина Данбара.

Разумное использование чисел Данбара

Конвей рассказывает нам о том, как команды и границы влияют на результат любого начинания. Поэтому появляется логический вопрос: как формируется команда и каков ее оптимальный размер? Многие наши клиенты обращаются за ответом к исследованию доктора Робина Данбара, проведенному в 1990 году.

В популярных статьях по социологии его теории о влиянии мозга на размер группы известны благодаря так называемому *числу Данбара* (<https://oreil.ly/WE3aO>). Оно устанавливает, что мы можем успешно отслеживать и поддерживать полезные отношения не более чем со 150 людьми. Любая группа с большим числом участников перегружает мозг людей, и они оказываются не способны управлять ею. Поэтому, как только группа перерастает это количество, координация и сотрудничество между ее членами усложняются.

Есть много подтверждений «силы 150» при взаимодействиях с группами. Уильям «Билл» Гор, основатель компании W.L. Gore с 1970 по 1986 год, установил правило: как только на отдельном предприятии появлялось более 150 сотрудников, эту группу разделяли и строили новое здание, иногда рядом со старым¹. Пэтти Маккорд из Netflix называет это количество числом «встать на стул»: как только подчиненных становится больше 150, руководитель уже не может просто встать на стул, чтобы обратиться ко всей группе².

Хотя 150 по Данбару — важное число, исследования, лежащие в его основе, еще более значимы. Суть теории Данбара в том, что мы должны тратить время и энергию, чтобы успешно общаться в группах, и что размер группы влияет на количество усилий для поддержания успешных связей. Его раннее исследование показало, что командам из 150 сотрудников «нужно 42 % рабочего времени на “социальные ухаживания”». Другими словами, при росте команд нужно больше времени посвящать усилиям по сплочению участников. В современных офисных условиях «социальные ухаживания» принимают форму собраний, e-mail, звонков, переписки в мессенджерах, ежедневных пятиминуток, планерок и т. д. Координация больших команд дорого стоит.

Хорошая новость — у Данбара было не только это число. Он определил серию чисел, начиная с пяти: 15, 50, 150 и т. д., до превышающих 1000. Внизу шкалы (5 и 15) стоимость координации (социальных ухаживаний) невысока. В группе из пяти человек все хорошо знакомы друг с другом, каждый знает свою задачу, и почти всегда все знают, кто не выполняет свою часть работы (если такой человек есть). Время на социальные ухаживания минимально. Даже при увеличении группы до 15 человек затраты на коммуникацию невысоки. Вы могли заметить, что рекомендованные в этой книге размеры групп приближаются к отметке пять человек (плюс-минус два-три).

¹ *Dunbar R.* Friends to Count On («Друзья, на которых можно положиться») // Guardian, 25 апреля 2011 г. <https://oreil.ly/1YNzp>.

² *Delaney K.J.* Something Weird Happens to Companies when They Hit 150 People («Что-то странное происходит с компаниями, когда численность достигает 150 человек») // Quartz, 29 ноября 2016 г. <https://oreil.ly/Djiz9>.

Эту небольшую команду (от пяти до семи человек) мы называем командой Данбара первого уровня. Это наиболее распространенный размер для ранних стартапов. Команды второго уровня по Данбару, до 15 человек, часто можно встретить в новых компаниях, прошедших первый этап и активно строящих свой бизнес. Многие IT-компании, с которыми мы общались, стараются сохранять размеры своих команд на первом и втором уровнях по Данбару. Это позволяет снизить затраты на коммуникацию и максимально увеличить их общую эффективность. Например, Spotify стремится к созданию «отрядов» в пять-семь человек (см. раздел «Команды и обязанности в Spotify» на с. 243).

Данбар показывает нам, что размер команды важен и что общение внутри небольших команд может быть более эффективным. Конвей напоминает, что взаимодействие *между* командами определяет итоговый результат. Итак, задача создания успешной культуры управления API в том, чтобы управлять *множеством* команд внутри крупной организации. Как в работе с ландшафтом API вы сталкиваетесь с задачами, отличными от задач, которые решаются в ходе работы с одним или несколькими API, так и работая с «ландшафтом команд», вы встречаетесь с уникальными аспектами. С этой проблемой «команды команд» могут помочь работы архитектора Кристофера Александера.

Работа с культурной мозаикой по Александеру

Руководство одной командой и/или ее поддержка — уже непростая задача. Мотивировать группу работать, помогать им найти опору, свой стиль и научиться вносить вклад в миссию компании — это тяжелый, но благодарный труд. Люди, которые этим занимаются, знают, что любая команда уникальна. Каждой приходится проходить один и тот же путь по-своему. Различия между командами и станут ключом к появлению разнообразия и силы в вашей компании. Хотя может показаться, что проще требовать от всех команд «одинаковости», это не будет признаком здоровой экосистемы.

Такой «ландшафт команд» обладает теми же сложностями и возможностями, что и ландшафт API (см. главу 9). Многие аспекты ландшафта из этой главы применимы и к ландшафту команд. По мере роста предприятия вы столкнетесь с появлением разнообразия, объема, непредсказуемости и других элементов здоровой экосистемы.

На самом деле системы людей (то есть команды) обычно становятся *лучше* с возникновением разнообразия. Многие из нас бывали в ситуации, когда появление в команде человека со стороны усилило ее. По этому поводу есть много афоризмов, включая «То, что нас не убивает, делает нас сильнее», — о том, что

неожиданные сложности помогают нам расти над собой. Книга Нассима Талеба *Antifragile*¹, выпущенная в 2012 году, основана именно на этой идее.

Другую точку зрения на силу «команды команд» можно найти в работах архитектора и мыслителя Кристофера Александера. Его книга *A Pattern Language* («Язык шаблонов»), изданная в 1977 году, считается толчком к появлению шаблонов в ПО. Она описывает понятие «мозаика субкультур» как способ организации здоровых и стабильных сообществ.

ВЛИЯНИЕ АЛЕКСАНДЕРА НА ПО

Хотя Кристофер Александер и проектирует здания, его работы и идеи сильно повлияли на архитектуру программного обеспечения. Его книга «Язык шаблонов» ввела понятие шаблонного мышления при построении больших систем и часто упоминается как катализатор движения программных шаблонов. Читать книгу про шаблоны тяжело, и только один из нашей четверки заявляет, что прочел ее целиком. Но у Александера есть книга меньше и доступнее — *The Timeless Way of Building* («Вневременной способ строительства»). Мы часто рекомендуем работы Александера архитекторам ПО, работающим с очень большими системами.

Шаблон мозаики субкультур по Александеру (<https://oreil.ly/SQ7lt>) описывает три основных способа работы с большими группами людей и меньшими, возникающими внутри них. Работы Александера посвящены городам, но в них прослеживаются важные параллели с руководством ИТ, работающим с организациями на глобальном уровне и на уровне предприятия.

Александер выделяет три подхода к появлению подгрупп внутри больших сообществ²:

- *гетерогенный*. Люди смешиваются независимо от их образа жизни и культуры, сводя все стили жизни к общему знаменателю, который оказывается однообразным и скучным;
- *гетто*. Люди группируются по основным и банальным отличительным признакам — расе и/или экономическому положению. Создаются изолированные группы, которые по-прежнему однообразны внутри каждого гетто;
- *мозаика*. Несколько небольших областей с четкими границами, между которыми можно свободно перемещаться, чтобы познакомиться с интересующими и вдохновляющими видами образа жизни и культурами.

¹ Талеб Н. Антихрупкость.

² Alexander C. et al. Mosaic of Subcultures («Мозаика субкультур») // A Pattern Language. Oxford: Oxford University Press, 1977. P. 42–50.

Может быть сложновато разобраться в примерах из области градостроительства, которые приводит Александер, но мы наблюдали, что принципы гомогенности («Мы все используем одинаковые инструменты и процессы для всей компании»), гетто («Мы здесь все — специалисты по базам данных») и мозаики («Я присоединился к этой группе, потому что хотел поработать над нашим мобильным приложением») преобладают и в IT-компаниях.

Внутри каждой компании появляются общие культурные элементы и субкультуры. Знание об этих составляющих — первый шаг на пути к работе с ними и их использованию для создания здоровой и стабильной культуры управления API.

Мы говорили, что недостаточно просто понимать динамику взаимодействия внутри одной команды (как утверждает Данбар). И подчеркнули важность связей между командами, описанных Мэлом Конвеем. Наконец, мы познакомили вас с понятием «ландшафт команд» и тем, как важно обращать внимание на способ формирования команд (по Александеру) и систему, где они работают. Но как это все окупится? Зачем тратить время на это, особенно в сфере управления API?

Корпоративная культура определяет уровень инноваций, экспериментов и креативности ваших команд и сотрудников. Это ключ к успеху компании.

Этот последний аспект культуры мы сейчас и обсудим.

Поддержка экспериментов

Важный повод тратить время на выстраивание корпоративной культуры — поддержка инноваций и преобразования повседневной работы организации. Основная причина этого выражена цитатой, приписываемой гуру управления бизнесом Питеру Дракеру: «Культура ест стратегию на завтрак».

Действительно, при любой стратегии компанию ведет вперед преобладающая в ней *культура*. Так, если вы хотите сменить направление своей команды, группы по разработке продукта или даже целой организации, нужно сосредоточиться на культуре. Этому нас учит и Конвей: именно организация и ее границы влияют на результат работы группы.

Ключ к инновациям — созданию новых продуктов, методов и идей — возможность безопасно экспериментировать. Эксперимент — это не запуск в производство непродуманной идеи. Как и многое, что мы обсудили в этой книге, он начинается с малого и каждый раз проходит повторяющиеся стадии разработки, чтобы определить связанные с ним идеи, научиться на них, отбросить лишнее

и найти что-то важное, что будет стоить потраченных на его реализацию времени и ресурсов.

В книге *Direct from Dell* («Руководство от Делла», 2006) бизнесмен и филантроп Майкл Делл сформулировал эту идею так: «Чтобы сотрудники внедряли больше инноваций, нужно обеспечить им безопасность в случае провала». Главная мысль в том, что провалить задачу должно быть не просто легко, но и *безопасно*. Нужно обеспечить атмосферу, позволяющую командам свободно экспериментировать, но остерегающую от ошибок, которые дорого вам обойдутся. Один из способов создания такой системы — использование элементов решения, выделенных ранее в этой книге (см. раздел «Элементы решения» на с. 52). Зная границы своей деятельности, команды четче видят, какие эксперименты могут ставить, чтобы понять, как улучшить свою работу.

Важно понимать, что гораздо лучше, когда эксперименты проводят *многие* команды, чем несколько или всего одна. В разделе «Центр поддержки» на с. 290 мы обсудили преимущества наличия специальной команды, способной помочь установить руководство и барьеры для предприятия.

Но это не то место, где *происходят* все эксперименты. Как и в других аспектах ИТ, слишком сильная централизация может привести к повышению уязвимости и нестабильности. В то же время распределение действий среди множества команд и групп по разработке продуктов повышает вероятность появления ценных идей и *снижает* опасность того, что эти эксперименты нанесут компании серьезный ущерб.

Последний пункт может показаться нелогичным некоторым руководителям в сфере ИТ. Можно подумать, что большое количество экспериментов усиливает непредсказуемость. Но так бывает, только если все они проводятся в одном месте. Такую концентрацию рисков Нассим Талеб обсуждает в книге *Skin in the Game*¹. Автор «Черного лебедя», «Антихрупкости» и других бестселлеров напоминает читателям: «Вероятность успеха для группы людей неприменима к [одному человеку]». Иначе говоря, *сообщество* из 100 команд, проводящих эксперименты с новыми API, — это не то же самое, что одна команда, проводящая 100 экспериментов. Вы можете использовать свои знания об элементах принятия решений, чтобы снизить риск, одновременно увеличивая количество экспериментов.

Увеличение количества экспериментов означает больше попыток инноваций, что приводит к модели непрерывного управления API (см. раздел «Жизненный цикл API-продукта» на с. 198), которую проще поддерживать со временем.

¹ Талеб Н. Рискую собственной шкурой.

Чтобы все это работало стабильно и экономично (но необязательно *эффективно*), вам нужно многообразное сообщество команд, работающих над проектами, которыми они увлечены. Здесь-то в игру и вступает мозаика Александра.

Итоги главы

В этой главе мы затронули много тем. Во-первых, определили набор обязанностей, охватывающий принятие решений и ответственность, необходимую, чтобы разработать, создать и поддерживать API. Мы обсудили, как эти обязанности можно использовать, чтобы собрать команду людей, которые работают над самим API. А еще увидели, что один человек может выполнять несколько обязанностей в одной или нескольких командах.

Мы также рассмотрели, как разные этапы жизненного цикла API могут повлиять на состав и потребность в различных ролях в команде разработчиков API. Оказывается, команды динамичны, а обязанности отражают число вовлеченных людей и зрелость API. Мы рассмотрели командную модель компании Spotify, учитывающую взаимодействие между командами на разных уровнях внутри компании. Также мы отметили, что можно применить централизованный или децентрализованный подход к обмену опытом и сотрудничеству между командами внутри организации.

Наконец, мы исследовали степень влияния корпоративной культуры в работе команд. Такие факторы, как размер команды, могут влиять на качество взаимодействия и точность результатов деятельности. А если вы не наладите коммуникацию между командами, внутри организации могут появиться «технические гетто», которые будут тормозить внедрение инноваций и «глушить» творческий подход к работе.

Последний раздел, посвященный значению культуры и улучшению коммуникации между командами, подводит нас к важному моменту. До этого момента мы фокусировались на управлении *одним* API и всем, что нужно для обеспечения его соответствия потребностям клиентов: понимании типичных навыков для создания и обслуживания API, обеспечении здорового управления изменениями.

Но есть и другой аспект этой темы — сотрудничество между командами и продуктами. Во всех компаниях, которые мы посетили, больше одного API, больше одной команды и больше одного способа совместной работы. Мы называем этот мир множества API и множества команд ландшафтом API вашей компании. А управление ландшафтом сильно отличается от управления одним объектом или API.

Когда ваша сфера ответственности выходит за рамки одного API или продукта, нужно изменить свой взгляд на проблемы и способы их решения. Прочитируем Стэнли Маккриснала (снова)¹:

Искушение вести себя как гроссмейстер, контролирующий каждое движение организации, должно уступить место подходу садовника, который позволяет, а не руководит. Подход садовника к руководству вовсе не пассивен. Руководитель действует как посредник с открытыми глазами, но связанными руками, который создает и поддерживает систему, где работает организация.

В следующих нескольких главах мы рассмотрим то, что нужно для того, чтобы обеспечить благоустройство ландшафта API вашей компании.

¹ *McChrystal et al.* Team of Teams («Команда команд»).

ГЛАВА 9

Ландшафты API

Теория эволюции путем нарастающего естественного отбора — единственная известная нам теория, способная объяснить существование организованной сложности.

Ричард Докинз

С ростом API становится все важнее управлять развивающимися программами так, чтобы польза и ценность *всех API организации* максимально выросли. Помните об этом балансе, потому что лучший (или достаточно хороший) способ предоставления отдельного сервиса через API может оказаться не таким полезным, если смотреть на него через призму легкости использования этого сервиса как части ландшафта.

ЛАНДШАФТ API

Ландшафт API — это совокупность всех API, опубликованных организацией (рис. 9.1). API в ландшафте могут находиться на разных стадиях зрелости (создание, публикация, окупаемость, поддержка, удаление) и быть нацеленными на разные аудитории (внутренние, партнерские, внешние). Могут быть и другие различия, например стиль или способ реализации.

Другие термины, которые можно отметить, говоря о ландшафтах API, — это *портфель API*, *каталог API* или *область API*.

Цель ландшафта API — создание среды, способствующей повышению эффективности разработки дизайна, реализации и взаимодействия с API. В итоге ландшафт API может помочь организации достичь таких бизнес-целей, как ускорение циклов продукта, упрощение тестирования и изменения продукта, создание среды, где бизнес-идеи и инициативы максимально быстро отражаются в IT.

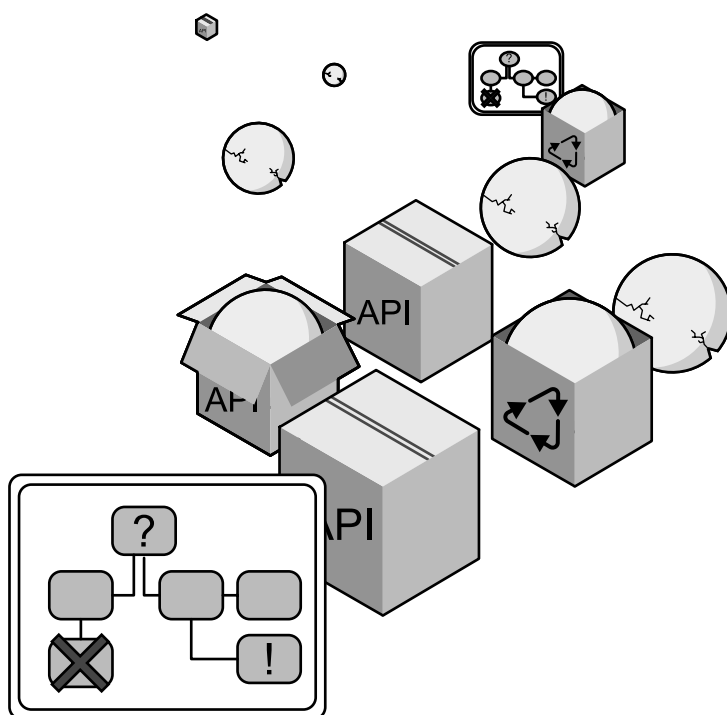


Рис. 9.1. Ландшафт продуктов API

Современные ландшафты API непрерывно растут в смысле общего числа API и количества API, используемых новыми сервисами. При таком увеличении числа зависимостей становится ясно, что разработчику нового сервиса полезно не разбираться в совершенно разных конструкциях API и не использовать их. Различия могут быть кардинальными — например, задействуют ли API ресурсный стиль или событийно-ориентированный (см. главу 6). Но даже если стили совпадают, могут обнаружиться такие технические различия, как использование формата JSON или XML.

С точки зрения пользователя API полезно было бы иметь общую терминологию. Например, при задействовании нескольких API, предоставляющих пользовательские данные, будет проще, если у них будет одна основная пользовательская модель (даже если она представлена слегка по-разному).

Мысль о пользе стандартизации очень ясно указывает нам на то, что чем ее больше, тем лучше. В какой-то мере так и есть, но в то же время известно, что на стандартизацию нужны время и ресурсы. Она обычно не сводится к «единственно

верной и лучшей модели», а просто создает ту, с которой все могут как-то смириться. Поэтому в целом эта инвестиция имеет как преимущества, так и риски.

Возможно, использование уникального формата для каждого API будет не лучшим решением. Разумнее выбрать из существующих, например JSON или XML. Тогда плюсы применения существующих стандартов перевешивают вероятные выгоды от уникальных форматов. Но может быть очень дорого стандартизировать какие-то элементы разных сервисов, например уже упомянутую пользовательскую модель. В этом случае не стоит идти на такие неоправданные расходы ради поиска единственно верной пользовательской модели. Лучше просто остановиться на модели предметной области.

В целом в идеале для каждого сервиса мы хотим не изобретать велосипед, когда это не нужно, а использовать повторно элементы дизайна, сокращающие затраты ресурсов на его создание, понимание и реализацию. Если мы сможем достичь такого идеального уровня повторного применения или хотя бы приблизиться к нему, создатели сервиса смогут сосредоточиться на тех аспектах дизайна, на которых им нужно, не отвлекаясь на уже решенные проблемы.

Все больше организаций поступают именно так. Но самое важное здесь — удостовериться, что руководства для дизайнеров постоянно обновляются: оцениваются и утверждаются новые методики, старые выходят из обращения, а главной движущей силой этих изменений служит постоянно развивающийся набор методик в организации.

Важно понимать, что ландшафт API — это подвижная и постоянно меняющаяся среда. Для поддержания ее в рабочем состоянии архитектура должна следовать по такому же пути непрерывного развития. Тогда ландшафт становится похож на очень масштабную систему, например интернет, который, с одной стороны, работает все время, а с другой — непрерывно меняется: в нем постоянно возникают новые стандарты и технологии.

Археология API

Хотя сейчас мы наблюдаем множество организаций, которые только начинают создавать API-программы, важно помнить, что в любой компании, где использовались ИТ, почти наверняка уже есть какие-то API, применяющиеся долгое время.

Если исходить из определения, то API — это любой интерфейс, позволяющий двум программным компонентам взаимодействовать. Если сузить определение до современного понимания сетевого API, то это *любой* интерфейс, позволяющий двум программным компонентам взаимодействовать через сеть.

ОПРЕДЕЛЕНИЕ АРХЕОЛОГИИ API

Археология — это практика раскопок артефактов и их понимания в контексте их происхождения во времени и месте. Такая концепция может быть применена и к API. *Археология API* — это поиск средств интеграции, исследование того, зачем и как они были созданы, и их документации, чтобы лучше понимать историю и структуру сложных IT-систем. Заниматься археологией API в организациях может быть очень полезно с точки зрения изучения существующих способов взаимодействия IT-компонентов.

Другой термин, который иногда используют люди, — *инвентаризация API*. Но тогда охват часто ограничивается перечислением существующих API вместо рассмотрения способов соединения компонентов, не относящихся к ним.

Во многих организациях такие интерфейсы не называются API и не созданы для повторного применения (см. раздел «Устав Безоса» на с. 78). Но чаще всего эти интерфейсы существуют, даже если были созданы и использовались для интеграции «один к одному» (подрывая этим одно из главных преимуществ API — возможность повторного использования).

Эти интерфейсы — первые признаки необходимости объединения компонентов. Поэтому мы называем их *прото-API*, используя в качестве приставки греческое слово *protos* (означающее «первый»), которое можно увидеть в слове *прототип*.

Поиск и использование таких прото-API могут быть полезны, ведь они показывают, где возникала потребность в интеграции (даже если она создавалась способами, не связанными с API). Не все эти прото-API стоит заменять обычными, но просто разобравшись в истории, можно получить некоторое представление о том, как наблюдалась и удовлетворялась потребность в интеграции, что принималось в этой области и где может появиться необходимость в дополнительной интеграции.

ПРОТО-API

Необходимость во взаимодействии компонентов существует во всех сложных системах. API — один из способов сделать это, но есть и другие. С точки зрения API любой механизм, не являющийся им и применяемый для взаимодействия компонентов, может считаться прото-API. В ландшафте без изъянов с помощью API выполняются абсолютно все взаимодействия компонентов. Если помнить об этом идеальном образе, любое взаимодействие без API становится кандидатом на модернизацию — на то, чтобы его заменили на API. Поэтому любой механизм взаимодействия компонентов без помощи API можно считать прото-API.

В некоторых организациях, обычно с крупными унаследованными системами, есть специальные *библиотекари API*. Это сотрудники организации, владеющие историей устаревшей архитектуры. Они знают, где находятся сервисы, их API,

как они работают и как к ним получить доступ. В общем, библиотекари API занимаются археологией API и делятся ценными результатами.

Археология API может помочь вам лучше понять IT-ландшафт, даже если сейчас он состоит в основном из прото-API. Она служит отправной точкой для понимания необходимости интеграции в прошлом и того, с помощью каких инвестиций в API проще всего распутать проблемную сеть из множества пользовательских интеграций. С опытом и со временем заменять интеграции до появления API современными моделями на основе API становится проще.

Управление API в больших масштабах

Управление API в масштабе — это баланс между введением каких-то правил проектирования на уровне ландшафта и максимальной свободой отдельных проектов на уровне API. Это классическая борьба между централизованной интеграцией для согласованности и оптимизации и децентрализацией ради гибкости и возможности развития.

- *Централизованная* интеграция принесла нам типичную IT-архитектуру прошлого. Главной движущей силой была стандартизация *исполнения* функций, чтобы предоставлять их оптимизированно и недорого. Высокий уровень интеграции упрощает оптимизацию, но в то же время влияет на возможность внесения изменений и развития итоговой системы.
- *Децентрализация* — это противоположный подход. Самый доступный и масштабный его пример — интернет. Главная движущая сила здесь — стандартизация *доступа* к функциям для их предоставления множеством разных способов. При этом функции остаются доступными, так как доступ основан на общих соглашениях об их взаимодействиях. Главная цель децентрализации — *ослабление связанности* (<https://oreil.ly/GhN2X>) — облегчение изменений в отдельных частях общей системы без необходимости менять другие.

Перспективы и сложности управления ландшафтом API в том, чтобы запомнить эту проблему и избежать ловушки, в которую попал SOAP. В нем говорилось, что только доступность сервисов имеет значение. Это был важный первый шаг, но он не позволил справиться со *слабой связанностью функций*. API и микросервисы¹, фокусируясь на техниках реализации и развертывания, позволяют

¹ Подробное описание стиля архитектуры микросервисов есть в книге Иракли Надарейшвили, Ронни Митра, Мэтта Макларти и Майка Амундсена *Microservice Architecture* («Архитектура микросервисов: соединение принципов, методик и культуры»), O'Reilly, <https://oreil.ly/2DDqCDi>.

нам еще раз обдумать, что важно для больших систем сервисов и как создавать системы, не попадающие в ловушку SOAP.

ДЕЦЕНТРАЛИЗАЦИЯ И ИСПОЛНЕНИЕ

Если мы можем чему-то научиться у еще не совсем реализованных обещаний сервис-ориентированной архитектуры (SOA) на основе SOAP, так это тому, что *тщательно управляемое исполнение* — основной аспект реализации перспектив ориентированности на сервисы. SOAP исполнял обещание предоставить доступ к функциям, но не справлялся с такой же важной задачей *правильного управления их исполнением*. И хотя SOAP и был полезен (ранее недоступные возможности предоставлялись в качестве сервисов), он не соответствовал потребностям в более гибком и развивающемся ландшафте.

Принцип платформы

Многие говорят о платформах, обсуждая API и основные бизнес-цели, но при этом могут иметь в виду разные вещи. Важно помнить: если на уровне бизнеса кажется удачной идеей создавать что-то как платформу, это не значит, что именно так это надо разрабатывать на техническом уровне.

На уровне бизнеса платформа обеспечивает основу, объединяющую стороны, чтобы они могли обмениваться ценностями, и глубже этой абстрактной формулировки мы зайти не сможем. Часто на привлекательность платформы влияют два основных фактора.

- *Какой у платформы охват?* То есть сколько пользователей можно охватить, участвуя в этой платформе? Это определяется по числу ее пользователей или подписчиков. Это самый важный параметр, вычисляемый по общему количеству или по качественным факторам, определяющим пользователей, которых можно охватить с помощью этой платформы.
- *Какие функции у платформы?* Если что-то создавать на ней, как она помогает вам или ограничивает вас с точки зрения дохода? Легко ли можно менять платформу, чтобы добавлять новые функции, не вредя ее пользователям?

Хотя эти бизнес-показатели важны, есть фактор, который часто забывают, когда речь идет о платформах: они всегда заставляют пользователей придерживаться определенных ограничений, но делают это по-разному.

- *Веб-приложения* могут использоваться кем угодно и чем угодно, если это соответствует базовым веб-стандартам. Тогда это будет современный браузер с поддержкой скриптов. Кто угодно может создавать веб-приложения и открывать к ним доступ, и любой человек может их задействовать. Нет центрального элемента, контролирующего работу *веб-платформы*.

- *Собственные приложения в App Store* внешне и по функциям могут быть похожи на веб-приложения, но предоставляются и используются иначе. Их часто можно загрузить только из централизованных магазинов приложений. Это значит, что только владелец магазина может решать, что устанавливают пользователи. К тому же их можно запускать только на определенных гаджетах.

Приложения в App Store создаются специально для конкретных устройств, то есть вложения в их создание ограничены только этой платформой. Чтобы использовать приложение на другой платформе, включая интернет, его нужно воссоздать в другой среде разработки и даже с другим языком программирования. Это значит, что клиентскую сторону приложения необходимо воссоздать почти с нуля.

Следуя этому шаблону для ландшафтов API и идее «предоставления платформы API для приложений», можно применить тот же подход.

Иногда «платформа API» воспринимается как конкретная среда, где доступны API. Вскоре это начинает напоминать сервисную шину предприятия (enterprise service bus, ESB), где такая платформа должна предоставлять инфраструктуру, а API становятся доступными благодаря тому, что могут ее задействовать.

В других случаях, говоря о платформе API, подразумевают общий набор принципов, используемый и предоставляемый сервисами. В таком случае присоединение к платформе не имеет ничего общего с тем, где и как предоставляются отдельные сервисы. Если сервисы придерживаются одних принципов, протоколов и шаблонов, они предоставляют API на этой платформе, становясь частью ландшафта API.

Второй тип платформы более абстрактный, но у него больше возможностей. Разделяя то, что выполняют функции, и то, как они это делают, он облегчает вклад пользователей в платформу. Он открывает много путей для инноваций, позволяя приложениям экспериментировать с методами реализации, не ставя под удар их возможности вносить вклад в ландшафт API.

И снова возьмем в качестве примера интернет. Сосредоточившись только на API, он позволяет многим вещам меняться со временем. Например, сеть передачи данных (Content Delivery Network, CDN) не встроена в сам интернет. Ее существование стало возможным из-за сложности его содержимого и гибкости браузера, позволяющего генерировать веб-страницы, основываясь на нескольких источниках из разных мест. Можно утверждать, что потенциал для создания CDN уже был заложен в принципах и протоколах самых первых веб-страниц, но шаблон CDN появился, только когда в нем возникла необходимость.

Именно это качество адаптации к новым вызовам мы хотим видеть в наших API-ландшафтах. Мы создаем ландшафт, основанный на открытых и расширяемых

принципах и протоколах, но можем и хотим что-то менять при необходимости. Также мы создаем шаблоны поддержки, помогающие приложениям более эффективно решать проблемы, и хотим развивать их со временем.

Принципы, протоколы и шаблоны

Главный вывод из прошлого раздела: платформа не должна требовать одного конкретного способа что-то делать (ответа на вопрос «как?») или одного конкретного места (ответа на вопрос «где?»). Хорошая платформа создается на основе принципов, протоколов и шаблонов. Мы можем проиллюстрировать эту мысль с помощью веб-платформы.

Давно известно, что интернет не только устойчив, но и гибок. За последние 30 лет фундаментальная архитектура интернета не изменилась, но, конечно, значительно эволюционировала. Как это явное противоречие стало возможным? Ведь другие системы сталкиваются с проблемами гораздо быстрее, двигаясь по менее радикальным траекториям.

Одна из главных причин в том, что в веб-платформе ничего не говорится о том, как служба реализована или используется. Сервисы могут быть реализованы на любом языке, и с годами меняются предпочтения по мере изменения языков программирования и сред. Они предоставляются с помощью любой рабочей среды: сначала это были серверы в подвалах, теперь — размещенные в самом интернете, а в последнее время появляются и облачные хранилища.

Услугами сервисов может пользоваться любой клиент, и они тоже радикально изменились: от простых браузеров с командной строкой до сложных графических на современных мобильных телефонах. Фокусируясь только на интерфейсе, определяющем, как информация идентифицируется, обменивается и представляется, веб-архитектура оказалась лучше приспособлена к естественному росту, чем любая сложная ИТ-система. Причины этого удивительно просты.

- *Принципы* — это фундаментальные понятия, встроенные в саму платформу. В случае интернета один из таких — то, что ресурсы распознаются по унифицированным идентификаторам ресурса (uniform resource identifier, URI), а затем протокол, идентифицирующий URI, позволяет взаимодействовать с этими ресурсами. Так, хотя мы и можем (как минимум в теории) перейти к интернету без HTTP (и в каком-то смысле уже переходим, потому что везде протоколы меняются на HTTPS), трудно представить переход к интернету без ресурсов. Принципы отражены в стилях API, потому что они опираются на разные фундаментальные концепции.
- *Протоколы* определяют конкретные механизмы взаимодействий, основанные на фундаментальных принципах. Хотя большинство взаимодействий

в интернете сейчас построено на HTTP, небольшая доля трафика еще относится к протоколу передачи файлов (File Transfer Protocol, FTP) и более специализированным протоколам (WebSockets и WebRTC). Согласование протоколов делает возможным взаимодействие, а тщательное проектирование платформы позволяет развиваться ландшафту протоколов, как мы наблюдаем сейчас с HTTP/2 и HTTP/3¹.

- *Шаблоны* — это конструкции более высокого уровня. Они объясняют, как сочетаются взаимодействия по нескольким протоколам для достижения целей приложения. Один из примеров — распространенный механизм OAuth. Это сложный шаблон на основе HTTP, предназначенный для достижения конкретной цели — трехступенчатой идентификации. Шаблоны — это способы решать распространенные проблемы. Они могут быть протоколами, как OAuth, или методиками, как приведенный ранее пример с CDN. Но, как и протоколы, со временем шаблоны развиваются, добавляются новые, а старые деактивируются и уходят в историю. Шаблоны — это общие методики решения проблем в пространстве решений, установленном принципами и протоколами.

Часто шаблоны развиваются со временем в ответ на меняющиеся требования. Например, аутентификация на основе браузера была популярна на заре интернета, потому что ее можно было легко контролировать с помощью конфигурации сервера и она хорошо работала для простых сценариев того времени. Но с ростом Сети ограничения такого подхода стали очевидны². Поддержка аутентификации стала стандартной функцией во всех популярных средах веб-программирования, а большая гибкость этого подхода заменила прежнюю практику, основанную на браузерах.

Важно помнить, что эта петля обратной связи и ответственна за успех интернета. Архитектура платформы начинается с простых решений. Появляются приложения, и какие-то из них расширяют границы того, что поддерживает платформа. При высоком спросе на платформу добавляются новые функции и возможности, что упрощает создание большего числа приложений, использующих новые функции. В обязанности архитекторов платформы входит наблюдать, где именно приложения расширяют границы, помогать разработчикам переходить их и развивать платформу, чтобы она лучше отвечала этим наблюдаемым потребностям разработчиков.

¹ HTTP/2 и HTTP/3 — хорошие примеры того, как веб-платформа может переходить от одной технологии к другой. Но они были специально разработаны почти без семантических отличий от HTTP/1.1, а большинство изменений направлены на более продуктивное взаимодействие.

² Идентификация через браузер была сложной для пользователя, например было не просто разлогиниться из какого-либо сервиса.

В ландшафтах API происходит такая же эволюция методов. И в ней надо видеть не *проблему*, а *особенность*, потому что методы можно корректировать по мере обучения команд и появления новых шаблонов, а иногда даже протоколов. Секрет успешных программ API — подходить к ним как к непрерывно развивающимся, разрабатывать их и управлять ими так, чтобы эта эволюция продолжалась.

Ландшафты API как языковые системы

Каждый API — это язык. То, как взаимодействуют поставщики услуг и потребители, когда дело доходит до раскрытия и использования определенных возможностей. Но важно помнить, что в этом разделе термин «язык» относится к взаимодействиям с API — к его дизайну, а не к внутренней работе API — его реализации на языке программирования.

Некоторые аспекты языка отдельного API определяются на фундаментальном уровне.

- *Стиль* API определяет базовые шаблоны диалогов (например, синхронный запрос/ответ или асинхронный на основе событий) и основные ограничения. Например, в API туннельного стиля основная абстракция диалога — запрос функции, а в API ресурсного стиля — понятие ресурсов.
- Затем *протокол* API определяет базовые языковые механизмы. Например, в API на основе HTTP при управлении основами диалога точно будут важны поля заголовка HTTP.
- Внутри протокола API часто есть много технологических подязыков в форме расширений основной технологии. Например, сейчас существует около 200 полей заголовка HTTP (<https://oreil.ly/B1PG0>), хотя основной стандарт определяет лишь небольшой их набор. API могут выбирать, какой из этих подязыков они будут поддерживать, основываясь на потребностях своего диалога.
- Определенные аспекты могут относиться к разным областям, и их можно использовать повторно в нескольких API (подробнее мы рассмотрим это в разделе «Словари» на с. 273). В качестве примера: эти многократно используемые части API могут быть определены как типы мультимедиа, а затем на них можно легко ссылаться и повторно использовать в API.

Главный вывод из всего этого в том, что *управление языками* — важная часть *управления ландшафтом*. Это сложная задача. Если языки слишком сильно унифицировать, пользователи системы будут чувствовать себя скованно и не смогут самовыражаться. Если же не пытаться поощрять возникновение общих языков,

в ландшафтах появится много решений одной и той же проблемы, что сильно их усложнит¹.

Один из самых распространенных шаблонов управления ландшафтом API — продвигать повторное использование языка с помощью пряника, а не кнута.

- *Метод кнута*: небольшая группа лидеров решает, какие языки должны задействоваться, и объявляет, что другие запрещены. Обычно это решение приходит сверху, и становится сложно экспериментировать с новыми решениями и применять новые методики.
- *Метод пряника*: можно предлагать для повторного использования любой язык, если с ним связаны инструменты и библиотеки, способные облегчить жизнь пользователям. Язык должен доказать свою полезность, и в случае положительного результата его можно добавить в список.

С применением метода пряника набор предложенных языков со временем будет и должен развиваться. Если возникают новые языки, должны появляться и новые способы доказать их пользу. И в случае успеха их тоже добавляют в список.

В итоге языки могут потерять популярность из-за появления более успешных конкурентов или из-за того, что пользователи просто начали применять другой метод работы. Это давно происходит в сфере XML/JSON. Хотя есть много XML-сервисов, сейчас выбор по умолчанию для API — это JSON (а через несколько лет, возможно, мы увидим, как он постепенно замещается другой технологией).

API через API

Масштабирование API предполагает, что, когда приходит время расширяться, у вас уже будет план автоматизации растущего количества задач для отдельных API и для ландшафта. Автоматизация требует четкого определения способов предоставления информации, ее использования и сбора. Если задуматься, то задача открытия доступа к информации и есть основное предназначение API! Это приводит нас к главной мантре «API через API»:

Выразите все, что хотите сказать об API, *через* API.

Можно сделать вывод, что неотъемлемая часть масштабируемого управления ландшафтами API вращается вокруг идеи использования «инфраструктурных

¹ Это хороший пример различия между комплексностью и сложностью. Комплексность ландшафта API определяется характеристиками разных API и их отражением в них. Сложность появляется, когда одна и та же проблема решается по-разному в разных API, создавая многообразие языков там, где оно может усложнить функциональность.

API» (или, скорее, инфраструктурной части существующих API). Простой пример такого API для инфраструктуры — способ предоставления информации о состоянии API. Если каждый API соответствует *стандартному шаблону* (подробнее об этом в разделе «Словари» на с. 273), автоматизировать задачу сбора информации о состоянии всех API очень легко. Выглядит это примерно так:

- начните с инвентаризации активных сервисов, посещайте каждый из них раз в десять минут;
- начните со стартовых страниц сервисов, перейдите по ссылке со словом **status**, чтобы найти информацию об их состоянии;
- соберите информацию о состоянии с каждого сервиса и обработайте/визуализируйте/заархивируйте ее.

По этому сценарию легко написать скрипт, регулярно собирающий эту информацию, и, основываясь на ней, создать инструменты и новые идеи. Это стало возможно, потому что *компонент API — стандартный способ доступа к определенным его аспектам*.

Теперь мы понимаем, что управление ландшафтом API и его развитие частично зависят от развития способов использования API для подобной автоматизации. При проектировании с учетом изменений эта информация может со временем добавляться, а существующие сервисы могут быть изменены при необходимости.

В этом примере открытие информации о состоянии стало новым шаблоном и появилась методика применения этой информации. Новая методика может из экспериментальной стать реализуемой, если в ландшафте API используются такие категории в указаниях по дизайну. В дальнейшем она может перейти в состояние увядающей и устаревшей, когда ее будут задействовать только старые сервисы, если в какой-то момент ландшафт перейдет на другой способ представления состояния API.

В последнем абзаце мы использовали слова «экспериментальная», «реализуемая», «увядающая» и «устаревшая» в качестве возможных состояний процесса руководства. Вы можете использовать другие слова, но важно понимать, что любое руководство развивается со временем. То, что когда-то было хорошим способом решения проблемы, может быть заменено на более быстрый и надежный.

Руководство должно помогать командам принимать решения о том, как решить проблему. Если они будут отслеживать состояние методик, им станет проще понимать, как те развиваются. Поэтому разумно следить за тем, какие решения удачны сейчас, какие могут стать удачными потом и какие были удачными раньше. В разделах «Структурирование руководства в ландшафте API» на с. 286 и «Жизненный цикл руководства» на с. 289 мы обсудим конкретные способы структурирования и развития руководства подробнее.

Решение этой проблемы так, чтобы оно стало *элементом дизайна* API, упрощает управление большими ландшафтами API, поскольку определенные элементы дизайна повторяются в разных API и их можно использовать для автоматизации.

Понимание ландшафта

Ландшафт API не отличается от других систем продуктов или функций, цель которых — развиваться легко и беспрепятственно, служа прочным фундаментом для создания новых внутренних или внешних функций. Во всех этих случаях достигают компромисса между оптимизацией ради одной известной цели и оптимизацией ради облегчения изменений. Оптимизация ради облегчения изменений всегда требует компромиссов по сравнению с фиксированными целями. Ключевые факторы изменчивости — открытость ландшафта для эволюции и управление им способом, позволяющим со временем вносить новые идеи в текущее состояние ландшафта и его динамику.

Здесь важное место занимает мысль, которую мы обсудили в прошлом разделе: выразите все, что хотите сказать об API, *через* API. Это может быть так же просто, как предоставление информации о состоянии, упомянутое ранее, или куда более сложно, как требование, по которому вся документация по API должна быть частью его самого, или управление безопасностью API через сам API. При таком подходе API переходят на самообслуживание, открывая доступ к информации, которая нужна для их понимания и использования.

Иногда такой подход может стоить дорого. Если рассматривать крайний случай, когда вы хотите создать API, открытые для доступа миллионам разработчиков, то имеет смысл сделать их максимально высокотехнологичными, чтобы разработчики могли легко понять и применить их. Тогда появляется продукт для массового рынка, и поэтому он оптимизирован именно для такого использования.

В большинстве ландшафтов API будут сотни или тысячи API, и нет необходимости вкладывать средства в доведение каждого из них до идеального продукта для массового рынка. Но и небольшая стандартизация может потребовать большой работы, например: убедиться, что команда API легко находит контактную информацию, предоставить минимальную документацию, машиночитаемое описание и несколько примеров для начала.

А когда кажется, что API нужно еще слегка «отполировать», может помочь эволюционная модель ландшафта: команды API начинают применять методы улучшения опыта разработчика, которые становятся устоявшимися и получают поддержку. Суть в том, чтобы наблюдать за меняющимися потребностями, за решениями, которые практикуются API, и поддерживать все, что можно, на уровне ландшафта.

Восемь аспектов ландшафта API

Управление ландшафтами API может быть непростой задачей. Это требует уравнивания вопросов скорости и независимости продукта с конфликтующими вопросами согласованности и устойчивости во времени. Но прежде чем подробно обсудить развитие ландшафта API (см. главу 10), мы подготовим базу для вопросов, которые имеют значение при долгосрочном развитии API-ландшафтов.

В модели восьми аспектов ландшафтов API мы предположили, что API проектируются и разрабатываются разными способами (и следуют разными путями через свои индивидуальные жизненные циклы, как обсуждалось в главе 7). Эти восемь аспектов подобны рычагам или циферблатам для вашей системы управления API. Вам нужно будет понаблюдать за ними и настроить так, чтобы получить лучший результат.

В частности, предполагается, что разработка и реализация стратегии ландшафта следует платформенной модели, как обсуждалось в разделе «Принцип платформы» на с. 263, где добавление API на платформу должно соответствовать принципам, протоколам и шаблонам этих платформ.

Говоря о такой открытой модели ландшафта API, важно рассмотреть следующие восемь аспектов, в той или иной форме взаимодействующие с *дизайном* и *реализацией* отдельных API и *организацией* всего ландшафта. Они также помогут *наблюдать* за системой и понять, как она эволюционирует.

В следующих разделах мы представим и опишем восемь важных аспектов, о которых нужно помнить при управлении ландшафтом API. Мы будем применять их и в главе 10 для модели зрелости ландшафта, которая использует риски, возможности и потенциальные вложения во все эти области как способ управления зрелостью ландшафта. Еще мы задействуем их в главе 11, чтобы объяснить, как управление жизненным циклом на уровне ландшафта поможет с жизненным циклом отдельного API.

Мы определили восемь основных «V»: variety (разнообразие), vocabulary (словари), volume (объем), velocity (скорость), vulnerability (уязвимость), visibility (видимость), versioning (контроль версий) и volatility (изменяемость), чтобы направлять и фокусировать управление ландшафтами API. Далее мы подробнее обсудим каждый аспект.

Разнообразие

Под *разнообразием* подразумевается то, что в ландшафтах API часто есть программы, созданные и разработанные разными командами на разных технологических платформах и для разных пользователей. Цель API — позволить этому разнообразию существовать и обеспечить командам большую автономию.

Например, может иметь смысл наличие одного руководства по дизайну, предлагающего API ресурсного стиля по умолчанию для сервисов на основной платформе, потому что они просты в применении и с ними может работать много пользователей. Но конкретно для серверной части мобильных приложений лучше предложить API, основанный на запросах и использующий технологию GraphQL, потому что так мобильные приложения смогут получать нужные им данные за одно взаимодействие.

Одна из целей управления ландшафтами API — контроль разнообразия и его ограничение, чтобы потребителям не нужно было учиться взаимодействовать со множеством разных стилей API. Ограничивать разнообразие одним вариантом дизайна — тоже не лучшая идея, если есть группы предпочтительных вариантов, подходящих для разных сценариев, чтобы лучшие продукты представлялись множеству пользователей.

Управление разнообразием API-ландшафтов — это баланс между ограничением числа вариантов, позволяющим избежать непродуктивного множества версий API, и открытостью для вариантов, которые увеличивают ценность ландшафта API.

Фундаментальный аспект таков: относитесь к управлению разнообразием как к долгосрочному руководству — не создавайте в своем ландшафте ничего, что помешает развивать понимание разнообразия, которое вы потом захотите подержать. Если в чем-то можно быть уверенным, так это в том, что ландшафты API через несколько лет точно не будут выглядеть как сейчас, поэтому не перекрывайте пути работы с развитием разнообразия, чтобы не загнать себя в угол.

ПРЕДПОЧТЕНИЯ В API

У вас могут быть определенные предпочтения в области дизайна API, которые можно использовать при руководстве разработкой. Можете поощрять разработчиков придерживаться этих предпочтений, потому что для ландшафта они кажутся лучшим соотношением доходов и расходов.

Но вы не должны ставить все на один набор предпочтений. Может появиться что-то лучшее, что заставит вас передумать. Или у вас просто могут быть потребители, запрашивающие определенные API, и вы хотите, чтобы они были довольны.

Рассмотрим GraphQL: неважно, что вы думаете об этой технологии, но если работаете над партнерскими или внешними API, то можете узнать, что определенное число пользователей предпочитает именно GraphQL. Важно иметь возможность поддерживать эти группы предпочтений, потому что со временем они разовьются и будут показывать, как развивается ваш ландшафт.

Не думайте, что есть единственно верный способ создавать API: любое ваше действие зависит от технологий и пользовательских предпочтений и со временем может измениться.

Поддержка и контроль разнообразия — длительный процесс, который должен быть встроен в ландшафт с самого начала. Ограничивать его с помощью принципов, протоколов и шаблонов — это баланс между пониманием того, как используются API и какую ценность они приносят, и принятием решений для увеличения дохода. С развитием ландшафта API (см. главу 10) вы можете начать лучше понимать его состояние, пути развития и применения. Тогда можно будет контролировать разнообразие с помощью баланса между затратами на его увеличение (что снижает согласованность внутри ландшафта) и специфичным дизайном API (что улучшает ценность API в подгруппе внутри ландшафта).

Словари

Каждый API — это язык (см. раздел «Ландшафты API как языковые системы» на с. 267). Он определяет способы применения сервиса разработчиками через шаблоны взаимодействий, базовые протоколы и обмен представлениями. Стандартизация структурных элементов API с помощью общей терминологии — действенный способ увеличить согласованность внутри ландшафта.

Для некоторых аспектов этого языка необязательно каждый раз изобретать велосипед. Простой пример — проблема сообщений об ошибках. Многие API на REST на основе HTTP составляют свои сообщения об ошибках, потому что хотят выйти за границы использования только стандартизированных кодов статусов HTTP. Такой формат можно определить индивидуально, но в формате «деталей проблемы» от RFC 7808 (<https://oreil.ly/xhg98>) уже есть стандартная форма (если API задействует JSON или XML). У применения терминологии отчетов об ошибках два преимущества.

- Командам, *разрабатывающим* API, не придется изобретать, составлять и документировать новую терминологию для сообщений об ошибках. Они могут адаптировать существующую и расширить ее, чтобы отразить специфические аспекты своих сообщений.
- Командам, *использующим* API, не нужно изучать проприетарный формат. Они поймут эту часть языка API, после того как впервые столкнутся с этой терминологией. Так разработчикам будет проще понять аспекты API, использующиеся и в других API.

Следующий пример взят из RFC 7807 и показывает, как такой формат может сочетать стандартную и специфическую терминологию. Здесь части **type**, **title**, **detail** и **instance** (тип, название, детали и программа) определены RFC 7807, а данные в **balance** и **accounts** (баланс, аккаунты) — специфические и определяются для конкретного API. Возможно, в своем ландшафте API вы всегда будете

использовать отчеты об ошибках от RFC 7807, но данные со временем меняются, когда API раскрывают конкретные детали проблемы:

```
{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
  "instance": "/account/12345/msgs/abc",
  "balance": 30,
  "accounts": ["/account/12345",
               "/account/67890"]
}
```

Часто такое повторное использование терминологии достигается с помощью стандартов — неважно, формальных на уровне интернета или неформальных внутри одного ландшафта. Важно не придумывать то, что уже придумали до вас.

На самом деле способность одинаково работать с формальными и неформальными стандартами очень важна, чтобы иметь возможность решить, когда лучше переключиться на один из стандартов ради какого-то аспекта языка API.

ЕИМ И API: ИДЕАЛ ПРОТИВ ПРАГМАТИЗМА

Хотя использование официальных стандартов — простой способ избежать повторного составления словаря, иногда организации идут дальше. Крайний случай — это идея *информационной модели предприятия* (enterprise information model, EIM). Ее цель — создание полной и согласованной модели всего, что должно быть в организации. Часто (обычно в больших организациях) эта цель недостижима. На документирование *всего* словаря крупной организации затрачивается очень много сил, а в конце оказывается, что реальность и системы уже изменились, превратив EIM в снимок из прошлого.

EIM должна развиваться вместе с компанией, но синхронизировать их очень сложно. Например, у организации может быть определенная модель пользователя и связанная с ним информация. Скорее всего, она все время дополняется, а разные продукты расширяют/дополняют модель пользователя как им удобно. Попытки убедиться в согласованности этих расширений и дополнений замедлят разработку и предоставление услуг. Это значит, что обычно приходится выбирать между EIM, отражающей статическую модель компании, и увеличением способности организации к изменению при необходимости, отказываясь от идеально разработанной и гармоничной модели всего.

Более реалистичный подход: предположить, что EIM — это объединение всех возможностей, доступных через API. Но необходимость решить, насколько вам нужна стандартизация словаря, лежит на руководстве ландшафтом API. Для некоторых вопросов, касающихся нескольких областей (например, вышеупомянутых сообщений об ошибках), лучше предпочесть стандартную терминологию.

Понятия с более четкой предметной областью будут выражены с помощью дизайна API, и тогда именно он станет EIM в этой области. Недостаток такого подхода в том, что у вас не будет одной согласованной и единообразной модели, которой часто стремится стать EIM. Но преимущество этого подхода в том, что модель предметной области теперь полностью управляема (с помощью API). А как сказано в определении этого подхода, то, что не выражено и/или не управляется с помощью API, — это не часть EIM.

Управление словарем успешнее всего, если оно сосредоточено на том, чтобы его было проще найти и использовать повторно, а не на создании единственно верной модели своей концепции. Если словарь легко найти и повторно применить, разработчики будут заинтересованы в том, чтобы пользоваться им, пока он отвечает их целям, потому что так им не придется разрабатывать свой.

Выбор словаря — сложная тема. Не будем углубляться в дебри UML и XML, определения, документации и составления словарей, но напомним, что цель API — *не открыть* всем модель реализации, а создать для нее *интерфейс*. Это часто отличается от внутренней модели сервиса или предметной области (как описано в главе 3, одна из стратегий предполагает *начать* с этой модели интерфейса, даже не задумываясь о ее реализации).

Словарями можно управлять по-разному. У каждого способа есть свои преимущества и ограничения.

- Когда словари используются для *полного отражения взаимодействий API*, они становятся точными моделями значения и идентификации разных концепций своей предметной области. Типичные примеры — XML- или JSON-схемы. Различается и *распознавание* этих словарей: иногда, например в интернет-API, применяются типы данных, а иногда — идентификаторы схем, которые затем ассоциируются с их использованием в API.
- Словари тоже часто задействуются в качестве *структурных элементов внутри представления*. Это позволяет API поддерживать представления, части которых применяют эту терминологию. У XML есть для этого довольно сложный механизм с пространствами для имен, а у JSON нет формального способа определения того, что какая-то часть представления использует стандартную терминологию. Мы рассмотрели пример из RFC 7808, где есть встроенные стандартные термины, и он позволяет API добавлять свои значения в формат сообщения об ошибке.
- Словарь может быть и *общим типом данных*. Тогда часто есть шаблон определения и поддержки развивающегося набора значения для типа данных с помощью *реестра*. Реестры позволяют пользователям делиться развивающимся набором известных значений для определенных типов данных. Это распространенный шаблон для фундаментальных технологий интернета. В качестве примера можно привести связи ссылок гипермедиа: есть реестр, позволяющий разработчикам узнавать о существующих отношениях или добавлять новые при необходимости.

Важно выбрать правильный способ утверждения терминологии и управления ею. Он определит, будет ли командам API проще (частично) собирать свои программы из существующих структурных элементов, а не создавать их с нуля. Но утверждать терминологию нужно только при наличии четкой модели, по

которой ее можно легко найти и применить повторно. Инструментарий — отличный способ использовать словари, чтобы у дизайнеров был четко определенный набор вариантов. Например, при создании ресурсно-ориентированного API у HTTP есть набор концепций, которые являются открытыми и развивающимися словарями.

Как показано на рис. 9.2, у HTTP много связанных с ним словарей (это скриншот «Веб-концепций» — открытого репозитория, дающего доступ к стандартным и распространенным значениям для этих словарей). Вряд ли в ландшафте дизайнерам API на HTTP понадобится около 200 существующих полей заголовков HTTP. Но на одной из *подгрупп* этих значений можно основывать инструменты для дизайна и реализации API, устанавливая общую методику выбора поля заголовка внутри компании.

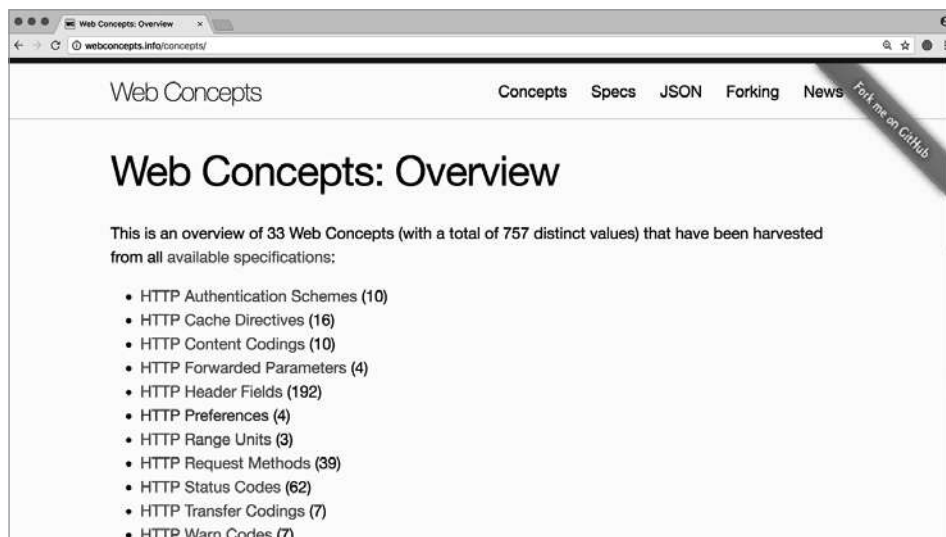


Рис. 9.2. Веб-концепции: терминология HTTP

Терминология HTTP — пример технической информации, где важно поделиться методами использования конкретной технологии. С точки зрения предметной области те же принципы могут применяться к предметным концепциям, таким как типы клиентов. К примеру, у компании может быть словарь по пяти разным типам клиентов, который со временем может вырасти и охватить дополнительные типы. Создать реестр терминов этой области будет полезно, чтобы общие значения были доступны разработчикам и инструментам и со временем могли развиваться.

Другой возможный способ эффективного управления терминологией — использование отраслевых стандартов. Хотя они не всегда идеально или полностью подходят для API, они все же могут быть полезны как структурный элемент. Подумайте, к примеру, о том, как представлять информацию о стране или языке (для этого есть стандарты ISO). Есть и более сложные (и часто более вертикальные) стандарты, такие как *Ресурсы быстрой совместимости в области здравоохранения* (Fast Healthcare Interoperability Resources, FHIR) для обеспечения взаимодействия электронных медицинских карт.

Объем

Как только организация начинает серьезно относиться к своей стратегии цифровой трансформации, *объем* существующих или доступных сервисов может резко вырасти. Одна из причин — то, что все бóльшая часть организации отражается в технологиях, появляется «цифровой отпечаток», поэтому число API в ней увеличивается. Оно легко может достичь сотен или тысяч для организаций, превышающих определенный размер и имеющих некоторую историю разработки API. Ландшафты API должны легко работать с таким масштабом, поэтому их размер становится прежде всего решением в интересах организации.

Вторая причина в том, что при работе со стратегией «API как продукт» (см. главу 3) все, что делается в организации, задумывается по принципу «сначала API». Потому что только так это может стать частью сетевого эффекта растущего использования API в организации. Это может означать, что многие такие инициативы при «API как продукт» далеко не пройдут, ведь API должны не только облегчать и ускорять сочетание услуг организации, но и помогать экономить на этом, чтобы продукты можно было быстро создавать и оценивать (и, возможно, отклонять).

Есть разные подходы к работе с объемом API в ландшафте. Иногда можно уменьшить объем насколько возможно и попытаться внимательно управлять ландшафтом API. В других случаях фокусируются на облегчении работы с этим объемом и поиске новых идей для ландшафта API. Поэтому объем становится в основном проблемой управления и с ним можно работать с помощью постепенных улучшений.

В любом случае рост объема естественен, так как ландшафт API развивается и в нем появляются новые сервисы. Поэтому так же естественно, что с ростом ландшафта работа с растущим объемом не должна становиться проблемой.

Скорость

Одно из важнейших преимуществ цифровой трансформации — ускорение разработки, реализации, тестирования и изменения продукта. Это происходит благодаря тому, что управление отдельными API в организации развивается и ландшафт API позволяет работать быстрее, чем раньше. Также компания становится сетью отдельных, но взаимозависимых сервисов. Если до этого было много структурных элементов, которые долго находились в стабильном состоянии, и новые добавлялись редко, то теперь все меняется и добавляется гораздо быстрее.

Дополнительная *скорость* — один из ключевых отличительных аспектов, приносящих выгоду организациям, но важно не забывать о *безопасности*. Иначе растущая скорость станет угрожать устойчивости операций, что недопустимо.

Для многих организаций скорость становится одним из главных мотивов для начала цифровой трансформации. Рынок сейчас меняется быстрее, и появление конкуренции ускоряется, поэтому важно иметь возможность действовать или реагировать как можно быстрее. Любые факторы, замедляющие скорость превращения идей в продукцию или препятствующие управлению постоянно растущим и развивающимся портфелем продукта, вредят и конкурентоспособности организации.

В ландшафтах API скорость достигается в основном за счет предоставления командам большей свободы проектирования и разработки в соответствии со своими предпочтениями и графиком. Это важно для минимализации всех частей процесса, которые могут замедлить выпуск API. Как мы уже говорили, важный урок, извлеченный из прежних подходов к API, в том, что *разделение* (то есть возможность изменять и размещать индивидуальные компоненты независимо от других) нужно для сокращения времени на создание API¹. Но если вы позволяете добавлять, изменять и исполнять отдельные функции независимо друг от друга, стоит изменить и традиционные методы тестирования API и работы с ним.

Как обсуждалось в разделе «Принцип платформы» на с. 263, один из распространенных подходов к тому, чтобы избежать связанности, приносящей потери в скорости, — это отказ от интеграции. Использовать платформу так, как мы описываем, означает уйти от интеграции и принять идею децентрализации. Это выгодно, ведь слабое связывание обеспечивает высокую скорость в отдельных частях, потому что уменьшаются затраты на координацию при внесении изменений. Минус этого подхода в том, что придется подстроить выпуск API

¹ Ранее мы сравнивали это с технологией SOAP, которая фокусировалась только на API и не указывала, как управлять растущим и меняющимся ландшафтом сервисов SOAP.

и операции под новую систему и убедиться, что она соответствует стандарту устойчивости, необходимой организации.

Если взять крайности, можно в очередной раз привести в пример интернет — это интересное упражнение. Он быстро меняется, потому что могут появиться новые сервисы, измениться старые, и все это способно повлиять или не повлиять на пользователей. Можно возразить, что в некоторой степени в интернете всегда что-то «сломано»: какой-нибудь сервис недоступен или на пользователя влияют его изменения. Но итоговая скорость всей системы сполна компенсирует присущую ей хрупкость, и с помощью качественного управления изменениями и правильных методов развертывания и тестирования можно найти в такой системе баланс между скоростью и выгодой.

Уязвимость

Только организации, где не применяются ИТ, могут не беспокоиться об уязвимости (по крайней мере напрямую) от информационных атак. Но с тенденцией к увеличению числа ИТ, большей согласованности между бизнесом и ИТ и открытию ИТ-возможностей через API создается множество уязвимостей, которыми нужно управлять. Важно убедиться, что опасность увеличения мишени для атаки в ландшафте API компенсируется преимуществами от возросших гибкости и скорости.

Экономический подход API выгоден тем, что организации могут быстрее реагировать и реструктурироваться, используя внутренние API, и передавать некоторые функции внешним работникам с помощью них. Но у всего этого есть и обратная сторона — появляются взаимозависимости.

В июне 2018 года Twitter купила компанию Smyte, предоставляющую технологии для борьбы с насилием. Многие компании пользовались услугами Smyte с помощью API, предлагавших инструменты для предотвращения оскорблений, сексуальных домогательств и рассылки спама в Сети. У этих компаний даже были заключены договоры со Smyte. Сразу же после приобретения Smyte без предупреждения Twitter закрыла свои Smyte API, создав проблемы для компаний, которые полагались на них.

Этот и подобные ему случаи учат нас всегда относиться к *внешним взаимозависимостям* как к уязвимым местам, всегда встраивать отказоустойчивость в сервисы, чтобы справляться с потенциальными прерываниями обслуживания, и сделать это базовой практикой разработки. Можно пойти дальше, распространив это правило на любую взаимозависимость, а не только внешнюю. Потому что при увеличении скорости возрастает и вероятность появления проблем с запущенными зависимостями, и любое неустойчивое использование сервиса может стать проблемой.

Реализовать эту устойчивость не так-то просто. Иногда взаимозависимости очень важны, и практически невозможно как-то компенсировать их недоступность. Но даже в таких случаях важно ответственно подойти к делу. Сервис не должен ломаться, зависеть или уходить в неопределенное операционное состояние. Он должен четко сообщить о ситуации, чтобы ее можно было проанализировать и исправить.

Помимо технической уязвимости проблема еще и в том, что с увеличением числа доступных сервисов растет и количество API, которые могут быть мишенью для атаки. Это важно, когда речь идет о попытках злоумышленников получить доступ к системе или нарушить ее работу. Также может возникнуть проблема, когда API-интерфейсы раскрывают информацию или возможности, которые по юридическим, нормативным или конкурентным причинам не должны быть доступны. Это может серьезно повлиять на отношение к организации, поэтому таким случаям нужно уделять не меньше внимания, чем прямым угрозам.

В целом с уязвимостью нужно справляться с помощью безопасной работы с API. Безопасность означает: лучше относиться ко всем взаимозависимостям как к брешам в защите и никогда не зависеть от доступности или конкретного поведения другого API. Это означает всегда следить за тем, чтобы злоумышленники не могли получить доступ к API-интерфейсам в ландшафте или нарушить их работу.

Видимость

Видимость и масштаб — практически естественные враги. Если у вас мало программ и все, кто их разрабатывает, использует и управляет ими, входят в одну небольшую команду, то все действия будут видны и легкообнаружимы. Нужно спросить окружающих, и они сразу расскажут, где найти искомое и как оно работает. Если вам нужен обзор всех составляющих, можете их просто просмотреть. Но все это не относится к большим ландшафтам API.

В больших и распределенных системах добиться видимости гораздо сложнее. Типичные схемы видимости в больших масштабах обычно сочетают в себе два аспекта (и тут мы снова вспомним интернет, потому что это самый большой пример видимости на данный момент).

- *Публиковать что-то* нужно так, чтобы это можно было легко обнаружить. В интернете для этого нужны рабочий веб-сервер и HTML для публикации, который используется для индексирования содержимого. Есть механизмы Sitemaps и Schema.org для облегчения обнаружения, но они были изобретены гораздо позже начала работы поисковых механизмов.

- *Поиск* — обычно больше, чем просто обнаружение. Он чаще всего связан с контекстом и тем, насколько «полезные» или «бесполезные» результаты были получены. Google произвела революцию своим алгоритмом PageRank (<https://oreil.ly/xnt6Z>), рассчитывающим релевантность по популярности. Во многих случаях поиск рассматривается как дополнительный сервис после первоначальной задачи по обнаружению и сбору информации (это действие в интернете называется *сканированием*).

Как мы обсудили в разделе «API через API» на с. 268, предварительное условие видимости и использования API в ландшафте — раскрытие информации об API через API. Именно этот ход заставил работать интернет. Вся информация о веб-странице находится на ней, и есть единый способ получить доступ ко всем веб-страницам.

Итак, видимость означает:

- отслеживание того, как помочь пользователям, сделав API заметнее;
- размышления о том, заключается ли проблема в возможности их найти (можно ли их вообще локализовать?);
- в представлении (доступна ли необходимая информация через API?) или поиске (применяют ли поисковые механизмы открытую информацию?);
- доработку тех факторов системы, которые нужно доработать для улучшения видимости.

Важна видимость не только самого API, но и его элементов. Например, такие аспекты, как стандартный формат деталей ошибки, который мы обсуждали в разделе «Словари» на с. 273, помогает улучшить видимость внутри API, поскольку теперь можно использовать инструменты, понимающие детали ошибки в разных API. В принципе, есть прочная связь между терминологией и видимостью: чем больше общей терминологии у разных API, тем проще пользоваться этим их общим аспектом. Для модели API через API видимость, полученная благодаря общей терминологии, очень полезна. Если вы хотите сказать что-то об API, выразите это через него и сделайте этот способ общим для нескольких API.

Контроль версий

Одна из проблем при переходе от интеграции к децентрализации — то, что изменения тоже происходят децентрализованно. Плюс здесь в том, что увеличивается скорость — одно из важных преимуществ ландшафтов API. Но чтобы разумно справляться с изменениями в ландшафте, к *контролю версий* нельзя подходить как в интегрированных системах.

Распространенный подход при этом — *избегать появления версий* как можно дольше или избегать описания того, как выпускаются разные версии и как они должны по-разному использоваться. Снова обратимся за примером к интернету. Немногие сайты выпускают новые версии, с которыми пользователям нужно учиться работать заново. Они стремятся вносить улучшения так, чтобы пользователи поняли, как применять новые функции не в ущерб установленному порядку действий, выполняемых при использовании сайта.

Главная цель создания версий в ландшафтах API должна быть похожа на цель создания версий сайтов: не навредить существующим пользователям и изначально разработать все API с функцией расширения, чтобы между ожидаемой и предоставляемой версией была слабая связь¹.

В такой модели не нужен жесткий контроль версий: к изменениям относятся как к *улучшенной/расширенной версии API*, которую потребителям не обязательно изучать, если они не хотят использовать новые функции. Несовместимые изменения вредят API. Нужно выпускать новую версию так, чтобы пользователи могли перейти на нее со старой. Тогда это новый API, а не обновленная версия старого.

Можно сказать, что версии API не имеют значения. Но со временем API развивается, поэтому полезно иметь возможность сделать «слепок» функций API в какой-то определенный момент, чтобы понять, что изменилось. Поэтому все-таки номера версий по-прежнему остаются полезной концепцией, ведь они помогают определить развивающиеся возможности API и ориентироваться в его истории.

Семантический контроль версий

Это простая схема контроля (<https://semver.org>), основанная на структурированном и осмысленном использовании номеров версий. Семантические номера версий структурированы по схеме `MAJOR.MINOR.PATCH` (Мажорная. Минорная. Патч). Эти части состоят из цифр и имеют следующее значение.

- `PATCH`. Версии-патчи просто исправляют ошибки, не влияя на конкретные интерфейсы. Они лишь корректируют неправильное поведение или вносят другие изменения, имеющие отношение только к реализации.
- `MINOR`. Минорные версии — это совместимые изменения в интерфейсе, означающие, что клиенты могут продолжать использовать его как есть, без

¹ Один из способов следовать шаблонам для надежной расширяемости (<http://bit.ly/2DDGmWR>) — использовать осмысленную базовую семантику, иметь четко определенную модель расширяемости и обработки взаимодействий с API и возможными расширениями.

необходимости приспосабливаться к изменениям в интерфейсе. Это минимизирует пользовательские усилия на адаптацию к новым версиям. Им нужно вносить изменения только после выхода новой минорной версии, чтобы воспользоваться преимуществами нового функционала.

- *MAJOR*. Мажорные версии вносят изменения, требующие обновления API. Клиенты не могут просто перейти с одной мажорной версии на другую.

При использовании семантического контроля версий продуктов API их номера становятся частью документации, так как несут в себе информацию о том, что изменилось между версиями. Полезно документировать и конкретные изменения, но семантический контроль версий — это отправная точка, где клиенты могут решить, хотят они исследовать обновления API или нет.

Изменяемость

Как уже говорилось в отношении аспектов *скорости и управления версиями*, динамика ландшафтов API отличается от подходов к интеграции, поэтому службы должны ее учитывать. *Изменяемость* неизбежна в больших распределенных системах. Сервисы меняются (при этом стоит избегать потенциально опасных изменений), перестают работать (при децентрализованной разработке и операциях создается менее централизованная модель доступа), могут даже исчезнуть (они не вечны). Ответственные методы разработки в ландшафте API учитывают эти соображения для всех зависимостей.

Изменяемость можно считать результатом децентрализации реализации и операций, а также признания того, что такой шаг приводит к более сложному набору сценариев отказа, чем двоичный код по сравнению с неработающими интегрированными монолитными системами. Это неизбежный побочный эффект радикального разделения компонентов, и решение этой дополнительной сложности, безусловно, связано с издержками (один из аспектов знаменитого «Устава API» Джеффа Безоса и его последствий (см. раздел «Устав Безоса» на с. 78)).

Разработчикам может быть непросто перейти от принципа «программирование как часть системы» к принципу «разработка как часть системы». Традиционные предположения о надежности и доступности становятся неверными, а переход к модели, где любая внешняя взаимозависимость приложения может привести к провалу, требует дисциплины при разработке.

В то же время прекрасно известны техники постепенного сокращения возможностей. Снова возьмем для примера интернет. Хорошо разработанные веб-приложения часто и устойчиво задействуют такую технику. Этот принцип применяется к существующим взаимозависимостям от других сервисов и зависимостям от среды, где выполняется программа (браузера). Веб-приложения

работают в среде, которую контролировать гораздо сложнее, чем обычную среду для запуска. Поэтому им нужна встроенная устойчивость, чтобы работать в большом количестве браузеров или сред.

Похожий подход к разработке нужен и для приложений в ландшафтах API. Чем более защищенным вы делаете приложение при программировании, тем более вероятна его устойчивость к изменениям в рабочей среде. Главная цель приложений в ландшафтах API — максимально устойчивая работа и отсутствие предположений, зависящих от доступности других компонентов.

Итоги главы

В этой главе мы углубились в идею ландшафтов API. Мы узнали, что существующие интеграционные решения можно рассматривать как «прото-API», что масштабирование практики API создает больше проблем, и узнали, как в идеале должна выглядеть платформа API.

Самое важное — это то, что для получения дохода от ландшафтов API нужно относиться к ним как к постоянно меняющейся среде, где изменения вызываются наблюдением за работой отдельных API. Роль ландшафта — создать на основе этих изменений принципы, протоколы и шаблоны, помогающие повысить продуктивность команд API.

В этой главе мы ввели восемь V-систем API — набор аспектов, о которых полезно помнить, работая с конкретными задачами ландшафта. Поскольку управление ландшафтом API — это всегда постепенный процесс, мы будем использовать эти аспекты в следующей главе, где обсудим, как они помогут обосновать инвестиции в ландшафт API и как это приведет к повышению зрелости всех аспектов.

Путешествие по ландшафту API

Настоящая проблема в том, что программисты слишком долго волновались о производительности не там и не тогда, когда нужно. Преждевременная оптимизация — корень всех зол (или большей их части) в программировании.

Дональд Кнут

В главе 9 мы подробно рассмотрели ландшафты API, сосредоточившись на их главных аспектах. Теперь перейдем к теме развития этих ландшафтов. Как и прежде, мы будем рассматривать это как путешествие, а не пункт назначения: ландшафт API никогда не бывает законченным, ведь он всегда будет продолжать развиваться в соответствии с эволюцией бизнеса и технологий (как обсуждалось в главе 7).

По нашей прошлой аналогии, такой взгляд на ландшафты API похож на постоянную эволюцию интернета. Он тоже никогда не закончит развиваться: появление новых технологий, сценариев и схем использования постоянно подпитывает его эволюцию. Это может показаться пугающим, но именно непрерывное развитие — причина успеха интернета. Без него он в какой-то момент стал бы неактуальным, и победил бы какой-нибудь другой подход.

Как и интернет, ландшафты API должны постоянно развиваться. Ландшафтная архитектура должна эволюционировать в соответствии с изменениями потребностей и развитием принципов, протоколов и шаблонов.

Но даже лучшим инженерам приходится сталкиваться с ограничением ресурсов, отведенных на изменения в архитектуре. Кроме того, команды API могут справиться только с максимальным уровнем изменений. Поэтому, если ландшафт хорошо работает в качестве платформы для API-продуктов, может быть

экономичнее повторно использовать установленные принципы, протоколы и шаблоны, а не пытаться создать идеальную платформу для каждой отдельной проблемы.

Поэтому, чтобы понять, как протекает развитие ландшафта, нужно знать, за чем наблюдать и во что вкладывать, чтобы улучшить его. Мы снова задействуем восемь аспектов из раздела «Восемь аспектов ландшафта API» на с. 271. Но сейчас мы сделаем это, чтобы показать потенциальную опасность в случае их неулучшения, и расскажем, как вносить эти изменения. Для этого мы определили несколько контрольных точек, которые помогут вам лучше понять уровень зрелости разных областей и куда направлять инвестиции, чтобы сделать развитие более последовательным и единообразным.

Прежде чем обсуждать все это, мы сначала рассмотрим некоторые организационные аспекты, связанные с повышением осведомленности и упреждающим управлением вашим ландшафтом API.

Структурирование руководства в ландшафте API

Создание руководства и управление им — важная часть управления ландшафтом API. Оно сообщает каждому, *зачем* что-то нужно сделать, *что* предпринимается для решения проблемы и *как* реализация может придерживаться руководства. Руководством нужно управлять как живым документом, который любой может прочесть, прокомментировать и обновить. Так каждый разработчик будет участником создания и развития этого документа.

Для повышения эффективности ландшафта API всегда нужно четко разделять ответы на вопросы «что?» и «как?» в теме требований к API, давать развернутый ответ на вопрос «зачем?», объясняющий причину действий, и обеспечивать инструменты и поддержку конкретных способов удовлетворения требований.

- «Зачем?» (*мотивация руководства*). Описывает обоснование требования или рекомендации, гарантируя, что это не надуманное правило, у которого есть объяснение. Документирование обоснования облегчает определение того, нацелены ли предложенные альтернативные решения на одно и то же обоснование.
- «Что?» (*руководство дизайном*). Объясняет подход, используемый для решения вопроса «почему?», чтобы стало ясно, что должны делать API для решения этих задач. Для этого нужно определить четкие требования

к самому API (а не к его реализации). Главный аспект ответа на этот вопрос — убедиться, что он не смешался с ответом на вопрос «как?», который задается отдельно.

- «Как?» (*руководство по реализации*). Рассказывает о конкретных способах решения проблем для реализации руководства. Это могут быть определенные инструменты или технологии. Возможны несколько подходов к выполнению одной задачи. Со временем, когда команды API найдут или создадут новые способы решения проблем, их добавят к существующим подходам, и постепенно эти способы смогут стать частью ландшафта.

Руководящие принципы должны помочь каждому быть эффективным командным игроком в общем ландшафте API. Они созданы для повышения продуктивности команд API и изменения культуры и методов разработки API, которые легко отслеживать и которыми просто управлять.

Далее приведем конкретный пример использования этой схемы. Здесь решается распространенная задача — выведение API из эксплуатации, а конкретно: как сообщить пользователям о приближающемся удалении. В этом примере есть один пункт «зачем?», два «что?» и три «как?».

- «Зачем?» (*мотивация руководства*). Пользователи сервиса должны знать о предстоящем выводе API из эксплуатации. Поэтому у API-интерфейсов должен быть механизм оповещения о том, что они выходят из строя.
- «Что?» (*руководство по дизайну № 1*). API могут использовать поле заголовка HTTP `Sunset` для оповещения о приближающемся списании. Нужно конкретизировать, какие ресурсы будут задействовать поле заголовка (чаще всего выбирают домашний) и когда оно будет появляться (обычно сразу после определения момента списания). Можно уточнить, что поле заголовка будет появляться до определенного срока перед списанием, чтобы дать пользователям время разобраться с последствиями этого решения и/или перейти на другой API.
- «Как?» (*руководство по реализации № 1 для руководства по дизайну № 1*). Один из методов реализации процесса — управление полем заголовка HTTP `Sunset` с помощью конфигурации. Как только она заканчивается, поле заголовка перестает высвечиваться в ответах. Когда становится известно о будущем списании, добавляется эта конфигурация, и поле заголовка появляется на ресурсах, определенных для этого API.
- «Как?» (*руководство по реализации № 2 для руководства по дизайну № 1*). Еще один метод реализации — добавить поле заголовка HTTP `Sunset` через шлюз API. Вместо самой реализации API поле заголовка добавляется шлюзом API, как только методика сконфигурирована и запущена. После этого поле заголовка появляется на ресурсах, определенных для API.

- «Что?» (*руководство по дизайну № 2*). API, у которых есть зарегистрированные пользователи, могут использовать для связи с ними внешние каналы, чтобы объявить о предстоящем удалении. Это руководство применимо, только если у вас есть такой список пользователей и канал для связи достаточно надежен.
- «Как?» (*руководство по реализации № 1 для руководства по дизайну № 2*). Один из методов реализации — связаться со всеми зарегистрированными пользователями по электронной почте. Она отлично подойдет для размещения ссылок на доступный ресурс (журнал изменений API или часть его документации) с информацией об удалении. У этого ресурса должен быть стабильный URI, чтобы на него можно было ссылаться при общении с пользователями.

Во многих компаниях, где API играют важную роль, есть руководство по API в какой-либо форме. Некоторые публикуются открыто, и их можно свободно просматривать. Например, вы можете посмотреть, что в качестве руководства используют такие организации, как Google, Microsoft, Cisco, Red Hat, PayPal, Adidas или Белый дом.



Гид по стилям API

Арно Лоре, известный как Мастер API, собрал несколько опубликованных руководств в своем «Гиде по стилям API» (<https://oreil.ly/x4NwM>), пользуясь информацией организаций, от Microsoft до Белого дома. Это интересный ресурс, который позволяет узнать, что именно крупные компании пишут в руководствах по API.

Еще до рассмотрения самих руководств вы можете многое узнать об их создании и управлении, глядя на способ их публикации (и об общей философии, стоящей за руководствами).

- Документы в PDF явно предназначены только для чтения. Они публикуются из некоего недоступного источника. Это способ собрать, отформатировать и опубликовать уже существующее содержимое. В этом случае вы почти не чувствуете вовлеченности в управление этим руководством и его развитие.
- HTML обычно воспринимается чуть лучше, потому что в большинстве случаев это и есть источник. Читатели видят сам источник документа, а не отформатированный и оторванный от него продукт, как PDF. Но управление HTML-источником все еще неочевидно, поэтому читатель ощущает, что он отделен от стадий создания и редактирования.
- У многих систем контроля версий есть функция публикации, поэтому их можно использовать для размещения руководств. Например, у GitHub

есть простые встроенные способы форматирования и публикации (файлы Markdown). Хотя, скорее всего, некоторых важных функций форматирования вы там не найдете¹. У GitHub удобные функции для комментирования, обсуждения вопросов и предложения изменений. Кроме того, большинство разработчиков уже привыкли к этой платформе, поэтому им удобно ее задействовать. Руководство, опубликованное так же, как и код, облегчает другим возможность внести в него свой вклад.

Есть еще одно правило для создания руководства: это может быть только то руководство, которое можно протестировать (то есть в нем должны быть инструменты, помогающие разработчикам определить, что они успешно воспользовались руководством). Это не только делает руководство более четким, а следование ему более объективным, но и означает, что его можно проверить автоматически. Полностью автоматизированное тестирование всех руководств может быть нерентабельным или вовсе невозможным, это идеальная модель. Поэтому мы предлагаем просто добавить к описанному ранее шаблону «зачем? что? как?» четвертый элемент.

«Когда?» (руководство по тестированию). Описывает все работы, которые нужно выполнить, когда все будет сделано. Для этого нужны способ проверки правильности выполнения этих работ и желательно автоматизированный тест в процессе развертывания, который будет контролировать выполнение работы и следование руководству.

Как и всё в хорошо управляемом ландшафте API, тесты можно со временем улучшить. Начните с простых тестов на правдоподобие, чтобы обеспечить минимальную уверенность и положительную обратную связь, показывающую, что к руководству обращались. Если со временем становится понятно, что это не так информативно, как должно быть, необходимо улучшить тесты, чтобы команды получили более полезную обратную связь и им стало легче проверять соответствие конкретному руководству.

Жизненный цикл руководства

Поскольку руководство — это развивающийся набор рекомендаций, у него тоже есть жизненный цикл. Сначала появляются предложения, некоторое время исследуются, а затем могут стать рекомендациями о том, что и как надо делать. Но, как и всё в ландшафте, они со временем будут заменены на новые, поэтому

¹ Содержание в формате Markdown создается напрямую в веб-представлении репозитория. Более амбициозные писатели выбирают GitHub Pages, где можно создать сайт прямо из репозитория.

эти рекомендации пройдут фазу заката и станут историческими. Стадии жизненного цикла руководства можно определить так.

- *Экспериментальная стадия.* В этой фазе руководство исследуется, то есть используется как минимум в одном продукте API. Это позволяет понять, имеет ли оно смысл в качестве руководства на уровне ландшафта. На этом этапе оно документируется, но вложений в него не делается.
- *Реализация.* После утверждения руководства на уровне ландшафта его нужно поддерживать (чтобы был хотя бы один ответ на вопрос «как?») и как минимум принимать во внимание, прежде чем отбросить. Некоторые руководства отбрасывать нельзя, поэтому команды обязаны им следовать.
- *Депрекация.* После появления более удачных способов выполнения определенной задачи руководство может войти в период депрекации, когда его еще можно придерживаться, но в идеале команды должны рассмотреть вариант следования руководству в стадии реализации.
- *Устаревание.* В итоге руководство устаревает, и его больше нельзя применять. Можно даже рассмотреть возможность рефакторинга существующих продуктов для перехода на более современный способ ведения дел. Устаревшее руководство все еще полезно сохранять для документирования того, как разрабатывались и использовались более старые API.

Эти стадии — лишь один из методов управления развитием руководства, и вы свободно можете определить для себя собственный. У вас даже могут быть способы маркировать его разными уровнями требований, например «опциональное» или «обязательное». Но в ходе работы с ними должны создаваться и исключения, чтобы, к примеру, обязательное руководство можно было пропустить, если есть достаточно доказательств, что следование ему вызовет проблемы.

Важный вывод из этой темы: важно понять, что руководство будет непрерывно меняться, поэтому нужно создать способ отслеживания этих изменений и управления ими в своей организации. Это мы обсудим в следующем разделе, где познакомим вас с широко применяемым в больших организациях способом решать проблему управления руководствами.

Центр поддержки

Организации по-разному называют команды по управлению руководством API, как правило, выполняющих и функцию управления программой API. Одно из популярных названий — *Центр передового опыта* (Center of Excellence, CoE), но он нравится не всем: многим кажется, что у него много недочетов. Поэтому

нам больше нравится название *Центр поддержки* (Center for Enablement, C4E), которое хорошо отражает меняющуюся роль современных ИТ-команд.

Управление руководствами может показаться всего лишь технической деталью, но на практике оно крайне важно. C4E должны собирать и редактировать информацию, которая будет опубликована, а отдельные команды API — предоставлять ее. C4E также отвечает за определение аспектов руководства, требующих инвестиций в поддержку инфраструктуры, чтобы то, что изначально было проблемой, которую должны были решать отдельные группы, теперь можно было решить с помощью доступных инструментов.

Другая часть обязанностей C4E — убедиться, что следование руководству по API не создает слабых мест. В идеале команды разобрались в руководстве, они знают, как реализовать обновления, обладают достаточным количеством навыков и имеют поддержку от C4E для использования инструментов, поэтому следование руководству по API их работу не замедляет. Все слабые места должны быть найдены, и с ними нужно разобраться, чтобы беспрепятственно реализовывать API.

Конечно, все это зависит от ограничений в организации. Например, в некоторых сферах есть внутренние инструкции или юридические нормы, требующие, чтобы организации проверяли и утверждали каждый релиз. Делать это нужно, и эти процессы нельзя полностью автоматизировать. Но это скорее исключения, поэтому большинство руководств действительно нужно воспринимать как то, чему *стоит* следовать. И главная роль C4E — дать командам API возможность эффективно и успешно это делать.

УПРАВЛЕНИЕ ИНЖЕНЕРАМИ: CHAOS MONKEY

Еще одна интересная роль C4E — определять способы, которыми нефункциональные требования могут быть перенесены в общую культуру разработки вашей организации. Один из примеров — популярный инструмент Chaos Monkey от Netflix. Его история такова: согласно общей методике разработки сервисы в такой среде, как сложный и взаимозависимый ландшафт API Netflix, должны быть максимально устойчивыми, чтобы проблемы отдельных сервисов затрагивали как можно меньше зависимых от них сервисов.

Проблема обязательного «устойчивого кода» в том, что его тяжело протестировать. Netflix решила ее с помощью гениального Chaos Monkey — инструмента, симулирующего изолированные и контролируемые проблемы в инфраструктуре и наблюдающего за поведением других сервисов в этот момент. Это позволяет инженерам контролировать устойчивость сервисов. Такая ситуация служит примером подхода, именуемого «управление инженерами»: создав инструмент, определяющий неустойчивый код, менеджеры добиваются от инженеров большей дисциплины при разработке кода. Если он не будет устойчивым, тестирование обнаружит проблемы до того, как они станут критическими, поэтому у разработчиков есть дополнительная возможность тестирования при запуске.

Такой подход упрощает С4Е масштабирование ландшафта для большего количества разрабатываемых и развертываемых API. Также отдельным командам проще понимать, какие есть требования, и получать (хотя бы частично) автоматизированные тесты на их знание. Конечно, для части руководства все равно будут нужны проверки и обсуждения, но чем легче сфокусироваться на этих аспектах, не подлежащих автоматическому тестированию, тем проще можно масштабировать С4Е.

В целом роль С4Е — быть распорядителем руководства на уровне ландшафта: максимально облегчить командам API *создание новых продуктов*, а пользователям — *использование API в ландшафте*. Поскольку С4Е должен поддерживать баланс между простотой создания и применения, его самые важные задачи — сбор отзывов от производителей и потребителей и поиск способа непрерывного развития ландшафта API для лучшего обслуживания обеих групп.

Постоянное развитие ландшафта API означает, что необходимо помнить о его аспектах из раздела «Восемь аспектов ландшафта API» на с. 271. Это значит, что С4Е приходится решать, когда в какие аспекты инвестировать, наблюдая, по какому из них можно не предоставлять высокотехнологичной поддержки, а на какой стоит потратить время и силы.

Например, в случае аспекта «Объем» некоторое время можно не тратить слишком много сил на масштабирование, но когда все больше и больше команд создают и используют API, расширение объема становится крайне важным и требует вложений.

Главная цель С4Е — помогать командам API вносить более эффективный вклад в ландшафт API. Мы рассматривали команды в главе 8. Дополняя это обсуждение, в следующем разделе поговорим о командах С4Е и о том, как с помощью управления отдельными API и их ландшафтом создать команду, максимально поддерживающую команды API.

Команда С4Е и контекст

Одна из ролей команды С4Е — обслуживать руководства всей системы и помогать командам исполнять их требования. Взаимодействуя с командами API, С4Е собирает обратную связь о том, какие новые шаблоны могут появиться, и узнает, как должны развиваться принципы, протоколы и шаблоны, чтобы улучшить ландшафт API.

Для этого С4Е должен развиваться вместе с ландшафтом. Возможно, изначально это будет даже не отдельная команда, а члены разных команд API, которые возьмут на себя эти обязанности (как обсуждалось в главе 8). Но со временем

в больших организациях центр подготовки превратится в настоящую команду со своими сотрудниками. И даже тогда важно помнить, что ее основная обязанность — поддерживать команды продуктов. Кевин Хикки пишет об этом так¹:

Теперь вы не централизованная группа [по архитектуре предприятия], отвечающая за решения команд разработчиков, вы более влиятельны и ответственны за сбор информации. Теперь вы должны не делать выбор, а помогать выбрать другим и затем распространить эту информацию.

Роли, которые мы определили в разделе «Обязанности в команде API» на с. 230, зачастую относятся и к С4Е или как минимум сильно влияют на их действия на уровне ландшафта. Появляются и новые обязанности, которых обычно нет на уровне команды.

Например, обязанности, связанные с соответствием. Многим организациям нужно убедиться, что они соответствуют нормам и законодательству, отслеживать изменения в требованиях и обеспечивать соответствие им. Для ландшафта API это часто приводит к существующим рекомендациям, которым нужно следовать (как мы обсуждали ранее). Чтобы это не стало уязвимым местом, в идеале соответствие должно быть тем, что команды API могут протестировать, чтобы оно могло стать частью конвейера доставки.

На практике это не всегда возможно или даже запрещено (может быть правило о том, что кто-то должен утвердить продукт после ознакомления с ним). Но при любых требованиях организации важно думать о соответствии для API, определить области, где нужно выпустить руководство по соответствию, и поддерживать команды API, чтобы максимально облегчить им создание продуктов.

Еще одна обязанность, обычно существующая только на уровне ландшафта, — предоставление инфраструктуры и инструментов. Как показано в разделе «Структурирование руководства в ландшафте API» на с. 286, типичное руководство в ландшафте API разделено на «зачем?», «что?» и «как?». Мы считаем, что каждому «зачем?» (мотивация руководства) должны соответствовать как минимум одно «что?» (руководство по дизайну) и одно «как?» (руководство по реализации) и возможная инфраструктура и инструменты тестирования, которые помогут командам проверять соответствие продукта существующему руководству.

При каждом тесте реализации должна существовать роль, отвечающая за повышение эффективности выполнения командами указаний и подтверждающая

¹ Hickey K. The Role of an Enterprise Architect in a Lean Enterprise («Роль корпоративного архитектора в рациональном предприятии»). 30 ноября 2015 г. <https://oreil.ly/OYmKt>.

соответствие руководству. Это может означать предоставление инструментов и/или инфраструктуры для реализации и проверки этого руководства. Создание и поддержка этих составляющих затем становится важной ролью на уровне ландшафта. Чем лучше помощь и инструменты на уровне ландшафта, тем больше команд могут сосредоточиться на удовлетворении своих бизнес-потребностей и продуктах, а не на том, чтобы вписаться в ландшафт. Любые препятствия, с которыми они сталкиваются, должны рассматриваться как важный сигнал о том, что где-то нужна более серьезная помощь и больше инструментов.

Один из примеров такого инструментария — *линтинг API*. Это процесс проверки описаний API (таких как OpenAPI или AsyncAPI) на соответствие правилам, формализующим определенные требования. Чтобы облегчить разработчикам API следование руководству по проектированию, C4E может предоставить инструменты или сервисы для проверки оформления кода, позволяющие проводить автоматическое тестирование. Эти инструменты можно внедрить в конвейеры CI/CD, что еще больше снижает усилия команд разработчиков для соблюдения имеющихся рекомендаций.

В общем, команда C4E очень важна для поддержки команд продуктов API. Она сообщает им о том, что нужно для создания руководства по основным моментам, требующим принятия решений, и помогает, предоставляя инфраструктуру и инструменты для эффективного выполнения распространенных задач API, чтобы большая часть энергии команд была потрачена на решение бизнес-задач. Другими словами, команда C4E ответственна за то, чтобы каждая из команд могла эффективно развивать свой API, принимая при этом верные решения, и за то, чтобы такая эффективность распространялась на множество API.

Зрелость и восемь аспектов

Восемь «V» из главы 9 очень важны, когда ландшафты API находятся на стадии планирования и эволюции. Они могут служить руководствами в момент определения уровня зрелости в этих сферах, когда нужно отразить мотивы и преимущества развития и принять решение о возможных инвестициях в них.

Важно понимать, что инвестиции в эти сферы должны быть эволюционными и соответствовать конкретным потребностям ландшафта. Если архитектура разработана качественно, инвестировать можно по необходимости и постепенно, и это не потребует повторной разработки ландшафтной архитектуры. Это означает, что зрелость самого развивающегося ландшафта API постоянно растет, внося улучшения по мере необходимости, и ландшафт постоянно совершенствуется на основе отзывов разработчиков и пользователей.

Как и любая эволюция, это постоянное улучшение не ведет к какой-либо определенной или предсказуемой цели. Ценность ландшафта определяется тем, насколько хорошо он поддерживает разрабатываемые в нем продукты и насколько они отвечают требованиям пользователей. Методы разработки и пользовательские нужды со временем меняются, поэтому улучшение неизбежно становится непрерывным процессом.

Главная цель архитектуры ландшафта — максимально упростить этот процесс, позволяя ему адаптироваться под меняющиеся потребности создателей и пользователей. Развитие ландшафта можно измерить объемом предоставляемой им поддержки. В случае вышеупомянутых восьми аспектов можно отдельно посмотреть на то, как для каждого из них определяется зрелость и выглядит стратегия управления ею.

Такая идея «зрелости ландшафта» несколько отличается от цикла зрелости продуктов API (см. главу 7). Продукты приходят и уходят, и все это обосновано их жизненным циклом, у которого есть начало и конец. Ландшафт существует, чтобы поддерживать продукты, и должен это делать, постоянно развиваясь. Здесь нет одного линейного пути или конечного состояния, поэтому и стадий тоже нет.

Мы решили эту проблему, изучив, как определенные нами восемь аспектов в разные моменты могут служить принципами для руководства при непрерывном улучшении ландшафта, разработке его стратегии и принятии решений по поводу инвестиций на ландшафтном уровне.

Разнообразие

Разнообразие ландшафта зависит от того, сколько ограничений накладывается, когда команды хотят разработать и внедрить API, и сколько свободы есть у них при разработке продуктов, которые они считают рентабельными (см. раздел «Разнообразие» на с. 271).

С разнообразием непросто работать, ведь в ландшафтах это всегда баланс между продвижением определенного уровня согласованности и повторного использования и попыткой не слишком ограничивать команды и не заставлять их применять неподходящие решения. Поэтому у разнообразия есть две плохие крайности.

- *Отсутствие разнообразия* означает, что выбранный шаблон становится «золотым молотком» (<https://oreil.ly/QKFSK>) — единственным путем решения всех проблем¹ (которое часто оказывается неподходящим как минимум для нескольких из них).

¹ Известное высказывание американского психолога Абрахама Маслоу гласит: «Когда у тебя в руках только молоток, все предметы вокруг становятся гвоздями».

- *Избыток разнообразия* приводит к «прекрасным снежинкам» — команды тратят силы на решение проблем, у которых уже есть решения. В результате пользователи затрачивают ненужные усилия, пытаясь разобраться в несогласованном API.

Соблюсти баланс между ними непросто, и единственно правильного решения для выбора точки на шкале между «золотым молотком» и «прекрасными снежинками» нет. Поэтому нецелесообразно определять зрелость разнообразия ландшафта API с точки зрения его величины. Во многих случаях оно может сложиться случайно из-за отсутствия гибкости (в результате низкого разнообразия) или невозможности управлять согласованностью (в результате высокого).

РАЗВИТИЕ РАЗНООБРАЗИЯ

Что значит управлять разнообразием с высокой степенью зрелости?

- Зрелость для разнообразия означает, что оно сознательно управляется в ландшафте API. Используемые варианты выбора и их обоснование четко задокументированы.
- Эти варианты должны развиваться по мере необходимости: разнообразие управляется и поддерживается балансом между поощрением повторного использования и предоставлением новых решений, если существующие бесполезны.
- Можно увеличить разнообразие, не нарушая ландшафта. Возможно, некоторые инструменты и поддержка в ландшафте потребуют корректировки, но вся инфраструктура инструментов и поддержки должна быть спроектирована так, чтобы расширение разнообразия могло осуществляться постепенно и было частью базовой архитектуры.

Есть много разных концепций в ландшафте API, в зависимости от способа организации ландшафта. Например, для ландшафтов, основанных на HTTP и использующих API в стиле ресурсов, одним из факторов разнообразия может быть выбор способа присвоения серийных номеров. Поскольку большинство современных ландшафтов задействуют и позволяют API поддерживать XML и JSON, это самые популярные варианты настоящего и недавнего прошлого.

Вполне вероятно, что благодаря дизайнерам API появятся новые способы присвоения номеров. Вопрос в том, считать ли новый формат ценным дополнением к ландшафту. У вас должна быть возможность начать с нескольких API и проверить, как они будут работать с новым вариантом. Возможно, они не смогут извлечь выгоду из использования существующих инструментов и поддержки, поскольку использование нового формата *экспериментально* (на этом этапе нет инвестиций на уровне ландшафта).

Когда новый вариант начинает считаться *продуктивным*, он может инициировать обновление доступных инструментов и поддержки. Зрелые ландшафты могут обрабатывать эти обновления как постепенные изменения, добавляемые при необходимости. Итак, расширение разнообразия — это лишь способ воспользоваться полезными функциями добавленного варианта и постепенный рост затрат на обновление инструментов и поддержки.

Из всего вышесказанного можно сделать важный вывод: все инструменты и поддержка должны быть способны к подобным обновлениям. Те, что не могут справиться с растущим разнообразием, ограничивают *доход, который обновление может принести ландшафту*. Такие инструменты и поддержка считаются способными вызвать проблемы для API.

Важное последствие стратегии разнообразия — анализ функций инструментов и поддержки API. Как говорилось в разделе «API через API» главы 8, когда все делается через API, включая взаимодействие инструментов и поддержки, расширять разнообразие становится проще. Если новые варианты поддерживают те же API, они все еще могут взаимодействовать с существующей инфраструктурой инструментов и поддержки.

СТРАТЕГИЯ РАЗВИТИЯ РАЗНООБРАЗИЯ

Вкладывая средства в инструменты и поддержку, всегда учитывайте, как эти инвестиции проявляются при увеличении разнообразия. Старайтесь избегать инструментов и поддержки, не имеющих четкой стратегии эволюции. В конце концов, инструменты и поддержка должны подстраиваться под ваш выбор продуктивного уровня ландшафтного разнообразия, а не диктовать его.

Словари

Как говорилось в разделе «Словари» на с. 273, многие API используют терминологию, определяющую некоторые аспекты их модели. Словари могут применяться по-разному, и во многих случаях API может использовать определенный словарь при первоначальном выпуске, но нужно быть готовым к тому, что со временем этот словарь может измениться. Тогда он становится частью модели расширения этого API, и возникает вопрос, как эта расширяемость проектируется и управляется.

То, что словари, используемые в ландшафте API, действительно эволюционируют, — результат того, что модели предметных областей API имеют тенденцию со временем развиваться. И эволюция словаря в API — просто отражение этого процесса. Они часто эволюционируют из-за уточнения понимания предметной

области, например если добавляются ссылки на соцсети в пользовательскую модель, где до этого была лишь основная личная информация. Тогда вопрос будет в том, как работать с данными (существующими пользовательскими записями без ссылок на соцсети) и кодом (приложениями без встроенной поддержки соцсетей), созданными до развития пользовательской модели. Дисциплинированное управление такой эволюцией словаря определяет развитие работы со словарем в ландшафте API.



Концепции терминов

В разделе «Словари» на с. 273 обсуждается, какая терминология может использоваться для API, включая независимую от предметной области (например, коды языков), специфичную для нее (домен, отраженный в API) и саму сферу концепций для дизайна API (например, коды статусов HTTP).

РАЗВИТИЕ СЛОВАРЯ

Отправная точка для каждого API — определение потенциальных словарей, с которыми он может развиваться. Это напрямую связано с определением точек расширения API: если команда API ожидает, что словарь будет развиваться, она должна определить это в самом API и предоставить пользователям модель обработки.

Когда эволюция словаря становится естественной частью API, становится важным ответственное управление ею. С одной стороны, это означает ответственный контроль версий API и их документирование, с другой — помощь пользователям в применении API, так чтобы правильно поддерживать эволюцию. Выбранный способ зависит в основном от того, как отдельные API решают реализовать эволюцию словаря.

Управление эволюцией словаря может быть передано отдельным API. Но есть и альтернативная модель, где ландшафт API позволяет словарям развиваться независимо. Типичная схема включает в себя использование реестров (<https://oreil.ly/VhE3a>), а их поддержка и управление ими сами по себе могут стать частью ландшафта.

Последний аспект зрелости требует дополнительных объяснений. Есть два способа «делегирования» эволюции словаря (то есть управления вне API): с помощью ссылки на внешний орган, ответственный за управление словарем, и с помощью управления им внутри ландшафта, но отделяя API от развивающихся словарей.

- *Внешний руководящий орган.* Типичный пример этого подхода — использование языковых тегов — идентификаторов человеческих языков, например «английский» или даже «американский английский». В большинстве слу-

чаев не стоит включать в API статический список языковых тегов. Вместо этого лучше дать ссылку на один из списков Международной организации по стандартизации (International Organization for Standardization, ISO), приведенный в стандарте ISO 639 (<https://oreil.ly/a23v2>). Используя этот шаблон, API может определить, что пространство значений языкового тега — это то, что ISO считает возможными языковыми тегами в любой момент времени. ISO гарантирует, что языковые теги будут развиваться, не вредя ландшафту, так как вам не понадобится удалять или менять определения существующих тегов.

- *Поддержка ландшафта API.* Не у всех концепций есть внешние сущности и менеджеры, такие как ISO для списка языковых тегов. Но вы можете использовать ту же схему и для других словарей. Ландшафты могут поддерживать реестры, позволяя API отделить свое определение от меняющегося пространства значений терминов. Работать с таким реестром несложно. Но эта работа не должна быть зоной ответственности отдельной команды API¹. Вместо этого на уровне ландшафта должна быть *поддержка реестров*. Так, например, Инженерный совет интернета (Internet Engineering Task Force, IETF) управляет своими спецификациями с помощью более чем 2000 реестров, находящихся в ведении Организации по управлению адресами (Internet Assigned Numbers Authority, IANA)².

Роль архитектуры в управлении словарем такая же, как и в других аспектах создания продуктивной и поддерживающей среды для API: следить за требованиями и методиками существующих API и предлагать удачные методы и поддержку, когда эволюция словаря начинает повторяться во многих из них.

Изначальный качественный метод должен заключаться в том, чтобы как минимум определить потенциально развиваемые словари в API и документировать их. Если повторяющиеся случаи изменения API — просто непреднамеренное следствие эволюции словаря, то, скорее всего, поддержка ландшафта API для эволюции словаря может помочь уменьшить потребность в обновлениях API.

¹ В конце концов, один из главных мотивов создания реестра — отделение управления списком общеизвестных значений от мест их использования.

² Модель реестров IANA очень хорошо иллюстрирует простоту и эффективность такой инфраструктуры. Это и отличная демонстрация того, как путем систематического применения этого шаблона проектирования во многих его спецификациях и определениях API можно было повысить их стабильность, ведь для многих изменений нужно обновлять только реестры.

Может оказаться труднее достичь высокого уровня зрелости словаря, потому что непросто придумать, как наблюдать за его использованием в API. Это может быть одним из случаев, когда инвестиции на раннем этапе помогают улучшить наблюдаемость. Например, создание инструментов для документирования словарей может упростить наблюдение за их использованием и эволюцией в разных API. Но для этого нужно, чтобы команды API сочли такую поддержку полезной для того, чтобы ее использовать. А для этого надо узнать, как они обычно документируют свои API. Как видите, развитие часто не сводится к поддержке и инструментам на ландшафтном уровне. Оно может начинаться с понимания того, за чем наблюдать, и последующей разработки способов наблюдения.

СТРАТЕГИЯ РАЗВИТИЯ СЛОВАРЯ

По возможности продвигайте передовой опыт, отделяющий дизайн API от эволюции словаря. Начните с поощрения повторного использования словарей, определенных и управляемых извне, таких как те, которые определены и управляются организациями, определяющими стандарты. Отслеживайте, сколько изменений API могут быть вдохновлены (в основном) необходимостью обновлять терминологию, и обдумайте обеспечение поддержки для управления словарем в ландшафте независимо от конкретных API, предоставив для этого инфраструктуру.

Объем

В разделе «Объем» на с. 277 предполагалось, что большее число API — лучше, чем меньшее. Это не всегда верно, но намекает на то, что решения по количеству API в ландшафте не должны зависеть от предположений о том, что система просто не справится с объемом. Большой объем — не всегда лучше, но и не нужно автоматически считать, что он хуже.

Вместо этого стремитесь к тому, чтобы API всегда можно было создавать, изменять и отзывать. Ландшафт API должен иметь возможность расширяться до любого уровня, и его способность справиться с объемом в идеале не должна влиять на стратегические решения по поводу размера и уровня изменений.

Управление объемом в ландшафте API в основном вращается вокруг соображений экономии за счет масштаба. То, что логично будет не поддерживать или не автоматизировать при небольших масштабах, может стать разумной целью по мере роста ландшафта. Это обычная схема получения дохода от инвестиций: вкладывать в поддержку или автоматизацию лучше после преодоления определенного порога, когда расходы на решение проблем по отдельности (снова и снова) начинают превышать расходы на поддержку и автоматизацию.

Как только объем начинает влиять на поддержку или автоматизацию, в ландшафте может появиться некоторая согласованность, поскольку все больше API начинают применять эти поддерживаемые механизмы. От этого они становятся более похожими, помогая пользователям понимать API и работать с ними.

Но, как упоминалось в разделе «Центр поддержки» на с. 290, важно помнить, что поддержка или автоматизация (ответ на вопрос «как?») никогда не должны быть единственным разрешенным способом выполнения задачи. С4Е должен определить и предоставить это как часть общей поддержки платформы API. Но всегда должно быть что-то, что можно заменить более подходящим способом решения той же проблемы, если такой был найден.

Как указывалось ранее, самый важный аспект зрелости объема — не позволять ему мешать принятию решений о возможности и способе расширения ландшафта API. Лучший способ сделать это — отслеживать текущую эволюцию ландшафта, того, *что* реализуют команды и *как* они это делают, и инвестиций, когда вам кажется, что поддержка или автоматизация могут повысить продуктивность команд.

РАЗВИТИЕ ОБЪЕМА

- Отслеживайте, как команды API решают проблемы, связанные с проектированием, созданием и эксплуатацией своих продуктов, и рассматривайте возможность инвестирования в поддержку или автоматизацию по мере необходимости (то есть когда это становится выгодно).
- Для потенциальной поддержки или автоматизации рассмотрите потенциальный доход как для команд API, так и для его пользователей. Общая прибыль от ввода поддержки или автоматизации будет складываться из суммы этих двух доходов¹.
- Самое важное для ландшафта API — определить повторяющиеся действия команд во время дизайна или реализации и исследовать возможность увеличения продуктивности с помощью вложений в поддержку или автоматизацию.

Этот подход подразумевает, что ландшафт API активно контролируется, что позволяет принимать решения на основе данных. Хорошая схема, позволяющая масштабировать этот подход, — придерживаться принципа «API через API» и убедиться, что сами API предоставляют информацию о себе. Так станет

¹ Можно принять в расчет и создание технического долга. Мы это пропустим, но рациональный подход, при котором вы всегда помните о том, что отойти от поддержки или автоматизации будет очень сложно, — тоже важный аспект управления ландшафтом.

возможно встраивать поддержку и автоматизацию в контроль API, что поможет решить, когда инвестировать в поддержку и автоматизацию дизайна и разработки API.

Один из хороших способов оценить зрелость аспекта объема — это подумать о том, какая информация о ландшафте API доступна для тех, кто оценивает ландшафт. Помните, что эту информацию можно собирать любым способом, лишь бы она была доступной. Ее можно предоставить через сами API (API через API), через инструментарий управления работающей инфраструктурой (например, собирать данные со шлюзов API) или инфраструктурой времен разработки этого API (например, собирать данные с общих платформ для разработки/развертывания продуктов API). Если информация доступна, становится проще понимать динамику развития ландшафта и управлять ею.

СТРАТЕГИЯ РАЗВИТИЯ ОБЪЕМА

Работа с объемом требует основы, которую можно применять для масштабируемого наблюдения за API, чтобы понимать динамику развития ландшафта. Наблюдаемость должна включать в себя информацию об API, которую можно использовать для принятия инвестиционных решений на основе тенденций в ландшафте API. Само управление объемом может быть масштабировано для работы с большими объемами, если важная для понимания ландшафта информация будет частью самих API. Подход «API через API» со временем будет развиваться, изменяя набор наблюдаемой информации, которая используется для понимания текущей эволюции ландшафта API.

Скорость

Как обсуждалось в разделе «Скорость» на с. 278, *скорость* отражает тот факт, что ландшафты API меняются постоянно и быстро. С одной стороны, это результат создания и использования все большего числа API, но с другой — это происходит из-за отношения к API как к продуктам (когда за ними наблюдают и меняют их в соответствии с отзывами и запросами пользователей (см. главу 3)). Эти изменения в большинстве случаев происходят нескоординированно, поскольку одна из целей ландшафтов API — позволить продуктам развиваться индивидуально, в отличие от сложных скоординированных процессов выпуска, которые позволяют продуктам развиваться только взаимозависимо.

Зрелое управление скоростью означает, что выпуски и обновления API могут выполняться по мере необходимости, а ландшафт способен поддерживать высокую скорость изменений. Все это должно происходить само собой. Изначально ландшафт API может быть довольно маленьким, чтобы даже при относительно большом числе изменений их количество в ландшафте API

оставалось небольшим. Но со временем это изменится. Сочетание увеличения объема (см. раздел «Объем» на с. 300) и скорости изменений API означает, что работа со скоростью становится все важнее, поскольку ландшафт API растет и развивается.

РАЗВИТИЕ СКОРОСТИ

- API всегда должны быть разработаны с учетом последующего внесения изменений. В зависимости от стиля API это может иметь разные значения, но узнать у команд об их планах по расширению API — прекрасный первый шаг к тому, чтобы взглянуть на эволюцию API как на естественную часть жизненного цикла API стал частью культуры дизайна API.
- Развитие API меняет и подход пользователей: применение API должно стать достаточно устойчивым для поддержки его эволюции, чтобы она и пользователи не были связаны.
- Увеличить скорость изменений API можно, убедившись, что затраты на координацию между реализациями снижены. Один из способов сделать это — принять микросервисы в качестве шаблона реализации сервисов.

Эти идеи показывают, что скорость влияет на производителей и на пользователей. С ростом размеров и популярности ландшафта API (и числа пользователей) высокоразвитая работа со скоростью становится все важнее. Координация обновлений между API и всеми пользователями вырастает в цене и быстро достигает той точки, где затраты на координацию могут заставить команды пересмотреть вопрос об усовершенствовании продукта.

Как говорилось в разделе «Контроль версий» на с. 281, есть разные стратегии работы с меняющимися API. Они могут меняться прозрачно, когда пользователи ощущают изменения в API, и они становятся понятны благодаря номеру семантической версии API (см. раздел «Семантический контроль версий» на с. 282). Другой вариант — обещание стабильных версий переводится в непрерывный поток новых, выпускаемых и доступных параллельно. Последняя схема подразумевает, что потребителям должно быть легко узнавать о новых версиях и находить информацию о них (см. раздел «Видимость» на с. 280).

Хотя важно обеспечить скорость, не менее важно ею управлять. Если она увеличивается, пользователи должны успевать за ней. Это можно сделать по-разному, например обещая стабильные API, которые будут работать в течение определенного периода времени¹, или постоянно развивающиеся API, что устраняет

¹ В этом случае удобнее пользователям, а создатели должны инвестировать в запуск нескольких версий API параллельно.

необходимость поддерживать работоспособность старых версий¹. Что бы вы ни выбрали, это область, где отдельные API могут извлечь выгоду из поддержки на уровне ландшафта, поэтому внедрение методов и их поддержка становятся выгодными инвестициями.

СТРАТЕГИЯ РАЗВИТИЯ СКОРОСТИ

Обеспечение гибкости (то есть возможности быстро вносить изменения на основе отзывов и требований) — один из основных движущих факторов API-ландшафтов. С одной стороны, проектирование с учетом будущих изменений означает разработку API, которые можно быстро и просто менять. С другой стороны, использование изменяющихся API означает, что должна существовать модель потребления, позволяющая потребителям обрабатывать меняющиеся услуги. Все, что затрудняет изменение вещей, должно быть тщательно выявлено и изучено. Этот процесс может проходить постепенно — вначале можно найти и улучшить один снижающий скорость фактор, затем при необходимости повторить процесс.

Уязвимость

Как говорилось в разделе «Уязвимость» на с. 279, растущая *уязвимость* — следствие увеличения ландшафта API. Отсутствие API означает отсутствие потенциальных уязвимостей, а любой новый API становится слабым местом вашей системы. Осознание этого простого факта — хороший первый шаг к зрелости в отношении аспекта ландшафта уязвимостей.

В зависимости от аудитории API могут быть открыты для внутренних (внутренние API) или для внешних (партнерские и внешние API) пользователей. Во многих случаях эти два или даже три сценария защищают по-разному, часто с использованием разных компонентов (рис. 10.1).

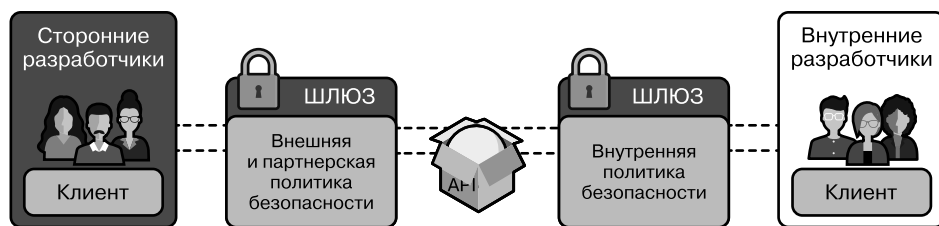


Рис. 10.1. Защита API с помощью шлюзов

¹ В этом случае удобнее создателям, а пользователи должны инвестировать в то, чтобы убедиться, что их приложения правильно поддерживают непрерывную эволюцию API.

Понятно, что для обеспечения безопасности этот процесс организован относительно централизованно, чтобы у вас была возможность наблюдать за трафиком и управлять им (и, если понадобится, прерывать его). Это позволит лучше понимать, как его применять, и выявить его потенциальные проблемы. Такая централизация вступает в конфликт с общими усилиями по децентрализации. Возникает вопрос о том, в какой степени отдельные продукты API контролируют (или могут контролировать) конфигурацию централизованной точки обеспечения безопасности. Здесь сложно сбалансировать скорость и безопасность, но опять же нужно ориентироваться на нужды организации и требования безопасности, а не на технические ограничения архитектуры.

Мы можем применить ту же схему, по которой смотрим на путь развития, и к уязвимости: важно наблюдать за развитием API в ландшафте API и выделять общие темы и области, в которых поддержка ландшафта и инструменты могут помочь. Единственное исключение в том, что уязвимость — это аспект с высоким уровнем риска, поэтому нужно жестко регламентировать наблюдение за ландшафтом и принятие различных мер.

Пример такого подхода — недавние разработки по поводу раскрытия персональных данных через API. Это опасно для организаций из-за возможных последствий с законодательством, нормами и репутацией. Эти риски не всегда могут быть видны командам, создающим продукты API. К тому же хотя информация, раскрытая с помощью одного API, может казаться анонимной, чтобы не считаться персональными данными, растущая доступность дополнительной информации через другие API означает появление риска раскрытия личности. И это чаще оценивается на уровне ландшафта, а не на уровне отдельных API.

Еще одна проблема — непреднамеренные последствия раскрытия определенных данных через API. В 2016 году Европейский союз выпустил Общий регламент о защите данных (GDPR). Документ относится к обработке личных данных и требует, чтобы все организации в ЕС предоставили информацию о персональных данных, которой они располагают, и открывали к ним доступ по запросу. Это означает, что создание продуктов API, управляющих персональными данными, имеет далеко идущие последствия для организации. В зависимости от размера и степени ее развития добиться соответствия регламенту может оказаться непросто.

Эти примеры показывают: несмотря на пользу скорости для ландшафта API и то, что это одна из причин, по которой организации и переходят на стратегии API, все равно важно управлять рисками. И в зависимости от сферы деятельности

организации, разработанных API и их ЦА во многих случаях для ответственного управления рисками нужно учитывать уязвимость и управлять ею.

РАЗВИТИЕ УЯЗВИМОСТИ

- API по умолчанию предоставляют бизнес-возможности, которые ранее были недоступны для потребителей. Оценивать риск каждого нового API нужно, чтобы избежать утечек информации или других проблем, которые создадут основные риски для организации.
- API-продукты должны документировать всю информацию, которую они хранят, и причину ее хранения. Информация потенциально ценна, но может и повышать риски. К управлению информацией всегда нужно относиться как к тому, что может создать риск в области законодательства, норм или репутации.
- Безопасность API необходима в ответственной стратегии API, и к ней нужно относиться как к важнейшему компоненту общей стратегии информационной безопасности вашей организации.

В сравнении с другими аспектами ландшафта уязвимость выделяется, так как увеличивает риск, поскольку API неизбежно создают проблемы, предоставляя доступ к бизнес-функциям.

Помимо вопросов безопасности от злоумышленников или потенциального правового, нормативного или репутационного риска, есть и вопрос стабильности и сервисного тестирования. Как говорится, с большой силой приходит большая ответственность, поэтому, если продукты API становятся автономными с точки зрения разработки и развертывания, появляются и вероятные новые сценарии провалов.

В своих знаменитых «Размышлениях о платформе Google» Стив Йегги писал: «Каждая из равных вам команд внезапно становится потенциальным DOS-злоумышленником. Никто не сможет добиться реального прогресса, пока в каждой службе не будут введены серьезные квоты и ограничения»¹. Эти слова подчеркивают, что надежность и отказоустойчивость важны для стабильности ландшафта API и что он становится уязвимым для моделей неисправностей, которые вводятся с децентрализацией. С ландшафтом будет проще работать при централизации.

Одна из главных сложностей в работе с уязвимостью ландшафта — приспособиться к новой реальности с более доступными бизнес-функциями. Поскольку у ландшафта API есть явная цель открыть к ним доступ, с этой реальностью придется работать. Оценивая и управляя уязвимостью каждого отдельного API

¹ Stevey's Google Platforms Rant («Разглагольствования Стиви о платформах Google») // GitHub Gist, 11 октября 2011 г. <https://oreil.ly/jxohc>.

и упрощая встраивание продуктов API в эту архитектуру, управление уязвимостями может вписаться в новый децентрализованный взгляд на IT-ландшафт, ориентированный на API.

СТРАТЕГИЯ РАЗВИТИЯ УЯЗВИМОСТИ

Переход к API-ландштафтам требует смены способа управления уязвимостями. Традиционная модель внутреннего/внешнего должна быть заменена моделью, которая рассматривает все службы как отдельные и потенциально внешние компоненты. Она позволяет применять ко всем сервисам одни и те же модели безопасности. Ландшафт должен позволять службам легко защищать себя от вредоносного или проблемного поведения. Могут быть «частные/партнерские/общедоступные» сервисы, но в идеале переключение категорий должно означать не что иное, как применение разных политик безопасности.

Видимость

Один из важнейших аспектов *видимости* — это *наблюдаемость* API (см. раздел «Видимость» на с. 280). Она соответствует принципу из раздела «API через API» на с. 268, который утверждает, что «все, что должно быть сказано об API, нужно выражать через этот API». Поэтому все, что нужно, становится видимым с помощью самого API, открывая доступ к информации, чтобы при необходимости на ее основе можно было создать дополнительные сервисы.

Это постепенный процесс, как и любой в ландшафте API. Изначально может быть не так много информации *об* API, которую нужно предоставить *через* API. Но со временем все может измениться, поскольку увеличение объема и скорости диктует необходимость в усилении поддержки и автоматизации определенных аспектов ландшафта. Если эту поддержку и автоматизацию можно создать на основе API, они становятся частью самого ландшафта. Это означает, что им не нужны способы взаимодействия с сервисами, не связанные с API.

В конце концов, одна из основных характеристик API — предоставление инкапсуляции. API должен инкапсулировать *все* относящееся к его реализации, становясь единственным интерфейсом для взаимодействия. Любой обходной путь, даже для внутренних целей, может считаться нарушением ландшафтного подхода.

Если API — это выбранный организацией подход для того, чтобы сделать *все* зависимости явными, четко определенными, видимыми и управляемыми, то *любая* практика создания невидимых зависимостей подрывает его. Именно поэтому заключительная мысль знаменитого «Устава API» Джеффа Безоса (см. раздел «Устав Безоса» на с. 78) такова — никакая практика обхода стратегии API недопустима: «Любой, кто не станет этого делать, будет уволен».

Распространенный способ создания взаимозависимостей, невидимых в ландшафте API, — использование библиотек¹ (потенциально общих). Многие разработчики считают, что их применение отличается от создания взаимозависимостей на уровне API, но это не так. Особенно это касается работы с библиотеками, которые или являются общими для нескольких продуктов API, или предполагают наличие работающих взаимозависимостей с другими компонентами. Так, относясь к библиотекам как к другим API, можно избежать сценариев, где в ландшафт возвращаются проблемы с управлением взаимозависимостями, находящимися даже не на видимом и управляемом уровне API.

РАЗВИТИЕ ВИДИМОСТИ

- Важный аспект видимости — раскрытие всего, что касается API: того, что нужно для его использования или управления им в ландшафте API. Эта информация, скорее всего, будет меняться, поэтому важно иметь возможность легко изменять API в ответ на возникающие потребности пользователей и менеджеров.
- Еще один важный аспект — раскрытие каждой зависимости через API, чтобы API стали точным их отражением и не было скрытых зависимостей, использующих побочные каналы.
- С ростом ландшафта видимость на уровне API нужно дополнять видимостью на ландшафтном уровне, то есть способностью находить API, основываясь на открытой информации о них.
- И наконец, видимость относится и к легкости применения информации об API. Когда API стандартизируют определенные функции (демонстрацию сообщения об ошибке или информацию о состоянии), такую информацию проще использовать и собирать на ландшафтном уровне, что повысит видимость этих аспектов API.

Видимость на уровне API хорошо влияет на видимость на уровне ландшафта: все, что сделано видимым на уровне API, может быть использовано на уровне ландшафта для улучшения обнаружения API. Это упрощает их поиск (и делает их заметнее) на уровне ландшафта.

К примеру, если API четко и наглядно отображают свои зависимости на уровне API (принимая во внимание принцип, по которому *все* взаимозависимости должны считаться взаимозависимостями API)², то эту информацию можно

¹ Забавно, но это изначальное значение термина «интерфейс прикладного программирования». API был создан как интерфейс между двумя запущенными параллельно компонентами ПО — обычно пользовательским кодом и библиотекой. В последнее время (и в этой книге) API называют сетевые интерфейсы, изменив значение термина. Теперь он больше не охватывает традиционный сценарий локального API.

² Показать взаимозависимости можно по-разному: разработчики могут открыто указать информацию о них, использовать инструменты, проверяющие код реализации, или наблюдать за работой API.

использовать для создания графа зависимостей и информации более высокого уровня, например высчитать популярность API.

В петле обратной связи потребности в видимости на уровне ландшафта могут учитываться в требованиях к видимости на уровне API, а наблюдения за потребностями пользователей и методиками создателей API позволят ландшафту адаптироваться к новым потребностям.

С развитием видимости до той стадии, когда API улучшаются как компоненты ландшафта (путем адаптации к требованиям видимости, вызванным проблемами на уровне ландшафта), отделение «ландшафтных» частей API может стать образцом с точки зрения его функциональных аспектов. Эта практика может сопровождаться методами, борющимися с уязвимостью, если часть «помощь ландшафту» должна быть ограничена и доступна только для инструментов ландшафта.

СТРАТЕГИЯ РАЗВИТИЯ ВИДИМОСТИ

Чтобы API приносили пользу, они должны быть полезными и легкодоступными. Чтобы быть полезными для ландшафта, им придется улучшить видимость некоторой информации. Любая проблема в ландшафте API должна вызывать вопрос: «Какая информация поможет решить ее?» А появление проблем с видимостью на уровне API или ландшафта должно привести к обновлению руководства (добавление видимости API) или инструментов ландшафта (добавление видимости ландшафта).

Контроль версий

Скорость — способность API-продуктов быстро меняться в ответ на отзывы или новые требования. Это важная мотивация перехода к ландшафту API. *Версии* — ее неотъемлемая часть, так как каждый раз при изменении API-продукта он становится новой версией. Это необязательно означает, что пользователь должен что-либо предпринимать по этому поводу (номер минорной версии означает, что изменения совместимы) или даже знать об этом (номер версии уровня «патч» означает, что интерфейс не изменился) (см. подраздел «Семантический контроль версий» на с. 282). Но управлять этими версиями для уменьшения отрицательного влияния на ландшафт обязательно, чтобы скорость не снижалась сильнее необходимого.

Контроль версий применяется к API всех стилей. В стилях туннеля, ресурса и гипермедиа это связано с изменением интерфейса ресурса для взаимодействий (либо процедуры для туннельных API, либо взаимодействия с ресурсами для API-интерфейсов ресурсов/гипермедиа). В API, построенных на запросах, контроль версий — это не часть самого интерфейса, представляющего собой общий

язык запросов, а наука управления схемой данных, которые нужно запросить таким образом, чтобы существующие запросы продолжили работать. В событийно-ориентированных API контроль версий относится к дизайну сообщений, чтобы потребители новых сообщений могли обращаться с ними как со старыми, а не отклонять их из-за изменений в схеме.

Управлять версиями API можно по-разному, и отчасти это продиктовано разными целями API. Обещание стабильных API, которые никогда не изменятся (как это делает Salesforce), привлекает пользователей и может стать хорошей инвестицией. Но при реализации этой стратегии возникают затраты на запуск нескольких разных версий параллельно. Другая стратегия — следовать по пути Google (<https://oreil.ly/YMIVU>) и не обещать полной стабильности API, но реализовывать методику контролируемых изменений API. Есть большой риск, что пользователи не поймут эту стратегию, но она снижает операционную сложность.

РАЗВИТИЕ КОНТРОЛЯ ВЕРСИЙ

- Убедитесь, что у каждого API есть стратегия управления версиями, — это первый шаг к развитию контроля версий. Она может включать в себя принятие того, что API делают сознательный выбор, не поддерживая контроль версий, и что любые обновления будут критическими изменениями и, по сути, новыми продуктами.
- Модели контроля версий во многом зависят от стиля, модели использования API и от баланса между расходами и доходами от того, что производители и потребители должны инвестировать в эти модели.
- Контроль версий сильно выигрывает от единообразия, если не для всего ландшафта API, то по крайней мере для определенных классов API и пользователей.
- В зависимости от модели контроля версий поддержка ландшафта API может помочь производителям и/или потребителям убедиться, что модель управления версиями ландшафта поддерживается и правильно используется.

Общие модели контроля версий делают лишь первые шаги и в плане стандартов, и в плане инструментов. Например, у популярного стандарта описаний OpenAPI нет модели версий или различий, что затрудняет его использование в качестве надежной основы для управления версиями. Вместо этого стандарт поощряет генерирование кода по описанию, не затрагивая вопрос о том, что делать, если API развивается и у него появляется новое описание. Это поднимает вопрос о том, как менять пользовательский код, чтобы адаптировать его к изменившемуся API.

Возможно, что со все более сложными и динамичными ландшафтами API важность управления версиями будет возрастать, а стандарты и инструменты будут адаптироваться к этому развитию. А пока развитие контроля версий все еще

требует внимания и качественного управления на уровне ландшафта. Отдельные API тоже выигрывают от руководства и/или инструментов, предоставленных им на этом уровне.

Как и ко всему на уровне ландшафта, к контролю версий не нужно относиться так, будто есть лишь один верный способ его выполнения. Нормально иметь стратегию и поддерживать ее, но еще важнее быть открытым для новых моделей и иметь возможность перейти к их использованию. Контроль версий может сильно различаться в зависимости от стиля, конкретного класса или определенной группы пользователей API. Поэтому важно уметь развивать эти стратегии и их применение со временем.

СТРАТЕГИЯ РАЗВИТИЯ КОНТРОЛЯ ВЕРСИЙ

Чтобы создание версий как можно меньше затрагивало ландшафт API, как можно реже задействуйте жесткий контроль версий и по возможности отдавайте предпочтение мягкому. Есть множество подходов к мягкому контролю версий, но основной принцип в том, что само по себе создание версий довольно вредно, поэтому согласованная стратегия их контроля (и возможные инструменты и поддержка для нее на уровне ландшафта) должна быть найдена как можно раньше.

Изменяемость

Модели программирования сложно изменить, особенно в мыслях тех, кто занимается программированием. Как мы видели в разделе «Изменяемость» на с. 283, программирование в распределенных системах — непростая задача, потому что в них есть много моделей неполадок, которых не было бы в более интегрированных системах, где модели неполадок проще. Работа с неизбежной *изменяемостью* сервисов в ландшафте API требует изменений в образе мышления разработчиков.

При первоначальном переходе к ландшафту API разработчики, вероятно, будут придерживаться своих моделей программирования и писать приложения, делающие слишком оптимистичные предположения о доступности компонентов. В ландшафтах со множеством зависимостей может быть особенно сложно найти источник проблем. При их появлении в одном из приложений для поиска причин может потребоваться отслеживать несколько сервисов — и это задание отличается от более традиционного подхода, заключающегося в последовательной отладке монолитного кода.

В идеале приложения будут обрабатывать все зависимости API и иметь встроенную отказоустойчивость, включая такие подходы, как *постепенное ухудшение*. Чтобы лучше справляться с изменяемостью, надо разрабатывать API в более

устойчивом стиле и пытаться максимально использовать операции в ландшафте. Не все взаимозависимости приложения важны, поэтому можно разработать приложение, поддерживающее откат к предыдущей версии и в то же время приносящее доход, даже если некоторые взаимозависимости недоступны.

РАЗВИТИЕ ИЗМЕНЯЕМОСТИ

- Локализация условий возникновения ошибок — основное требование, которое нужно выполнять в ландшафте API. Возможность отслеживать трафик, получая так информацию, крайне важна для локализации проблем.
- Изменение методов разработки для лучшей адаптации к децентрализованным режимам помогает убедиться, что отдельные сбои не обязательно превращаются в каскады по цепочке зависимостей.
- Чем большему числу разработчиков можно будет помочь или подтолкнуть к более устойчивым методам разработки, тем более надежным будет ландшафт API.

Поскольку изменяемость при децентрализации неизбежна, помните, что переход с интегрированной на распределенную модель меняет модель отказа. Учитывая, что отдельные сервисы теперь могут портиться, их надежность более глубоко влияет на надежность всего ландшафта. Общий эффект от отдельных неполадок увеличивается с ростом ландшафта API и изменением графика зависимости между ними.

Работа с потенциальным увеличением общей частоты отказов означает, что нужно минимизировать их распространение. Частично этого можно добиться, убедившись в устойчивости компонентов к сбоям, изолируя их, а не распространяя.

Изменяемость — это один из аспектов, который, как вам может показаться, можно отложить на время, но проблемы могут быстро выйти из-под контроля, особенно когда динамика ландшафта API начинает расти. И конечно, худший из вариантов, когда могут проявиться проблемы с надежностью, — это момент, когда ландшафт API и уровень его изменений резко увеличиваются. Поэтому хорошо бы начинать инвестировать в работу с изменяемостью на ранних этапах и считать это необходимым первым шагом на пути развития ландшафтов API.

СТРАТЕГИЯ РАЗВИТИЯ ИЗМЕНЯЕМОСТИ

Управление изменяемостью в ландшафте API требует изменения практики разработчиков для написания приложений, которые ведут себя ответственно в распределенной системе. Также нужно, чтобы ландшафт предоставлял инструменты для обработки изменяемости в распределенных системах, такие как отслеживание ошибок. Если вы не справитесь с изменяемостью, эффект почувствуете очень быстро, особенно когда ландшафт быстро растет и/или когда его службы имеют разную эксплуатационную стабильность.

Итоги главы

В этой главе мы исследовали путь зрелости ландшафта, используя его аспекты из главы 9 как способ сформулировать путь к более зрелому ландшафту API. Для каждого из аспектов мы рассмотрели отдельные факторы, влияющие на зрелость, и узнали, как инвестиции в ее повышение отразятся на ландшафтном уровне. Мы решили не указывать один линейный путь к развитию ландшафта API, используя вместо этого его аспекты, чтобы показать многогранность этого пути. Главное, о чем нужно помнить: все аспекты влияют на общее развитие ландшафта API, поэтому важно учитывать их все при оценке текущего уровня его развития и возможных путей его улучшения.

В следующей главе мы рассмотрим, как зрелость ландшафта API взаимодействует с жизненным циклом API-продукта из рис. 7.1. Ведь цель API-ландшафтов — обеспечивать удобные условия для разработки, использования и улучшения API, поэтому нужно понимать, как на методы создания отдельных продуктов влияет ландшафт, где они разрабатываются и развертываются.

Управление жизненным циклом API в меняющемся ландшафте

Из многих капель собирается ведро, из многих ведер собирается пруд, из многих прудов — озеро, а из многих озер — океан.

Перси Росс

Если вы дочитали книгу до этого момента, то, скорее всего, заметили, что есть некоторые ключевые различия между управлением жизненным циклом одного API и их ландшафта. По нашему опыту можем сказать, что у компаний, которые замечают различия между одним и множеством и действуют, учитывая их, выше шансы на долгосрочный успех в процессе цифровой трансформации.

В этой главе мы кратко затронем эти различия и углубимся в столпы жизненного цикла из главы 4. Сосредоточимся на ландшафте — множестве API из главы 9, а не на одном API. Это должно наглядно показать, как проблемы масштаба, объема и стандартов из главы 1 вступают в игру по мере роста ландшафта API.

На протяжении всего пути мы будем возвращаться к восьми аспектам ландшафта API (см. главы 9, 10) и затронем некоторые элементы принятия решений (см. главу 2). Собранные вместе, эти части помогут вам понять, как лучше всего применять эти схемы и методы в своих командах, для своих продуктов так, как будет оптимально для культуры и ценностей вашей компании.

Но сначала давайте рассмотрим некоторые прагматические подходы к тому, как заставить управление API работать на сложном уровне развивающегося ландшафта.

Управление развивающимся ландшафтом на практике

В любой средней или крупной организации в конечном итоге будет много API, у каждого из которых будут собственные динамики команд, микрокультуры и жизненные циклы продукта. Ранее мы коснулись свойств ландшафта API и того, о чем вам нужно будет думать по мере его роста.

Позже в этой главе мы попытаемся объединить макромодель построенного нами ландшафта с микромоделью продукта API, которую мы создали ранее. Но прежде чем мы это сделаем, давайте рассмотрим три практики, которые помогут вам начать управлять своим ландшафтом.

По правде говоря, управлять ландшафтом API-продуктов нелегко. Мы уже упоминали об их сложной природе. Но эта сложность действительно становится очевидной, когда именно вы отвечаете за то, чтобы сделать все API в вашей организации безопаснее, лучше и дешевле.

Учитывая это, мы изложили несколько подходов, которые помогли нам и практикующим специалистам, с которыми мы общались. Начнем с важного первого шага в любой работе по проектированию системы: определения ваших границ путем установления ваших «рамки дозволенного».

Организуйте в коллективе свои «красные линии»

В мире природы эволюция происходит потому, что одни вещи работают, а другие нет. То, что работает, сохраняется надолго, а что не работает — отмирает. Эволюция — мощная сила. Но с практической точки зрения вы не можете позволить себе, чтобы ваш продукт, бизнес или организация «отмерли» ради долгосрочного эволюционного успеха.

Вот почему в самом начале вы должны определить «красные линии» для вашего ландшафтного контекста. Какие вещи не подлежат обсуждению там, где вы работаете? Те, с которыми вы не можете позволить себе экспериментировать. Это может быть связано со слишком высоким риском, противоречием корпоративной культуре компании или потому что это мешает вам достичь известной бизнес-цели.

Такие ограничения в больших организациях часто не озвучиваются и четко не определяются. Это может привести к проблемам, когда приходят новые люди и тратят время на попытки внедрить инновации туда, где границы уже установлены. Полезно выразить их так, чтобы люди могли быстро их освоить — в виде принципов, политики или даже в виде «лучших практик».

Вот некоторые «красные линии», с которыми мы сталкивались в разных компаниях:

- перебои в обслуживании неприемлемы, и мы должны любой ценой обеспечить стопроцентное время безотказной работы клиентских приложений;
- мы не можем использовать инструменты компании X из-за недавнего спора;
- структура команды должна соответствовать недавно опубликованным приказам отдела кадров;
- не принимайте технических решений, противоречащих решениям основателя;
- мы не можем хранить какие-либо данные для пользователей из Свенборгии.

Не забывайте, что «красные линии» не для того, чтобы их оспаривали. Это означает, что вы ограничиваете свой инновационный потенциал всякий раз, когда внедряете их. Но, с другой стороны, ограничения могут быть лучшим другом дизайнера. Они могут помочь вам направить энергию организации в области с огромным потенциалом для улучшения.

Платформы важнее проектов (в конечном итоге)

Очень сложно получить максимальную отдачу от ландшафта API без постоянного улучшения. Это центральная тема книги, и она отражает наш реальный опыт. Но проблема в том, что многие организации не предназначены для такой работы. Постоянное совершенствование может означать и постоянные затраты, а это может стать большим и неожиданным изменением для организации, которая еще не осознала ценность управления API.

Другой способ сказать об этом в том, что организации следует принять продуктовый (или платформенный) образ мышления для управления API. Это означает, что постоянная команда проектирует, создает и поддерживает ландшафт API. Они принимают решения и несут ответственность за улучшение

продуктов API внутри него. Это отличается от проектного мышления, когда организация финансирует временные команды, выполняющие проекты по улучшению ландшафта.

Вот несколько примеров классических проектных подходов:

- ограниченные полугодовые инвестиции в финансирование для улучшения управления API;
- срочное привлечение консалтинговой фирмы для аудита и улучшения комплекса API;
- использование фонда централизованно управляемых групп доставки для краткосрочных проектов по совершенствованию технологий.

С точки зрения организации проектный подход эффективен, измерим и управляем. Но он создает препятствия для изменений, выполнения и принятия целостных решений. Без устойчивой команды ландшафтные особенности становятся разрозненными и противоречивыми. Без постоянного финансирования вводится меньше изменений или улучшений из-за накладных расходов, связанных с получением разрешений.

Если повезет, вы сможете создать постоянную команду, пользующуюся доверием организации для курирования вашего ландшафта с первого дня. Но во многих компаниях это доверие (и финансирование) нужно заслужить. Для этого вам нужно будет работать усерднее и адаптировать проектную перспективу к платформенной путем предоставления ценности. Иногда для этого придется вести переговоры. На основе прошлых примеров вы можете попробовать следующие подходы:

- использовать часть финансирования для создания бизнес-обоснования следующего цикла финансирования. Повторяйте этот процесс для создания постоянной команды;
- выберите фирму, которая будет рассматривать ваши API с точки зрения ландшафта, чтобы она могла дать рекомендации по организационным и операционным изменениям, которые потребуются для успеха платформы;
- определите специализацию в области управления API. Затем найдите и используйте группу людей, которую вы сможете вырастить в постоянную команду.

Ни один из них не идеален. Но они показывают, что вам может потребоваться практический подход для перехода вашей организации к платформенному мышлению снизу вверх.

Дизайн для потребителей, производителей и спонсоров

Будь то продукт или платформа, успех зависит от удовлетворения пользовательских потребностей. В ландшафте API вам нужно быстро определить ключевого потребителя, производителя и спонсора.

- *Обслуживание ваших потребителей.* Одной из ваших первых целей должно стать улучшение того, как ландшафт (см. главу 9) может отвечать потребностям пользователей API. Как вы будете формировать разнообразие, объем, видимость и другие его параметры, чтобы они отвечали нуждам потребителей? Хорошая отправная точка — определить, кто ваши основные потребители и чего они хотят. Затем вы можете проверить свои решения по проектированию и исполнению на соответствие этим потребностям. Например, предлагая услугу планирования встреч, мы бы отдавали приоритет не только потребностям разработчиков, использующих наш API для планирования, но и потребностям команд, которые применяют внутренние API для удовлетворения потребностей этих пользователей.
- *Обслуживание ваших производителей.* Ориентация на потребителя важна для продукта. Но чтобы охватить платформенную перспективу (см. раздел «Принцип платформы» на с. 263), нам нужно будет удовлетворить и потребности команд API. Как вы облегчите им создание API, отвечающих потребностям пользователей? Опять же, определите, кто ваши основные производители API. Это может означать выделение нескольких «архетипов», сообществ или «племен». Например, вы можете определить, что большинство ваших команд используют Java и фреймворк Spring Boot. Или что потребности команд, работающих над внешними API, отличаются от тех, кто работает над внутренними API. Как бы вы их ни определили, цель в том, чтобы удовлетворить потребности этих производителей с помощью принимаемых вами ландшафтных решений и сформировать API, которые они выпускают.
- *Обслуживание ваших спонсоров.* У каждой платформы есть как минимум один владелец или спонсор. Это люди, которые принимают решения о целях системы и обеспечивают финансирование ее функционирования, совершенствования и запуска. Когда мы говорим о теории платформы, об этой группе легко забыть. Но каждый, кто руководил программой управления API в крупной организации, прекрасно понимает, как важно удовлетворять потребности спонсоров. Если вы не подумаете о них при разработке своей платформы, она может просуществовать очень недолго. В крупных организациях лучше сделать ландшафт API проще для понимания и наблюдения для спонсоров, особенно если финансирование вашей платформы исходит

от нетехнической бизнес-команды. Насколько легко бизнесу понять имеющиеся у него возможности с поддержкой API? Насколько легко они могут сопоставить структуру вашего ландшафта со своими стратегиями бизнеса? Упрощение вашего ландшафта в соответствии с потребностями спонсора может помочь вашему долгосрочному успеху.

ПРОЕКТИРОВАНИЕ ПЛАТФОРМ

Перспектива платформы не уникальна для API. На самом деле уже есть несколько замечательных инструментов и методологий, которые можно использовать для разработки целостных решений, помогающих взглянуть на ситуацию с учетом потребностей. Мы успешно применяем подход к проектированию услуг (<https://oreil.ly/XvvdX>) в наших более крупных проектах. Есть и специальные инструменты, такие как Platform Design Toolkit (<https://oreil.ly/rfJsA>), способные помочь вам двигаться в правильном направлении.

Сосредоточение внимания на участниках вашего ландшафта поможет вам сохранить фокус на реальных потребностях и задачах ваших сотрудников и пользователей. Вы сможете принимать более обоснованные решения относительно ландшафта и ваших API-продуктов. Но для некоторых решений вам понадобится недюжинная смелость. Вот почему так важен дух тестирования, измерения и обучения.

Тестировать, измерять и учиться

Когда речь идет о решении больших, запутанных, сложных проблем, самый частый совет, который вы можете услышать, — это делать небольшие изменения и извлекать из них уроки. Такой подход «тестируй и учись» позволяет вам сделать небольшие инвестиции в изменение системы, которая вознаграждает вас информацией о том, как ее нужно улучшить. Ландшафт продуктов API квалифицируется как сложная система, которая нуждается в подходе «тестируй и учись». Советуем вам разработать стратегию для вашей программы управления API, ориентированную на эту идею.

Настоящая проблема, с которой вы столкнетесь, — определение правильных шагов и лучшего способа измерения вашего прогресса по мере продвижения. Ваши организационные цели, «красные линии», технологии и люди, с которыми вы работаете, будут определять лучший способ продвижения.

Рассмотрим эти два совершенно разных подхода к запуску ландшафта API — один для новой, облачной платформы «с нуля», а другой для сложной, уже существующей, «зрелой» платформы API.

Эволюция органического ландшафта «с нуля»

1. Начните разработку новых продуктов API. Сделайте это до создания каких-либо объектов на уровне ландшафта.
2. Создайте кросс-продуктовую команду специалистов API.
3. Используйте опыт и возможности отдельных API, делая их функциями ландшафтного уровня.
4. Предоставьте возможность специалистам API привнести ландшафтные функции в свои продукты.

Структурированная эволюция для «зрелого» ландшафта

1. Определите API-кандидаты, которые выиграли бы от улучшения (и которые безопасно изменять).
2. Определите оперативные меры и ключевые показатели эффективности для измерения прогресса.
3. Внедрите регламенты на уровне ландшафта и протестируйте их с помощью API-кандидатов.
4. Наблюдайте за результатами измерений и вносите коррективы в ландшафт.
5. Внедрите регламенты на уровне ландшафта для множества API.

Мы много говорили в этой книге о постоянном совершенствовании и управлении API. Но для этого вам нужно действительно хорошо и детально разбираться в этой сложной проблеме. Ничего страшного, если с первого раза у вас не все получится. Но важно придерживаться менталитета «тестировать, измерять и учиться», чтобы становиться лучше по мере роста и развития. Определить правильные меры и наметить верные шаги не так-то просто. Но хорошая новость в том, что вы можете использовать восемь V-элементов ландшафта и столпы продукта API как основы для руководства.

Давайте погрузимся в эти модели и посмотрим, как они работают вместе.

Продукты API и столпы жизненного цикла

Как отмечалось в главе 3, применение подходящего уровня мышления в интересах продукта API помогает вашим командам сосредоточиться на клиенто-ориентированном использовании интерфейса. Для вас это первая возможность разработать и реализовать API с помощью подхода Клейтона Кристенсена «Работа, которую нужно выполнить» (Jobs-to-be-Done, JTBD) (<https://oreil.ly/tuyRY>). Научите свои команды по дизайну и архитектуре задавать вопросы: «Как это

поможет нам достичь наших бизнес-целей?» и «Каких OKR мы хотим добиться с помощью этого дизайна?». На бизнес-уровне это сильно снизит вероятность того, что серверные команды просто выпустят интерфейсы, основанные на данных, без четкого пользовательского потока. Фокусируясь на JTBD и пользователях, руководителям IT проще опрашивать команды об их вкладе в общую работу и привязывать инициативы в сфере API не только к KPI, сконцентрированным на технологиях, но и к общим OKR на уровне бизнеса.

Помимо стратегии AaaP мы рассматривали и так называемые столпы жизненного цикла API (см. главу 4). Хотя это ненастоящие *циклы* (у них нет четкого порядка, повторяющегося раз за разом), столпы жизненного цикла определяют важнейшие элементы, поддерживающие жизнеспособную программу API. Их можно использовать как руководство по созданию API. Так модель AaaP и столпы жизненного цикла поддерживают друг друга на уровне одного API.

Но как вы видели в последних двух главах, знакомящих с понятием ландшафта API, компании редко работают с несколькими отдельными API. Напротив, большинство организаций создают системы взаимозависимых и функционально совместимых API, такие, которые помогут в получении дохода и снижении затрат и рисков при использовании API и сервисов для достижения OKR.

Ландшафты API

До сих пор эта книга была сосредоточена на том, как непрерывно управлять отдельными API и как они вписываются в общую картину ландшафта API. Восемь аспектов из глав 9 и 10 использовались для того, чтобы позволить вам внимательнее рассмотреть отдельные элементы этой картины и обдумать все важные задачи управления растущим и непрерывно развивающимся ландшафтом API.

Сейчас мы хотим соединить перспективу управления отдельными API и рабочее окружение ландшафта API, где они находятся. Мы сфокусируемся на основной теме — на том, что связь между отдельными API и ландшафтами всегда должна быть обоюдной. *API вносят свой вклад в ландшафт*, и за ними должно быть максимально просто наблюдать, чтобы получать информацию об их разработке и развитии. *Ландшафт направляет и поддерживает команды продуктов API*, предоставляя руководство, поддержку и информацию, формирующую общую картину принципов, протоколов и методов.

Восемь аспектов можно использовать, чтобы сосредоточиться на отдельных столпах жизненного цикла API. Мы соединим перспективы отдельного API и ландшафта, обсуждая, как нахождение API в нем влияет на эти столпы

и какие аспекты важнее, когда вы снова обдумываете этот конкретный столп (и как поддержать команды API-продукта, опираясь на него) в контексте всего ландшафта.

Момент принятия решения и развитие

В разделе «Решения» на с. 41 мы заявляли, что код и интерфейсы, создаваемые сегодня, — «глупые»: релизы выполняют то, что велено, и ничего больше. Код не способен *решать*, *исследовать* или *экспериментировать*, он может только *выполнять*. Решения и творческий подход исходят от человека, и их в какой-то момент нужно перевести в код и API, чтобы реализовать.

Для принятия решения важно знать, *когда* нужно это сделать. Откладывать решение зачастую разумно, если речь идет о реализации ландшафта API. В растущей системе так много взаимозависимых элементов, что поспешное решение может уменьшить число будущих возможностей или сделать невозможными некоторые из лучших вариантов решения проблем. Том и Мэри Поппендик, авторы нескольких книг, в том числе *Lean Software Development: An Agile Toolkit* («Бережливая разработка программного обеспечения: гибкий набор инструментов»), рекомендуют «откладывать подтверждение на позднейший ответственный момент, то есть момент, когда непринятие решения исключает важную альтернативу».

Но это бывает трудно сделать IT-менеджерам и дизайнерам системного уровня. Распространенный вопрос: «*Когда* наступает “последний ответственный момент”?» Цель главы — помочь вам распознать распространенные изменения формы вашего ландшафта API, пройдясь по столпам и продемонстрировав часто встречающиеся сложности, связанные с разными его аспектами. Это должно помочь вам сравнить аспекты, наблюдаемые в своей компании (разнообразие, объем и т. д.), с приведенными здесь примерами. Мы надеемся, что все это поможет вам понять, какие столпы требуют особого внимания из-за роста вашей системы.

Аспекты ландшафта и столпы жизненного цикла API

Столпы жизненного цикла и подход АааР совместно формируют методы, применимые в дизайне, разработке и выпуске API в вашей организации. И они же могут помочь в управлении растущим ландшафтом API. Можно создать простую матрицу (табл. 11.1), где вышеупомянутые аспекты и столпы жизненного цикла API сочетаются, чтобы показать предметные области, которыми нужно управлять в ландшафте.

Таблица 11.1. Столпы жизненного цикла и аспекты ландшафта API

Столп	Аспект		Словари	Скорость	Уязвимость	Видимость	Контроль версий	Изменяемость
Стратегия	✓	Разнообразие	✓	✓				
Дизайн	✓		✓					
Документация	✓		✓			✓		
Разработка	✓			✓				✓
Тестирование				✓	✓			✓
Развертывание	✓			✓				✓
Безопасность				✓	✓	✓		
Контроль						✓		
Обнаружение	✓	✓	✓			✓		
Управление изменениями			✓	✓		✓		

Каждый столп из главы 4 и каждый из восьми аспектов из главы 9 заслуживают внимания. Рост ландшафта — это не только про увеличение. Он *меняет форму*. В небольшом ландшафте (например, если одна команда работает над одним взаимосвязанным набором API) не придется тратить много времени на создание руководств и стандартов по стилям API или форматам сообщений. Все работают в одной команде, используют одни инструменты и стремятся к одним результатам.

Но по мере роста ландшафта API появятся новые команды с другими целями. Некоторые могут находиться далеко от вас, использовать другие наборы инструментов, стили API и руководства. В растущем ландшафте могут появиться большее разнообразие, более широкий словарь и разные уровни видимости, объема, скорости и уязвимости. Со временем растущие ландшафты меняют свою форму.

Рассмотреть каждый из аспектов ландшафта в применении к столпам жизненного цикла важно, но для этого нужна книга потолще той, что мы вам предлагаем. Вместо этого мы решили сосредоточиться на избранных аспектах каждого столпа, привести полезные примеры и посоветовать, как справиться с некоторыми сложностями.

В идеале вы сможете использовать эти идеи как руководство, когда ваш ландшафт начнет увеличиваться и станет проходить через стадии развития, описанные в прошлой главе. Поскольку в каждой компании своя культура, может оказаться полезно создать свои чистые шаблоны матриц. Так вы сможете начать дискуссию, которая позволит всем сотрудникам организации привести свои примеры и дать свои советы.

Теперь давайте пройдемся по столпам жизненного цикла API и выделим несколько самых распространенных аспектов, с которыми вы столкнетесь в ходе роста и изменения ландшафта.

Стратегия

Во время роста ландшафта API компании тактика и даже краткосрочные цели ее стратегии API могут изменяться. В разделе «OKR и KPI» на с. 193 мы показали, как OKR и KPI могут повлиять на общую цифровую стратегию, на использование API и их ЦА. Это все еще важно при расширении ландшафта, некоторые детали изменятся.

KPI для ваших первых API, скорее всего, фокусировались на надежности, увеличении использования внутри компании и вкладе в доход или снижение затрат. При расширении вашей программы до десятков, сотен или тысяч API

вам может понадобиться перестроить KPI, чтобы они сфокусировались на вопросах, уникальных для больших ландшафтов API. Здесь вы и можете применить аспекты из главы 9, чтобы сменить свою тактику и требования к реализации на соответствующие текущим задачам.

Среди аспектов, которые нужно учитывать при перестройке стратегии, особого внимания требуют три: разнообразие, объем и скорость. Кратко рассмотрим их.

Разнообразие

При добавлении в ландшафт новых групп по продукту, команд и пользователей ваша способность контролировать каждый элемент дизайна и реализации API станет менее эффективной из-за естественной тенденции к росту разнообразия. Заставить одну команду или небольшую группу параллельных команд выполнять одни указания по дизайну и использовать одинаковые форматы, инструменты и методы тестирования и релиза легко.

Но если вы добавите новые локации (например, офис, находящийся через полмира от вас), начнете поддерживать API групп других продуктов (например, приобретенных компаний) и работать с продуктами, использующими множество технологий (STFP, системы мейнфрейма и т. д.), то поймете, что больше не можете просто диктовать, как выполнять задания.

С увеличением ландшафта растет и разнообразие. Вместо того чтобы пытаться избежать его, важно изменить стратегию API так, чтобы учесть различия и сосредоточиться на приоритетных *принципах* всех команд, а не заставлять всех использовать одинаковые методики во всех спектрах технологий и группах продуктов (см. раздел «Принципы, протоколы и шаблоны» на с. 265).

Увеличение разнообразия — это нормально.

Объем

При увеличении ландшафта растет и объем многих элементов — становится больше API, трафика, команд и т. д. Но ваши ресурсы для управления этим ростом могут быть ограничены. Поэтому приходится выбирать, какие из инициатив поддержать, какие деактивировать, а какие API пока оставить без изменений.

При выборе сосредоточьтесь на API, которые приносят больше пользы бизнесу, которые легче обновлять и поддерживать, и на других бизнес-ориентированных целях, способных помочь вам справиться с растущим объемом. Возможно, придется начать инвестировать в платформы, лучше масштабируемые при высоком объеме трафика. Вам может понадобиться перейти с локальных релизов на виртуальные облачные устройства, которые проще масштабировать. Возможно,

некоторые из API будут работать лучше в среде «функции как услуга» и др. А иногда разумнее вернуть трафик обратно в локальную инфраструктуру для снижения затрат.

Объем — один из самых распространенных аспектов, с которыми вы столкнетесь при росте ландшафта. Есть много способов справиться с ним.

Скорость

Последний аспект, который мы здесь рассмотрим, — скорость. Некоторые пользователи сообщают нам, что «все становится быстрее» и им непросто угнаться. Иногда это действительно так, но иногда проблема не в скорости изменений, а в их *количестве*. Скорость может проявляться в нескольких формах.

При росте ландшафта понадобится вносить изменения во многие его части, поэтому вы будете чаще их замечать. Нужно перестроить стратегию API так, чтобы изменения не приносили вреда и становились более дешевыми и менее рискованными. Обычно это означает установку ограничений для масштабных изменений (например, формальных предложений, внимательного изучения и утверждения) и их ликвидацию для небольших коррективов (например, незначительных изменений макета UI, исправлений ошибок, изменений, не вредящих API, и т. д.).

Скорость может проявляться и в виде роста числа пользователей. Если ваши API успешны и приносят новые заказы, но серверные команды все еще выполняют такие задачи, как изучение и утверждение клиентов, вручную, то вскоре вы обнаружите множество невыполненных заказов, сулящих потерю части дохода. Стратегия вашего API должна простираться за пределы технических подробностей разработки и релиза кода — она должна включать в себя всех сотрудников организации.

Скорость выражается не только в простом увеличении темпа, она может создать и иллюзию замедления в некоторых частях компании.

Дизайн

Дизайн интерфейса — важный столп работы над API-продуктом (см. раздел «Дизайн» на с. 116). А когда API разрабатываются в ландшафте, появляются новые сложности. Часть ландшафта — это уже не отдельный продукт, а часть семьи продуктов. Насколько такое видение должно влиять на дизайн отдельных API, во многом зависит от предполагаемого их применения.

- API для многократного использования проектируются с высоким уровнем видимости и будут задействоваться в качестве контактной точки отдельного

пользователя. Их можно по-прежнему разрабатывать как отдельные продукты, оптимизируя дизайн в первую очередь для пользователей. Но с точки зрения реализации, невидимой пользователям, лучше синхронизировать их с ландшафтом, позволяя разработчикам применять все преимущества поддержки и инструментов.

- API, предназначенный для использования в качестве части ландшафта, например в соединении с другими API той же семьи продуктов, нужно разрабатывать, помня об этой схеме использования. Знакомый дизайн будет играть важную роль. Для разработчиков, использующих и сочетающих разные API одного ландшафта, полезнее синхронизация на уровне дизайна API.

Глядя на два этих сценария, становится понятно, что дизайн API в обоих случаях важен, но по-разному. В первом случае синхронизация важна в основном при реализации, хотя и при создании дизайна можно пользоваться советами из руководства, определяя и решая проблемы. Во втором — синхронизация важна и на уровне дизайна, и на уровне реализации, поэтому руководство здесь важнее.

Когда речь идет о подходе к дизайну в ландшафте API, сказанное здесь можно дополнить с помощью аспектов из главы 9. Из них на дизайн сильнее всего влияют разнообразие, словарь и контроль версий.

Разнообразие

Разнообразие — естественный результат развития ландшафта API. Причина в изменении схем дизайна со временем, отчего одна проблема решается по-разному в зависимости от методик, принятых на момент создания дизайна. Вторая причина в том, что разные продукты соответствуют разным потребностям и нацелены на разных пользователей. Поэтому одной идеальной для всех групп пользователей методики нет.

По этим причинам разнообразие нужно разрешить, а не пытаться ограничить пространство решений одним возможным дизайном. Но руководство, управляемое C4E (см. раздел «Центр поддержки» на с. 290) и влияющее на сферу дизайна, позволяет ограничивать его так, чтобы помочь дизайнерам выбрать самый подходящий вариант дизайна из всех, основываясь на том, как те применяются и какие инструменты поддерживают.

Одна из антисхем разнообразия появляется, когда одну проблему в разных API решают по-разному без причины. Это не только пустая трата ресурсов команды — это отрицательно влияет на продуктивность пользователей, потому что им приходится заново учиться работать с каждым API. Такое отношение к отдельному API часто называется «прекрасные снежинки» — каждый из них уникален

и отличается самыми тонкими деталями. Этот подход совершенно не продвигает и не поддерживает повторное использование там, где это необходимо.

В общем, разнообразие дизайна важно и должно приниматься там, где на это есть *причина, основанная на самом продукте*. Иначе экономичнее и полезнее будет использовать устоявшиеся схемы дизайна.

Словари

Согласование словарей дизайна в организации поможет создать более последовательную практику проектирования и избежать повторных (и в худшем случае конфликтующих) попыток создать модели для одной и той же области. Но определение и синхронизация словарей требуют затрат, поэтому попытка синхронизировать все внутри организации может дорого обойтись¹.

Обычно доменные концепции, которые можно безопасно внедрить в сервис (то есть те, которые не отображаются с помощью API этого сервиса), вообще не нужно синхронизировать. Это детали реализации сервиса, и их видимость за его пределами будет противоречить принципу инкапсуляции.

Концепции, важные для API сервиса, можно разделить на два типа.

- *Словари, не зависящие от домена*, должно быть легко найти внутри организации. Простой пример — список стран или языков. Любой API, использующий словари, должен пытаться разграничить API и словарь, а затем составить список словарей, чтобы можно было наблюдать за их применением.
- *Словари, зависящие от домена*, должны быть установлены (а не просто оформлены в виде ссылки) как часть дизайна API. В зависимости от развития этого аспекта (см. раздел «Словари» на с. 297) ландшафт может обеспечивать поддержку, чтобы облегчить командам определение и распространение нового словаря.

В общем, управление словарями для разработки API следует идее о том, что гармонизация словарей — это хорошо и что наблюдение и поддержка могут помочь в этом. API, фиксирующий использование словарей, помогает ландшафтным менеджерам наблюдать за их применением и развитием. Это может привести к упрощению дизайна API через использование устоявшихся словарей вместо создания новых.

¹ Примером могут служить многочисленные попытки крупных организаций создать информационную модель предприятия (см. врезку «EIM и API: идеал против прагматизма» на с. 274). В некоторых случаях предприятие меняется довольно медленно для выполнения этой задачи, но даже тогда инициативы создания EIM редко считаются удачными.

Контроль версий

Одна из главных целей высокоразвитой стратегии API — отделить эволюцию сервисов, чтобы они могли развиваться по отдельности с оптимальной для них скоростью. Такая эволюция означает, что реализации сервисов могут меняться, как и сами API. С точки зрения дизайнера первое все же важнее, так как может означать, что команда продукта решила справиться с проблемой по-новому.

Чтобы узнать уровень изменений ландшафта API, важно отслеживать версии (см. раздел «Контроль версий» на с. 309). По принципу выражения API через API (см. раздел «API через API» на с. 268) это означает, что они должны показывать номера своих версий, чтобы изменения в их реализации и дизайне стали видимыми.

Управление версиями тоже важно для клиента — об этом подробнее в разделе «Управление изменениями» на с. 357. Так, важной частью дизайна становится следование его принципам, облегчающим работу с новыми версиями для пользователей. В идеале это должно быть так, чтобы они всегда могли узнать об открытии доступа к новой версии, но чтобы при этом им приходилось что-то делать, только если они решают воспользоваться новыми функциями.

В общем, при разработке дизайна в ландшафтах API всегда нужно помнить о том, что сервисы будут непрерывно развиваться. В таком случае *проектирование с учетом будущих изменений* означает упрощение получения информации о новых версиях и для руководства, и для пользователей. Это значит и что дизайн должен требовать от пользователей как можно меньше усилий при адаптации к новым версиям.

Документация

Документация сильно зависит от зрелости API (см. раздел «Документация» на с. 119), хотя наличие базовой документации — это всегда неплохо, а в случае подхода AaaP (см. главу 3) с нее и нужно начинать.

Документация может быть в самых разных формах. К примеру, минимальную справочную документацию можно сгенерировать с помощью технических приспособлений, например описаний OpenAPI. Документацию можно дополнить комментариями, примерами, учебными пособиями и руководствами по использованию. Ее даже можно внедрить в сам API, так чтобы он стал самостоятельно описываемым. Это поможет улучшить опыт разработчика так, чтобы убрать почти все препятствия для использования и сделать API оптимизированным самодостаточным продуктом.

Но это очень дорогостоящий шаг, а инвестиция считается полезной, только когда усилия по созданию идеальной документации перевешиваются количеством разработчиков, которым она принесла пользу.

Уровень инвестиций в документацию отдельных API зависит от их развития и намеченной целевой аудитории (см. главу 7). Но в таком случае ландшафт должен предоставить руководство и поддержку. Из всех аспектов самые важные при поддержке документации отдельных API — это *«Разнообразие»*, *«Словари»*, *«Контроль версий»* и *«Видимость»*.

Разнообразие

Поддержка разных стилей документации поможет каждой команде выбрать оптимальный для своего API. Выбор будет зависеть как от *стиля* API, так и от развития и предполагаемой ЦА отдельных API.

Предоставление командам возможности выбирать инструменты и глубину документации, соответствующие их дизайну API, стадии его зрелости и аудитории, поможет им публиковать ту документацию, что отвечает их текущим потребностям.

Если у вас есть «кластеры» требований к документации, полезно иметь конкретные руководства и инструменты для утверждения, позволяющие командам внедрять проверку документации в процесс создания API. В большинстве случаев нельзя автоматически проверить все аспекты документации, но часто можно включить хотя бы несколько проверок на адекватность (есть ли в ней разделы о расширении и создании версий и точно ли они явно подчеркнуты и выделены). Тогда команды будут лучше понимать, чего от них ожидают.

Возможно, со временем разнообразие документации будет увеличиваться. Некоторые ее стили могут устареть, а другие — появиться. В идеале разнообразие в стилях документации должно быть отделено от ее рабочего окружения. Так, лучше сделать, чтобы какие-нибудь важные принципы, например указания по разделам, посвященным расширению и контролю версий, можно было распространять на разные стили документации.

Словари

Один из признаков развития ландшафта API — управление словарями нескольких API, чтобы создателям было проще найти и повторно задействовать существующие словари, а пользователям — применить свое знание терминологии в нескольких API (см. раздел «Словари» на с. 297).

Управление словарями для улучшения документации означает *управление документацией этих словарей*, чтобы любой API, использующий словарь, мог повторно использовать существующую документацию. В зависимости от способа управления повторное использование документов может быть «паутинным» и представлять собой просто ссылки на документацию, доступную в другом месте. А если стиль чуть менее «паутинный» и относится скорее к созданию автономной документации для API, тогда повторное использование может означать включение ее в документацию API¹.

Важно помнить, что управление словарями документации очень полезно для наблюдения. Если API документируют используемые термины так, что это *поддерживается* или как минимум *наблюдается* в ландшафте, становится гораздо проще разбираться в использовании терминов в разных API, предлагать новые словари и понимать, какие уже не используются. Еще раз: *наблюдение за API для поддержки ландшафта и поддержка API, чтобы сделать вещи наблюдаемыми*, — это две стороны медали, обеспечивающие лучший способ объединения преимуществ для отдельных API и для их ландшафта.

Контроль версий

Одна из главных целей API и их ландшафтов — разделить реализации, чтобы отдельные продукты могли развиваться индивидуально. Такой рост скорости продукта ведет к росту числа его *версий* (см. раздел «Контроль версий» на с. 309). Хотя в идеале большинство версий не должны вредить API, каждую, что меняет младшую версию при использовании семантического контроля, все же нужно документировать. Это помогает пользователям понимать, какие изменения могли произойти, а создателям — делать документацию полезной для нескольких версий.

Как сказано в разделе «API через API» на с. 268, документация, как и любая информация об API, должна быть его частью. Согласно этому принципу, *история документации* тоже должна быть частью API, позволяя пользователям понимать эволюцию API, отслеживая ее².

¹ «Паутинный» способ, который мы упомянули, называется трансклюзией. Это значит, что он доступен через документацию API, но сохраняет свою идентичность как документация словаря.

² По крайней мере, это верно для некритических изменений в API. Для критических (мажорная версия в семантическом контроле версий) может сработать другая схема — архивирование документации старых версий, что само по себе может быть сервисом, предоставляемым ландшафтом API.

Руководства по управлению версиями могут упростить использование API и помочь пользователям получать сведения о новых версиях и решать, когда и как они хотят подстроиться под обновление. Предоставление командам разработчиков API рекомендаций о том, как создавать и публиковать документацию для разных версий, поможет упростить для них следование этим практикам. Ландшафт может предоставлять поддержку и инструменты для тестирования, чтобы разработчики могли сразу же получить обратную связь по своей документации для нескольких версий.

Если ваш ландшафт использует семантический контроль версий и придерживается «паутинных» принципов, рекомендуем создать навигацию по всем версиям документации API с помощью ссылок RFC 5829 (<https://oreil.ly/byW2L>). Эта схема включает в себя навигацию по истории документации и документацию отдельных версий со ссылками друг на друга. Тестирование на наличие этих ссылок можно провести после развертывания API и их документации, чтобы можно было утвердить хотя бы схематическую часть этой методики.

Таким образом, документация в ландшафтах API будет содержать аспект управления версиями, и, предоставляя руководство и поддержку для документирования версий наблюдаемым способом, управление ландшафтом может получить некоторое представление о версиях API и их документации.

Видимость

Документация сама по себе может быть довольно сложным ресурсом, содержащим много информации и структур. Как мы уже обсудили, полезно иметь существующие инструменты и поддержку документации, чтобы команды API могли положиться на определенные процессы при ее создании, а не выбирать или создавать свои.

Важно помнить (см. раздел «Разнообразие» на с. 295), что главной целью документации по руководству и поддержке должно быть объяснение командам не *как* что-то создавать, а *что* нужно создавать. У них должен быть поддерживаемый инструментарий, позволяющий легко ответить на вопрос «что?», следуя инструкции. Но важно отделить его от того, что он должен создавать.

Документация API всегда должна быть видимой, а вместе с ней — все аспекты, значимые для ландшафта. Это важно как для пользователей, так и для самого ландшафта, который затем может использовать эту видимость как способ получить подробное представление об API и при необходимости предоставить ссылки на документацию. Поэтому важно обозначить в руководстве, какие требования должны быть наблюдаемыми с точки зрения ландшафта, и добавить эти аспекты к руководству и инструментам.

Разработка

Может показаться, что *разработка* не так уж важна в ландшафтах API. В конце концов, задача API — *инкапсулировать* реализации, поэтому способ разработки не рассматривается с точки зрения чистого API. Но без разработки реализации API точно не было бы, поэтому столп «Разработка», который мы обсуждали на с. 122, — важнейшая часть ландшафтов API. Повышение общей эффективности ландшафтов — одна из главных целей управления ими, поэтому важно изучать, как разработка встроена в их перспективу.

И снова посмотрим на интернет как на крупнейший из известных нам ландшафтов API. Если бы кто-то ввел правило, что все веб-приложения должны программироваться на одном языке с помощью одних инструментов разработки, это быстро погубило бы его. В конце концов, один из главных рецептов успеха Сети — то, что здесь командам разработчиков обеспечена полная свобода реализации выбранных решений, если их продукт в итоге работал как приложение, используемое через браузер.

С другой стороны, хотя для успеха интернета было важно, чтобы языки и инструменты разработки не зависели от веб-архитектуры, а веб-приложения стали массовыми, поддержка разработки стала основным фактором повышения ее эффективности. PHP, ASP, JSP, JSF, Django, Flask, Ruby on Rails, Node.js и многие другие сформировали способ разработки веб-приложений в прошлом и настоящем. Но они и сами проходят свой жизненный цикл, поэтому можно точно сказать, что интернет не только пережил множество языков и инструментов для разработки веб-приложений, но переживет в дальнейшем.

Урок, который можно извлечь из такого взгляда на методы разработки и поддержки интернета, напрямую применим к ландшафтам API. Существование языков и инструментов, поддерживающих разработку отдельных продуктов, увеличивает продуктивность и помогает с определенными протоколами и шаблонами. Но важно помнить, что раз протоколы и шаблоны меняются, то меняются и языки, и инструменты. И даже без изменения протоколов и шаблонов все равно будет постоянный поток языков и инструментов, претендующих на лучшее решение существующих задач.

Поэтому, рассматривая столп разработки с точки зрения ландшафта, вы упустите возможности получить большую экономию, установить методы разработки и поделиться ими со всеми командами, оценить и принять новые языки и инструменты, когда они становятся доступными. Чтобы ландшафт поддерживал и реализовывал эти возможности, нужно помнить о самых важных аспектах — разнообразии, скорости, контроле версий и изменямости.

Разнообразие

Если смотреть с точки зрения подхода «API как продукт» (как обсуждалось в главе 3), начальные этапы концепции API не касаются деталей реализации или разработки продукта. Всех волнуют только дизайн, обсуждение протоколов и наблюдение за тем, как первая обратная связь может повлиять на ранние этапы проектирования.

Когда становится ясно, как должен выглядеть API-продукт, следующий шаг — продумать оптимальный способ самого создания продукта. Это решение должно быть основано на дизайне продукта (убедитесь в выборе правильных инструментов для работы) и команде (убедитесь, что команду устраивает выбор инструментов).

Поскольку разные API служат разным целям, ориентированы на разные группы потребителей и создаются разными командами разработчиков, вполне вероятно, что нет такого понятия, как лучшие язык и набор инструментов для разработки любого API. Поэтому очень важно поддерживать разнообразие в ландшафте и позволять командам экспериментировать с новыми подходами, если они чувствуют в этом необходимость.

Сбалансировать разнообразие с необходимостью не создавать ландшафт реализаций — непростая задача. Прежде всего нужно принимать во внимание необходимость *согласованности* в отношении аспекта разнообразия. Использовать разные языки и инструменты — вполне нормально, но мы советуем ограничить число вариантов для обеспечения согласованности между ними. Так инвестиции и образование окупятся за счет экономии на масштабе. И поэтому всегда есть некая «критическая масса» вокруг вариантов развития. Если какими-то языками или инструментами разработки пользуется больше определенного числа API-продуктов, они могут сменить статус экспериментальных на статус вариантов для реализации.

Скорость

Для отдельных API скорость показывает, как быстро продукт может быть реализован и его можно корректировать и адаптировать к меняющимся требованиям. На скорость влияет весь процесс разработки и развертывания (см. раздел «Развертывание» на с. 129). Хотя важно использовать языки и инструменты, которые являются хорошим решением проблемы реализации, ландшафт может помочь, предоставляя поддержку и инструменты, чтобы сделать процесс разработки и развертывания более плавным и быстрым.

На скорость влияет и размер сообщества разработчиков: чем больше команд используют языки и инструменты, тем быстрее они развиваются и тем быстрее их

проблемы могут быть решены. Поэтому скорость зависит не только от выбора подходящих языков и инструментов. На нее влияет и управление разнообразием, которое обсуждалось в прошлом разделе. Если разнообразие допускает наличие критической массы для распознавания и решения вопросов, то управление скоростью можно описать как лучшее решение этой проблемы, *когда на уровне организации появляется критическая масса*.

Контроль версий

Методы контроля версий определенно влияют на методы разработки API. Это важный аспект ландшафтов API. Ответственный подход к контролю версий становится необходим по мере роста сложности API-ландшафта с точки зрения размера, зависимостей служб и числа вносимых в него изменений (см. раздел «Контроль версий» на с. 309).

С точки зрения пользователя контроль версий может быть очень полезным (когда API улучшают для предоставления услуг, интересных потребителям) или разрушительным (когда API меняются, но пользователи не хотят менять способ использования сервиса). Как сказано в подразделе «Скорость» на с. 326, если при разработке скорость позволяет быстро вносить изменения и уменьшается отрицательный эффект от новых версий, общая ценность от гибкости сервиса максимально увеличивается.

На уровне ландшафта важно убедиться, что за скоростью разработки можно наблюдать. Так, вы можете следить за уровнем изменений и открывать доступ к информации о разных версиях. Для начала можно использовать стандартные способы осмысленной идентификации из подраздела «Семантический контроль версий» на с. 282 и продемонстрировать номера версий. Это уже даст информацию о динамике всего ландшафта API.

Изменяемость

Изменяемость сервисов в ландшафте API (см. подраздел «Изменяемость» на с. 341) создает ряд сложностей для существующих методов разработки. Главная проблема в том, что по определению ландшафты API децентрализованы и сталкиваются со всеми основными сложностями децентрализованной модели. Ответственное отношение к изменяемости ландшафтов требует другого подхода к программированию в условиях с более прочной связанностью. А если не обратить на это внимания, пострадает общая стабильность ландшафта, например произойдет каскад ошибок.

В правильной работе с изменяемостью большую роль играют языки, инструменты и методы разработки. Поэтому роль ландшафта в том, чтобы распознавать

и поощрять разработку, подходящую для присущей ему изменчивости. Возможно, это нужно не во всех сценариях, но так вы можете убедиться, что правильно работаете с изменчивостью.

GraphQL и доступность API

Изменяемость можно изолировать. Это значит, что некоторым командам придется подходить к ней более ответственно, а некоторым — менее. Так, если следовать шаблону «серверная часть для клиентской части» (Backend for Frontends, BFF) (<https://oreil.ly/qouqo>), одно приложение для серверной части будет служить агрегатором для нескольких API, а затем передавать *один* из них внешнему интерфейсу приложения с помощью гибкой модели API, основанной на запросах (например, GraphQL).

В этом сценарии у клиентской части очень простая модель доступа к API: GraphQL или доступен, или нет. Но серверной части придется столкнуться с гораздо более сложной моделью. При переводе запроса GraphQL в разные запросы к API все эти API должны считаться изменчивыми. Хорошо написанный распознаватель GraphQL сможет справиться с неполным отключением базовых API, отвечая с помощью частичных ответов GraphQL. В этом сценарии управление изменчивости *на уровне API* было передано серверной части BFF, а его клиентской части нужно работать только с изменчивостью *на уровне данных* (если брать сценарий частичных ответов GraphQL). Это значительно облегчает работу команды по разработке.

Управление присущей API-ландштафтам изменчивостью в рамках процесса разработки API может сильно повлиять на качество продуктов API и общую стабильность ландшафта. Важно убедиться, что языки, инструменты и методы разработки учитывают этот аспект, чтобы продукты начали лучше работать в постоянно изменчивой среде ландшафта API.

Тестирование

В разделе «Тестирование» на с. 126 мы объяснили, почему оно так важно, и установили довольно высокую планку, заявив: «Ни один API не должен отпираться в производство без тестирования». При росте ландшафта это может стать сложной задачей. Время и сроки выполнения работы всегда были весомым фактором в разработке ПО, поэтому от вас могут потребовать не только ускорить тестирование, но и *сократить* его, пропустив какие-то шаги. Это крайне неудачная мысль. Но сейчас мы покажем, как развивать методы тестирования при росте ландшафта.

Еще одна сложность, с которой вы столкнетесь при расширении ландшафта API, — повышение затрат на тестирование. Это касается не только фактической стоимости выполнения тестов, но и издержек при их *невыполнении* (например,

если вы не найдете проблем при тестировании). Другими словами, рост цены неудачи. Это может особенно беспокоить архитекторов системного уровня, так как неполадки в системе обычно четко видны и, если с ними не справиться, могут дорого обойтись вашей компании.

К счастью, опытные компании часто сталкивались с этой проблемой, и решения проблемы проведения тестирования в ландшафте API уже найдены и доступны. Здесь мы выделим самые распространенные из них. Надеемся, что это поможет вам верно смотреть на задачи тестирования и находить решения, подходящие вашей компании. Важнейшие аспекты для тестирования ландшафта — это объем, скорость, уязвимость и изменяемость.

Объем

Общая проблема по мере роста вашего API-ландшафта в том, что само число тестов, необходимых для «покрытия», начинает быстро расти. А поскольку большинство ландшафтов API полагаются на вызовы других API, количество тестов растет *нелинейно*. При добавлении одной конечной точки API, используемого 12 другими, не просто добавляется набор тестов — нужно модифицировать еще 12 таких наборов! Если ваш метод тестирования опирается на тесты, управляемые человеком (то есть когда сотрудники что-то печатают на экране и записывают результаты), спрос на тестирование в растущем ландшафте API быстро превысит возможности вашего отдела ОК.

Нелинейный рост количества тестов — один из основных поводов для введения автоматического тестирования. Гораздо проще масштабировать автоматические тесты, например запуская параллельно больше тестов, чем принимать новых сотрудников в команду по ОК и обучать их. Конечно, автоматическое тестирование требует затрат на начальном уровне, но они быстро окупятся при росте системы.

Другая сложность, связанная с объемом, — это количество трафика для того, чтобы ваши тесты выдавали надежные результаты. На ранних этапах авантюры с API симуляция 100 запросов в секунду (requests per second, RPS) может отражать трафик производства. Но при появлении новых команд API, которые начинают чаще отправлять друг другу запросы, общий объем трафика быстро раздуется. Когда трафик производства достигает 1000 RPS, по тестированию на 100 RPS уже нельзя предсказать, что будет после публикации этого компонента. Важно четко отслеживать уровень запросов во время производства и убеждаться, что тестовая среда все еще соответствует этим требованиям после расширения экосистемы.

Скорость

Как мы уже упоминали, скорость тестирования — способность выполнять тесты за разумное время — может стать проблемой по мере роста вашей экосистемы. Есть проверенное правило: тест одной единицы или одного блока должен выполняться за несколько секунд, тест поведения или соответствия целям бизнеса — менее чем за 30 секунд, тест на интеграцию — менее чем за 5 минут, а тест на масштаб/производительность — менее чем за 30 минут. Если платформа для тестирования не может поддерживать такой темп, а команды по разработке обновляют их в стиле непрерывного изменения, то множество запросов в области тестирования/ОК останутся без ответа.

Есть три основных способа решения проблемы с объемом:

- параллельное тестирование;
- виртуализация;
- канареечный тест.

Первый способ повысить скорость тестирования — создать *параллельные* тесты. Самый прямой метод — распределить автоматические тесты по нескольким устройствам и запустить их одновременно. Например, если после создания каждого компонента нужно запустить 35 тестов, их можно запустить подряд на одном устройстве или по одному параллельно на 35 устройствах. Предполагая, что каждый тест выполняется за 10 секунд или меньше, при последнем подходе вы перейдете от теста, занимающего около 5 минут, к тесту, занимающему менее 10 секунд. Это и есть скорость. Конечно, это предполагает, что все тесты *могут* выполняться параллельно, то есть при отсутствии зависимости от порядка тестов (тест 13 *должен* запускаться перед тестом 14 и т. д.), что, кстати, тоже неплохая практика. Параллельное тестирование помогает увеличить его скорость *перед* запуском в производство.

Параллельное тестирование может помочь со скоростью на уровне тестов отдельных единиц и поведения, а с тестированием на взаимодействие и производительность можно справиться с помощью элементов виртуализации. Проводить тестирование нового компонента на совместимость в сравнении с другими сервисами может быть и дорого, и рискованно. А если новый компонент не справится с данными о производстве? Или при взаимодействии с другими компонентами он неожиданно навредит им? В небольших экосистемах использование *фиктивных* сервисов в качестве замены компонентов производства хорошо масштабируется. Но при росте экосистемы имитациям может быть сложно успевать за скоростью изменений.

Удачное решение для повышения скорости тестирования при сохранении безопасности — использовать *виртуальные* сервисы. Это можно делать через общую платформу виртуализации, которая может задействовать трафик производства и имитировать его по запросу в безопасной тестовой среде. Это позволяет разработчикам сократить свои усилия по синхронизации фиктивных сервисов с производственными функциями и характеристиками, увеличивая число реальных производственных компонентов. В качестве дополнительного преимущества хорошие платформы виртуализации позволяют разработчикам создавать *искусственный трафик*, имитирующий не только правильное поведение производственных сервисов, но и искаженные или мошеннические взаимодействия в сети, чтобы можно было протестировать API и сервисы в неидеальных условиях. Это помогает справляться с аспектами уязвимости и изменчивости, о которых мы поговорим далее.

Еще один способ повысить скорость тестирования — выполнить некоторые тесты уже после запуска сервиса в производство. Иногда это называют *канареечным тестом*, или *канареечным развертыванием* (<https://oreil.ly/DZKNC>). Тогда после основных тестов на поведение вы запускаете новый сервис для ограниченного набора аккаунтов (которые захотели стать бета-тестерами). После такого частичного запуска вы отслеживаете результаты (см. раздел «Мониторинг» на с. 350) и, если все хорошо, выкатываете обновление для более широкой аудитории.

При проведении канареечного теста нужно учесть несколько важных моментов:

- вы по-прежнему запускаете базовые тесты для проверки компонента;
- у вас есть возможность выполнить частичный выпуск подраздела вашей экосистемы;
- у вас надлежащий мониторинг, позволяющий оценить влияние частичного выпуска;
- вы можете в течение нескольких секунд откатить изменения и вернуться к предыдущей версии, не вредя функциональности и хранению данных.

Уязвимость

По мере увеличения объема вашего API-ландшафта (например, увеличения числа конечных точек) и роста масштаба (например, увеличения числа команд, использующих эти конечные точки) ваша экосистема становится более уязвимой. Мы обсудим это подробнее в разделе «Безопасность» на с. 346, а сейчас важно сфокусироваться на том, как рост ландшафта ведет к увеличению уязвимости и что с этим делать.

Мы уже упоминали, что при масштабировании тестов нужно убедиться, что объем трафика в тестах соответствует объему производства. Это относится и к увеличению количества команд или других сервисов, которые будут использовать конкретный API. Ваш режим тестирования должен учитывать, что много *разных* пользователей API будут делать запросы одновременно. Например, иногда какое-то состояние, на которое влияют действия пользователей, передается от одного компонента к другому.

В случае запуска API для более широкой аудитории стоимость поддержания этого состояния резко вырастет. Вам может понадобиться и больше тестировать, чтобы убедиться, что это состояние остается изолированным между пользователями API и что большие объемы не переполняют оперативную память при росте числа пользователей. Уязвимость может обнаружиться при увеличении объемов использования. Она возрастает также и при изменении *типа* пользователей API.

В какой-то момент вашими пользователями становятся не только внутренние команды, но и ключевые внешние партнеры, и даже сторонние разработчики, над которыми у вас мало контроля. Тесты должны отражать эти изменения в экосистеме и разрабатываться так, чтобы защитить ее и ваши данные от ошибочных или злонамеренных действий сторонних пользователей.

В последние годы многие опираются на «устав», в котором Джефф Безос учил свои команды в том числе тому, что «все интерфейсы сервисов без исключения должны изначально разрабатываться для внешнего использования»¹. Хотя, по нашему опыту, можем сказать, что вам не всегда нужно делать это в первый же день планирования API, но вы все же *должны* быть готовы разбираться с этим при росте ландшафта.

Наконец, стоит упомянуть, что один из самых эффективных способов улучшить результаты тестирования — писать код и псевдокод API так, чтобы уменьшить вероятность невыполнения теста. Поэтому многие компании, с которыми мы сотрудничаем, способные правильно подстроиться под рост объемов и масштаба, набирают в команды разработчиков экспертов в области тестирования. Другими словами, они «сдвигаются влево» в момент тестирования. Добавляя человека с навыками тестирования в команды, пишущие код и разрабатывающие дизайн, можно избежать ошибок, из-за которых тесты долго запускаются и неточно отражают производственные условия. Подумайте над тем, чтобы снизить уязвимость своей системы, улучшая навыки тестирования команды по разработке.

¹ Kim J. The API Manifesto Success Story («История успеха манифеста API») // ProFocus (блог), обновлено 26 сентября 2019 г. <https://oreil.ly/AAmSO>.

Изменяемость

Как мы уже упоминали, растущий ландшафт API означает увеличение сложности, а не только рост экосистемы. То, что было мелкой ошибкой, когда в мире API вашей компании была лишь горсточка конечных точек, управляемая одной командой, может вывести из строя большую часть системы, если окажется, что все ваши сервисы зависят от одного из них, запущенного на одном устройстве где-то далеко.

Большой ландшафт API рискует стать более изменчивым. Это часто происходит из-за того, что в систему незаметно проникают фатальные взаимозависимости. Простой пример такой ситуации мы можем найти в кризисе из-за left-pad (<https://oreil.ly/5OnKJ>), произошедшем в 2016 году в сообществе Node.js.

Не вдаваясь в детали (если вам интересно, прочитайте статью по ссылке), можем сказать, что небольшая библиотека за короткий период времени была включена в тысячи проектов на Node.js. Спор с автором этой библиотеки привел к тому, что он удалил библиотеку из обращения, и почти сразу тысячи сборок потерпели крах, включая сборку для самого Node.js! Хотя проблему нашли и исправили меньше чем за час, это событие стало суровым напоминанием о том, как большие системы со временем могут стать более уязвимыми¹.

Такие уязвимости не ограничиваются случаями, когда компоненты «пропадают» или каким-то образом становятся недоступны. Возможно также, что API или компонент, от которого зависит ваша система, *изменится*, нарушив важные функции. Вы можете научить команды снижать вероятность этого при обновлении кода, но у вас не будет контроля над сторонними библиотеками или платформами, которые появятся в вашей экосистеме. При росте ландшафта вы станете чаще пользоваться внешними API, которые повысят его изменяемость.

Это означает, что важно добавить тесты, выявляющие фатальные зависимости и подчеркивающие стоимость критических сбоев ключевых компонентов. Лучше всего делать это на стадии тестирования на совместимость или производительность, когда проверяются ссылки на другие API и сервисы. Как мы уже упомянули ранее, лучший способ уменьшить уязвимость — добавить эксперта по тестированию в команды по дизайну и разработке. Так вы сможете разобраться с этими проблемами *до того*, как компонент или процесс будут запущены в производство.

¹ Williams C. How One Developer Just Broke Node, Babel and Thousands of Projects in 11 Lines of JavaScript («Как один разработчик просто сломал Node, Babel и тысячи проектов за 11 строк JavaScript») // The Register, 23 марта 2016 г. <https://oreil.ly/5OnKJ>.

В больших системах даже мелкие ошибки могут иметь далеко идущие последствия. Обязательно тестируйте на наличие моментов, когда важные компоненты пропадают или меняются и ими невозможно воспользоваться, и исправляйте эти проблемы.

Развертывание

Один из важнейших столпов любой API-программы — это развертывание. Вне зависимости от применяемых процессов дизайна или сборки программа не становится реальной, пока API или компонент не опубликован (см. раздел «Стадия 2: публикация» на с. 203), а развертывание — это и есть процесс публикации. На начальном этапе программы вы можете сосредоточиться на простом процессе запуска API в производство. Многие организации даже запускают их вручную (щелкают на инструментах запуска, используют отбор и исполнение скриптов вручную и т. д.) на первом этапе. Но когда ландшафт начинает расти, запуск вручную становится сложно масштабировать, и появляется новый уровень ненужной изменчивости в экосистеме.

Наиболее распространенная тактика масштабирования развертывания — максимально автоматизировать его. Это одна из ключевых идей DevOps и движения за непрерывное развертывание. Джез Хамбл, соавтор книги «Непрерывное развертывание» (Continuous Delivery, Addison-Wesley), говорил: «Наша цель — развертывать программы. Будь то крупномасштабная распределенная система, сложная производственная среда, встроенная система или приложение — это предсказуемое и рутинное дело, которое нужно выполнять по запросу»¹.

У автоматизации развертывания много преимуществ, особенно при масштабировании развертывания в ландшафте API. Мы сфокусируемся на четырех его аспектах: разнообразии, скорости, контроле версий и изменчивости.

Разнообразие

В значительной части этой книги мы подчеркивали ценность поддержки разнообразия по мере роста вашей экосистемы. Но во время сборки и разработки оно может стать реальной угрозой ее жизнеспособности и стабильности. Запуск процесса развертывания должен быть устойчивым, определенным и повторяемым. Если сегодня ваша команда выполняет процесс `installOnboardingAPIs`, он должен выдать *точно такой же результат* и при повторном запуске. Развертывание не должно варьироваться.

¹ Humble J. What Is Continuous Delivery? («Что такое непрерывное развертывание?»), <https://oreil.ly/KVxP8>.

Для этого нужно удалить разнообразие из системы. При сборке и развертывании полезно применять такие подходы, как «Шесть сигм», «Кайдзен», «Бережливая разработка» и т. д. То есть сфокусироваться на удалении из релиза неважных вариаций и убедиться, что технология развертывания собирает *все* артефакты релиза (код, конфигурацию и т. д.) в одном месте для облегчения публикации. Нужно внимательно отслеживать ОС и другие взаимозависимости, чтобы они оставались неизменными при каждом повторе одного развертывания. Хорошие платформы CI/CD позволяют разработать и реализовать надежный и повторяемый процесс развертывания.

«ШЕСТЬ СИГМ», «БЕРЕЖЛИВАЯ РАЗРАБОТКА», «КАЙДЗЕН»

Есть ряд моделей, направленных на постоянное совершенствование и устранение изменчивости при одновременном повышении качества. «Шесть сигм», «Бережливая разработка» и «Кайдзен» — самые известные из них. У каждой из этих моделей есть несколько вариаций (например, «Бережливые шесть сигм» и т. д.). Если у вашей компании еще нет программы, работающей по этим моделям, советуем рассмотреть их. На Formaspace есть интересная статья (<https://oreil.ly/DAMop>), где сравниваются три основные модели. Можно начать их изучение с нее.

Хотя важно удалить разнообразие из процесса развертывания, вам все равно может понадобиться поддерживать разные инструментарии CI/CD. Нет жестких требований к тому, чтобы во всех частях вашей организации от центрального сервера до КПК по всему миру использовалась *всего одна платформа* для развертывания. Но советуем максимально ограничить число вариантов платформ, так как большинство из них требует много времени и денег.

Скорость

Ускорение развертывания часто считается главной целью компаний, преобразовывающих свои IT-процессы. Есть два аспекта, которые важно учитывать по мере расширения вашего ландшафта. Первый мы называем *типом 1*: сокращение промежутков между релизами одного API/компонента. Второй — *типом 2*: наращивание общей скорости всех циклов релиза в вашей IT-группе.

Первый случай (тип 1) — тот, о котором думает большинство людей, когда речь идет о скорости развертывания. Он очень важен. Мы часто обсуждаем с пользователями снижение числа случаев появления петли «от обратной связи к функции» или, для новых проектов, о более быстром переходе от «идеи к установке». Ускоренное развертывание может снизить риски и затраты на эксперименты с новым продуктом или сервисом. Это улучшит способность вашей компании обучаться и внедрять инновации. А автоматизированное и определенное развертывание поможет повысить скорость релиза.

Второй случай (тип 2) сильно отличается от первого. Вам придется запускать в производство *больше* программ за тот же период времени. Для этого нужно, чтобы больше команд выпускали релизы, больше релизов запускалось в производство и больше изменений вносилось в ландшафт. Если вы полагаетесь на одну центральную команду по релизам для развертывания всей продукции, вам будет сложно набрать скорость второго типа. Есть пределы расширения одной команды по релизам.

Лучший подход — распределять ответственность за релизы среди более широкого сообщества разработчиков. Это еще одна причина, по которой автоматизация максимального объема процессов релиза крайне полезна. Чем значительнее автоматизация, тем больше людей смогут сосредоточиться на крайних случаях и ожиданиях, чтобы все работало как следует и единообразно. Так же, как и в случае других столпов из этой главы, можно безопасно ускорить процесс, распределив его или запустив параллельные процессы. Результат: *больше* число релизов в целом без ускорения каждого отдельного цикла.



Управление распределенными релизами в Etsy

Майк Бриттен, технический директор Etsy, создал очень хороший набор слайдов и видеопрезентаций по версии распределенных выпусков Etsy под названием «Управление распределенными релизами» (<https://oreil.ly/ixT3G>). Если вы хотите реализовать эту идею, советуем начать с презентации Бриттена.

Ускоряя развертывание, обязательно помните о ценности скорости первого и второго типов.

Контроль версий

Контроль версий в целом обсуждался в главе 9 (см. раздел «Контроль версий» на с. 281). Хотя мы заявили, что при разработке дизайна и реализации нужно избегать создания версий *внешнего интерфейса* API, с *внутренним интерфейсом* все иначе. Пользователей API не стоит беспокоить исправлением ошибок или мелкими изменениями, не нарушающими их работы. Этим нужно заняться, только если работа интерфейса нарушается и/или становятся доступными новые функции. Но внутренние пользователи (разработчики, дизайнеры, специалисты по архитектуре и пр.) должны иметь возможность видеть каждое мелкое отклонение и изменение в пакете релиза. Даже такие небольшие, как изменения в ресурсах поддержки, например в логотипах, и т. п.

Один из способов убедиться, что вы предоставляете небольшие изменения в пакетах развертывания, — *создать версию релиза* с помощью схемы семантического

контроля версий (см. раздел «Семантический контроль версий» главы 8): **MAJOR** (мажорная, то есть серьезные изменения), **MINOR** (минорная (новые функции, совместимые с предыдущими версиями)) и **PATCH** (патч (без изменений в интерфейсе, исправление ошибок)). Иногда пользователи добавляют уровень **RELEASE** (релиз, то есть **MAJOR.MINOR.PATCH.RELEASE**). С таким дополнительным значением проще отследить каждую сборку и/или цикл релиза до малейшего изменения. А это может быть важно при неожиданном поведении релиза продукта. Возможность отследить пакет с помощью номера релиза может пригодиться для определения его отличительных черт.

Большинство инструментов позволяют присваивать дополнительный номер сборки каждому релизу. Так вам не нужно править схему семантического контроля версий, и при этом вы все равно получаете подробное отслеживание каждого пакета сборки и производства. Что бы вы ни решили, помните, что *внутренние* релизы получают детальные идентификаторы, а вот идентификаторы *внешних* релизов лучше исправлять только в случае серьезных изменений.

Изменяемость

Как вы могли подумать, увеличение скорости развертывания по обоим типам (см. пункт «Скорость» на с. 343) вызывает риск роста общей изменяемости системы. Поэтому многие организации пытаются замедлить темп релиза. Но лучше так не делать. Вместо этого вы можете сделать три вещи, чтобы убедиться, что ваш компонент развертывания не вносит неожиданную нестабильность в экосистему:

- убедиться, что изменения в релизах не нарушают работу программы;
- поддержать определенные пакеты развертывания без вариаций;
- поддержать мгновенную обратимость установки.

Развертывание должно по возможности *избегать контроля версий*, в том смысле, в каком это видит большинство из нас (см. раздел «Контроль версий» на с. 281). По нашему опыту, можно вносить важные изменения в работающую систему без необходимости ее нарушать. Джейсон Рэндольф из GitHub называет такой подход эволюционным дизайном и объясняет его ценность так:

Когда люди создают что-то на основе нашего API, мы просим их доверять нам, ведь они вкладывают время в создание своих приложений. И чтобы заслужить их доверие, мы не можем вносить изменения [в API], способные нарушить работу их кода.

Рэндольф объясняет, что можно использовать элементы *дизайна*, чтобы упростить внесение не критичных изменений. Можно создать тесты (см. раздел «Тестирование» на с. 126), проверяющие наличие таких изменений, и включить их в процесс сборки для снижения вероятности нарушения производства. Своим обещанием не нарушать работы программы вы ограничите возможность появления изменяемости при масштабировании развертывания.

Еще один ключ к снижению изменяемости при развертывании — убедиться, что каждый пакет релиза автономен и, как мы уже говорили ранее, определен (см. подраздел «Разнообразие» на с. 342). Когда вы способны предсказать результаты развертывания (например, когда вы уверены, на какие элементы производства повлияет этот релиз), вы можете снизить вероятность неожиданностей в обновлении.

Кроме того, ответственный за развертывание пакета в рабочей среде должен иметь возможность запустить релиз в этой или другой среде (на устройстве разработчика, тестовом сервере и т. д.) и получить те же самые результаты. Это важно для тестирования релиза и поиска ошибок совместимости и других внешних неполадок, способных появиться в производстве.

Наконец, важный аспект изменяемости развертывания связан с *обратимостью*. Как мы уже говорили, скорость 2-го типа, где *больше* общее число релизов, может угрожать стабильности, увеличивая изменяемость. То, что вы убедитесь, что изменения не нарушают работу системы, а развертывание не имеет вариаций, поможет уменьшить число неполадок, но не предотвратит их на 100 %.

Если в релизе появляется неожиданная ошибка, важно иметь возможность *моментально отменить изменение*, откатить его в течение *нескольких секунд*. Это решение на самый крайний случай. Откат изменения означает, что собранные или сохраненные данные не повреждены. Другими словами, все ваши развертывания должны рассчитывать на обратимость любых изменений в схеме или модели данных. Для этого нужно изменить способ обновлений разработки дизайна и реализации.

Безопасность

В разделе «Безопасность» на с. 132 мы говорили о важности основных элементов безопасности (идентификация, распознавание и авторизация). Еще мы обсудили, как снизить общую поверхность, которая может подвергнуться атаке, и добавить в каждый компонент и/или API, запущенный в производство, устойчивость и изоляцию (см. раздел «Уязвимость» на с. 279).

Все эти элементы становятся все важнее по мере роста вашего ландшафта. И с каждым из них возникают проблемы. Безопасность — обширная и сложная

тема, которую мы не сможем здесь подробно рассмотреть. Но здесь мы выделим три самых важных аспекта ландшафта и обсудим их влияние на общую безопасность системы.

Скорость

Большая проблема для поддержания надлежащей безопасности в расширяющейся экосистеме API — скорость изменения самого ландшафта. Добавляются новые компоненты, обычно ускоренными темпами, новые взаимосвязи и новые пользователи. Многие из наших клиентов используют инфраструктуру безопасности, требующую явных определений управления доступом в ландшафте, прежде чем компонент или интерфейс можно будет запустить в производство. Это работает, когда темп изменений и охват экосистемы невысоки. Но по мере роста экосистемы определение контроля доступа на раннем этапе способно стать уязвимым местом. Оно может сдерживать релизы, замедлять выпуск новых функций и исправление ошибок.

Распространенный способ справиться с проблемой скорости для безопасности — убедиться, что компоненты разработаны для надежной работы, даже если профилей контроля доступа еще нет.

Еще один способ — ввести автоматическое тестирование безопасности. Скриптовые тесты безопасности — не идеальное решение (сложно проверить наличие злонамеренных попыток в скриптах), но они могут помочь. Тестирование безопасности во время цикла сборки поможет найти проблемы на ранней стадии, снизить затраты на их исправление и вероятность ущерба при запуске.

Уязвимость

При росте ландшафта API увеличивается поверхность, а следовательно, и уязвимость. Если много команд быстро выпускают много компонентов, становится трудно следить за возможными уязвимыми местами вашей системы. Как мы только что сказали, может помочь добавление тестов на безопасность в фазе сборки. Но это не единственный способ разобраться с растущей уязвимостью ландшафта API.

Когда каждый выпуск компонента рассматривается как «одноразовое» событие безопасности, может быть сложно отслеживать и проверять пространство уязвимостей. Еще один способ справиться с увеличением масштаба и объема таков:

- полагаться на общие политики в качестве основы для профилей безопасности на уровне компонентов;
- переложить ответственность за отслеживание и отчет о действиях в области безопасности на уровень, максимально близкий к командному.

У использования реализаций безопасности на основе политик (а не реализаций, специфичных для кода) есть несколько преимуществ. Во-первых, описательные алгоритмы проще читать и отлаживать, чем предписывающий код. Во-вторых, большинство прокси-серверов позволяют относиться к любым алгоритмам как к единицам, которые можно использовать повторно и сочетать в мощном пакете профилей, который проще контролировать и отслеживать.

Наконец, многие платформы безопасности позволяют управлять алгоритмами с помощью скриптов и/или инструментов командной строки, совместимых с системами CI/CD, чтобы сделать свой алгоритм безопасности элементом пакета релиза, научиться работать с которым должны все команды.

Это приводит ко второй части подхода: перекладывание ответственности за отслеживание и отчетность на команды разработчиков. Вряд ли одна команда по безопасности (особенно находясь на корпоративном уровне в глобальном предприятии) сможет достаточно успешно контролировать события на уровне компонентов и реагировать на них. Гораздо более разумно, если команда, работавшая и выпустившая этот компонент или API, будет следить за ним. Но делать это качественно они могут только при наличии соответствующих инструментов и поддержки.

По мере роста вашей экосистемы важно преобразовать централизованный опыт в распределенные инструменты и методы. Для этого можно перевести некоторых экспертов по безопасности в команды по разработке. Другой способ расширить навыки безопасности — создать такие инструменты, как методы на уровне дизайна, тестирование на уровне сборки и компактное отображение информации на уровне производства. Инвестируя в инструменты, помогающие масштабировать имеющиеся у вас знания о безопасности, вы сможете успешно расширить сферу своей деятельности и повысить общую безопасность своего API-ландшафта.

Видимость

Это приводит к последнему аспекту, который нужно здесь выделить, — видимости. В мире безопасности именно то, чего вы не знаете, может навредить вам. Все предусмотреть нельзя. Вместо этого, наряду с внедрением таких методов, как «нулевое доверие», правила безопасности на основе политик и тесты безопасности во время сборки, вы можете повысить прозрачность за счет использования журналов и информационных панелей.

Информационная панель предлагает обзор активности в сети в реальном времени. Благодаря ей возможность увидеть свои интерфейсы и компоненты

в действии появляется у команд из всех отделов компании, в том числе и из отдела безопасности. Вначале информационные панели полезны, потому что делают видимым общий трафик вашей сети. Со временем команды могут создавать фильтры, чтобы сосредоточиться на трафике, который, как они узнали, является важным индикатором работоспособности системы (или ее отсутствия).

Часто точки ввода данных, которые появляются на информационных панелях, — это своего рода KPI и OKR (см. раздел «OKR и KPI» на с. 193), которые мы обсуждали ранее. Команды по безопасности могут сами определить, за какими значениями стоит следить, и упростить передачу этой информации для всех команд разработчиков. Чаще всего ее получают от шлюзов и прокси-серверов, используемых по всей экосистеме, но иногда нужно разработать индивидуальные компоненты, чтобы собирать и показывать важные параметры запросов на распознавание, одобрение, разрешение доступа, отказ в доступе и т. д.

Предоставляя командам разработчиков точки ввода данных и шаблоны, которые эксперты по безопасности ожидают увидеть, и обеспечивая согласованность на уровне компонентов во время тестирования перед запуском, вы добьетесь того, что ваши действия в области безопасности будут соответствовать новому масштабу и объему растущего ландшафта API.

Обычно системные журналы можно использовать как часть анализа произошедшего, которая поможет вам и вашей команде по безопасности понять, что произошло, и в идеале подскажет, как избежать этих проблем в будущем. Внедрение практики ведения журналов для всех ваших команд разработчиков — ключ к обеспечению сбора информации для возможного использования в последующем обсуждении безопасности. Но просто *собрать* информацию в виде журналов недостаточно. Нужно убедиться, что она хранится в форме, облегчающей доступ к данным, фильтрацию и корреляцию, чтобы вы могли найти и изучить только нужные записи.

Для этого можно применить метод *децентрализованного сбора* (каждая команда отвечает за сбор информации по результатам отслеживания) и *централизованного хранения* (есть единая платформа, куда посылают все системные журналы, чтобы позже отфильтровать и изучить их).

Центральное руководство по безопасности тоже может предоставить рекомендуемые требования точки ввода данных и действия по отслеживанию. А еще можно написать тесты на уровне сборки, чтобы убедиться в согласованности работы всех команд с этим руководством, прежде чем запустить ее в производство.

Мониторинг

У мониторинга несколько полезных целей, включая поиск уязвимостей, отслеживание внутренних KPI, внешних OKR и предупреждение об аномалиях в работе программы (необычных пиках трафика, неожиданных разрешениях на доступ и др.). На раннем этапе существования вашей API-программы контролем можно заниматься с помощью небольшого количества специальных инструментов, нескольких простых информационных панелей и методов изучения системных журналов. Но, как и в случае многих других столпов API, с ростом ландшафта проблемы объема, видимости и изменяемости могут превысить возможности ваших инструментов. Мы назовем несколько распространенных проблем и поделимся способами их решения.

Объем

Распространенная сложность мониторинга при расширении ландшафта API в том, что общий объем информации перерастает вашу способность с ней справляться. Так происходит, когда у организации есть централизованная модель управления мониторингом, где небольшая группа специалистов выполняет задачи по сбору данных системных журналов, как поступающих в реальном времени, так и старых, управлению ими и их интерпретации.

В какой-то момент одна команда уже не может справиться с объемом, и ее увеличение не улучшает положения. Вместо этого, как мы предложили в разделе «Уязвимость» на с. 347, можно делегировать задачи по мониторингу командам, разрабатывающим API и компоненты. Первый шаг здесь — распределение работы по сбору данных отслеживания и управлению ими. После появления *собственных* данных отслеживания команды могут начать разрабатывать фильтры и корреляции для получения полезной информации об этих данных.

Но контроль одного компонента или API — это лишь часть проблемы. При росте ландшафта важно контролировать и взаимодействие между этими компонентами. Хорошей тактикой будет создание центрального репозитория отслеживания, где можно сфокусироваться на корреляциях между разными API в вашей компании. В этом случае центральный репозиторий часто служит отфильтрованным и коррелированным подразделом (обычно сжатым по времени) всех данных отслеживания команд разных компонентов или API. Централизованные данные — это избранное, выдержка из всего объема, которую можно использовать, чтобы увидеть повторяющиеся схемы и аномалии. После их определения команды могут использовать детали отслеживания, чтобы искоренить проблемы и подтвердить информацию.

Для этого метода нужно:

- дать командам задание по сбору данных отслеживания и управлению ими;
- установить центральный репозиторий, отбирающий отфильтрованные данные из хранилищ отдельных команд;
- при появлении тенденций или проблем на центральном уровне опираться на детали на уровне команд, чтобы подтвердить или решить любые проблемы.

Видимость

Практика создания центрального репозитория отфильтрованных/коррелированных данных — ключевой элемент в поддержании и улучшении видимости мониторинга. В разделе «Безопасность» на с. 346 мы говорили, что нельзя предусмотреть все возможные проблемы в сфере безопасности. Отсюда можно сделать важный вывод: невозможно знать все точки ввода данных, которые надо отследить.

Поэтому ранние попытки регистрации и мониторинга часто включают множество значений, которые не имеют никакого отношения к текущему состоянию вашей сети. Но связь есть, просто такая, о которой человек в принципе не может *узнать*. Поэтому команды могут урезать сбор данных, выбрасывая якобы несвязанные значения из записей. Это плохая идея.

Вместо этого разумнее записывать все, а отслеживать только отдельный набор точек ввода данных. Это соответствует предыдущему совету — поощрять команды собирать и хранить свои данные, позволяя объектам централизованного мониторинга отбирать отфильтрованные и коррелированные версии этих данных в общий набор открытых информационных панелей. Пока команды следят за более подробным (но ограниченным) представлением системы, операции централизованного мониторинга могут иметь более широкое (но менее подробное) представление о ней. Это простой пример принципа неопределенности Гейзенберга.

ПРИНЦИП НЕОПРЕДЕЛЕННОСТИ ГЕЙЗЕНБЕРГА

В 1927 году Вернер Гейзенберг опубликовал работу по физике, где было следующее наблюдение: чем точнее определено положение частицы, тем менее точно мы знаем ее импульс, и наоборот.

В сфере IT это воспринимается как компромисс между детальным пониманием небольшой части системы и знанием того, как она влияет на всю систему. Попытки получить и детальное понимание каждой части, и полное представление об их взаимодействии при росте экосистемы становятся все менее и менее результативными. Вот почему мы советуем командам поддерживать детальное представление системы, а операторам службы централизованного мониторинга — широкое и менее подробное.

Еще один важный аспект видимости в системах мониторинга — это связанное с ним качество *наблюдаемости*. При росте ландшафтов становится все сложнее наблюдать за их поведением. И для улучшения наблюдаемости можно использовать мониторинг, точнее, публикацию данных отслеживания. Неожиданные результаты, ошибки и другие странности часто появляются, потому что человек не видит, как работает система и/или как ее компоненты связаны друг с другом. По нашему опыту, в случае обнаружения странной ошибки в сложной системе люди склонны говорить что-то вроде «Я и не знал, что такое возможно» или «Я не думал, что это так работает». Улучшение мониторинга и компактное отображение информации не предотвращают неполадки, но могут помочь не удивляться, когда что-то пойдет не по плану.

Изменяемость

Наконец, снова повторим то, что заявили в разделе «Безопасность» на с. 346: в больших системах более вероятен высокий уровень изменяемости. Это наблюдение сделал специалист по теории вычислительных машин и систем Мэл Конвей в своей работе 1967 года «Как комитеты создают новое?»: «Структуры больших систем стремятся к распаду... значительно чаще, чем структуры небольших»¹.

Хотя это наблюдение относилось прежде всего к фазе разработки больших проектов, мы видим то же поведение в работе больших систем. Одна маленькая ошибка способна нарушить работу всей системы, а с ростом последней такие нарушения обходятся все дороже. Из-за этого некоторые компании могут предположить, что качество их систем со временем падает, но на самом деле ошибки, скорее всего, всегда там были. Просто с увеличением системы возросло и их влияние, а с ним и опасность. Вы можете снизить риск нарушения работы всей системы из-за одной ошибки и лучше увидеть ее качество, если будете поддерживать надежную программу мониторинга.

Обнаружение

Столп *обнаружения и продвижения* жизненного цикла API состоит из всех действий, которые помогают пользователям найти и использовать API. Для отдельных API нужно понимать, в каком контексте их можно будет обнаружить, и упрощать этот процесс (см. раздел «Обнаружение» на с. 138).

¹ Conway M. E. How Do Committees Invent? («Как комитеты изобретают?») // Datamation, апрель 1968 г. <https://oreil.ly/PXGIIt>.

В сложных и постоянно растущих ландшафтах API обнаружение развивается с той же общей динамикой, что и сайты, и веб-страницы. Вначале достаточно было иметь тщательно составленный список сайтов и страниц, распределенных по категориям для упрощения их поиска. Этот подход использовала Yahoo! как изначальный механизм обнаружения в Сети. Он хорошо работал, пока количество сайтов и страниц было небольшим, как и уровень изменений, и было достаточно одной схемы деления на категории для всей информации в интернете.

Конечно, этот подход плохо масштабировался и был быстро забыт из-за невероятного роста интернета, частоты его изменений и разнообразия содержимого. Начиная с 1996 года Google, которая изначально называлась *BackRub* и была сервисом Стэнфордского университета, радикально изменила обнаружение, введя два главных изменения.

- Поиск по категории был заменен *поиском по содержимому*. То есть вместо того, чтобы опираться на схему деления на категории от сторонних разработчиков, поисковик задействовал полнотекстовый поиск по самому содержимому.
- Ранжирование вручную было заменено *ранжированием по популярности*. В итоге контент стали сортировать в зависимости от его релевантности по популярности в Сети, полученной благодаря внутренним ссылкам.

Конечно, история развития поиска в интернете гораздо длиннее, но важно помнить об общей динамике. Хотя у этого подхода и есть издержки (популярные сайты становятся еще популярнее, а менее популярные найти все труднее), он работал достаточно хорошо, чтобы пользователи сочли его полезным, поэтому большинство задач по поиску в интернете сегодня выполняется по этой модели.

В сфере API *контент* имеет другое значение, ведь API сами по себе — это не столько контент, сколько *описание сервисов*. Но *популярность* — это концепция, которую довольно просто перенести в мир API, и график зависимости API вполне может стать эквивалентом структуры ссылок в интернете.

Пока неясно, кто может стать «Google в мире API». И поскольку многие ландшафты в основном содержат внутренние или партнерские, а не внешние API, не совсем понятно, возникнет ли для API та же динамика поиска, что и для интернета. Но чтобы поиск и продвижение могли масштабироваться, важно, чтобы релевантная информация была доступна в таком виде, чтобы ее можно было использовать для автоматизации. Главные аспекты ландшафта в этой области — разнообразие, объем, словарь, видимость и контроль версий.

Разнообразие

Документацию API можно создавать в разных формах, продиктованных решениями по инвестициям в определенной области жизненного цикла. Инвестиции зависят от развития API и предполагаемой ЦА. Они зависят от поддержки и инструментов, которые могут стать доступными на уровне ландшафта.

С ростом популярности и важности API становятся доступными сложные решения для их документирования и обнаружения. Часто это интегрированные пакеты, сочетающие аспекты документации, обнаружения, генерации кода и других факторов DX. Хотя такие наборы очень полезны, важно помнить, что в них обязательно встроены какие-то предпочтения (например, по стилям API). И, как и любые инструменты, их следует использовать так, чтобы можно было при необходимости увеличить или удалить.

Идеальное решение, позволяющее убедиться в том, что инструментарий помогает обнаружению, оставаясь при этом открытым и декларативным, — выразить всю нужную для него информацию *с помощью самого API* (этот принцип рассматривается в разделе «API через API» на с. 268) и позволить поддержке и инструментам собрать эту информацию, используя ее для обнаружения с помощью ландшафта. Это соответствует общему принципу раскрытия всей релевантной информации через сам API. Мы обсудим это подробнее в разделе «Словари» далее.

Объем

С ростом объема ландшафта API важно обеспечить легкость обнаружения как аспект, который поможет не только *найти* API, но и *ранжировать* их. Находить их важно, но этого недостаточно, и это понимает каждый, кто когда-либо использовал интернет-поисковик, который выдает миллионы ссылок на один запрос.

Поэтому для обнаружения нужны не только способы найти API, но и способы их лучше понять и ранжировать. Вероятно, что понимание «полезного ранжирования» со временем будет меняться в любом ландшафте. Изначально оно может быть даже ненужным из-за малого количества API. Затем из-за растущего их объема такая необходимость возникает. Полезные способы ранжирования тоже могут со временем изменяться, ведь объем растет, и определение того, что лучше всего соответствует поисковому запросу, развивается.

Чтобы иметь возможность расти по оси постоянно меняющегося и улучшающегося обнаружения, важно помнить, что API в любой момент должны

выдавать максимум информации о себе. Эта информация может постоянно эволюционировать, поскольку ландшафту нужно, чтобы API со временем менялись.

С точки зрения ландшафта важно обеспечить поддержку и инструменты, облегчающие обнаружение в нем. В зависимости от модели обнаружения этот набор задач может быть очень похож на оптимизацию сайта в поисковых системах SEO в интернете, где есть баланс между информацией, которую могут собрать и использовать сервисы по обнаружению, и уровнем сотрудничества, которого хотят отдельные партнеры.

Словари

Обнаружение означает упрощение поиска API. И хотя общий принцип, обсуждаемый здесь, соответствует схеме перехода обнаружения в больших структурах с модели категоризации Yahoo! к полнотекстовому поиску Google и модели ранжирования на основе популярности, полезно будет рассмотреть и другие веб-разработки.

В 2011 году в рамках сотрудничества основных поисковых систем Bing, Google и Yahoo! был основан сайт Schema.org (<https://oreil.ly/NiCHO>). Это была попытка создать единую схему по ряду тем, включавшую людей, места, события, продукты, предложения и т. п. Главная цель этого проекта — позволить людям, публикующим информацию в интернете, пометить ее так, как они считают нужным, и позволить поисковикам использовать эти метки как дополнительный вклад в алгоритмы поиска и ранжирования.

Стоит отметить, что это конкретно относится к веб-контенту, а не к API. Но главное — *принцип*: Schema.org продолжает развиваться как словарь терминов в интернете, которые можно использовать, чтобы пометить свою информацию. Это стало возможным, поскольку сам словарь и его применение разъединены. С развитием словаря информацию можно помечать более сложными способами, а создание и использование его терминов связаны слабо.

Для ландшафта API можно установить похожие принципы. Он может поддерживать и продвигать использование каких-либо терминов, чтобы облегчить обнаружение и обеспечить поддержку для создания маркированного содержимого (например, в документации и/или базовых документах API) и его одобрения. Конвейеры развертывания могут даже автоматизировать тесты на наличие некоторых терминов (например, тестировать «стиль API») и в случае отрицательного результата выдавать предупреждение и просить команду API включить эту информацию, сделав ее доступной для обнаружения.

Видимость

Как говорилось в разделе «Объем» на с. 354, один из главных аспектов, о которых надо помнить, думая об обнаружении в ландшафтах API, — это объем. И как мы увидели, главный способ управлять им — сделать видимым через API достаточное количество информации о нем. Конечно, ее набор со временем будет меняться. Поэтому главное, что надо учитывать с точки зрения видимости, — следует начинать с малого и иметь план по развитию количества информации, раскрытой в отдельных API.

Помня, что набор видимой информации обычно со временем меняется, можно непрерывно совершенствовать обнаружение в ландшафте API так же, как оно постоянно улучшается в интернете. Легкость обнаружения никогда не бывает окончательной, а достичь ее можно с помощью развития информации, доступной инструментам для обнаружения, наблюдения за развитием ландшафта и размышлений о том, какие изменения помогут улучшить обнаружение.

Контроль версий

API в ландшафте обычно меняются, и одна из основных целей — упрощение этих изменений. Возможность вносить изменения, не нарушая работы пользователей, с каждым новым релизом — тоже цель многих API и их ландшафтов. Применяя качественные методы управления изменениями, можно четче разьединить создателей и пользователей API, позволив им развиваться независимо друг от друга.

В идеале команды API могут развивать свои продукты и выпускать новые версии в собственном темпе. Так проще для команд, но пользователям API сложнее уследить за выходом версий, найти старые и понять разницу между ними.

Один из способов упрощения просмотра и понимания API и их версий в ландшафтах — потребовать от API документирования всех версий. Так их историю можно будет легко обнаружить. Это позволяет пользователям находить информацию о версии API и понимать ее эволюцию на протяжении всей истории продукта.

Этот аспект может усложниться при серьезных изменениях API, когда реализация, возможно, стала совершенно иной и труднее обеспечить обнаружение документации и версий. Тогда может быть полезно обеспечить поддержку команд API, чтобы им не приходилось тратить дополнительные усилия на сохранение устаревших версий. Вместо этого они смогут положиться на ландшафт, оставив доступной лишь часть информации или сообщив пользователям, что старая версия все еще есть, но детальные сведения получить уже нельзя.

Управление изменениями

Один из важных столпов жизненного цикла API — *управление изменениями* (см. раздел «Управление изменениями» на с. 141). Часть общего пути API должна заключаться в том, чтобы минимизировать нарушения экосистемы API по мере их развития. Часто для этого нужно придерживаться принципов управления изменениями, которые могут вращаться вокруг моделей расширения и внесения в соответствии с ними только безопасных изменений. Другой подход — никогда не менять запущенные в производство API, а вместо этого создать операционную модель, позволяющую запускать много разных версий параллельно.

Планирование изменений — одна из центральных проблем ландшафтов API и их непрерывного развития. Управление изменениями сильно зависит от того, как ландшафт API поддерживает их с помощью руководства и инструментария. Чтобы помочь отдельным API с управлением изменениями, нужно учитывать аспекты словаря, скорости, видимости и контроля версий. Мы обсудим это далее.

Словари

При управлении изменениями API важно помнить о влиянии *эволюции словаря* на *эволюцию API*. Словарь — это распространенный и известный аспект дизайна API, а его разработка всегда подразумевает, что его обновления приведут к обновлениям API. Хотя при правильной модели расширения эти обновления могут не нарушать работы системы, они все же запускают целую цепь обновлений API и связанных с ним ресурсов, провоцируя такие же действия у пользователей API.

Управление словарем можно отделить от API, превратив его в ресурс, способный развиваться, позволяя API оставаться стабильным даже при изменении словаря. Как сказано в разделе «Словари» на с. 297, распространенный способ сделать это — обращение к *реестру*, который либо управляется внешним органом, либо управляется и размещается как часть ландшафта. В этой модели изменения в словаре не обязательно запускают изменения в API, и так проще *делиться словарем* с другими API.

Поэтому рассмотрение словарей на ландшафтном уровне важно и может поддержать и упростить работу по проектированию и развитию отдельных групп разработчиков API. В каком-то смысле идея поддержки словаря на уровне ландшафта связана с концепцией *словаря базы данных*. Но этот термин чаще применяют к конкретным аспектам схем БД, а словари независимы от реализации и являются просто самоуправляемыми типами данных, которые могут повторно использоваться разными API.

Скорость

Внесение изменений в первую очередь должно быть обусловлено планированием и итерацией продукта на основе отзывов и развертывания функций. Управление изменениями нужно для облегчения этой задачи и не должно мешать внесению изменений. Скорость — это одна из главных целей перехода на API и их ландшафт. Важно понять, как ландшафт помогает или мешает увеличить скорость (см. раздел «Скорость» на с. 302).

Следует поощрять команды разработчиков API к предоставлению отзывов о том, ощущают ли они, что их ускоряли или замедляли. Эти отзывы нужно учитывать при создании рекомендаций по улучшению управления изменениями и поддержки и инструментов на уровне ландшафта. Управление изменениями — это один из столпов, к которым надо всегда относиться серьезно, в том числе из-за его прямой связи со скоростью.

Опять же важно помнить о контексте управления изменениями и по возможности сдвигать применение более сложных способов на поздние стадии развития API. Ответ на вопрос: «Важны ли кому-то изменения API в ландшафте?» зависит от того, кто зависит от самого API.

Если (пока) API никто не использует, можно утверждать, что любое управление изменениями в этот момент — пустая трата времени и поэтому влияет на скорость. Но с приходом пользователей возникает парадокс: при большем объеме использования хорошее управление изменениями помогает удерживать скорость, но менять API для поддержки самого управления изменениями становится сложнее. Так, чтобы поддерживать скорость, особенно на поздних этапах развития, полезно вкладываться в управление изменениями и поддерживать его на ландшафтном уровне с самого начала.

Контроль версий

В целом API должны следовать модели семантического контроля версий из раздела «Семантический контроль версий» на с. 282. Необязательно применять одну и ту же схему, но полезно разделять уровни изменений:

- в случае *патчей* на уровне API нет заметных изменений, поэтому они интересны только потребителям, чтобы проверить, была ли изменена реализация для устранения ошибки;
- в случае *минорных* версий, которые по определению обратно совместимы, пользователям может быть интересно узнать об изменениях, но они могут решить и игнорировать их, если состояние сервиса их устраивает;

- в случае *мажорных* версий пользователи должны что-то предпринять, поскольку те вносят изменения, нарушающие работу программы. Нужно уведомлять пользователей о выпуске этих версий и о том, когда текущая будет удалена.

Есть много разных способов работы с этими механизмами с помощью отдельных API. Поскольку управление изменениями и взаимозависимостями играет важную роль в устойчивости ландшафта API, советуем согласовать эти два вопроса, чтобы пользователям было проще справляться со скоростью изменений в этом ландшафте.

В конце концов, возможность быстро менять и обновлять API очень полезна, если контролировать негативные побочные эффекты.

Видимость

Управлять видимостью сложно, когда речь идет об управлении изменениями в ландшафтах API. В целом пользователи хотели бы работать с API так, чтобы их как можно реже беспокоили, за исключением сообщений о новых функциях, которые им нужны, — в этом случае они готовы инвестировать в адаптацию своего API под использование изменений. Обеспечение видимости управления изменениями и предоставление потребителям возможности писать код, способный реагировать на эту информацию, помогают повысить устойчивость API-ландшафта.

Как мы сказали ранее, важно встроить управление изменениями в ландшафт API, чтобы их скорость не нарушала работу сервисов чаще, чем нужно. Один из главных вопросов, которые мы обсудили, — *что именно* делать видимым. Лучше использовать модель, отражающую схему семантического контроля версий: *патчи*, *минорные* и *мажорные* версии.

С точки зрения безопасности лучше не демонстрировать версии-патчи внешним пользователям и даже партнерам. Эти изменения должны не менять сам API или его поведение, а только решать проблемы реализации.

Минорные версии нужно делать видимыми, ведь это помогает пользователям понять, что они доступны. У потребителя *всегда* должен быть способ исследовать историю версий, и различия между минорными всегда нужно документировать. А мажорные версии всегда должны быть видимы, поскольку они вводят серьезные изменения.

Унифицированное отображение версий помогает потребителям API адаптироваться к этой модели управления изменениями в ландшафте и даже может

позволить им применять и повторно использовать инструменты, чтобы реагировать на это. Поэтому одинаковый подход к версиям всех API сильно увеличивает ценность (в форме улучшения стабильности) ландшафта API. Предоставление руководства, говорящего, что делать, поддержки того, как это делать, и инструментов для контроля того, что API действительно следуют этому руководству, поможет упростить управление изменениями для создателей API. Пользователям же будет полезно наличие устойчивого управления изменениями во всем ландшафте.

Итоги главы

В этой главе мы решили объединить столпы жизненного цикла из главы 4 с аспектами ландшафта из главы 9. Самой важной целью этого упражнения было подчеркнуть переход от работы с одним API к работе со многими. Так, с одной стороны, отдельные API могут процветать внутри ландшафта, а с другой — ландшафт, ограничивающий и поддерживающий API, может постоянно развиваться. Основные факторы этой эволюции — *обратная связь через наблюдение*, позволяющая ландшафту понять, как развиваются методы API, и *поддержка с помощью руководств и инструментов*, позволяющая ему превратить наблюдения в способы упростить внесение изменений.

Матрица ландшафта/жизненного цикла (см. раздел «Аспекты ландшафта и столпы жизненного цикла API» на с. 322) — это способ показать взаимосвязь между аспектами ландшафта и основными элементами жизненного цикла. Разобравшись в получившейся таблице, мы сосредоточились на тех комбинациях аспектов, которые заслуживают особого внимания.

Основная модель объединения аспектов ландшафта и столпов жизненного цикла заключается в рассмотрении *наблюдаемости* отдельных API, чтобы с точки зрения ландшафта было можно наблюдать за их поведением. Это соответствует общему смыслу «API через API». Второй шаг — использовать эти наблюдения для определения областей, где разработку API можно направить и поддержать с лучшим результатом, если инвестировать в эти столпы. Мы всегда применяем модель «зачем? что? как?» (см. раздел «Центр поддержки» на с. 290), чтобы убедиться, что *руководство API* и *руководство по реализации* четко разделены. Это позволяет методам создания API и реализации развиваться независимо друг от друга, что обеспечивает большую согласованность в ландшафте, ведь реализации могут меняться, не затрагивая при этом методы на уровне API.

Наконец, эта глава позволила совместить несколько элементов, затронутые в других главах. Мы коснулись понятия зрелости как индикатора изменения

формы вашего ландшафта и процесса распределения принятия решений как инструмента для разделения ответственности и направления действий в растущей организации. У каждой компании есть свои производственная культура, общепринятые практики и уровни ожиданий. В идеале этот материал даст вам несколько идей по развитию своей уникальной матрицы ландшафта или жизненного цикла и поможет научиться использовать внутренний процесс принятия решений в компании, чтобы непрерывно улучшать свою экосистему, пока она развивается.

Продолжение путешествия

Мы требуем строго определенных зон сомнения
и неопределенности!

Дуглас Адамс

Управление API — сложная тема, и нам пришлось охватить в этой книге много вопросов, чтобы изучить ее. После краткого обсуждения проблем и перспектив программ API (глава 1) мы рассмотрели основополагающую концепцию *руководства* API (глава 2), выполнение работы, основанной на принятии решений. Сосредоточение внимания на решениях привело нас к модели их принятия с элементами, которые мы могли распространять или отображать. Планирование решений обеспечило нам полезный и детальный способ развития API.

Сосредоточившись на принятии решений в качестве основы, мы начали наше путешествие по API, представив первый важный фактор управления API — *перспективу продукта* (глава 3). Подход к API как к продукту, который решает проблемы ЦА, помогает принять наиболее значимые решения. Мы начали с этого продуктового подхода, сосредоточившись на работе по созданию *единого* продукта API. Наш опыт свидетельствует, что контекст *локальной оптимизации* важен для определенного варианта использования (например, «Работа, которую нужно выполнить» Клейтона Кристенсена). Начать с одного варианта использования проще, чем разбираться со сложным ландшафтом, который неизбежно возникает по мере добавления в систему новых API.

В этом первом наборе глав мы изучили локальный контекст API, ознакомившись с их основными столпами (глава 4), изучили стили API (глава 6) и проработали все этапы жизненного цикла вашего продукта API (глава 7). С помощью этих инструментов вы сможете заложить прочный фундамент, ведущий к созданию последовательных, согласованных API, построенных так, чтобы можно было

поддерживать отслеживание и управление потоком работ от этапа создания до вывода из эксплуатации.

Следующий важный фактор управления API, который мы рассмотрели, — общая организация и культура вашей компании. Это слишком большая тема (и важная), чтобы ограничиться парой глав, но мы хотим обязательно выделить два основных организационных элемента, с которыми мы все сталкиваемся, когда дело доходит до внедрения и поддержки здоровой программы API. Первый из них — возвращение в компании духа непрерывного совершенствования (глава 5).

Создание общеорганизационной среды с нужным уровнем психологической безопасности и неустанным стремлением к экспериментам, направленным на улучшение ситуации, — непростая задача. И это важный элемент для успешной программы API в долгосрочной перспективе. Но усилия на уровне компании — это только начало. Вам нужно создать аналогичный уровень доверия и экспериментирования в командах.

Поэтому мы посвятили отдельную главу концепции команд API (глава 8). Здесь мы обсудили роли внутри команды и более масштабную задачу по созданию и поддержке самих команд. Если вы в процессе руководства будете стремиться к постоянному совершенствованию, подкрепляя это целенаправленными усилиями по поддержке эффективности команд, то со временем вы сможете вырастить в своей компании сильную, здоровую культуру, ведущую к созданию качественных API.

Наконец, мы добавили третий фактор управления API — масштаб. Мы познакомили вас с ландшафтом API и обзором сложной системы с высоты. Последняя часть книги сосредоточена на объеме *системной оптимизации* и сопутствующих ей решениях. Мы ввели понятие ландшафтов API (глава 9) и их влияния на жизненный цикл этих API (глава 11). Работа на ландшафтном уровне может быть довольно сложной задачей. Здесь все ранее рассмотренные нами элементы — управление, продукты, культура и масштаб — объединяются в сложное сочетание взаимодействий.

В последних двух главах мы надеялись дать вам некоторые рекомендации и поделиться советами о том, каких проблем ожидать при создании уникального сочетания API-интерфейсов вашей компании, составляющих единый ландшафт системного уровня вашей организации.

В этой книге много информации, структур и моделей, но в основе управления API лежат четыре основных понятия: руководство, продукты, культура и масштаб. Неважно, какие у вас API, в какой сфере вы работаете и какого размера ваша компания. Все равно придется управлять API с учетом этих понятий. В идеале мы дали в книге инструменты, которые помогут вам начать уже сегодня.

Продолжайте готовиться к будущему

Будущее сложно предсказать, но мы уверены, что тенденция к связи между программами не исчезнет. Поскольку архитектура все больше опирается на взаимосвязанные или интегрированные компоненты, спрос на управление API будет расти, несмотря на изменения и развитие протоколов, форматов, стилей и языков, с помощью которых они создаются.

Действительно, с тех пор как мы выпустили первое издание этой книги всего несколько лет назад, наблюдается всплеск интереса к управлению группой API (мы называем ее *ландшафтом*). Все больше внимания уделяется снижению барьеров для использования, повторного применения и интеграции — тому, что мы обсуждали в 2019 году.

Мы попытались написать эту книгу так, чтобы она была вам полезна независимо от выбора технологий, с которыми вы сталкиваетесь. Ключевые концепции — руководство, продукт, культура и масштаб — всегда будут важны в сфере управления API. Поэтому, даже если все вокруг изменится, у вас будут концепции и модели, которые помогут с этим разобраться.

Вы можете использовать подход «API как продукт», чтобы упростить выбор в сфере дизайна и реализации. Он даст вам свободу создавать API, подходящие требованиям пользователей, а не полагаться на милость краткосрочных тенденций и популярных веяний в индустрии. Вы можете распределять решения по API, основываясь на целях, талантах сотрудников и обстановке в организации. Не пытайтесь скопировать корпоративную культуру последнего успешного стартапа.

Мы призываем вас принять сложность системы, которой вы управляете, а не бороться с ней. Поймите, как время и масштаб меняют принцип работы. Используйте жизненный цикл API-продукта и путешествие по ландшафту, чтобы очертить свой контекст. Поиграйте в «а что, если?..» с переменными ландшафта, чтобы оценить его. А что, если вырастет разнообразие? А что, если скорость больше не будет важна? Ответы на эти вопросы, может, и не будут провидческими, но точно дадут новую информацию. Они, несомненно, приведут к возможностям постоянного совершенствования вашей системы.

Не останавливайтесь

Сталкиваясь с большой проблемой в сложной и запутанной области, легко почувствовать себя подавленным. В начале книги мы обсуждали качество решений. Один из самых важных элементов принятия качественного решения — это доступная информация. К ней относятся знание того, как другие люди решали похожие проблемы, понимание контекста и эффекта, производимого вашим решением.

Очень важно потратить время на сбор информации об управлении API, чтобы принимать лучшие решения. Но если времени на это будет потрачено слишком много, вы сможете научиться на практике. Сталкиваясь с неопределенностью в сложной адаптивной системе, примите разумное решение — разбираться с проблемой поэтапно. Займитесь одним небольшим делом, извлеките из него уроки и переходите к следующему.

Лучший способ делать это — применить такие техники, как PDSA Деминга (см. раздел «Постепенное улучшение» на с. 157). Используйте полученные данные и создайте теорию. В соответствии с ней спланируйте эксперимент, найдите в организации безопасное место и проведите его. Измерьте результаты и начните снова. Для старта управления API не нужны гибкие процессы, методы бережливой разработки, доски технологии «Канбан», инструменты DevOps или микросервисная архитектура. Это полезно и уместно, но не нужно для начала работы. Все, что вам нужно, — это теория, точные измерения и желание работать и экспериментировать безопасно и последовательно.

Столкнувшись с такой сложной сферой, как управление API, продвигаться так — самое разумное решение. Вы можете начать прямо сейчас. Найдите в своем ландшафте API то, что можно улучшить, и используйте информацию из этой книги, чтобы провести эксперимент. Учитесь на ходу и растите над собой по мере обучения. Вскоре вы получите систему управления API, которая работает на вас так же, как вы работаете над ней.

Еще более хорошая новость: вы можете продолжать этот цикл решения небольших проблем так долго, как захотите. Это и есть *непрерывная часть непрерывного развития API*. Проблемы всегда будут разными, а решения будут развиваться вместе с технологиями и опытом. Но общий подход останется прежним.

Это долгий путь. Но он не обязательно должен быть трудным. Если вы вооружены знаниями об управлении API из этой книги и стремитесь найти решение, подходящее для уникальных потребностей вашей компании, у вас будет большое преимущество. Ваш путь будет ясным, а шансы на продвижение по нему — высоки.

Чем дальше мы продвигаемся, тем мы ближе к достижению целей. И продолжать это делать полезно для всех.

Об авторах

Мехди Меджуи — предприниматель в сфере API, соучредитель OAuth.io и создатель конференций APIDays — всемирной серии конференций по API, которые ежегодно проводятся в семи странах. Как ведущий экономист по API, Мехди консультирует лиц, принимающих решения в отношении API, о влиянии внедрения API на их стратегии цифровой трансформации на микро- и макроуровнях. Он разработал отраслевой ландшафт API, был соавтором доклада *Banking APIs: State of the Market* («API для банков: состояние рынка»), является экспертом Европейской комиссии по API для проекта «Цифровое правительство». Кроме того, он читает лекции по предпринимательству в цифровую эпоху в парижской бизнес-школе НЕС и является советником в нескольких стартапах по инструментам для API.

Эрик Уайлд — эксперт в разработке протоколов и структурированных данных, помогает организациям получить максимальную отдачу от API, консультируя их в отношении стратегии и программы. Эрик занимается разработкой инновационных технологий с момента появления интернета и активно сотрудничает с IETF и W3C. Он получил докторскую степень в Высшей технической школе Цюриха, преподавал в ней и в Университете Беркли, после работал в компаниях EMC, Siemens, CA Technologies и с недавнего времени в Axway.

Ронни Митра работает консультантом, помогая технологическим и бизнес-лидерам проводить масштабные цифровые преобразования. Соавтор книг «Микросервисы. От архитектуры до релиза»¹ и *Microservice Architecture* («Архитектура микросервисов: соединение принципов, методик и культуры»).

Майк Амундсен — всемирно известный писатель и спикер, консультирует организации по всему миру по вопросам сетевой архитектуры, веб-разработки и пересечению технологий и общества. Он работает с крупными и небольшими компаниями, чтобы помочь им извлечь выгоду и получить прибыль от возможностей API, микросервисов и цифровой трансформации, предоставляемых как потребителям, так и предприятиям.

¹ Митра Р., Надареишвили И. Микросервисы. От архитектуры до релиза. — СПб.: Питер, 2023.

Иллюстрация на обложке

На обложке изображена валлийская овчарка (вал. *Ci Defaid Cymreig*) — представитель породы домашних пастушьих собак типа колли, родом из Уэльса. Эти собаки бывают черно-белого, рыже-белого и трехцветного окраса, часто с мраморными отметинами. Конечности у валлийских овчарок длиннее, а грудная клетка и морда шире, чем у бордер-колли.

Валлийские овчарки — очень волевые и энергичные собаки. Обученные пастушьим обязанностям, могут исполнять их почти без человеческого вмешательства. Но у них нет низкой стойки и сильного визуального контакта, как у бордер-колли (характерные для волков черты хищника, облегчающие собаке управление стадом), поэтому их реже выбирают для присмотра за скотом.

Из-за того что их разводят ради поведенческих качеств, а не внешних черт, и часто скрещивают с бордер-колли, валлийские овчарки не признаются основными кинологическими организациями стандартизированной породой. В последние годы были предприняты попытки сохранить ее для домашних целей.

Большинство животных на обложках книг издательства O'Reilly находятся под угрозой исчезновения, и все они очень важны для нашего мира.

Изображение взято из книги Дж. Дж. Вуда *Animate Creation*.

Мехди Меджуи, Эрик Уайлд, Ронни Митра, Майк Амундсен

**Непрерывное развитие API. Правильные решения
в изменчивом технологическом ландшафте**

2-е издание

Перевел с английского С. Черников

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>Т. Сажина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>М. Молчанова, Н. Терех</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 12.2022. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 12.10.22. Формат 70×100/16. Бумага офсетная. Усл. п. л. 29,670. Тираж 500. Заказ 0000.